[강의] 나를 바꾸는 C++3

작성자: 오희성[Heesung Oh]

3부. 제네릭 프로그래밍과 표준 라이브러리 **403**
10. 템플릿과 제네릭 프로그래밍 **403**
10.1 제네릭 프로그래밍 404
10.1.1 코드 일반화 404
10.1.2 타입에 독립적인 코드와 요구 조건 404
10.1.3 객체지향 다형성과 정적 다형성 405
10.2 함수 템플릿 407
10.2.1 함수 템플릿 선언 407
10.2.2 템플릿 인수 추론 408
10.2.3 명시적 템플릿 인수 (Explicit Template Argument) 409
10.2.4 함수 오버로딩과 템플릿 410
10.2.5 템플릿 인스턴스화 410
10.3 클래스 템플릿 411
10.3.1 클래스 템플릿 선언 412
10.3.2 멤버 함수 정의 413
10.3.3 템플릿과 헤더 파일 414
10.3.4 클래스 템플릿 인수 추론 416
10.3.5 기본 템플릿 인수 417
10.3.6 별칭 템플릿 418
10.3.7 비 타입 템플릿 매개변수 418
10.4 템플릿 특수화 419
10.4.1 전체 특수화 420
10.4.2 부분 특수화 421
10.4.3 함수 템플릿 오버로딩과 특수화 423
10.4.4 특수화 사용 시 주의점 424
10.5 가변 인수 템플릿 425
10.5.1 매개변수 팩 425
10.5.2 팩 확장 425
10.5.3 재귀적 전개 427
10.5.4 폴드 표현식 427
10.5.5 여러 인수 전달 428
10.6 Concepts와 제약 조건 429
10.6.1 템플릿 제약의 필요성 429
10.6.2 표준 Concept 430
10.6.3 requires 절 432
10.6.4 requires 표현식 433
10.6.5 사용자 정의 Concept 434
10.6.6 제약 조건과 오버로딩 435
10.7 타입 특성과 조건부 코드 436
10.7.1 type_traits 437
10.7.2 타입 판정과 타입 변환 438
10.7.3 if constexpr 439
10.7.4 스피네이 (SFINAE) 와 enable_if 개요 441
10.7.5 Concepts와 기존 제약 방식의 선택 442
10.8 전달 참조 (Forwarding Reference) 와 완벽 전달 (Perfect Forwarding) 443
10.8.1 참조 축약 443
10.8.2 전달 참조 443
10.8.3 std::forward 445
10.8.4 완벽 전달 446
10.8.5 std::move와 std::forward의 차이 448
10.8.6 완벽 전달의 사용 기준과 종합 예제 449
10.9 학습 정리 453
10.10 학습 과제 454
11. 표준 라이브러리와 순차 컨테이너 **456**
11.1 표준 라이브러리의 구조 456
11.1.1 표준 라이브러리와 STL 456
11.1.2 컨테이너 457
11.1.3 반복자 457
11.1.4 알고리즘 458
11.1.5 함수 객체 (Function Object) 458

11.1.6 자료구조와 알고리즘의 분리 459
11.2 컨테이너 공통 기능 459
11.2.1 원소 타입과 크기 459
11.2.2 원소 접근 460
11.2.3 삽입과 제거 460
11.2.4 begin과 end 461
11.2.5 복사와 이동 462
11.2.6 반복자·포인터·참조의 무효화 462
11.3 std::array 463
11.3.1 C 배열과 std::array 464
11.3.2 고정 크기 464
11.3.3 반복자 지원 464
11.3.4 배열 대입과 비교 465
11.3.5 함수 인수로 전달하기 465
11.4 std::vector 466
11.4.1 연속 메모리 466
11.4.2 원소 추가와 제거 466
11.4.3 size와 capacity 467
11.4.4 재할당 467
11.4.5 reserve와 resize 468
11.4.6 반복자·포인터·참조의 무효화 468
11.4.7 push_back과 emplace_back 469
11.5 std::deque 470
11.5.1 양쪽 끝 삽입과 제거 470
11.5.2 메모리 구조 471
11.5.3 임의 접근 471
11.5.4 vector와의 차이와 무효화 471
11.6 std::list와 std::forward_list 472
11.6.1 연결 리스트 472
11.6.2 양방향 리스트와 단방향 리스트 472
11.6.3 삽입과 제거 472
11.6.4 순회 비용 473
11.6.5 연결 리스트 선택 기준 473
11.7 std::string 474
11.7.1 문자열 생성과 대입 474
11.7.2 연결과 비교 474
11.7.3 검색과 부분 문자열 475
11.7.4 삽입과 제거 475
11.7.5 숫자와 문자열 변환 476
11.7.6 문자열의 저장 공간 476
11.8 순차 컨테이너 선택 477
11.8.1 고정 크기와 가변 크기 477
11.8.2 연속 메모리와 캐시 지역성 477
11.8.3 삽입과 제거 위치 477
11.8.4 임의 접근 478
11.8.5 컨테이너 선택 기준 478
11.9 학습 정리 478
11.10 학습 과제 479
12. 연관 컨테이너와 컨테이너 어댑터 **481**
12.1 std::set과 std::multiset 481
12.1.1 정렬된 키와 키 동등성 481
12.1.2 중복 허용 여부 482
12.1.3 삽입, 탐색, 범위 검색과 제거 482
12.1.4 비교 함수와 정렬 기준 483
12.1.5 원소 수정 제한과 노드 핸들 484
12.2 std::map과 std::multimap 484
12.2.1 키와 값 485
12.2.2 operator[]와 at 485
12.2.3 insert, emplace와 try_emplace 486
12.2.4 insert_or_assign과 값 갱신 486
12.2.5 구조적 바인딩을 이용한 순회 487
12.3 비 정렬 연관 컨테이너 488
12.3.1 std::unordered_set과 std::unordered_multiset 488
12.3.2 std::unordered_map과 std::unordered_multimap 488
12.3.3 해시 함수, 동등 비교와 버킷 489
12.3.4 충돌, 적재율과 재 해시 489

12.3.5 사용자 정의 키 490
12.4 정렬 연관 컨테이너와 비 정렬 연관 컨테이너 490
12.4.1 정렬 여부와 순회 순서 490
12.4.2 평균 탐색 비용과 최악 탐색 비용 491
12.4.3 메모리 구조와 캐시 지역성 491
12.4.4 범위 검색과 선택 기준 491
12.5 std::stack 492
12.5.1 후입선출 492
12.5.2 기반 컨테이너와 제한된 인터페이스 492
12.5.3 스택 활용 493
12.6 std::queue 493
12.6.1 선입선출 493
12.6.2 기반 컨테이너와 제한된 인터페이스 494
12.6.3 작업 대기열과 메시지 처리 494
12.7 std::priority_queue 494
12.7.1 우선순위 기반 처리 494
12.7.2 힙 구조와 top 495
12.7.3 비교 함수 495
12.7.4 최대값과 최소값 우선순위 496
12.8 사용자 정의 타입과 연관 컨테이너 497
12.8.1 비교 연산자 497
12.8.2 비교 함수 객체 497
12.8.3 해시 함수 498
12.8.4 동등 비교와 해시 계약 498
12.9 컨테이너와 객체 수명 499
12.9.1 값 객체 저장 499
12.9.2 원시 포인터 저장 500
12.9.3 unique_ptr 저장 500
12.9.4 shared_ptr 저장 501
12.9.5 다형 객체 저장 501
12.9.6 반복자·포인터·참조의 유효성과 무효화 501
12.10 컨테이너 선택 기준 503
12.11 학습 정리 503
12.12 학습 과제 504
13. 반복자**,** 함수 객체**,** 람다와 알고리즘 **505**
13.1 반복자의 개념 505
13.1.1 반복자와 포인터 505
13.1.2 반복 구간과 반 개방 (Half-open intervals) 구간 506
13.1.3 begin, end와 범위 506
13.1.4 const_iterator와 반복자의 const 508
13.2 반복자 범주 508
13.2.1 입력 반복자와 단일 순회 508
13.2.2 출력 반복자와 단일 순회 509
13.2.3 순방향 반복자와 다중 순회 509
13.2.4 양방향 반복자 509
13.2.5 임의 접근 반복자 509
13.2.6 연속 반복자 510
13.3 반복자 도구 510
13.3.1 advance, next와 prev 510
13.3.2 distance 511
13.3.3 역방향 반복자 511
13.3.4 삽입 반복자 511
13.3.5 스트림 반복자 512
13.3.6 반복 중 변경과 반복자 무효화 513
13.4 Callable과 함수 유틸리티 514
13.4.1 함수 포인터 514
13.4.2 함수 객체와 상태 515
13.4.3 표준 함수 객체 515
13.4.4 std::function과 Callable 저장 516
13.4.5 std::invoke와 멤버 포인터 517
13.4.6 std::mem_fn 518
13.4.7 std::bind와 std::placeholders 519
13.4.8 참조 바인딩과 std::ref 520
13.4.9 std::bind_front 520
13.4.10 Callable의 선택 521
13.5 람다 표현식 521

13.5.1 람다의 기본 구조 521
13.5.2 값 캡처와 참조 캡처 522
13.5.3 초기화 캡처 523
13.5.4 mutable 523
13.5.5 반환 형식 524
13.5.6 제네릭 람다 524
13.5.7 클로저 타입과 클로저 객체 525
13.5.8 람다 캡처와 객체 수명 526
13.6 검색과 판정 알고리즘 527
13.6.1 find와 find_if 527
13.6.2 count와 count_if 528
13.6.3 all_of, any_of와 none_of 529
13.7 정렬과 탐색 알고리즘 529
13.7.1 sort와 stable_sort 529
13.7.2 partial_sort와 nth_element 530
13.7.3 lower_bound, upper_bound와 equal_range 531
13.7.4 binary_search와 정렬 사전 조건 532
13.8 복사, 이동, 변환과 제거 532
13.8.1 copy와 move 532
13.8.2 transform 533
13.8.3 remove와 remove_if 534
13.8.4 erase-remove와 std::erase_if 534
13.8.5 unique와 연속 중복 제거 535
13.9 수치 알고리즘 536
13.9.1 accumulate 536
13.9.2 inner_product 536
13.9.3 partial_sum과 adjacent_difference 537
13.9.4 reduce와 transform_reduce 537
13.10 std::function과 콜백 538
13.10.1 호출 가능한 객체 저장 538
13.10.2 std::function 539
13.10.3 타입 소거 540
13.10.4 콜백 인터페이스 설계 541
13.10.5 std::function의 비용과 선택 기준 542
13.11 Ranges 543
13.11.1 Range의 개념 543
13.11.2 std::ranges 알고리즘 543
13.11.3 Projection 544
13.11.4 View와 지연 평가 545
13.11.5 View 조합 545
13.11.6 View와 반환 반복자의 수명 546
13.12 알고리즘 선택 546
13.12.1 반복자 범주와 사전 조건 546
13.12.2 시간 복잡도와 메모리 사용 547
13.12.3 직접 반복문과 표준 알고리즘 547
13.12.4 가독성과 성능 548
13.13 학습 정리 548
13.14 학습 과제 549
14. 유틸리티 타입과 데이터 표현 **551**
14.1 std::pair와 std::tuple 551
14.1.1 두 값과 여러 값을 묶는 타입 551
14.1.2 생성과 std::get 552
14.1.3 std::tie, std::ignore와 구조적 바인딩 552
14.1.4 여러 반환값과 std::apply 553
14.2 std::optional 554
14.2.1 값의 부재 554
14.2.2 생성, 대입, reset과 emplace 555
14.2.3 값 접근과 기본값 556
14.2.4 포인터와 optional의 차이 556
14.3 std::variant 557
14.3.1 여러 타입 중 하나 557
14.3.2 대안 타입과 std::monostate 558
14.3.3 index, holds_alternative, std::get과 std::get_if 558
14.3.4 std::visit 559
14.3.5 상태 표현 560
14.3.6 valueless_by_exception 561

14.3.7 상속 기반 다형성과의 차이 561
14.4 std::any 562
14.4.1 복사 가능한 임의 타입 저장 562
14.4.2 빈 상태와 저장 타입 확인 562
14.4.3 any_cast 562
14.4.4 variant와 any의 차이 563
14.4.5 런타임 타입 저장의 비용 564
14.5 std::reference_wrapper 564
14.5.1 참조를 복사 가능한 객체로 저장 564
14.5.2 컨테이너와 참조 565
14.5.3 std::ref, std::cref와 get 565
14.5.4 대상 객체의 수명 566
14.6 std::span 566
14.6.1 포인터와 길이를 묶는 비 소유 구간 566
14.6.2 배열과 컨테이너 전달 567
14.6.3 정적 크기와 동적 크기 567
14.6.4 span의 const와 원소의 const 568
14.6.5 부분 구간 568
14.6.6 수명과 무효화 569
14.7 std::string_view 569
14.7.1 비 소유 문자열 569
14.7.2 문자열 복사 방지 570
14.7.3 부분 문자열과 함수 매개변수 570
14.7.4 널 종료와 data 571
14.7.5 댕글링 뷰 571
14.8 비트와 바이트 572
14.8.1 std::byte 572
14.8.2 std::bitset 573
14.8.3 비트 연산과 비트 플래그 573
14.8.4 C++20 비트 유틸리티 574
14.8.5 std::bit_cast 575
14.8.6 엔디언 (Endian) 575
14.8.7 객체 표현 시 주의점 575
14.9 타입 안전한 데이터 표현 576
14.9.1 불리언 플래그의 한계 576
14.9.2 enum class와 의미 타입 577
14.9.3 optional, variant와 expected의 선택 577
14.9.4 유효하지 않은 상태 줄이기 578
14.10 학습 정리 579
14.11 학습 과제 579
15. 입출력과 표준 유틸리티 **583**
15.1 스트림 입출력 583
15.1.1 입력 스트림과 출력 스트림 583
15.1.2 스트림 상태와 오류 플래그 584
15.1.3 입력 실패 복구 585
15.1.4 형식 지정과 스트림 상태 586
15.1.5 사용자 정의 타입의 입출력 587
15.1.6 버퍼링과 출력 시점 588
15.1.7 형식 입력과 행 입력의 혼합 588
15.2 문자열 스트림 589
15.2.1 std::istringstream 589
15.2.2 std::ostringstream 590
15.2.3 문자열 파싱과 조립 590
15.2.4 입력 전체와 스트림 상태 확인 591
15.2.5 문자열 스트림 재사용 592
15.3 형식화된 출력 593
15.3.1 iomanip 593
15.3.2 std::format 594
15.3.3 형식 문자열과 서식 지정 594
15.3.4 std::format_to 595
15.3.5 사용자 정의 형식화 개요 595
15.4 텍스트 파일 입출력 596
15.4.1 std::ifstream과 std::ofstream 596
15.4.2 std::fstream과 파일 열기 모드 597
15.4.3 행 단위 입력 598
15.4.4 구분자 기반 입력 599

15.4.5 파일 오류 처리와 자원 정리 600
15.5 바이너리 파일 입출력 601
15.5.1 바이너리 모드 601
15.5.2 read와 write 601
15.5.3 파일 위치 602
15.5.4 고정 길이 데이터와 문자열 603
15.5.5 객체를 그대로 저장할 때의 문제 604
15.5.6 파일 형식과 버전 관리 605
15.6 파일 시스템 606
15.6.1 std::filesystem::path 606
15.6.2 파일과 디렉터리 확인 607
15.6.3 디렉터리 생성과 순회 607
15.6.4 파일 복사, 이름 변경과 삭제 608
15.6.5 플랫폼 독립 경로 609
15.6.6 예외와 std::error_code 609
15.7 난수 610
15.7.1 의사 난수 610
15.7.2 난수 엔진과 분포 610
15.7.3 시드와 엔진 상태 611
15.7.4 재현 가능한 난수 612
15.7.5 대표 분포와 셔플 613
15.7.6 rand와 random 라이브러리의 차이 613
15.8 시간 614
15.8.1 std::chrono::duration 614
15.8.2 std::chrono::time_point 615
15.8.3 시계의 종류와 선택 615
15.8.4 시간 단위 변환 616
15.8.5 경과 시간 측정 616
15.8.6 시간 기반 처리 617
15.9 학습 정리 618
15.10 학습 과제 619
16. 동시성 프로그래밍 **621**
16.1 동시성과 병렬성 621
16.1.1 프로세스와 스레드 621
16.1.2 동시성과 병렬성의 차이 621
16.1.3 공유 상태와 데이터 경쟁 622
16.1.4 메모리 가시성과 happens-before 622
16.2 std::thread와 std::jthread 623
16.2.1 스레드 생성과 함수 실행 623
16.2.2 인수 전달과 객체 수명 624
16.2.3 join, joinable과 detach 625
16.2.4 예외 경로와 스레드 수명 626
16.2.5 std::jthread와 자동 결합 626
16.2.6 stop_token을 이용한 협력적 중지 627
16.3 뮤텍스와 잠금 627
16.3.1 std::mutex와 임계 구역 627
16.3.2 std::lock_guard 628
16.3.3 std::scoped_lock 628
16.3.4 std::unique_lock 629
16.3.5 RAII와 잠금 해제 629
16.4 교착 상태 630
16.4.1 교착 상태의 네 조건 630
16.4.2 잠금 순서 631
16.4.3 여러 뮤텍스의 안전한 잠금 631
16.4.4 잠금 범위와 외부 호출 631
16.5 조건 변수 632
16.5.1 std::condition_variable 632
16.5.2 공유 상태와 조건식 632
16.5.3 대기, 통지와 가짜 깨어남 (spurious wakeup) 633
16.5.4 생산자와 소비자 633
16.5.5 시간 제한 대기 635
16.6 비동기 작업과 결과 전달 635
16.6.1 std::future와 공유 상태 635
16.6.2 std::promise 636
16.6.3 std::packaged_task 636
16.6.4 std::async와 실행 정책 637

16.6.5 결과와 예외 전달 637
16.7 원자적 연산 638
16.7.1 std::atomic 638
16.7.2 원자적 읽기, 쓰기와 읽기-수정-쓰기 639
16.7.3 비교와 교환 639
16.7.4 atomic wait와 notify 639
16.7.5 메모리 순서 개요 640
16.7.6 뮤텍스와 원자적 연산의 선택 641
16.8 C++20 동기화 도구 641
16.8.1 std::latch 641
16.8.2 std::barrier 642
16.8.3 std::counting_semaphore 642
16.9 병렬 알고리즘 개요 643
16.9.1 실행 정책 643
16.9.2 병렬 실행의 조건 643
16.9.3 부작용, 데이터 경쟁과 예외 644
16.9.4 지원 범위와 성능 측정 644
16.10 동시성 설계 기준 644
16.10.1 공유 상태 줄이기 644
16.10.2 작업 단위와 소유권 645
16.10.3 스레드 안전한 인터페이스 645
16.10.4 종료와 취소 646
16.10.5 성능 측정과 과도한 병렬화 피하기 646
16.11 학습 정리 647
16.12 학습 과제 647

3부. 제네릭 프로그래밍과 표준 라이브러리

10. 템플릿과 제네릭 프로그래밍

지금까지 클래스를 이용해 객체의 책임을 나누고, 상속과 가상 함수를 통해 서로 다른 객체를 하나의 기반 타입으로 다루는 방법을 학습했다. 가상 함수 기반 다형성은 실행 시간에 실제 객체의 타입을 확인하여 재정의된 함수를 호출한다. 이 방식은 서로 다른 구체 객체를 하나의 컨테이너에 저장하거나 실행 중 구현을 교체할 때 유용하다.

C++은 컴파일 시점에 타입을 결정하여 코드를 생성하는 또 다른 일반화 방법도 제공한다. 템플릿(template)을 사용하면 함수나 클래스가 특정 타입 하나에만 의존하지 않고, 여러 타입에 공통으로 적용되는 구조를 정의할 수 있다. 컴파일러는 실제로 사용된 타입과 값에 맞는 구체 코드를 생성한다.

두 값을 비교하여 큰 값을 반환하는 함수가 있다고 하자.

```
int GetMax(int left, int right)
{
    return left < right ? right : left;
}
double GetMax(double left, double right)
{
    return left < right ? right : left;
}
```

두 함수는 매개변수와 반환 타입만 다르고 수행하는 알고리즘은 같다. C에서는 이러한 중복을 줄이기 위해 함수형 매크로를 사용할 수 있다.

```
#define GET_MAX(left, right) ((left) < (right) ? (right) : (left))
```

매크로는 전달된 코드를 전처리 과정에서 그대로 치환하므로 여러 타입에 사용할 수 있다.

```
int maxCount = GET_MAX(10, 20);
double maxSpeed = GET_MAX(3.5, 7.2);
```

그러나 매크로는 타입 검사를 받지 않으며 실제 함수도 아니다. 인수로 전달한 표현식이 여러 번 평가될 수 있어 예상하지 못한 결과가 발생할 수도 있다.

```
int value = 10;
int result = GET_MAX(value++, 20);
```

GET_MAX의 정의에서 선택된 인수가 다시 사용되므로 value++가 두 번 평가될 가능성이 있다. 괄호를 빠뜨리면 연산자 우선순위로 인한 문제도 발생할 수 있다.

C++은 이러한 매크로의 한계를 피하면서 타입에 독립적인 코드를 작성할 수 있도록 템플릿을 제공한다. 타입을 템플릿 매개변수로 분리하면 여러 타입에 사용할 수 있는 하나의 함수 템플릿으로 표현할 수 있다.

```
template <typename T>
T GetMax(T left, T right)
{
    return left < right ? right : left;
}
```

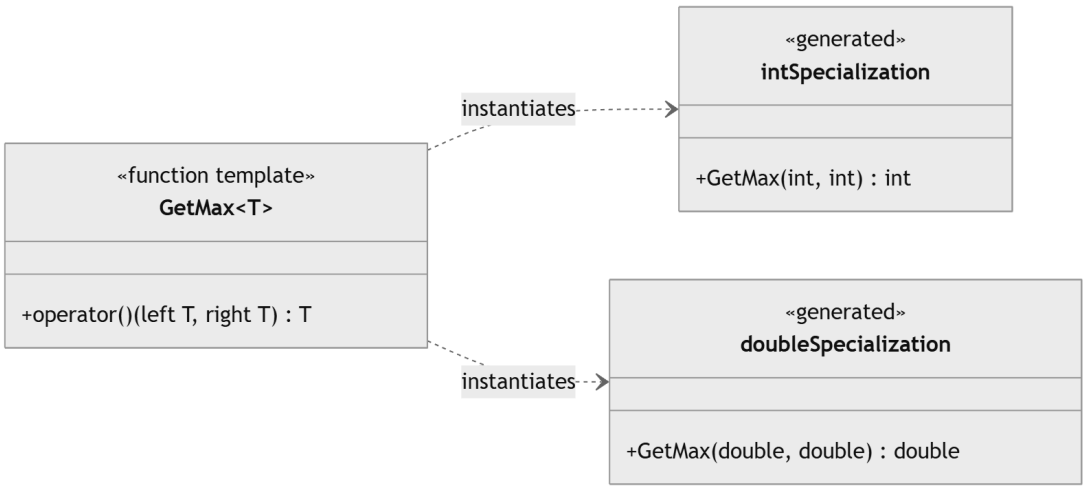
GetMax()를 호출하면 컴파일러는 전달된 인수의 타입을 추론하고, 해당 타입에 맞는 함수를 생성한다.

```
int maxCount = GetMax(10, 20);
double maxSpeed = GetMax(3.5, 7.2);
```

GetMax(10, 20)에서는 T가 int로 추론되고, GetMax(3.5, 7.2)에서는 T가 double로 추론된다. 함수 템플릿은 매크로와 달리 C++의 타입 검사를 받으며, 인수도 일반 함수 호출과 같은 규칙으로 평가된다.

이렇게 특정 타입이 아닌 일반적인 타입에 같은 알고리즘을 여러 타입에 적용할 수 있도록 코드를 일반화하는 프로그래밍 방식을 제네릭 프로그래밍이라고 한다.

제네릭 프로그래밍은 단순히 타입 이름을 T로 바꾸는 작업이 아니다. GetMax()에 사용하는 타입은 두 값을 < 연산자로 비교할 수 있어야 하며, 비교를 통해 선택한 값을 반환할 수 있어야 한다. 따라서 제네릭 코드는 특정 타입의 이름이 아니라 그 타입이 제공해야 하는 연산과 의미를 기준으로 작성한다.



이제 제네릭 프로그래밍의 개념과 코드 일반화의 의미를 좀 더 구체적으로 살펴보자.

10.1 제네릭 프로그래밍

제네릭 프로그래밍(generic programming)은 타입이나 값의 차이를 매개변수로 분리하고, 여러 대상에 공통으로 적용되는 알고리즘과 자료 구조를 작성하는 방식이다. 함수 템플릿은 알고리즘을 일반화하고, 클래스 템플릿은 저장 구조와 객체 타입을 일반화한다.

제네릭 코드의 핵심은 모든 타입을 무조건 허용하는 데 있지 않다. 알고리즘이 요구하는 연산을 제공하는 타입이라면 같은 구현을 사용할 수 있도록 만드는 데 있다.

10.1.1 코드 일반화

플레이어 점수와 몬스터 체력에 값을 제한하는 함수를 각각 작성할 수 있다.

```
int ClampScore(int value, int minimum, int maximum)
{
    return (value < minimum)? minimum
          :(maximum < value)? maximum : value;
}

float ClampSpeed(float value, float minimum, float maximum)
{
    return (value < minimum)? minimum
          :(maximum < value)? maximum : value;
}
```

두 함수는 매개변수와 반환 타입만 다르다. 비교와 반환 과정은 동일하므로 타입을 템플릿 매개변수로 분리할 수 있다.

```
template <typename T>
T Clamp(T value, T minimum, T maximum)
{
    return (value < minimum)? minimum
          :(maximum < value)? maximum : value;
}
```

호출할 때 사용한 인수의 타입으로 T가 결정된다.

```
int score = Clamp(1200, 0, 999);
float speed = Clamp(12.5f, 0.0f, 10.0f);
```

첫 호출에서는 T가 int, 두 번째 호출에서는 float로 추론된다. 컴파일러는 각 타입에 맞는 함수 특수화를 인스턴스화한다.

코드 일반화는 형태가 비슷하다는 이유만으로 수행해서는 안 된다. 두 코드가 우연히 현재 같은 모양을 가졌더라도 변경 이유와 의미가 다르면 하나로 묶지 않는 편이 낫다. 체력 제한과 좌표 제한이 같은 비교 규칙을 사용하고 같은 의미의 범위 보정을 수행한다면 일반화할 수 있다. 반면 세금 계산과 공격력 계산이 현재 같은 곱셈식을 사용한다는 이유만으로 하나의 템플릿으로 묶으면 의미가 불분명해진다.

10.1.2 타입에 독립적인 코드와 요구 조건

템플릿은 구체 타입의 이름에서 독립적이지만 타입의 능력과 의미에서는 독립적이지 않다. Clamp()는 세 값을 비교할 수 있어야 한다. 정확히는 value < minimum과 maximum < value가 논리 조건으로 사용할 수 있어야 하고, minimum, maximum, value를 T로 반환할 수 있어야 한다.

사용자 정의 타입도 이러한 요구를 만족하면 사용할 수 있다.

```
class Level
{
public:
    explicit Level(int value) : value(value)
    {
    }
    int GetValue() const
    {
        return value;
    }
    friend
    bool operator<(const Level& left, const Level& right)
    {
        return left.value < right.value;
    }

private:
    int value;
};
```

Level은 < 연산을 제공하므로 Clamp()에 전달할 수 있다.

```
Level current(120);
Level minimum(1);
Level maximum(99);

Level result = Clamp(current, minimum, maximum);
```

Clamp()는 Level의 내부에 정수 멤버가 있다는 사실을 알지 못한다. 비교 가능한 값이라는 요구 조건만 사용한다.

연산이 문법적으로 가능하더라도 의미가 올바르다는 보장은 없다. 예를 들어 < 연산자가 매번 임의의 결과를 반환한다면 코드는 컴파일되지만 범위 제한 알고리즘의 의미는 성립하지 않는다. Concepts는 문법적인 요구 조건과 일부 타입 관계를 표현할 수 있지만 연산의 수학적 의미나 도메인 규칙까지 자동으로 검증하지는 않는다.

<제네릭 알고리즘 설계 관점>

```
- 연산 식별: 실제 사용하는 연산 파악
- 조건 최소화: 불필요하게 강한 타입 제약 배제
- 중요도 평가: 반환 타입 및 예외 발생 여부 검증
- 명시적 정의: Concept 및 문서화 적극 활용
- 경계 재설정: 타입별 예외 증가 시 일반화 수준 재평가
```

10.1.3 객체지향 다형성과 정적 다형성

객체지향 다형성과 템플릿 기반 정적 다형성은 서로 다른 시점에 구현을 선택한다.
가상 함수 기반 공격 행동을 구성해 보자.

```
class IAttackBehavior
{
public:
    virtual ~IAttackBehavior() = default;
    virtual int CalculateDamage() const = 0;
};

class MeleeAttack : public IAttackBehavior
{
public:
    explicit MeleeAttack(int damage) : damage(damage)
    {
    }
    int CalculateDamage() const override
    {
        return damage;
    }
private:
    int damage;
```

```
};
```

호출 코드는 기반 클래스 참조만 알고 있다.

```
int ExecuteAttack(const IAttackBehavior& behavior)
{
    return behavior.CalculateDamage();
}
```

실행 시간에 객체의 동적 타입을 확인하여 MeleeAttack::CalculateDamage()가 호출된다.
정적 다형성에서는 공격 정책을 템플릿 인수로 받는다.

```
#include <utility>

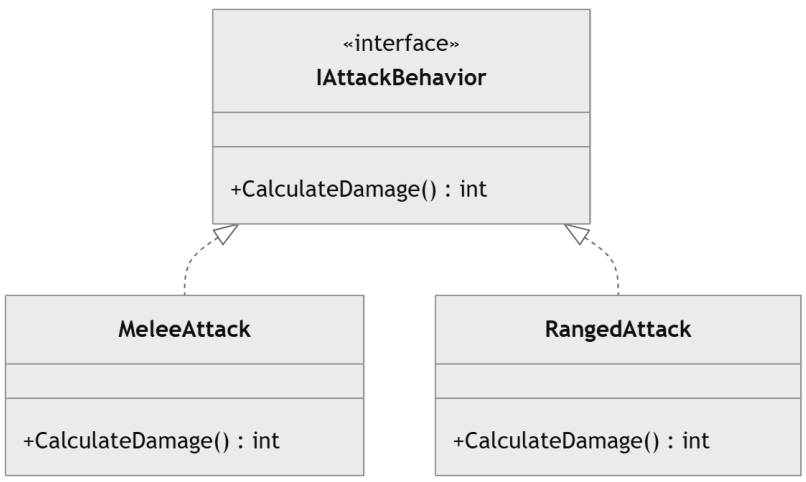
template <typename AttackPolicy>
class AttackProcessor
{
public:
    explicit AttackProcessor(AttackPolicy policy)
        : policy(std::move(policy))
    {
    }

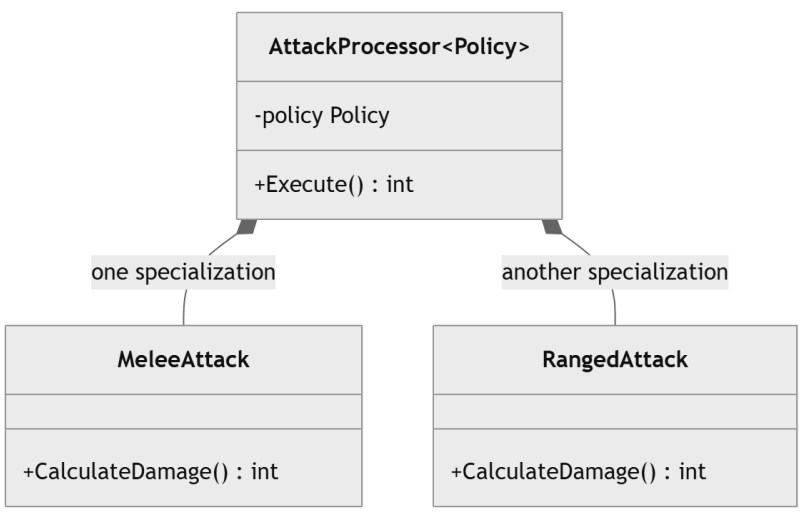
    int Execute() const
    {
        return policy.CalculateDamage();
    }

private:
    AttackPolicy policy;
};

MeleeAttack meleeAttack(20);
AttackProcessor<MeleeAttack> processor(meleeAttack);
int damage = processor.Execute();
```

AttackProcessor<MeleeAttack>의 구체 타입은 컴파일 시점에 결정된다. 컴파일러는 AttackPolicy가 MeleeAttack인 코드를 생성하며, 호출을 인라인으로 전개할 수도 있다.





<객체지향 다형성, 정적 다형성 비교>

기준	객체지향 다형성	정적 다형성
구현 선택 시점	실행 시간	컴파일 시점
공통 형식	기반 클래스와 가상 함수	템플릿과 요구 조건
서로 다른 객체의 단일 컨테이너 저장	자연스러움	별도 래퍼나 타입 소거가 필요할 수 있음
실행 중 구현 교체	기반 포인터 교체로 가능	구체 템플릿 타입이 달라지면 객체 타입도 달라짐
호출 비용	간접 호출이 발생할 수 있음	인라인과 최적화 가능성이 큼
코드 크기	공통 가상 함수 호출 코드 공유	타입별 인스턴스화로 증가할 수 있음
컴파일 시간	비교적 작음	템플릿 분석과 인스턴스화 비용이 큼
ABI 경계	안정적인 인터페이스 설계에 유리	구현 노출과 재컴파일 의존이 큼

객체지향 다형성은 실행 중 서로 다른 타입을 같은 역할로 관리할 때 적합하다. 정적 다형성은 구체 타입이 컴파일 시점에 알려지고 타입별 최적화가 중요한 경우에 적합하다. 하나의 프로그램에서 두 방식을 함께 사용할 수도 있다. GameObject 계층은 가상 함수로 관리하면서, 각 객체 내부의 수치 계산이나 정책은 템플릿으로 구성하는 방식이 가능하다.

10.2 함수 템플릿

함수 템플릿(function template)은 함수의 타입이나 값을 템플릿 매개변수로 분리한 정의이다. 함수 템플릿 자체는 특정 타입의 함수가 아니라 함수를 생성하기 위한 틀이다.

10.2.1 함수 템플릿 선언

함수 템플릿은 template 선언과 템플릿 매개변수 목록으로 시작한다.

```
template <typename T>
T Square(T value)
{
    return value * value;
}
```

typename T는 T가 타입 템플릿 매개변수임을 나타낸다. 이 위치에서 class T를 사용해도 의미는 같다.

```
template <class T>
T Square(T value)
{
    return value * value;
}
```

모던 C++ 코드에서는 타입이라는 의미가 더 직접적으로 드러나는 typename을 많이 사용한다.
템플릿 매개변수는 여러 개 선언할 수 있다.

```
template <typename Left, typename Right>
auto Add(const Left& left, const Right& right)
{
    return left + right;
}

int integerResult = Add(10, 20);
double realResult = Add(10, 2.5);
```

두 번째 호출에서는 Left가 int, Right가 double로 추론되고, left + right의 타입으로 반환 타입이 결정된다.
타입 이름은 T, U처럼 짧게 적을 수 있지만 역할이 분명한 경우 의미 있는 이름이 더 좋다.

```
template <typename ValueType, typename Predicate>
bool Check(const ValueType& value, Predicate predicate)
{
    return predicate(value);
```

```
}
```

템플릿 정의에서도 일반 함수와 같은 범위 규칙이 적용된다. 템플릿 매개변수 이름은 템플릿 선언이 적용되는 함수 선언과 정의 안에서 사용할 수 있다.

10.2.2 템플릿 인수 추론

함수 템플릿을 호출할 때 템플릿 인수를 생략하면 컴파일러가 함수 인수로부터 타입을 추론한다.

```
template <typename T>
T GetMin(T left, T right)
{
    return right < left ? right : left;
}

int first = GetMin(10, 20);          // T = int
double second = GetMin(3.5, 1.2);    // T = double
```

하나의 T가 두 매개변수에 사용되면 두 인수에서 같은 타입이 추론되어야 한다.

```
// auto value = GetMin(10, 2.5);
//      T를 int와 double 중 하나로 결정할 수 없음
```

함수 호출의 일반적인 암시적 변환은 템플릿 인수 추론이 끝난 뒤 적용된다. 두 인수에서 서로 다른 타입이 추론되면 컴파일러가 임의로 공통 타입을 선택하지 않는다.

타입을 직접 지정하거나 템플릿 매개변수를 두 개로 분리할 수 있다.

```
double value = GetMin<double>(10, 2.5);

template <typename Left, typename Right>
auto GetMinMixed(const Left& left, const Right& right)
{
    return right < left ? right : left;
}
```

조건 연산자의 두 피연산자에서 공통으로 사용할 수 있는 타입이 결정되면 혼합 타입 결과를 반환한다. 반환 타입과 변환 규칙이 의도에 맞는지 별도로 확인해야 한다.

값 매개변수에서는 인수의 최상위 const와 참조성이 T에 포함되지 않는다.

```
#include <type_traits>

template <typename T>
void InspectValue(T value)
{
    static_assert(std::is_same_v<T, int>);
    value = 20;
}

const int number = 10;
InspectValue(number);
```

number는 const int이지만 복사된 값 매개변수의 T는 int이다. 원본 객체의 const는 유지되며 복사본의 타입 추론에서는 최상위 const가 제거된다.
참조 매개변수에서는 참조 대상의 const가 보존된다.

```
#include <type_traits>

template <typename T>
void InspectReference(T& value)
{
    static_assert(std::is_same_v<T, const int>);
}

const int number = 10;
```

```
InspectReference(number);
```

T& 매개변수에 const int 좌측값을 전달하면 T는 const int로 추론된다.
배열을 값 형태의 매개변수로 받으면 포인터 변환이 일어난다.

```
#include <type_traits>

template <typename T>
void InspectArrayValue(T value)
{
    static_assert(std::is_same_v<T, int*>);
}

int values[5]{};
InspectArrayValue(values);
```

배열 참조를 사용하면 원소 타입과 크기를 보존할 수 있다.

```
#include <cstddef>

template <typename T, std::size_t Size>
constexpr std::size_t ArraySize(const T (&)[Size]) noexcept
{
    return Size;
}

int values[5]{};
static_assert(ArraySize(values) == 5);
```

const T&는 복사를 피하면서 상수 객체와 임시 객체를 받을 수 있어 읽기 전용 제네릭 함수에서 자주 사용된다.

```
template <typename T>
const T& GetLarger(const T& left, const T& right)
{
    return left < right ? right : left;
}
```

이 함수는 전달된 객체 중 하나를 참조로 반환한다. 임시 객체를 인수로 전달하고 반환 참조를 저장하면 호출 표현식의 전체 표현식이 끝난 뒤 댕글링 참조가 될 수 있으므로 수명 관계를 확인해야 한다.

10.2.3 명시적 템플릿 인수 (Explicit Template Argument)

템플릿 인수를 함수 이름 뒤의 꺾쇠괄호 (Angle braket) 에 직접 지정할 수 있다.

```
template <typename T>
T Convert(int value)
{
    return static_cast<T>(value);
}

// <double> : 명시적 템플릿 인수
// 전달된 인수(10) 은 반환 타입 T 추론 불가. <double> 사용
double realValue = Convert<double>(10);
```

T가 함수 매개변수에 나타나지 않아 호출 인수만으로 추론할 수 없으므로 명시적 템플릿 인수가 필요하다.
일부 템플릿 인수만 왼쪽부터 지정하고 나머지를 추론할 수도 있다.

```
template <typename Result, typename Left, typename Right>
Result AddAs(const Left& left, const Right& right)
{
    return static_cast<Result>(left + right);
}

double result = AddAs<double>(10, 2.5f);
```

Result는 double로 지정하고 Left와 Right는 각각 int, float로 추론한다.

명시적 인수는 추론 실패를 해결할 수 있지만 강제로 변환을 일으킬 수도 있다.

```
int result = GetMin<int>(3.8, 2.2);
```

두 실수는 int 매개변수로 변환되어 소수 부분을 잃는다. 컴파일된다는 사실과 의도한 결과를 만든다는 사실은 다르므로 명시적 인수의 변환 효과를 확인해야 한다.

10.2.4 함수 오버로딩과 템플릿

일반 함수와 함수 템플릿은 같은 이름으로 오버로드될 수 있다.

```
#include <iostream>

void Print(int value)
{
    std::cout << "int: " << value << '\n';
}

template <typename T>
void Print(const T& value)
{
    std::cout << "template: " << value << '\n';
}

Print(10);       // 일반 함수 Print(int)
Print(3.5);      // 함수 템플릿 Print<double>
```

두 후보가 같은 수준의 변환으로 호출될 수 있다면 일반 함수가 우선된다. 그러나 템플릿 후보가 더 좋은 변환을 제공하면 템플릿이 선택될 수 있다.

```
void Show(long value)
{
    std::cout << "long: " << value << '\n';
}

template <typename T>
void Show(T value)
{
    std::cout << "template: " << value << '\n';
}

Show(10);      // 함수 템플릿 Show<int>
```

일반 함수는 int를 long으로 변환해야 하지만 템플릿은 T = int로 정확히 일치하므로 템플릿이 선택된다.
꺾쇠괄호를 사용하면 템플릿 호출을 명시할 수 있다.

```
Print<>(10);
Print<int>(10);
```

두 호출은 일반 함수가 아니라 함수 템플릿을 사용한다.

함수 오버로딩과 템플릿을 함께 사용할 때는 호출 후보가 지나치게 많아지지 않도록 주의해야 한다. 암시적 변환, 템플릿 인수 추론과 제약 조건이 결합되면 어떤 후보가 선택되는지 이해하기 어려울 수 있다. 구체 타입에 특별한 의미가 있을 때만 일반 오버로드를 추가하고, 타입 범주에 따른 선택은 Concepts를 이용해 조건을 드러내는 편이 좋다.

10.2.5 템플릿 인스턴스화

함수 템플릿 정의만으로 모든 타입의 함수가 미리 생성되는 것은 아니다. 실제 사용된 템플릿 인수 조합에 대해 함수 특수화가 만들어지는 과정을 인스턴스화(instantiation)라고 한다.

```
template <typename T>
T Multiply(T left, T right)
{
    return left * right;
}
```

```
int integerResult = Multiply(3, 4);
double realResult = Multiply(1.5, 2.0);
```

개념적으로 컴파일러는 이런 함수에 해당하는 코드를 생성한다.

```
int Multiply<int>(int left, int right)
{
    return left * right;
}

double Multiply<double>(double left, double right)
{
    return left * right;
}
```

위 코드는 원본 소스에 직접 작성하는 실제 문법 예제가 아니라 인스턴스화 결과를 설명하기 위한 개념 표현이다.
템플릿 본문 안의 모든 연산이 템플릿 정의 시점에 완전히 검사되는 것은 아니다. 템플릿 인수에 의존하는 표현은 해당 특수화가 인스턴스화될 때 검사된다.

```
template <typename T>
void PrintName(const T& value)
{
    std::cout << value.GetName() << '\n';
}
```

GetName()이 없는 타입으로 호출하지 않는 동안 이 템플릿 정의가 존재할 수 있다. 해당 타입으로 호출하면 인스턴스화 과정에서 오류가 발생한다.

```
class Player
{
public:
    const char* GetName() const
    {
        return "Player";
    }
};

Player player;
PrintName(player);

// PrintName(10); // int에는 GetName()이 없음
```

특정 타입의 인스턴스화를 명시적으로 요청할 수 있다.

```
template <typename T>
T Multiply(T left, T right)
{
    return left * right;
}

template int Multiply<int>(int, int);
```

마지막 선언은 Multiply<int>의 명시적 인스턴스화 정의이다. 하나의 소스 파일에서 코드를 생성하고 다른 번역 단위에서는 중복 인스턴스화를 억제하려면 명시적 인스턴스화 선언을 사용할 수 있다.

```
extern template int Multiply<int>(int, int);
```

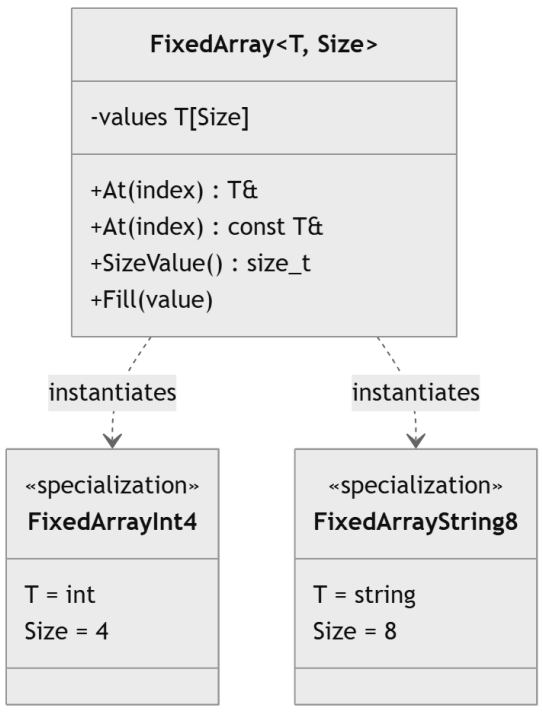
extern template은 해당 번역 단위에서 암시적 인스턴스화를 만들지 말고 다른 번역 단위에 존재하는 명시적 인스턴스화 정의를 사용하겠다는 의미이다. 이 방식은 대규모 프로젝트의 컴파일 시간을 줄이거나 템플릿 코드 생성을 통제할 때 사용할 수 있지만 선언과 정의의 관리 부담이 생긴다.

함수 템플릿을 일반적으로 사용할 때는 정의를 헤더에 둔다. 호출 지점에서 템플릿 정의를 볼 수 있어야 필요한 타입의 코드를 인스턴스화할 수 있기 때문이다.

10.3 클래스 템플릿

클래스 템플릿(class template)은 데이터 멤버와 멤버 함수에서 사용할 타입이나 값을 템플릿 매개변수로 분리한다. 저장할 원소 타입과 크기를 매개변수화하면 같은 구조를 여러 타입과 크기에 재사용할 수 있다.

이 절에서는 고정 크기 배열을 표현하는 FixedArray를 중심으로 클래스 템플릿의 구조를 살펴본다.



10.3.1 클래스 템플릿 선언

클래스 선언 앞에 템플릿 매개변수 목록을 작성한다.

```
#include <cstddef>
#include <stdexcept>

template <typename T, std::size_t Size>
class FixedArray
{
public:
    T& At(std::size_t index)
    {
        if (index >= Size)
        {
            throw std::out_of_range("FixedArray index out of range");
        }

        return values[index];
    }

    const T& At(std::size_t index) const
    {
        if (index >= Size)
        {
            throw std::out_of_range("FixedArray index out of range");
        }

        return values[index];
    }

    constexpr std::size_t SizeValue() const noexcept
    {
        return Size;
    }

    void Fill(const T& value)
    {
        for (std::size_t index = 0; index < Size; ++index)
        {
            values[index] = value;
        }
    }
```

```
private:
    static_assert(Size > 0, "FixedArray size must be positive");
    T values[Size]{};
};
```

T는 원소 타입을 나타내는 타입 템플릿 매개변수이고, Size는 컴파일 시점의 크기를 나타내는 비 타입 템플릿 매개변수이다.
클래스 템플릿의 객체를 만들 때 템플릿 인수를 지정한다.

```
FixedArray<int, 4> scores;
FixedArray<double, 8> weights;
```

FixedArray<int, 4>와 FixedArray<double, 8>은 서로 다른 구체 타입이다. 같은 클래스 템플릿에서 만들어졌지만 원소 타입이나 크기가 다르면 대입할 수 없다.

```
FixedArray<int, 4> first;
FixedArray<int, 8> second;

// first = second; // 서로 다른 타입
```

FixedArray<int, 4>의 모든 객체는 동일한 타입이며 같은 멤버 함수 특수화를 사용한다.

```
FixedArray<int, 4> first;
FixedArray<int, 4> second;

first.Fill(10);
second = first;
```

원소 타입 T는 기본 생성과 대입이 가능해야 한다. T values[Size]{}에서 원소가 값 초기화되고, Fill()에서 복사 대입이 사용되기 때문이다.
클래스 템플릿의 요구 조건은 멤버 함수별로 다를 수 있다. 객체를 생성할 때는 기본 초기화 가능성이 필요하고, Fill()을 호출할 때는 복사 대입 가능성이 추가로 필요하다.

10.3.2 멤버 함수 정의

멤버 함수를 클래스 안에서 정의하면 템플릿 매개변수를 바로 사용할 수 있다. 클래스 밖에서 정의할 때는 템플릿 선언과 완전한 클래스 템플릿 이름을 함께 작성해야 한다.
선언과 정의를 분리한 Box를 구성해 보자.

```
template <typename T>
class Box
{
public:
    explicit Box(const T& value);

    const T& GetValue() const;
    void SetValue(const T& value);

private:
    T value;
};
```

생성자 정의에는 반환 타입을 작성하지 않는다.

```
template <typename T>
Box<T>::Box(const T& value)
    : value(value)
{
}
```

일반 멤버 함수 정의에서는 반환 타입과 클래스 범위를 모두 템플릿 인수와 함께 적는다.

```
template <typename T>
const T& Box<T>::GetValue() const
{
    return value;
}
```

```
template <typename T>
void Box<T>::SetValue(const T& value)
{
    this->value = value;
}
```

클래스 이름이 멤버 선언 안에서 사용될 때는 현재 인스턴스화를 뜻하는 축약된 클래스 이름으로 사용할 수 있다.

```
template <typename T>
class Box
{
public:
    Box(const Box& other) = default;
    Box& operator=(const Box& other) = default;
};
```

이 위치의 Box는 Box<T>를 의미한다. 클래스 밖의 정의나 다른 범위에서는 일반적으로 Box<T>처럼 템플릿 인수를 명시해야 한다.
클래스 템플릿의 정적 데이터 멤버는 각 특수화별로 별도 상태를 가진다.

```
#include <cstddef>

template <typename T>
class ObjectCounter
{
public:
    ObjectCounter()
    {
        ++count;
    }

    ObjectCounter(const ObjectCounter&)
    {
        ++count;
    }

    ~ObjectCounter()
    {
        --count;
    }

    static std::size_t GetCount()
    {
        return count;
    }

private:
    inline static std::size_t count = 0;
};

ObjectCounter<int> first;
ObjectCounter<int> second;
ObjectCounter<double> third;

// ObjectCounter<int>::GetCount()는 2
// ObjectCounter<double>::GetCount()는 1
```

ObjectCounter<int>와 ObjectCounter<double>은 서로 다른 타입이므로 정적 멤버도 공유하지 않는다.

클래스 템플릿 특수화가 사용되었다고 해서 모든 멤버 함수 정의가 즉시 코드로 생성되는 것은 아니다. 일반적으로 사용된 멤버 함수의 정의가 필요할 때 해당 멤버가 인스턴스화된다.
이 특성 때문에 특정 연산을 지원하지 않는 타입이라도 그 연산을 요구하는 멤버 함수를 호출하지 않으면 클래스의 다른 기능을 사용할 수 있는 경우가 있다. 다만 클래스의 데이터 멤버 구성과 객체 생성에 필요한 조건은 객체를 만들 때 충족해야 한다.

10.3.3 템플릿과 헤더 파일

일반 클래스는 선언을 헤더 파일에 두고 멤버 함수 정의를 소스 파일에 둘 수 있다. 호출 코드는 함수의 선언만 알아도 되고, 링커가 소스 파일에서 컴파일된 함수 정의를 연결한다.

템플릿은 실제 템플릿 인수에 맞는 코드를 호출 지점에서 인스턴스화해야 한다. 일반적으로 컴파일러가 템플릿 정의 전체를 볼 수 있어야 한다.

Box.h에 선언만 두었다고 하자.

```h
// Box.h
#pragma once

template <typename T>
class Box
{
public:
    explicit Box(const T& value);
    const T& GetValue() const;

private:
    T value;
};
```

Box.cpp에 멤버 함수 정의를 두는 일반 클래스 방식은 문제가 될 수 있다.

```cpp
// Box.cpp
#include "Box.h"

template <typename T>
Box<T>::Box(const T& value)
    : value(value)
{
}

template <typename T>
const T& Box<T>::GetValue() const
{
    return value;
}
```

다른 번역 단위에서 Box<int>를 사용해도 Box.cpp가 어떤 타입의 특수화를 생성해야 하는지 알지 못할 수 있다. 사용 파일에서는 정의를 볼 수 없으므로 인스턴스화하지 못하고 링크 오류가 발생할 수 있다.

가장 일반적인 해결 방식은 선언과 정의를 모두 헤더에 두는 것이다.

```h
// Box.h
#pragma once
template <typename T>
class Box
{
public:
    explicit Box(const T& value) : value(value)
    {
    }
    const T& GetValue() const
    {
        return value;
    }
private:
    T value;
};
```

클래스 선언과 멤버 정의를 시각적으로 분리하고 싶다면 구현 파일을 헤더에서 포함하는 방식도 사용할 수 있다.

```h
// Box.h
#pragma once
template <typename T>
class Box
{
public:
    explicit Box(const T& value);
```

```
    const T& GetValue() const;
private:
    T value;
};

#include "Box.inl"
```

```
// Box.inl

template <typename T>
Box<T>::Box(const T& value) : value(value)
{
}
template <typename T>
const T& Box<T>::GetValue() const
{
    return value;
}
```

파일이 나뉘어 있어도 전처리 후에는 템플릿 정의가 헤더를 포함한 번역 단위에 보인다.

지원할 타입이 제한되어 있다면 명시적 인스턴스화를 이용해 소스 파일로 분리할 수 있다.

```
// Box.cpp
#include "Box.h"
#include <string>

template <typename T>
Box<T>::Box(const T& value) : value(value)
{
}
template <typename T>
const T& Box<T>::GetValue() const
{
    return value;
}

template class Box<int>;
template class Box<double>;
template class Box<std::string>;
```

이 구조는 명시한 타입만 사용할 수 있도록 제한하는 효과가 있다. 새로운 타입이 필요하면 소스 파일에 명시적 인스턴스화를 추가해야 한다.
일반 목적의 제네릭 라이브러리는 헤더에 정의를 제공하는 방식이 자연스럽다.

10.3.4 클래스 템플릿 인수 추론

C++17부터 생성자 인수로 클래스 템플릿 인수를 추론할 수 있다. 이를 클래스 템플릿 인수 추론(class template argument deduction, CTAD)이라고 한다.

```
#include <utility>

template <typename T>
class ValueBox
{
public:
    explicit ValueBox(T value)
              : value(std::move(value)) { }
    const T& GetValue() const
    {
        return value;
    }
private:
    T value;
};

ValueBox first(10);       // ValueBox<int>
```

```
ValueBox second(3.5);     // ValueBox<double>
```

생성자 ValueBox(T)에서 T를 추론한다. 생성자의 매개변수가 템플릿 인수와 직접 연결되어 있으면 컴파일러가 암시적 추론 가이드를 만든다. 문자열 리터럴을 전달하면 배열에서 포인터로 변환되어 const char*가 추론될 수 있다.

```
ValueBox name("Knight"); // ValueBox<const char*>
```

문자열을 소유하는 std::string으로 저장하고 싶다면 명시적 추론 가이드를 작성할 수 있다.

```
#include <string>
ValueBox(const char*) -> ValueBox<std::string>;
ValueBox name("Knight"); // ValueBox<std::string>
```

추론 가이드는 함수처럼 보이지만 호출할 수 있는 일반 함수가 아니다. 객체 생성 표현식에서 클래스 템플릿 인수를 결정하는 규칙이다.

여러 생성자가 있으면 각 생성자에서 암시적 추론 가이드가 만들어질 수 있다. 사용자가 작성한 추론 가이드와 함께 오버로드 결정이 이루어진다. 추론 규칙이 복잡해지면 명시적으로 템플릿 인수를 적는 편이 더 읽기 쉬울 수 있다.

```
ValueBox<std::string> explicitName("Knight");
```

CTAD는 변수 선언의 객체 생성에 적용된다. 함수 매개변수 타입을 ValueBox만으로 작성하는 기능은 아니다.

```
// void PrintBox(const ValueBox& box);
// ValueBox만으로는 완전한 타입이 아님

template <typename T>
void PrintBox(const ValueBox<T>& box)
{
    std::cout << box.GetValue() << '\n';
}
```

10.3.5 기본 템플릿 인수

클래스 템플릿 매개변수에도 기본 인수를 지정할 수 있다.

```
#include <cstddef>
#include <stdexcept>

template <typename T = int
          , std::size_t Width = 10
          , std::size_t Height = 10>
class Grid
{
public:
    T& At(std::size_t x, std::size_t y)
    {
        if (x >= Width || y >= Height)
        {
            throw std::out_of_range("Grid index out of range");
        }

        return values[y][x];
    }

private:
    T values[Height][Width]{};
};
```

기본 인수를 모두 사용하려면 빈 꺾쇠괄호를 적는다.

```
Grid<> defaultGrid;              // Grid<int, 10, 10>
Grid<float> floatGrid;           // Grid<float, 10, 10>
Grid<float, 20> wideGrid;        // Grid<float, 20, 10>
Grid<float, 20, 30> customGrid;  // Grid<float, 20, 30>
```

클래스 템플릿에서 기본 인수를 가진 매개변수 뒤의 매개변수도 일반적으로 기본 인수를 가져야 한다. 마지막에 매개변수 팩이 오는 경우는 예외이다.

```
// template <typename T = int, typename U>
// class InvalidPair;
```

기본 템플릿 인수는 자주 사용하는 구성을 간단히 표현하지만 실제 타입이 눈에 잘 보이지 않을 수 있다. 라이브러리 사용자가 기본값을 자연스럽게 예상할 수 있고 대부분의 호출이 같은 구성을 사용할 때 적합하다.

10.3.6 별칭 템플릿

using으로 클래스 템플릿의 일부 인수를 고정하거나 긴 타입 표현에 의미 있는 이름을 붙일 수 있다.

```
template <std::size_t Size>
using IntArray = FixedArray<int, Size>;

IntArray<4> scores;
IntArray<16> tiles;
```

원소 타입은 int로 고정하고 크기만 템플릿 인수로 받는다.
두 인수의 위치나 의미를 재구성할 수도 있다.

```
template <typename T>
using SmallArray = FixedArray<T, 8>;

SmallArray<float> weights;
SmallArray<std::string> names;
```

별칭 템플릿은 새로운 타입을 만드는 것이 아니다.

```
#include <type_traits>

static_assert(std::is_same_v<SmallArray<int>, FixedArray<int, 8>>);
```

두 표기는 동일한 타입을 가리킨다. 별칭에 멤버를 추가하거나 별도로 특수화할 수는 없다. 새로운 동작과 별도 타입 정체성이 필요하면 클래스 템플릿으로 래핑해야 한다.

타입 특성과 함께 사용하면 긴 변환 타입을 간결하게 표현할 수 있다.

```
#include <type_traits>

template <typename T>
using RemoveConstReference =
    std::remove_const_t<std::remove_reference_t<T>>;
```

C++20에서는 같은 목적에 std::remove_cvref_t<T>를 사용할 수 있다.

10.3.7 비 타입 템플릿 매개변수

템플릿 매개변수는 타입뿐 아니라 컴파일 시점의 값도 받을 수 있다. 이런 매개변수를 비 타입 템플릿 매개변수(non-type template parameter)라고 한다.

```
template <typename T, std::size_t Size>
class StaticBuffer
{
public:
    constexpr std::size_t GetCapacity() const noexcept
    {
        return Size;
    }

    T& operator[](std::size_t index)
    {
        return values[index];
    }
```

```
    const T& operator[](std::size_t index) const
    {
        return values[index];
    }

private:
    T values[Size]{};
};

StaticBuffer<int, 32> packet;
StaticBuffer<float, 4> position;
```

Size는 런타임 데이터 멤버가 아니다. 타입의 일부이므로 객체마다 별도 저장 공간을 차지하지 않는다.

```
static_assert(StaticBuffer<int, 32>{}.GetCapacity() == 32);
```

크기가 다르면 타입도 다르다.

```
StaticBuffer<int, 4> small;
StaticBuffer<int, 8> large;

// small = large; // 서로 다른 타입
```

비 타입 템플릿 인수는 컴파일 시점에 사용할 수 있는 상수 표현식이어야 한다.

```
constexpr std::size_t PlayerCount = 4;
StaticBuffer<int, PlayerCount> scores;

std::size_t runtimeCount = 8;
// StaticBuffer<int, runtimeCount> values; // 런타임 값은 사용할 수 없음
```

C++20에서는 정수와 열거형뿐 아니라 특정 조건을 만족하는 구조적 타입도 비 타입 템플릿 매개변수로 사용할 수 있다. 기본적인 자료 구조에서는 크기, 차원, 정렬 단위와 정책 열거값처럼 의미가 분명한 값에 사용하는 편이 좋다.
열거형 정책을 템플릿 인수로 받을 수 있다.

```
enum class OverflowPolicy
{
    Ignore,
    Throw
};

template <typename T, std::size_t Size,
          OverflowPolicy Policy = OverflowPolicy::Throw>
class LimitedBuffer
{
public:
    void Push(const T& value)
    {
        if (count == Size)
        {
            if constexpr (Policy == OverflowPolicy::Throw)
            {
                throw std::overflow_error("LimitedBuffer is full");
            }
            else
            {
                return;
            }
        }

        values[count++] = value;
    }

private:
    T values[Size]{};
    std::size_t count = 0;
};
```

정책이 타입에 포함되므로 LimitedBuffer<int, 8, OverflowPolicy::Throw>와 LimitedBuffer<int, 8, OverflowPolicy::Ignore>는 서로 다른 타입이다. 실행 중 정책을 바꿔야 한다면 데이터 멤버나 객체지향 전략 객체가 더 적합할 수 있다.

10.4 템플릿 특수화

기본 템플릿을 대부분의 타입에 적용하면서 특정 타입이나 타입 형태에 다른 구현을 제공할 수 있다. 전체 특수화는 템플릿 인수를 모두 고정하고, 부분 특수화는 일부 조건만 고정한다.

특수화는 일반화된 규칙에 실제로 예외가 존재할 때 사용한다. 타입마다 특수화가 계속 늘어난다면 기본 템플릿의 책임이나 타입 모델이 잘못되었는지 검토해야 한다.

10.4.1 전체 특수화

기본 클래스 템플릿을 정의한다.

```
#include <string_view>

template <typename T>
struct TypeName
{
    static constexpr std::string_view Get()
    {
        return "unknown";
    }
};
```

int에 대한 전체 특수화는 빈 템플릿 인수 목록과 구체 타입을 사용한다.

```
template <>
struct TypeName<int>
{
    static constexpr std::string_view Get()
    {
        return "int";
    }
};
```

double도 별도로 특수화할 수 있다.

```
template <>
struct TypeName<double>
{
    static constexpr std::string_view Get()
    {
        return "double";
    }
};

static_assert(TypeName<int>::Get() == "int");
static_assert(TypeName<double>::Get() == "double");
static_assert(TypeName<char>::Get() == "unknown");
```

전체 특수화는 기본 템플릿과 완전히 다른 멤버 구성을 가질 수도 있다.

```
template <typename T>
class Storage
{
public:
    void Set(const T& value)
    {
        this->value = value;
    }

    const T& Get() const
    {
```

```
        return value;
    }

private:
    T value{};
};
```

bool은 한 비트 상태만 필요하다는 이유로 별도 표현을 사용할 수 있다.

```
#include <cstdint>

template <>
class Storage<bool>
{
public:
    void Set(bool value)
    {
        bits = value ? 1u : 0u;
    }

    bool Get() const
    {
        return bits != 0;
    }

private:
    std::uint8_t bits = 0;
};
```

이 특수화는 인터페이스를 비슷하게 유지하지만 내부 표현은 다르다. 템플릿 사용자에게 같은 의미의 타입으로 보이게 하려면 특수화에서도 기본 템플릿의 공개 계약을 유지해야 한다.

함수 템플릿도 전체 특수화를 정의할 수 있다.

```
template <typename T>
void PrintValue(const T& value)
{
    std::cout << value << '\n';
}

template <>
void PrintValue<bool>(const bool& value)
{
    std::cout << (value ? "true" : "false") << '\n';
}
```

함수 템플릿에서는 전체 특수화보다 일반 함수 오버로딩이 더 자연스러운 경우가 많다. 이 차이는 10.4.3에서 다룬다.

10.4.2 부분 특수화

부분 특수화(partial specialization)는 클래스 템플릿 인수의 일부 패턴에 별도 구현을 제공한다.
기본 템플릿으로 타입의 형태를 설명해 보자.

```
#include <string_view>

template <typename T>
struct TypeCategory
{
    static constexpr std::string_view Get()
    {
        return "value";
    }
};
```

모든 포인터 타입에 적용되는 부분 특수화를 정의할 수 있다.

```cpp
template <typename T>
struct TypeCategory<T*>
{
    static constexpr std::string_view Get()
    {
        return "pointer";
    }
};

static_assert(TypeCategory<int>::Get() == "value");
static_assert(TypeCategory<int*>::Get() == "pointer");
static_assert(TypeCategory<double*>::Get() == "pointer");
```

포인터가 가리키는 타입은 여전히 T로 남아 있으므로 int*, double*, 사용자 정의 타입 포인터에 모두 적용된다.
배열 형태도 부분 특수화할 수 있다.

```cpp
#include <cstddef>

template <typename T, std::size_t Size>
struct TypeCategory<T[Size]>
{
    static constexpr std::string_view Get()
    {
        return "fixed array";
    }

    static constexpr std::size_t GetSize()
    {
        return Size;
    }
};

static_assert(TypeCategory<int[5]>::Get() == "fixed array");
static_assert(TypeCategory<int[5]>::GetSize() == 5);
```

여러 템플릿 인수 중 일부만 고정할 수도 있다.

```cpp
#include <string>

template <typename Key, typename Value>
struct PairStorage
{
    static constexpr std::string_view Kind()
    {
        return "general";
    }
};

template <typename Value>
struct PairStorage<std::string, Value>
{
    static constexpr std::string_view Kind()
    {
        return "string key";
    }
};
```

두 번째 정의는 키 타입이 std::string인 모든 Value에 적용된다.

부분 특수화 후보가 여러 개 일치하면 더 구체적인 특수화가 선택된다. 어느 쪽이 더 구체적인지 결정할 수 없으면 모호성 오류가 발생한다.

함수 템플릿은 부분 특수화를 지원하지 않는다.

```cpp
// 함수 템플릿의 부분 특수화는 허용되지 않음
// template <typename T>
// void Process<T*>(T* value);
```

함수는 오버로딩으로 같은 목적을 표현할 수 있다.

```cpp
template <typename T>
void Process(const T& value)
{
    std::cout << "value: " << value << '\n';
}

template <typename T>
void Process(T* value)
{
    if (value != nullptr)
    {
        std::cout << "pointer: " << *value << '\n';
    }
}
```

10.4.3 함수 템플릿 오버로딩과 특수화

함수 템플릿의 기본 구현이 특정 타입에 적합하지 않거나 다른 처리가 필요하면 명시적 특수화(explicit specialization) 또는 함수 오버로딩으로 별도의 구현을 제공할 수 있다. 기본 구현에서 사용하는 연산을 특정 타입이 지원하지 않아 컴파일할 수 없는 경우에도 별도 구현이 필요할 수 있다.

함수 템플릿의 명시적 특수화는 특정 타입에 대해 기본 함수 템플릿을 대신할 구현을 정의한다. 전체 타입을 지정하므로 전체 특수화(full specialization) 라고도 한다.

다음 예제는 Serialize()가 일반 타입은 그대로 출력하고 std::string만 별도로 처리한다고 가정한다.

```cpp
#include <iostream>
#include <string>

template <typename T>
void Serialize(const T& value)
{
    std::cout << value;
}

template <>
void Serialize<std::string>(const std::string& value)
{
    std::cout << '"' << value << '"';
}
```

template <>는 명시적 특수화를 나타내며 Serialize<std::string>은 T가 std::string일 때 사용할 구현을 지정한다. 다른 타입은 기본 함수 템플릿을 사용한다.

특정 타입에 대한 별도 동작은 일반 함수 오버로딩으로도 제공할 수 있다.

```cpp
#include <iostream>
#include <string>

template <typename T>
void Write(const T& value)
{
    std::cout << value;
}

void Write(const std::string& value)
{
    std::cout << '"' << value << '"';
}
```

두 방법은 비슷한 결과를 만들 수 있지만 함수가 선택되는 과정에는 차이가 있다. 함수 호출의 오버로드 결정에서는 일반 함수와 기본 함수 템플릿이 후보가 된다. 명시적 특수화는 독립된 오버로드 후보로 참여하지 않는다. 기본 함수 템플릿과 템플릿 인수가 결정된 후 해당 조합에 대한 명시적 특수화가 존재하면 특수화된 구현을 사용한다.

명시적 특수화는 해당 타입으로 기본 함수 템플릿이 암시적으로 인스턴스화되기 전에 선언되어 있어야 한다. 여러 파일에서 템플릿을 사용할 때는 특수화의 선언 위치와 가시성도 고려해야 한다.

함수 템플릿은 부분 특수화(partial specialization) 를 지원하지 않는다. 특정 타입이나 매개변수 형태에 별도 동작을 제공하려면 일반 함수 오버로딩을 이용할 수 있다. 일반 함수는 오버로드 후보에 직접 참여하므로 매개변수 타입을 이용한 구분을 자연스럽게 표현할 수 있다.

```
#include <iostream>
#include <string>

template <typename T>
void Save(const T& value)
{
    std::cout << value;
}

void Save(const char* value)
{
    std::cout << '"' << value << '"';
}

void Save(const std::string& value)
{
    std::cout << '"' << value << '"';
}
```

const char*와 std::string은 각각의 오버로드가 처리하고 그 밖의 타입은 함수 템플릿이 처리한다. 특정 타입이나 매개변수 형태를 구분하는 목적이라면 일반 함수 오버로딩을 우선해서 고려할 수 있다.

하나의 타입이 아니라 정수형이나 실수형처럼 타입 범주에 따라 동작을 구분하려면 Concepts를 이용한 제약 오버로딩으로 조건을 직접 표현할 수 있다.

```
#include <concepts>
#include <iostream>

template <std::integral T>
void Save(T value)
{
    std::cout << "integer:" << value;
}

template <std::floating_point T>
void Save(T value)
{
    std::cout << "real:" << value;
}
```

std::integral과 std::floating_point 는 C++20 표준 라이브러리에서 제공하는 Concept이다. std::integral은 int, long 과 같은 정수형을, std::floating_point 는 float, double과 같은 부동소수점형을 조건으로 지정한다. 함수 호출에 사용된 타입이 해당 Concept 의 조건을 만족해야 그 함수 템플릿을 사용할 수 있다.
따라서 특정 타입 하나에 별도의 구현을 제공하는 명시적 특수화와 달리, Concepts를 이용하면 같은 성질을 가진 타입의 범주를 기준으로 함수의 동작을 구분할 수 있다.

10.4.4 특수화 사용 시 주의점

특수화는 강력하지만 기본 템플릿의 규칙을 여러 위치로 분산시킨다. 사용자가 기본 템플릿만 읽고 예상한 동작과 특정 타입의 실제 동작이 달라질 수 있다.

<특수화 사용 기준>

```
- 특정 타입이 기본 구현을 사용할 수 없는 근본적인 이유 확인
- 기본 템플릿과 공개 인터페이스 및 의미의 일관성 유지
- 오버로딩이나 if constexpr로 더 직접적으로 표현 가능한지
검토
- 특수화 정의 전에 암시적 인스턴스화가 발생하지 않는지 확인
- 여러 부분 특수화 사이의 모호성 여부 확인
```

명시적 특수화는 기본 템플릿이 선언된 이름 공간에서 선언해야 한다. 헤더에 함수나 변수의 전체 특수화 정의를 둘 때는 여러 번 정의되는 문제를 피하도록 inline 여부를 확인해야 한다.

특수화는 사용 지점에서 볼 수 있어야 한다. 어떤 번역 단위에서는 기본 템플릿이 인스턴스화되고 다른 번역 단위에서는 특수화가 사용되면 프로그램 전체의 정의가 일관되지 않을 수 있다.

표준 라이브러리의 템플릿을 임의로 특수화해서는 안 된다. C++ 표준이 사용자 정의 타입에 대한 특수화를 명시적으로 허용한 경우에만 가능하다. 표준 타입에 대한 특수화나 허용되지 않은 템플릿 특수화는 정의되지 않은 동작이나 컴파일 오류를 만들 수 있다.

타입별 차이가 간단한 조건 분기라면 if constexpr가 더 읽기 쉬울 수 있다.

```
#include <type_traits>

template <typename T>
void Describe(const T& value)
{
    if constexpr (std::is_pointer_v<T>)
    {
        if (value != nullptr)
        {
            std::cout << "pointer value: " << *value << '\n';
        }
    }
    else
    {
        std::cout << "value: " << value << '\n';
    }
}
```

기본 템플릿과 특수화의 경계를 설계할 때는 타입의 차이가 자료 구조 전체를 바꾸는지, 함수 하나의 동작만 바꾸는지 구분해야 한다.

10.5 가변 인수 템플릿

가변 인수 템플릿(variadic template)은 개수가 정해지지 않은 템플릿 인수와 함수 인수를 받을 수 있다. 여러 타입을 하나의 묶음으로 표현하는 템플릿 매개변수 팩과 여러 값을 하나의 묶음으로 표현하는 함수 매개변수 팩을 사용한다.
가변 인수 템플릿은 로그 출력, 객체 생성, 콜백 조합, 여러 기반 클래스 상속과 같은 구조에 활용된다. 팩은 컨테이너처럼 실행 시간에 원소를 저장하는 객체가 아니다. 컴파일 시점에 여러 타입이나 표현식을 문법적으로 펼치기 위한 언어 기능이다.

10.5.1 매개변수 팩

타입 매개변수 팩은 typename... Types 형태로 선언한다.

```
template <typename... Types>
class TypeList
{
};

TypeList<> emptyTypes;
TypeList<int> oneType;
TypeList<int, double, const char*> threeTypes;
```

Types는 0개 이상의 타입을 나타낸다.
함수 템플릿에서는 타입 매개변수 팩과 함수 매개변수 팩을 함께 사용할 수 있다.

```
#include <iostream>

template <typename... Args>
void CountArguments(const Args&... args)
{
    std::cout << sizeof...(Args) << '\n';
    std::cout << sizeof...(args) << '\n';
}
```

sizeof...는 팩에 포함된 원소 개수를 컴파일 시점 상수로 반환한다.

```
CountArguments();
CountArguments(10);
CountArguments(10, 3.5, "Knight");
```

세 번째 호출에서는 Args가 int, double, char[7]에 대응하는 타입 묶음으로 추론되고 원소 수는 3이다. const Args&... 형태는 각 타입에 대해 상수 좌측값 참조 매개변수를 만든다.

비 타입 매개변수 팩도 선언할 수 있다.

```
template <int... Values>
struct IntegerSequence
{
    static constexpr std::size_t Size = sizeof...(Values);
};

static_assert(IntegerSequence<1, 2, 3, 4>::Size == 4);
```

템플릿 매개변수 팩은 일반적으로 템플릿 매개변수 목록의 끝에 배치한다. 함수 템플릿에서는 뒤쪽 매개변수를 추론할 수 있거나 기본 인수가 있는 경우 더 유연한 배치가 가능하지만, 읽기 쉬운 인터페이스를 위해 끝에 두는 편이 좋다.

10.5.2 팩 확장

팩 이름 뒤에 ...를 붙인 표현은 팩의 각 원소에 패턴을 반복하는 팩 확장(pack expansion)이 된다.
각 인수를 한 번씩 출력하는 보조 함수를 정의한다.

```
#include <iostream>

template <typename T>
void PrintOne(const T& value)
{
    std::cout << value << '\n';
}
```

초기화 목록 안에서 팩을 확장할 수 있다.

```
#include <initializer_list>
template <typename... Args>
void PrintLines(const Args&... args)
{
    (void)std::initializer_list<int>
    {
        (PrintOne(args), 0)...
    };
}
```

(PrintOne(args), 0)...는 각 args에 대해 PrintOne()을 호출하고 정수 0을 생성한다. 초기화 목록의 원소 평가는 왼쪽에서 오른쪽 순서로 이루어지므로 인수 순서대로 출력된다. C++17 이후에는 폴드 표현식으로 더 간단하게 작성할 수 있다.
함수 호출의 인수 목록에서도 팩을 확장할 수 있다.

```
template <typename Function, typename... Args>
auto Call(Function function, const Args&... args)
{
    return function(args...);
}
```

args...는 함수 매개변수 팩의 각 원소를 별도 함수 인수로 펼친다.

```
int AddThree(int first, int second, int third)
{
    return first + second + third;
}

int result = Call(AddThree, 10, 20, 30);
```

클래스 템플릿의 기반 클래스 목록과 생성자 초기화에서도 팩을 확장할 수 있다.

```
template <typename... Bases>
class Combined : public Bases...
{
public:
    explicit Combined(Bases... bases)
        : Bases(std::move(bases))...
```

```
    {
    }
};
```

이 코드는 각 Bases를 기반 클래스로 상속하고 각 기반 클래스 생성자를 호출한다. 여러 함수 객체의 호출 연산자를 결합하는 구조에서 활용할 수 있지만, 다중 상속의 책임과 이름 충돌을 함께 검토해야 한다.
팩 확장의 핵심은 ...가 붙은 전체 패턴이다.

```
template <typename... Args>
void Example(const Args&... args)
{
    PrintOne((args + args)...);
}
```

이 코드는 PrintOne()을 여러 번 호출하는 것이 아니라 args + args를 각각 펼쳐 하나의 PrintOne() 호출에 여러 인수를 전달하려고 한다. PrintOne()이 인수 하나만 받으므로 팩의 크기가 1이 아니면 오류가 발생한다. 반복하려는 단위가 함수 호출 전체인지 함수 인수인지 구분해야 한다.

10.5.3 재귀적 전개

C++17의 폴드 표현식이 도입되기 전에는 가변 인수 팩을 첫 원소와 나머지 팩으로 나누어 재귀적으로 처리하는 방식이 널리 사용되었다.
재귀 종료를 위한 인수 없는 함수를 정의한다.

```
void PrintRecursive()
{
    std::cout << '\n';
}
```

첫 인수와 나머지 인수를 받는 함수 템플릿을 정의한다.

```
template <typename First, typename... Rest>
void PrintRecursive(const First& first, const Rest&... rest)
{
    std::cout << first;

    if constexpr (sizeof...(Rest) > 0)
    {
        std::cout << ' ';
    }

    PrintRecursive(rest...);
}

PrintRecursive(10, 3.5, "Knight");
```

호출 과정은 이런 구조로 축소된다.

```
PrintRecursive(10, 3.5, "Knight")
→ 10 출력
→ PrintRecursive(3.5, "Knight")
→ 3.5 출력
→ PrintRecursive("Knight")
→ Knight 출력
→ PrintRecursive()
```

재귀적 전개는 각 단계에서 하나 이상의 함수 템플릿 특수화를 만들 수 있다. 코드가 길고 컴파일 오류의 호출 경로도 깊어질 수 있다. 현재 C++에서는 단순한 결합 연산에 폴드 표현식을 우선한다. 재귀 방식은 각 인수에 서로 다른 단계 처리가 필요하거나 기존 코드를 이해할 때 유용하다.

재귀 함수에서 종료 오버로드의 선언 위치도 중요하다. 템플릿 정의 안의 이름 탐색 규칙 때문에 재귀 템플릿보다 종료 함수를 먼저 선언하는 편이 안전하다.

10.5.4 폴드 표현식

폴드 표현식(fold expression)은 매개변수 팩의 원소 사이에 이항 연산자를 반복 적용한다. C++17부터 사용할 수 있다.
합계를 계산하는 단항 왼쪽 폴드를 작성해 보자.

```cpp
template <typename... Args>
auto Sum(const Args&... args)
{
    return (... + args);
}

int integerSum = Sum(10, 20, 30);
double mixedSum = Sum(10, 2.5, 3.5f);
```

(... + args)는 개념적으로 ((arg1 + arg2) + arg3)처럼 왼쪽부터 결합한다.
폴드 표현식에는 네 형태가 있다.

형태	문법	개념적 결합
단항 왼쪽 폴드	(... op pack)	((a op b) op c)
단항 오른쪽 폴드	(pack op ...)	(a op (b op c))
이항 왼쪽 폴드	(init op ... op pack)	((init op a) op b)
이항 오른쪽 폴드	(pack op ... op init)	(a op (b op init))

빼기와 나누기처럼 결합 방향에 따라 결과가 달라지는 연산은 왼쪽과 오른쪽 폴드를 구분해야 한다.

```cpp
template <typename... Args>
auto SubtractLeft(const Args&... args)
{
    return (... - args);
}

template <typename... Args>
auto SubtractRight(const Args&... args)
{
    return (args - ...);
}

int left = SubtractLeft(100, 20, 5);   // (100 - 20) - 5 = 75
int right = SubtractRight(100, 20, 5); // 100 - (20 - 5) = 85
```

초기값을 가진 이항 폴드는 빈 팩도 처리하기 쉽다.

```cpp
template <typename... Args>
constexpr auto SumFromZero(const Args&... args)
{
    return (0 + ... + args);
}

static_assert(SumFromZero() == 0);
static_assert(SumFromZero(1, 2, 3) == 6);
```

모든 조건이 참인지 확인할 수 있다.

```cpp
template <typename... Conditions>
bool All(Conditions... conditions)
{
    return (... && conditions);
}
```

&&의 단항 폴드는 빈 팩에서 true, ||의 단항 폴드는 빈 팩에서 false가 된다. 쉼표 연산자의 빈 단항 폴드는 void()가 된다. 다른 연산자는 빈 단항 폴드에 항등값이 정의되지 않을 수 있으므로 초기값을 제공해야 한다.
쉼표 연산자를 사용하면 각 인수에 함수를 순서대로 적용할 수 있다.

```cpp
template <typename... Args>
void PrintFold(const Args&... args)
```

```
{
    ((std::cout << args << ' '), ...);
    std::cout << '\n';
}
```

괄호는 폴드 범위와 연산 순서를 분명하게 한다. 출력 연산 자체를 폴드할 수도 있다.

```
template <typename... Args>
void PrintCompact(const Args&... args)
{
    (std::cout << ... << args) << '\n';
}
```

이 방식은 구분자를 각 인수 사이에 넣는 처리가 어렵다. 형식이 복잡해지면 보조 함수나 명시적인 반복 구조가 더 읽기 쉬울 수 있다.

10.5.5 여러 인수 전달

가변 인수 템플릿은 여러 값을 동일한 목적의 함수에 전달할 때 유용하다. 로그 함수는 값의 타입과 개수에 관계없이 하나의 출력 메시지를 만들 수 있다.

```
#include <iostream>
#include <string_view>

enum class LogLevel
{
    Info,
    Warning,
    Error
};

std::string_view ToString(LogLevel level)
{
    switch (level)
    {
    case LogLevel::Info:
        return "INFO";
    case LogLevel::Warning:
        return "WARNING";
    case LogLevel::Error:
        return "ERROR";
    }

    return "UNKNOWN";
}

template <typename... Args>
void Log(LogLevel level, const Args&... args)
{
    std::cout << '[' << ToString(level) << "] ";
    ((std::cout << args), ...);
    std::cout << '\n';
}

Log(LogLevel::Info, "Player hp=", 100);
Log(LogLevel::Warning, "Low stamina: ", 12.5f, '%');
Log(LogLevel::Error, "Cannot open file: ", "save.dat");
```

각 인수는 operator<<로 출력할 수 있어야 한다. 이 요구 조건은 아직 함수 선언에 드러나지 않는다. Concepts를 사용하면 출력 가능성을 검사하거나 더 구체적인 인터페이스를 제공할 수 있다.

여러 인수를 다른 함수로 그대로 전달할 때는 값 범주를 보존해야 할 수 있다. const Args&...는 읽기 전용 출력에는 적합하지만 생성자 인수 전달에는 적합하지 않다. 우측값을 상수 참조로 받아 전달하면 이동 생성자 대신 복사 생성자가 선택될 수 있다. 이런 문제는 10.8의 전달 참조와 완벽 전달에서 해결한다.

가변 인수 템플릿은 인수 개수를 제한하지 않으므로 잘못 사용하면 함수의 책임이 모호해질 수 있다. 인수의 위치마다 의미가 다르고 호출자가 순서를 기억해야 한다면 구조체나 명명된 옵션 객체가 더 적합하다.

10.6 Concepts와 제약 조건

템플릿은 실제 타입이 인스턴스화될 때 본문의 연산을 검사한다. 요구 조건을 만족하지 않는 타입이 전달되면 오류가 템플릿 내부의 긴 인스턴스화 경로에서 발생할 수 있다. C++20의 Concepts는 템플릿이 받을 수 있는 타입의 조건을 선언에 표현한다.

Concept는 컴파일 시점의 논리 조건이다. 타입이 특정 연산을 제공하는지, 특정 타입 관계를 만족하는지, 표현식의 결과가 원하는 타입인지 검사할 수 있다.

10.6.1 템플릿 제약의 필요성

제약이 없는 합계 함수를 작성해 보자.

```
#include <string>

template <typename T>
T Add(T left, T right)
{
    return left + right;
}
```

int, double, std::string은 + 연산을 제공하므로 사용할 수 있다.

```
int integer = Add(10, 20);
double real = Add(1.5, 2.5);
std::string text = Add(std::string("Game"),
std::string("Object"));
```

함수의 목적이 수치 합산이라면 문자열 연결까지 허용하는 것이 의도와 다를 수 있다. + 연산이 존재한다는 사실만으로 알고리즘의 도메인 조건이 충족되지는 않는다.
+ 연산이 없는 타입을 전달하면 함수 본문에서 오류가 발생한다.

```
struct Position
{
    int x;
    int y;
};

// Position value = Add(Position{1, 2}, Position{3, 4});
```

오류 메시지는 Add()의 선언보다 left + right 표현식과 여러 후보 연산자를 중심으로 나타날 수 있다. 템플릿 인터페이스에서 요구 조건을 먼저 검사하면 호출 지점에 가까운 오류를 만들 수 있다.

```
#include <concepts>

template <std::integral T>
T AddInteger(T left, T right)
{
    return left + right;
}

int value = AddInteger(10, 20);
// double real = AddInteger(1.5, 2.5); // std::integral을
만족하지 않음
```

제약 조건은 오류 메시지를 개선하는 기능에만 그치지 않는다. 오버로드 후보를 제한하고, 더 구체적인 제약을 가진 구현을 선택하며, 템플릿의 공개 계약을 문서화한다.

10.6.2 표준 Concept

표준 라이브러리의 <concepts> 헤더는 자주 사용하는 타입 관계와 연산 조건을 Concept로 제공한다.
정수형과 부동소수점형을 제한할 수 있다.

```
#include <concepts>

template <std::integral T>
T DoubleValue(T value)
```

```
{
    return value * 2;
}

template <std::floating_point T>
T HalfValue(T value)
{
    return value / static_cast<T>(2);
}
```

std::same_as<T, U>는 두 타입이 같은지 검사한다.

```
template <typename T, typename U>
requires std::same_as<T, U>
bool AreSameTypeValues(const T& left, const U& right)
{
    return left == right;
}
```

std::convertible_to<From, To>는 한 타입을 다른 타입으로 변환할 수 있는지 검사한다.

```
template <typename From, typename To>
requires std::convertible_to<From, To>
To ConvertValue(const From& value)
{
    return static_cast<To>(value);
}
```

std::derived_from<Derived, Base>는 공개적이고 모호하지 않은 파생 관계를 검사한다.

```
class GameObject
{
public:
    virtual ~GameObject() = default;
};

class Player : public GameObject
{
};

template <typename T>
requires std::derived_from<T, GameObject>
void RegisterGameObject(T& object)
{
    GameObject& base = object;
    (void)base;
}
```

호출 가능 객체 조건도 제공한다.

```
#include <concepts>
#include <functional>
#include <utility>

template <typename Function, typename... Args>
requires std::invocable<Function, Args...>
decltype(auto) Invoke(Function&& function, Args&&... args)
{
    return std::invoke(
        std::forward<Function>(function),
        std::forward<Args>(args)...);
}
```

이 예제의 std::forward는 10.8에서 자세히 다룬다.

<자주 사용하는 표준 Concept>

Concept	의미
std::same_as<T, U>	두 타입이 같음
std::derived_from<D, B>	D가 B의 공개 파생 타입임
std::convertible_to<F, T>	F에서 T로 변환 가능
std::integral<T>	정수형 범주
std::signed_integral<T>	부호 있는 정수형 범주
std::unsigned_integral<T>	부호 없는 정수형 범주
std::floating_point<T>	부동소수점형 범주
std::constructible_from<T, Args...>	지정한 인수로 T 생성 가능
std::default_initializable<T>	기본 초기화 가능
std::copyable<T>	복사 가능한 값 타입 요구 조건
std::movable<T>	이동 가능한 값 타입 요구 조건
std::invocable<F, Args...>	지정한 인수로 호출 가능
std::predicate<F, Args...>	논리 판정 함수로 사용 가능

표준 Concept는 문법적 가능성뿐 아니라 라이브러리 알고리즘이 기대하는 의미 조건을 함께 문서화한다. 컴파일러는 모든 의미 조건을 검사할 수 없으므로 타입 작성자가 계약을 지켜야 한다.

10.6.3 requires 절

requires 절은 템플릿 선언에 논리 제약을 추가한다.

```
#include <concepts>

template <typename T>
requires std::integral<T>
T AbsValue(T value)
{
    return value < 0 ? -value : value;
}
```

템플릿 매개변수 목록 뒤에 작성하는 형태이다. 함수 선언 뒤에 작성할 수도 있다.

```
template <typename T>
T AbsValue(T value)
    requires std::integral<T>
{
    return value < 0 ? -value : value;
}
```

두 형태의 의미는 같다. 반환 타입과 함수 선언이 긴 경우 뒤쪽 requires가 읽기 쉬울 수 있다.
여러 조건은 논리 연산자로 결합한다.

```
template <typename T>
requires std::integral<T> && std::signed_integral<T>
T Negative(T value)
{
    return -value;
}
```

std::signed_integral<T> 자체가 std::integral<T>을 기반으로 정의되므로 이 예제의 첫 조건은 중복이다. 실제 코드에서는 더 간결하게 작성한다.

```
template <std::signed_integral T>
```

```
T Negative(T value)
{
    return -value;
}
```

Concept 이름을 템플릿 매개변수 자리에 직접 사용할 수 있다.

```
template <std::integral T>
T Increment(T value)
{
    return value + 1;
}
```

축약 함수 템플릿 문법도 제공한다.

```
auto Increment(std::integral auto value)
{
    return value + 1;
}
```

매개변수가 여러 개라면 각 auto는 독립적인 타입으로 추론된다.

```
auto AddNumbers(std::integral auto left,
                std::integral auto right)
{
    return left + right;
}
```

두 타입이 같아야 한다면 별도 템플릿 매개변수와 std::same_as 조건을 사용한다.

```
template <std::integral T, std::integral U>
requires std::same_as<T, U>
auto AddSameType(T left, U right)
{
    return left + right;
}
```

제약은 템플릿의 목적을 드러낼 만큼 구체적이어야 하지만 실제 구현보다 강한 조건을 요구해서는 안 된다. 함수가 복사하지 않고 이동만 사용한다면 std::copyable보다 std::movable이 적절할 수 있다.

10.6.4 requires 표현식

requires 표현식은 타입과 표현식이 특정 요구 조건을 만족하는지 검사하여 bool 상수 결과를 만든다.

<기본 형태>

```
requires (매개변수 목록)
{
    요구 조건들
}
```

단순 요구 조건(simple requirement)은 표현식이 유효한지 검사한다.

```
template <typename T>
concept HasSize = requires(const T& value)
{
    value.size();
};
```

세미콜론으로 끝나는 value.size();는 실제로 실행되지 않는다. 해당 표현식이 올바르게 컴파일될 수 있는지만 검사한다.
타입 요구 조건(type requirement)은 중첩 타입이나 템플릿 특수화가 존재하는지 검사한다.

```
template <typename T>
concept HasValueType = requires
{
```

```
    typename T::value_type;
};
```

복합 요구 조건(compound requirement)은 표현식의 유효성, noexcept 여부와 결과 타입을 함께 검사할 수 있다.

```
#include <concepts>
template <typename T>
concept ReadableHealth = requires(const T& value)
{
    { value.GetHp() } noexcept -> std::same_as<int>;
};
```

중괄호 안의 표현식 뒤에 noexcept를 적으면 예외를 발생시키지 않는 표현식이어야 한다. -> std::same_as<int>는 표현식 결과 타입이 정확히 int인지 검사한다.

중첩 요구 조건(nested requirement)은 requires 키워드 뒤에 추가 논리 조건을 작성한다.

```
template <typename T>
concept SmallObject = requires
{
    requires sizeof(T) <= 64;
};
```

네 요구 조건을 하나의 Concept에 함께 사용할 수 있다.

```
#include <concepts>

template <typename T>
concept Damageable = requires(T& target,
                              const T& constTarget,
                              int damage)
{
    typename T::HealthType;
    { target.TakeDamage(damage) } -> std::same_as<int>;
    { constTarget.GetHp() } noexcept -> std::same_as<int>;
    { constTarget.IsDead() } noexcept -> std::same_as<bool>;
    requires std::default_initializable<typename
T::HealthType>;
};
```

<Concept 조건 표현 요약>

- T::HealthType 중첩 타입이 존재함
- TakeDamage(int)를 호출할 수 있고 결과가 int임
- GetHp()와 IsDead()가 예외 없이 호출됨
- 반환 타입이 각각 int, bool임
- HealthType을 기본 초기화할 수 있음

요구 조건에 사용하는 지역 매개변수는 검사 목적의 가상 매개변수이다. 저장 공간을 만들거나 생성자를 실제로 호출하지 않는다. requires 표현식 자체는 일반 상수 표현식으로도 사용할 수 있다.

```
template <typename T>
constexpr bool HasUpdate = requires(T& value)
{
    value.Update();
};
```

Concept 정의에 사용하면 이름과 의도를 제공할 수 있으므로 반복되는 요구 조건은 Concept로 분리하는 편이 좋다.

10.6.5 사용자 정의 Concept

게임 객체가 피해를 받을 수 있는 조건을 단순한 형태로 정의해 보자.

```
#include <concepts>

template <typename T>
```

```
concept DamageableObject = requires(T& target, int damage)
{
    { target.TakeDamage(damage) } -> std::convertible_to<int>;
    { target.IsDead() } -> std::convertible_to<bool>;
};
```

피해를 적용하는 함수에 제약을 사용한다.

```
template <DamageableObject T>
int ApplyDamage(T& target, int damage)
{
    if (damage <= 0 || target.IsDead())
    {
        return 0;
    }

    return target.TakeDamage(damage);
}
```

Player와 Monster가 같은 기반 클래스를 상속하지 않아도 요구되는 함수를 제공하면 사용할 수 있다.

```
class Player
{
public:
    int TakeDamage(int damage)
    {
        const int applied = damage > hp ? hp : damage;
        hp -= applied;
        return applied;
    }

    bool IsDead() const
    {
        return hp == 0;
    }

private:
    int hp = 100;
};

class BreakableBox
{
public:
    int TakeDamage(int damage)
    {
        const int applied = damage > durability ? durability : damage;
        durability -= applied;
        return applied;
    }

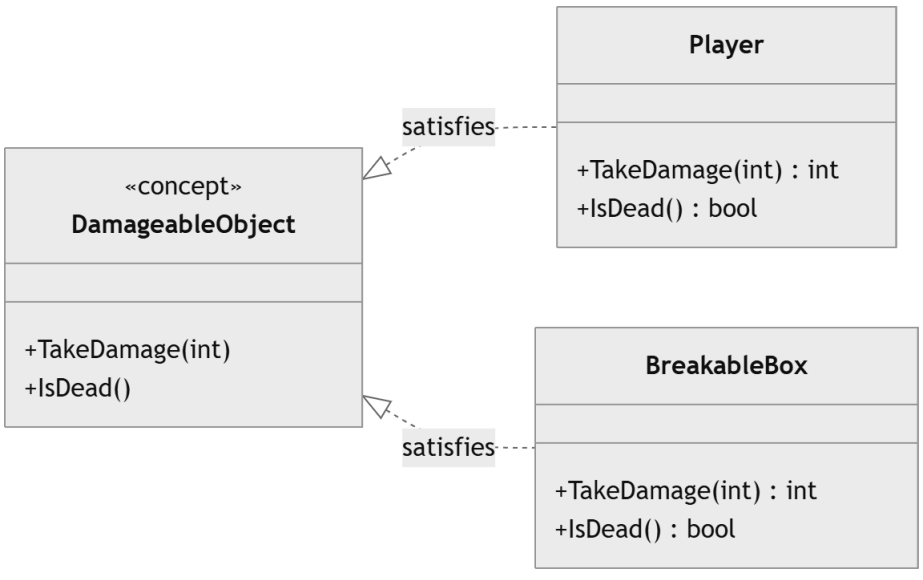
    bool IsDead() const
    {
        return durability == 0;
    }

private:
    int durability = 40;
};

Player player;
BreakableBox box;

ApplyDamage(player, 25);
ApplyDamage(box, 10);
```

DamageableObject는 상속 계층이 아니라 정적 인터페이스를 표현한다.



Concept를 만족한다고 선언하는 별도 상속 문법은 없다. 컴파일러가 사용 지점에서 요구 표현식을 검사한다. 따라서 클래스 정의만 보고 어떤 Concept를 만족하는지 모두 알기 어려울 수 있다. 중요한 정적 계약은 문서와 함수 선언에서 분명하게 드러내야 한다.
Concept 이름은 타입의 문법적 형태보다 역할과 의미를 나타내는 편이 좋다. HasTakeDamageFunction보다 DamageableObject가 사용 목적을 더 잘 표현한다.

10.6.6 제약 조건과 오버로딩

제약 조건이 다른 함수 템플릿을 오버로드할 수 있다.

```
#include <concepts>
#include <iostream>

template <std::integral T>
void DescribeNumber(T value)
{
    std::cout << "integral: " << value << '\n';
}

template <std::signed_integral T>
void DescribeNumber(T value)
{
    std::cout << "signed integral: " << value << '\n';
}

DescribeNumber(10);
DescribeNumber(10u);
```

int는 두 조건을 모두 만족하지만 std::signed_integral이 std::integral보다 더 구체적인 제약이므로 두 번째 오버로드가 선택된다. unsigned int는 첫 오버로드만 만족한다.
사용자 정의 Concept로 포함 관계를 표현할 수 있다.

```
template <typename T>
concept Number = std::integral<T> || std::floating_point<T>;

template <typename T>
concept SignedNumber = Number<T> &&
    (std::signed_integral<T> || std::floating_point<T>);

template <Number T>
void ProcessNumber(T value)
{
    std::cout << "number: " << value << '\n';
}

template <SignedNumber T>
void ProcessNumber(T value)
{
    std::cout << "signed number: " << value << '\n';
}
```

이름 있는 Concept를 재사용하면 컴파일러가 제약 조건의 포함 관계를 판단하기 쉬워지고 코드의 의미도 분명해진다.

논리적으로 같은 조건처럼 보이는 표현을 각 오버로드에서 별도로 반복하면 원자 제약의 정체성이 달라 포함 관계가 기대대로 성립하지 않을 수 있다. 반복되는 제약은 Concept로 정의하여 같은 조건을 재사용하는 편이 좋다.

제약 오버로딩은 함수 본문 안의 if constexpr 분기와 목적이 다르다.

```
template <Number T>
void PrintNumber(T value)
{
    if constexpr (std::integral<T>)
    {
        std::cout << "integer: " << value << '\n';
    }
    else
    {
        std::cout << "real: " << value << '\n';
    }
}
```

제약 오버로딩은 서로 다른 인터페이스 후보를 제공한다. if constexpr는 하나의 선택된 템플릿 구현 안에서 타입에 따라 코드를 분기한다. 함수 시그니처, 반환 타입이나 사용 가능 조건이 다르면 제약 오버로딩이 적합하고, 전체 알고리즘 중 일부 단계만 다르면 if constexpr가 적합하다.

10.7 타입 특성과 조건부 코드

타입 특성(type traits)은 컴파일 시점에 타입의 성질을 확인하거나 다른 타입으로 변환하는 템플릿이다. 표준 라이브러리의 <type_traits>는 정수형 여부, 참조 여부, 상속 관계, 생성 가능성 같은 정보를 제공한다.

타입 특성은 if constexpr의 조건, static_assert, Concept 정의와 기존 SFINAE 코드에 사용된다. C++20에서는 공개 인터페이스의 제약을 Concepts로 표현하고, 구현 내부의 타입 계산과 분기에 타입 특성을 사용하는 구성이 자연스럽다.

10.7.1 type_traits

표준 타입 특성은 대부분 클래스 템플릿으로 정의되어 있다. std::is_integral<T>은 T가 정수형인지 나타내는 value 정적 멤버를 제공한다.

```
#include <type_traits>

static_assert(std::is_integral<int>::value);
static_assert(!std::is_integral<double>::value);
```

C++17부터 _v 접미사의 변수 템플릿을 사용할 수 있다.

```
static_assert(std::is_integral_v<int>);
static_assert(!std::is_integral_v<double>);
```

std::is_integral_v<T>는 std::is_integral<T>::value 를 간결하게 표현한다.

타입 특성의 기본 구조를 직접 작성해 보면 원리를 이해할 수 있다.

```
template <typename T>
struct IsPointer
{
    static constexpr bool value = false;
};

template <typename T>
struct IsPointer<T*>
{
    static constexpr bool value = true;
};

static_assert(!IsPointer<int>::value);
static_assert(IsPointer<int*>::value);
```

변수 템플릿을 추가할 수 있다.

```cpp
template <typename T>
inline constexpr bool IsPointerValue = IsPointer<T>::value;

static_assert(IsPointerValue<double*>);
```

표준 타입 특성은 std::integral_constant를 기반으로 구성되는 경우가 많다.

```cpp
#include <type_traits>

using TrueType = std::integral_constant<bool, true>;
using FalseType = std::integral_constant<bool, false>;

static_assert(TrueType::value);
static_assert(!FalseType::value);
```

std::true_type과 std::false_type은 각각 참과 거짓을 나타내는 별칭이다.

```cpp
static_assert(std::true_type::value);
static_assert(!std::false_type::value);
```

<자주 사용하는 판정 타입 특성>

타입 특성	확인하는 내용
std::is_void_v<T>	void인지 여부
std::is_integral_v<T>	정수형인지 여부
std::is_floating_point_v<T>	부동소수점형인지 여부
std::is_array_v<T>	배열 타입인지 여부
std::is_pointer_v<T>	포인터 타입인지 여부
std::is_reference_v<T>	좌측값 또는 우측값 참조인지 여부
std::is_const_v<T>	최상위 const가 있는지 여부
std::is_enum_v<T>	열거형인지 여부
std::is_class_v<T>	클래스 또는 구조체인지 여부
std::is_same_v<T, U>	두 타입이 같은지 여부
std::is_base_of_v<B, D>	B가 D의 기반 클래스인지 여부
std::is_constructible_v<T, Args...>	지정한 인수로 생성 가능한지 여부
std::is_copy_constructible_v<T>	복사 생성 가능한지 여부
std::is_move_constructible_v<T>	이동 생성 가능한지 여부
std::is_nothrow_move_constructible_v<T>	예외 없이 이동 생성 가능한지 여부

타입 특성은 컴파일 시점 상수이므로 일반 실행 시간 분기보다 먼저 오류를 검출하거나 서로 다른 구현을 선택할 수 있다.

```cpp
template <typename T>
void RequireMovable()
{
    static_assert(std::is_move_constructible_v<T>,
                  "T must be move constructible");
}
```

static_assert는 조건이 거짓일 때 해당 특수화의 컴파일을 중단한다. Concepts가 함수 후보를 제한하는 반면 static_assert는 선택된 템플릿 본문에서 추가 계약을 검사하는 데 적합하다.

10.7.2 타입 판정과 타입 변환

<type_traits>는 타입을 판정하는 기능뿐 아니라 타입의 한정자와 형태를 변환하는 기능도 제공한다.

<참조 제거>

```
#include <type_traits>

using First = std::remove_reference_t<int&>;
using Second = std::remove_reference_t<const int&&>;

static_assert(std::is_same_v<First, int>);
static_assert(std::is_same_v<Second, const int>);

// const와 volatile 제거
using Value = std::remove_cv_t<const volatile int>;
static_assert(std::is_same_v<Value, int>);
```

C++20의 std::remove_cvref_t<T>는 참조를 제거한 뒤 최상위 cv 한정자를 제거한다.

```
using Raw = std::remove_cvref_t<const int&>;
static_assert(std::is_same_v<Raw, int>);
```

포인터를 추가하거나 제거할 수도 있다.

```
using Pointer = std::add_pointer_t<int>;
using Pointee = std::remove_pointer_t<int*>;

static_assert(std::is_same_v<Pointer, int*>);
static_assert(std::is_same_v<Pointee, int>);
```

std::decay_t<T>는 함수에 값을 전달할 때와 비슷한 변환을 적용한다.

- 참조 제거
- 최상위 cv 한정자 제거
- 배열을 원소 포인터로 변환
- 함수 타입을 함수 포인터로 변환

```
using ArrayType = int[5];
using DecayedArray = std::decay_t<ArrayType>;

static_assert(std::is_same_v<DecayedArray, int*>);
```

제네릭 코드에서 저장 타입을 만들 때 std::decay_t나 std::remove_cvref_t를 사용한다. 두 변환은 배열과 함수 처리에서 차이가 있으므로 목적에 맞게 선택해야 한다.
조건에 따라 타입을 선택할 수 있다.

```
#include <cstdint>
#include <type_traits>

using Selected = std::conditional_t<
    (sizeof(void*) == 8),
    std::uint64_t,
    std::uint32_t>;
```

두 타입의 공통 타입을 구할 수 있다.

```
using Common = std::common_type_t<int, double>;
static_assert(std::is_same_v<Common, double>);
```

혼합 타입 계산 함수의 반환 타입에 활용할 수 있다.

```
#include <type_traits>

template <typename Left, typename Right>
std::common_type_t<Left, Right>
AddCommon(const Left& left, const Right& right)
{
```

```
    using Result = std::common_type_t<Left, Right>;
    return static_cast<Result>(left) + static_cast<Result>(right);
}

double result = AddCommon(10, 2.5);
```

공통 타입이 알고리즘의 의미와 항상 일치하는 것은 아니다. 서로 다른 단위나 도메인 값을 자동으로 공통 타입에 넣으면 정보가 손실될 수 있다. 타입 계산은 언어 변환 규칙을 제공하며 도메인 의미는 별도로 설계해야 한다.

타입 특성 결과는 Concept 정의에도 사용할 수 있다.

```
template <typename T>
concept SmallTrivialValue =
    std::is_trivially_copyable_v<T> && sizeof(T) <= 16;
```

이 Concept는 작은 자명 복사 타입이라는 구현 조건을 표현한다. 이런 저수준 조건을 공개 인터페이스에 과도하게 노출하면 구현 변경을 어렵게 할 수 있으므로 성능상 실제로 필요한 경우에 사용한다.

10.7.3 if constexpr

if constexpr는 컴파일 시점 조건문이다. 조건은 상수 표현식이어야 하며 선택되지 않은 분기는 해당 템플릿 특수화에서 인스턴스화되지 않는다.
타입에 따라 출력 방식을 바꿔 보자.

```
#include <iostream>
#include <type_traits>

template <typename T>
void PrintCategory(const T& value)
{
    if constexpr (std::is_integral_v<T>)
    {
        std::cout << "integer: " << value << '\n';
    }
    else if constexpr (std::is_floating_point_v<T>)
    {
        std::cout << "real: " << value << '\n';
    }
    else
    {
        std::cout << "object: " << value << '\n';
    }
}

PrintCategory(10);
PrintCategory(3.5);
PrintCategory(std::string("Knight"));
```

T = int 특수화에서는 첫 분기만 사용되고 나머지 분기는 버려진 문장이 된다. T = std::string에서는 마지막 분기만 인스턴스화된다.
일반 if는 모든 분기가 문법과 타입 면에서 유효해야 한다.

```
template <typename T>
void Reset(T& value)
{
    if constexpr (std::is_pointer_v<T>)
    {
        value = nullptr;
    }
    else
    {
        value = T{};
    }
}
```

T가 포인터일 때 value = T{}도 유효하지만, 더 복잡한 경우 선택되지 않은 분기에 해당 타입에서 사용할 수 없는 코드가 있어도 템플릿 인수에 의존하는 표현이라면 인스턴스화되지 않는다.

```
template <typename T>
void Clear(T& value)
{
    if constexpr (requires { value.clear(); })
    {
        value.clear();
    }
    else
    {
        value = T{};
    }
}
```

clear() 멤버가 있는 타입은 해당 함수를 호출하고, 그렇지 않은 타입은 값 초기화된 객체를 대입한다.

버려진 분기라도 C++ 문법으로 분석할 수 있어야 한다. 템플릿 인수와 무관하게 항상 잘못된 표현은 컴파일러가 오류로 진단할 수 있다.

if constexpr는 하나의 알고리즘 안에서 작은 구현 차이를 처리할 때 적합하다. 타입마다 클래스의 저장 구조와 공개 인터페이스가 크게 달라지면 부분 특수화가 더 적합할 수 있다. 함수 자체의 사용 가능 조건이 다르면 Concept 제약 오버로딩이 더 분명하다.

반환 타입 추론과 함께 사용할 수 있다.

```
template <typename T>
auto GetValue(T value)
{
    if constexpr (std::is_pointer_v<T>)
    {
        return *value;
    }
    else
    {
        return value;
    }
}
```

각 특수화에서 선택된 return 문만 반환 타입 추론에 참여한다. 포인터 타입에서는 가리키는 값을, 일반 타입에서는 값 자체를 반환한다. 널 포인터 처리와 참조 반환 여부는 실제 인터페이스 요구에 맞게 보완해야 한다.

10.7.4 스피네이 (SFINAE) 와 enable_if 개요

SFINAE는 substitution failure is not an error의 약자이다. 함수 템플릿의 인수 대입 과정에서 즉시 문맥에 있는 타입이나 표현식이 유효하지 않으면 전체 컴파일을 바로 실패시키지 않고 해당 함수 템플릿을 후보에서 제거한다.
C++20 이전에는 std::enable_if를 이용해 함수 템플릿의 사용 조건을 표현하는 방식이 널리 사용되었다.

```
#include <type_traits>

template <typename T>
std::enable_if_t<std::is_integral_v<T>, T>
DoubleValue(T value)
{
    return value * 2;
}
```

T가 정수형이면 std::enable_if_t<true, T>가 T가 되어 함수가 유효하다. 정수형이 아니면 type 멤버가 존재하지 않아 대입 실패가 발생하고 함수 후보에서 제거된다.

```
int value = DoubleValue(10);
// double real = DoubleValue(1.5); // 사용할 수 있는 후보가 없음
```

반환 타입 대신 템플릿 매개변수에 조건을 둘 수 있다.

```
template <typename T,
          std::enable_if_t<std::is_integral_v<T>, int> = 0>
T TripleValue(T value)
{
    return value * 3;
```

```
}
```

정수형과 부동소수점형 오버로드를 구성할 수 있다.

```
#include <iostream>
#include <type_traits>

template <typename T,
    std::enable_if_t<std::is_integral_v<T>, int> = 0>
void Describe(T value)
{
    std::cout << "integer: " << value << '\n';
}

template <typename T,
    std::enable_if_t<std::is_floating_point_v<T>, int> = 0>
void Describe(T value)
{
    std::cout << "real: " << value << '\n';
}
```

같은 목적을 Concepts로 더 직접적으로 표현할 수 있다.

```
template <std::integral T>
void DescribeModern(T value)
{
    std::cout << "integer: " << value << '\n';
}

template <std::floating_point T>
void DescribeModern(T value)
{
    std::cout << "real: " << value << '\n';
}
```

SFINAE는 템플릿 인수 대입과 관련된 즉시 문맥에서만 적용된다. 함수 본문을 인스턴스화한 뒤 발생한 오류까지 후보 제거로 처리하는 것은 아니다.

```
template <typename T>
auto GetSize(const T& value) -> decltype(value.size())
{
    return value.size();
}
```

후행 반환 타입의 value.size()가 유효하지 않으면 이 함수 템플릿은 후보에서 제거될 수 있다. 본문 안에서만 value.size()를 호출하고 반환 타입에 조건이 드러나지 않으면 후보 선택 뒤 오류가 발생할 수 있다.

기존 라이브러리와 C++17 코드를 읽기 위해 SFINAE의 원리를 이해할 필요가 있다. 신규 C++20 코드에서는 Concepts가 의도를 더 잘 드러내고 오류 메시지도 읽기 쉬운 경우가 많다.

10.7.5 Concepts와 기존 제약 방식의 선택

타입에 따른 제어 기능은 역할이 겹쳐 보이지만 사용 위치가 다르다.

기능	주된 역할	적합한 위치
타입 특성	타입 판정과 타입 변환	구현 내부와 Concept 정의
static_assert	선택된 템플릿의 필수 조건 검사	클래스나 함수 본문
if constexpr	하나의 구현 안에서 컴파일 시점 분기	함수와 멤버 함수 본문
SFINAE	부적합한 함수 템플릿 후보 제거	C++17 호환 인터페이스
Concepts	템플릿 요구 조건과 제약 오버로딩	C++20 공개 인터페이스
특수화	타입 형태에 따른 전체 구현 변경	클래스 저장 구조와 정책

C++20을 기준으로 새 인터페이스를 작성할 때는 Concepts를 우선한다.

```
template <typename T>
concept HealthValue =
    std::integral<T> &&
    std::signed_integral<T>;

template <HealthValue T>
class Health
{
public:
    explicit Health(T maxHp)
        : hp(maxHp), maxHp(maxHp)
    {
    }

private:
    T hp;
    T maxHp;
};
```

구현 내부에서 타입별로 작은 차이가 있으면 if constexpr를 사용한다.

```
template <HealthValue T>
void PrintHealth(T hp)
{
    if constexpr (sizeof(T) <= sizeof(int))
    {
        std::cout << static_cast<int>(hp) << '\n';
    }
    else
    {
        std::cout << static_cast<long long>(hp) << '\n';
    }
}
```

타입의 참조와 cv 한정자를 제거하거나 공통 타입을 계산할 때는 타입 변환 특성을 사용한다. 같은 조건을 함수 선언에서 다시 SFINAE로 표현할 필요는 없다.
라이브러리가 C++17과 C++20을 모두 지원해야 한다면 내부 구현을 타입 특성으로 공유하고 공개 선언만 버전에 따라 SFINAE와 Concepts로 나눌 수 있다. 지원 표준과 유지보수 비용을 고려하여 한 프로젝트 안에서는 일관된 방식을 사용하는 편이 좋다.

10.8 전달 참조 (Forwarding Reference) 와 완벽 전달 (Perfect Forwarding)

제네릭 함수가 받은 인수를 다른 함수나 생성자에 전달할 때 타입뿐 아니라 값 범주도 보존해야 할 수 있다. 좌측값은 좌측값으로 전달하고, 우측값은 우측값으로 전달해야 복사와 이동 선택이 호출자의 의도와 일치한다.

전달 참조(forwarding reference)는 함수 템플릿 인수 추론과 참조 축약 규칙을 이용해 좌측값과 우측값을 모두 받을 수 있다. std::forward는 원래 인수의 값 범주를 복원하여 전달한다.

10.8.1 참조 축약

C++에서는 참조에 참조를 직접 선언할 수 없다.

```
// int& & invalidReference;
```

템플릿 인수 대입과 타입 별칭을 처리하는 과정에서는 참조 형태가 겹칠 수 있다. 이때 참조 축약(reference collapsing) 규칙이 적용된다.

겹친 형태	축약 결과
T& &	T&
T& &&	T&
T&& &	T&
T&& &&	T&&

우측값 참조끼리 결합한 경우에만 우측값 참조가 유지되고, 좌측값 참조가 하나라도 포함되면 좌측값 참조가 된다.
타입 별칭으로 확인할 수 있다.

```
#include <type_traits>

using LValue = int&;
using RValue = int&&;

using First = LValue&;
using Second = LValue&&;
using Third = RValue&;
using Fourth = RValue&&;

static_assert(std::is_same_v<First, int&>);
static_assert(std::is_same_v<Second, int&>);
static_assert(std::is_same_v<Third, int&>);
static_assert(std::is_same_v<Fourth, int&&>);
```

참조 축약은 전달 참조의 타입 추론에서 핵심 역할을 한다. 좌측값을 **T&&** 매개변수에 전달하면 **T**가 좌측값 참조로 추론되고, **T&&**에 축약 규칙이 적용되어 최종 매개변수 타입이 좌측값 참조가 된다.

10.8.2 전달 참조

함수 템플릿의 매개변수가 정확히 **T&&** 형태이고 T가 호출 인수에서 추론되면 **T&&**는 전달 참조가 된다. 전달 참조는 좌측값과 우측값을 모두 받을 수 있으며, 호출 인수의 참조성과 const 여부를 보존한다.

```
template <typename T>
void Relay(T&& value)
{
}
```

일반 변수와 상수 변수는 이름이 있는 객체이므로 모두 좌측값이다.

```
int number = 10;
const int limit = 20;

Relay(number);
Relay(limit);
```

number를 전달하면 T는 int&로 추론되고, 참조 축약에 따라 매개변수 타입도 int&가 된다.

```
T = int&
T&& = int& &&
     = int&
```

limit를 전달하면 T는 const int&로 추론되고, 매개변수 타입도 const int&가 된다.

```
T = const int&
T&& = const int& &&
     = const int&
```

정수 리터럴 10은 우측값이므로 T는 참조가 아닌 int로 추론된다.

```
Relay(10);
T = int
T&& = int&&
```

호출	인수의 값 범주	T	매개변수 타입
Relay(number)	좌측값	int&	int&
Relay(limit)	상수 좌측값	const int&	const int&
Relay(10)	우측값	int	int&&

추론 결과는 타입 특성으로 확인할 수 있다.

```cpp
#include <iostream>
#include <type_traits>

template <typename T>
void InspectForwarding(T&& value)
{
    using ValueType = std::remove_reference_t<T>;

    if constexpr (std::is_lvalue_reference_v<T>)
    {
        std::cout << "lvalue";
    }
    else
    {
        std::cout << "rvalue";
    }

    if constexpr (std::is_const_v<ValueType>)
    {
        std::cout << ", const";
    }

    std::cout << '\n';

    (void)value;
}

int main()
{
    int number = 10;
    const int limit = 20;

    InspectForwarding(number);
    InspectForwarding(limit);
    InspectForwarding(10);
}

// 출력 결과
// lvalue
// lvalue, const
// rvalue
```

T&&라고 해서 항상 전달 참조가 되는 것은 아니다.

```cpp
template <typename T>
void ReceiveConstRvalue(const T&& value)
{
}
```

const T&&는 정확한 T&& 형태가 아니므로 전달 참조가 아니라 상수 우측값 참조이다. 좌측값은 전달할 수 없다.
클래스 템플릿의 타입 매개변수가 객체 생성 시 이미 결정된 경우에도 전달 참조가 아니다.

```cpp
template <typename T>
class Holder
{
public:
    void Set(T&& value)
    {
    }
};
```

Holder<std::string>에서 T는 이미 std::string으로 결정되어 있으므로 Set()의 매개변수는 std::string&&이다.
멤버 함수가 별도의 타입 매개변수를 추론하면 전달 참조가 된다.

```cpp
template <typename T>
class Holder
{
public:
```

```
    template <typename U>
    void Set(U&& value)
    {
    }
};
```

U는 Set()의 호출 인수로부터 추론되므로 U&&는 전달 참조이다.
auto&&도 비슷한 추론 규칙을 사용한다.

```
int number = 10;
const int limit = 20;

auto&& first = number;  // int&
auto&& second = limit;  // const int&
auto&& third = 10;      // int&&
```

범위 기반 for문에서 auto&&를 사용하면 원소의 참조성과 const 여부를 유지할 수 있다.

```
for (auto&& value : values)
{
    // 원소의 참조성과 const 여부를 보존한다.
}
```

중괄호 초기화 목록과 함께 사용하는 auto&&에는 별도의 추론 규칙이 적용되므로 일반적인 전달 참조와 동일하게 보아서는 안 된다.

10.8.3 std::forward

이름이 있는 변수 표현식은 선언 타입이 우측값 참조여도 좌측값이다.

```
void Use(const std::string& value)
{
    std::cout << "copy-like use: " << value << '\n';
}

void Use(std::string&& value)
{
    std::cout << "move-like use: " << value << '\n';
}
```

전달 참조 함수에서 매개변수 이름을 그대로 사용하면 항상 좌측값 오버로드가 선택된다.

```
template <typename T>
void RelayWrong(T&& value)
{
    Use(value);
}

std::string name = "Knight";

RelayWrong(name);                  // Use(const std::string&)
RelayWrong(std::string("Mage"));   // 역시 Use(const std::string&)
```

두 번째 호출에서 value의 선언 타입은 std::string&&이지만 이름이 있는 표현식 value는 좌측값이다.
std::forward<T>(value)는 추론된 T에 따라 값 범주를 복원한다.

```
#include <utility>

template <typename T>
void RelayCorrect(T&& value)
{
    Use(std::forward<T>(value));
}

RelayCorrect(name);                // Use(const std::string&)
RelayCorrect(std::string("Mage")); // Use(std::string&&)
```

좌측값 호출에서는 T = std::string&이므로 std::forward<T>(value)의 결과도 좌측값이 된다. 우측값 호출에서는 T = std::string이므로 결과가 우측값이 된다.

std::forward는 인수를 무조건 우측값으로 바꾸지 않는다. value를 T&&로 형 변환하여 템플릿 인수 T에 기록된 값 범주를 복원한다.

표준 라이브러리의 실제 구현에는 여러 오버로드와 제약이 포함된다. 설명용 코드에는 좌측값 매개변수에 적용되는 형 변환만 남겼다.

```
template <typename T>
T&& Forward(std::remove_reference_t<T>& value) noexcept
{
    return static_cast<T&&>(value);
}
```

T가 std::string&이면 T&&는 참조 축약에 따라 std::string&가 된다. T가 std::string이면 T&&는 std::string&&가 된다.

실제 표준 std::forward에는 잘못된 사용을 막기 위한 오버로드와 검사도 포함되어 있다. 사용자는 직접 구현하지 않고 <utility>의 std::forward를 사용한다.

10.8.4 완벽 전달

완벽 전달(perfect forwarding)은 함수가 받은 인수의 타입, cv 한정자와 값 범주를 가능한 한 그대로 다른 함수에 전달하는 기법이다. 객체 생성 함수를 작성해 보자.

```
#include <memory>
#include <utility>

template <typename T, typename... Args>
std::unique_ptr<T> CreateObject(Args&&... args)
{
    return std::make_unique<T>(
        std::forward<Args>(args)...);
}
```

Args&&...는 각 인수에 대한 전달 참조 팩이다. std::forward<Args>(args)...는 각 인수의 원래 값 범주를 보존하여 생성자로 전달한다.

```
class Character
{
public:
    Character(std::string name, int hp)
        : name(std::move(name)), hp(hp)
    {
    }

private:
    std::string name;
    int hp;
};

std::string name = "Knight";

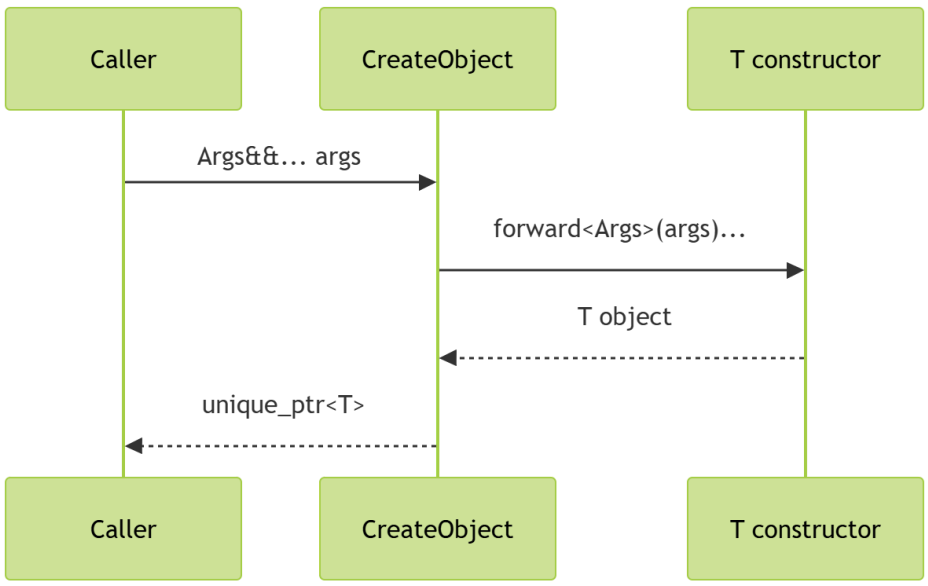
auto first = CreateObject<Character>(name, 100);
auto second = CreateObject<Character>(std::string("Mage"), 80);
```

첫 호출의 name은 좌측값으로 전달되므로 Character 생성자의 값 매개변수로 복사된 뒤 멤버로 이동된다. 두 번째 호출의 임시 문자열은 우측값으로 전달되어 값 매개변수 생성에 이동을 사용할 수 있다.

완벽 전달이 없다면 모든 인수를 값으로 받거나 상수 참조로 받을 수 있다.

```
template <typename T, typename... Args>
std::unique_ptr<T> CreateObjectByConstReference(
    const Args&... args)
{
    return std::make_unique<T>(args...);
}
```

이 함수는 이동 전용 인수를 전달할 수 없고, 우측값도 생성자에 좌측값으로 전달한다.



Concept를 추가하면 생성 가능성을 인터페이스에 표현할 수 있다.

```
#include <concepts>

template <typename T, typename... Args>
requires std::constructible_from<T, Args...>
std::unique_ptr<T> CreateChecked(Args&&... args)
{
    return std::make_unique<T>(
        std::forward<Args>(args)...);
}
```

그러나 std::constructible_from<T, Args...>의 Args...는 전달 표현식의 참조 축약 결과와 관계가 있으므로 실제 구현에서는 함수 선언과 표준 라이브러리 함수의 제약을 함께 확인해야 한다. std::make_unique 자체도 생성자 호출이 유효하지 않으면 컴파일 오류를 제공한다. Concept는 함수 목적을 드러내는 데 의미가 있다.

10.8.5 std::move와 std::forward의 차이

std::move와 std::forward는 모두 실제 이동 연산을 수행하지 않는다. 두 함수는 표현식의 값 범주를 바꾸거나 보존하는 형 변환 도구이다. std::move(value)는 인수를 무조건 우측값으로 취급할 수 있는 xvalue로 변환한다.

```
std::string source = "Knight";
std::string target = std::move(source);
```

이동 생성자가 선택될 수 있도록 source를 우측값으로 표현한다. 실제 자원 이동은 std::string의 이동 생성자가 수행한다. std::forward<T>(value)는 T의 추론 결과에 따라 좌측값 또는 우측값을 유지한다.

```
template <typename T>
void ForwardToUse(T&& value)
{
    Use(std::forward<T>(value));
}
```

<std::move, std::forward 비교>

항목	std::move	std::forward
목적	더 이상 유지할 필요가 없는 객체를 이동 가능한 표현식으로 변환	호출자가 전달한 원래 값 범주 보존
주 사용 위치	일반 코드와 이동 연산 구현	전달 참조를 받는 제네릭 함수
결과	항상 우측값 범주의 표현식	T에 따라 좌측값 또는 우측값
템플릿 인수	필요 없음	원래 추론된 타입 인수 필요
사용 후 주의	원본이 이동된 상태일 수 있음	우측값으로 전달한 경우 이동된 상태일 수 있음

전달 함수에서 std::move를 사용하면 좌측값까지 우측값으로 전달해 버린다.

```
template <typename T>
void RelayWithMove(T&& value)
{
    Use(std::move(value));
}

std::string name = "Knight";
RelayWithMove(name);
```

호출자는 좌측값을 전달했지만 Use(std::string&&)이 선택된다. 함수가 실제로 자원을 이동하면 호출자의 name이 변경될 수 있다. 전달이 목적이라면 std::forward<T>를 사용해야 한다.

반대로 함수가 매개변수를 자신의 멤버로 확실히 이동하려는 상황에서는 std::move가 적합하다.

```
class Player
{
public:
    explicit Player(std::string name)
        : name(std::move(name))
    {
    }

private:
    std::string name;
};
```

값 매개변수 name은 함수 내부의 지역 객체이며 더 이상 원래 값을 유지할 필요가 없다. 멤버로 이동한다는 의도가 분명하므로 std::move를 사용한다.

10.8.6 완벽 전달의 사용 기준과 종합 예제

완벽 전달(perfect forwarding)은 생성자나 다른 호출 대상에 인수를 중계하는 함수에 적합하다. 모든 제네릭 함수에 전달 참조를 사용할 필요는 없다.

읽기만 하는 함수는 const T&가 더 분명하다.

```
template <typename T>
void PrintValue(const T& value)
{
    std::cout << value << '\n';
}
```

값을 복사하여 보관하는 함수나 클래스는 값 매개변수를 사용할 수 있다.

```
template <typename T>
class ValueHolder
{
public:
    explicit ValueHolder(T value)
        : value(std::move(value))
    {
    }

private:
    T value;
};
```

```
<완벽 전달 적용 기준>
- 다른 함수나 생성자에 인수를 중계할 때 사용
- 전달 참조와 추론된 템플릿 인수를 함께 사용
- std::forward<T>(value)로 전달한 값은 다시 사용하지 않음
- 우측값 인수는 호출 대상에서 이동될 수 있음
- 읽기, 계산, 비교만 수행하는 함수에는 사용하지 않음
- 전달 생성자가 복사 생성자나 다른 오버로드보다 우선 선택되지 않는지 확인
```

전달 생성자는 거의 모든 타입을 받을 수 있으므로 의도하지 않은 호출까지 선택될 수 있다.

```cpp
template <typename T>
class Wrapper
{
public:
    template <typename U>
    explicit Wrapper(U&& value)
        : value(std::forward<U>(value))
    {
    }

private:
    T value;
};
```

Wrapper의 복사나 변환 과정에서 이 생성자가 예상보다 좋은 후보가 될 수 있다. std::constructible_from<T, U&&> 같은 제약과 자기 타입 제외 조건을 추가하거나 값 매개변수 생성자로 단순화할 수 있다.

중괄호 초기화 목록은 이름 있는 단일 타입이 아니므로 일반 전달 함수에서 Args를 추론하지 못할 수 있다.

```cpp
void UseList(std::initializer_list<int> values)
{
}

template <typename T>
void RelayList(T&& value)
{
    UseList(std::forward<T>(value));
}

// RelayList({1, 2, 3}); // T를 추론할 수 없음
```

명시적으로 std::initializer_list<int> 객체를 만들거나 별도 오버로드를 제공해야 한다.

```cpp
RelayList(std::initializer_list<int>{1, 2, 3});
```

오버로드된 함수 이름도 단일 타입으로 결정되지 않아 추론이 실패할 수 있다. 목표 함수 포인터 타입으로 형 변환하거나 별도 람다로 감싸는 방식이 필요하다. 비트 필드는 비상수 참조에 직접 바인딩할 수 없으므로 전달 참조로 받을 수 없는 경우가 있다.

제네릭 프로그래밍의 여러 기능을 하나의 게임 예제로 연결해 보자. 이 예제는 객체지향 다형성과 정적 제약을 함께 사용한다.

```cpp
#include <algorithm>
#include <concepts>
#include <iostream>
#include <memory>
#include <string>
#include <string_view>
#include <utility>

class HealthComponent
{
public:
    explicit HealthComponent(int maxHp)
        : hp(maxHp), maxHp(maxHp)
    {
    }

    int TakeDamage(int damage)
    {
        if (damage <= 0 || IsDead())
        {
            return 0;
        }

        const int oldHp = hp;
        hp = std::max(hp - damage, 0);
        return oldHp - hp;
    }
```

```
    int GetHp() const noexcept
    {
        return hp;
    }

    bool IsDead() const noexcept
    {
        return hp == 0;
    }

private:
    int hp;
    int maxHp;
};

class GameObject
{
public:
    virtual ~GameObject() = default;

    virtual void Update() = 0;
    virtual std::string_view GetName() const noexcept = 0;
};

class Player : public GameObject
{
public:
    Player(std::string name, int maxHp)
        : name(std::move(name)), health(maxHp)
    {
    }

    void Update() override
    {
        std::cout << name << " handles player input\n";
    }

    std::string_view GetName() const noexcept override
    {
        return name;
    }

    int TakeDamage(int damage)
    {
        return health.TakeDamage(damage);
    }

    int GetHp() const noexcept
    {
        return health.GetHp();
    }

    bool IsDead() const noexcept
    {
        return health.IsDead();
    }

private:
    std::string name;
    HealthComponent health;
};

class Monster : public GameObject
{
public:
    Monster(std::string name, int maxHp, int attackPower)
        : name(std::move(name)),
          health(maxHp),
          attackPower(attackPower)
```

```
    {
    }

    void Update() override
    {
        std::cout << name << " runs AI behavior\n";
    }

    std::string_view GetName() const noexcept override
    {
        return name;
    }

    int TakeDamage(int damage)
    {
        return health.TakeDamage(damage);
    }

    int GetHp() const noexcept
    {
        return health.GetHp();
    }

    bool IsDead() const noexcept
    {
        return health.IsDead();
    }

    int GetAttackPower() const noexcept
    {
        return attackPower;
    }

private:
    std::string name;
    HealthComponent health;
    int attackPower;
};

template <typename T>
concept Damageable = requires(T& target,
                              const T& constTarget,
                              int damage)
{
    { target.TakeDamage(damage) } -> std::same_as<int>;
    { constTarget.GetHp() } noexcept -> std::same_as<int>;
    { constTarget.IsDead() } noexcept -> std::same_as<bool>;
};

template <Damageable Target>
int ApplyDamage(Target& target, int damage)
{
    return target.TakeDamage(damage);
}

template <typename T, typename... Args>
requires std::derived_from<T, GameObject> &&
         std::constructible_from<T, Args...>
std::unique_ptr<GameObject> CreateGameObject(Args&&... args)
{
    return std::make_unique<T>(
        std::forward<Args>(args)...);
}

template <typename... Objects>
requires (std::derived_from<Objects, GameObject> && ...)
void UpdateObjects(Objects&... objects)
{
    (objects.Update(), ...);
```

```
}

int main()
{
    Player player("Knight", 150);
    Monster monster("Goblin", 80, 12);

    UpdateObjects(player, monster);

    const int appliedDamage = ApplyDamage(monster, 35);

    std::cout << monster.GetName()
              << " damage=" << appliedDamage
              << " hp=" << monster.GetHp()
              << '\n';

    std::unique_ptr<GameObject> objects[]
    {
        CreateGameObject<Player>("Mage", 100),
        CreateGameObject<Monster>("Orc", 120, 18)
    };

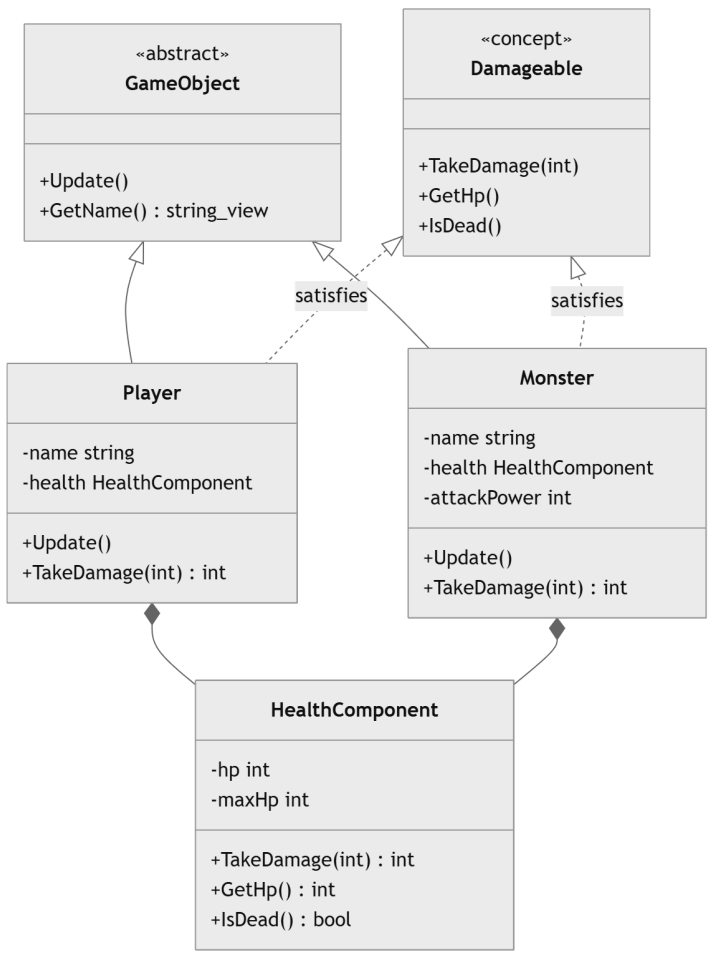
    for (const auto& object : objects)
    {
        object->Update();
    }
}
```

GameObject 계층은 실행 시간 다형성을 제공한다. 서로 다른 구체 객체를 std::unique_ptr<GameObject> 배열에 저장하고 가상 Update()를 호출한다.

Damageable Concept는 상속 여부와 관계없이 피해 처리에 필요한 정적 인터페이스를 표현한다. ApplyDamage()는 구체 타입이 컴파일 시점에 알려지므로 가상 함수 없이 TakeDamage()를 호출한다.

UpdateObjects()는 가변 인수 템플릿과 폴드 표현식을 사용한다. 각 타입이 GameObject의 파생 타입인지 제약하고, 여러 구체 객체의 Update()를 순서대로 호출한다. 이 함수는 객체 타입이 컴파일 시점에 알려진 경우의 정적 다형성을 보여 준다.

CreateGameObject()는 구체 타입 T가 GameObject의 파생 타입인지 확인하고 생성자 인수를 완벽 전달한다. 반환 타입은 std::unique_ptr<GameObject>이므로 호출 코드는 생성된 구체 객체를 공통 소유 타입으로 관리할 수 있다.



이 구조는 객체지향과 제네릭 프로그래밍이 서로 대체 관계만은 아니라는 점을 보여 준다. 객체의 런타임 종류를 다룰 때는 기반 클래스와 가상 함수를 사용하고, 컴파일 시점에 타입별 공통 연산을 적용할 때는 템플릿과 Concept를 사용할 수 있다.

10.9 학습 정리

제네릭 프로그래밍은 구체 타입의 이름보다 타입이 제공하는 연산과 의미에 의존하여 알고리즘과 자료 구조를 작성하는 방식이다. 템플릿을 사용하면 타입과 컴파일 시점 값을 매개변수로 분리하고 여러 구체 특수화를 생성할 수 있다.

함수 템플릿은 함수 인수로부터 템플릿 인수를 추론한다. 값 매개변수에서는 최상위 const와 참조성이 제거되고, 참조 매개변수에서는 참조 대상의 한정자가 보존된다. 배열 참조를 이용하면 배열 크기를 비 타입 템플릿 인수로 추론할 수 있다. 추론할 수 없는 타입은 명시적 템플릿 인수로 지정할 수 있다.

일반 함수와 함수 템플릿은 함께 오버로드될 수 있다. 오버로드 결정은 변환 순위를 먼저 비교하며, 같은 수준이라면 일반 함수가 우선될 수 있다. 명시적 꺾쇠괄호를 사용하면 함수 템플릿 호출을 지정할 수 있다.

템플릿은 사용된 템플릿 인수 조합에 대해 인스턴스화된다. 템플릿 인수에 의존하는 표현은 실제 특수화가 만들어질 때 검사될 수 있다. 일반적인 템플릿 정의는 호출 지점에서 보여야 하므로 헤더에 둔다. 제한된 타입만 지원한다면 명시적 인스턴스화로 코드 생성을 분리할 수 있다.

클래스 템플릿은 원소 타입, 정책과 크기를 매개변수화한다. 각 템플릿 인수 조합은 서로 다른 구체 타입이다. 클래스 템플릿 인수 추론은 생성자 인수와 추론 가이드를 이용해 템플릿 인수를 결정한다. 별칭 템플릿은 기존 특수화에 간결한 이름을 제공하지만 새로운 타입을 만들지는 않는다.

비 타입 템플릿 매개변수는 배열 크기와 정책 값처럼 컴파일 시점에 결정되는 값을 타입의 일부로 만든다. 크기나 정책 값이 다르면 서로 다른 타입이 되며 실행 중 값을 바꾸는 용도에는 적합하지 않다.

전체 특수화는 모든 템플릿 인수를 고정한 별도 구현을 제공하고, 부분 특수화는 특정 타입 형태나 일부 인수 조건에 적용된다. 함수 템플릿은 부분 특수화를 지원하지 않으므로 일반 오버로딩이나 Concept 제약 오버로딩을 우선한다. 특수화는 기본 템플릿의 의미와 인터페이스를 가능한 한 유지해야 한다.

가변 인수 템플릿은 타입과 값의 개수가 정해지지 않은 팩을 받는다. 팩 확장은 문법적 패턴을 각 원소에 반복하며, 폴드 표현식은 팩 원소 사이에 연산자를 결합한다. 단순 합계, 논리 조건과 순차 호출에는 재귀적 전개보다 폴드 표현식이 간결하다.

Concepts는 템플릿이 요구하는 타입 조건을 선언에 표현한다. 표준 Concept는 정수형, 부동소수점형, 생성 가능성, 파생 관계와 호출 가능성 같은 조건을 제공한다. requires 표현식은 단순, 타입, 복합과 중첩 요구 조건으로 표현식과 타입의 유효성을 검사한다.

제약 조건이 다른 오버로드가 모두 사용 가능하면 더 구체적인 제약을 가진 후보가 선택될 수 있다. 반복되는 조건을 이름 있는 Concept로 분리하면 포함 관계와 인터페이스 의미가 분명해진다.

타입 특성은 컴파일 시점에 타입의 성질을 판정하거나 참조와 cv 한정자를 제거하고 공통 타입을 계산한다. if constexpr는 선택된 템플릿 구현 안에서 타입별 코드를 분기한다. SFINAE와 enable_if는 기존 C++ 코드에서 부적합한 함수 템플릿 후보를 제거하는 방식이며, C++20 신규 인터페이스에서는 Concepts가 더 직접적이다.

전달 참조는 추론되는 T&& 매개변수이다. 좌측값을 받으면 T가 좌측값 참조로 추론되고 참조 축약으로 최종 매개변수가 좌측값 참조가 된다. 우측값을 받으면 우측값 참조가 유지된다.

이름이 있는 매개변수는 항상 좌측값 표현식이므로 원래 값 범주를 전달하려면 std::forward<T>(value)를 사용한다. std::move는 인수를 무조건 우측값으로 표현하고, std::forward는 호출자가 전달한 값 범주를 보존한다. 완벽 전달은 생성자나 다른 함수에 인수를 중계하는 제네릭 인터페이스에 제한적으로 적용한다.

객체지향 다형성과 정적 다형성은 목적이 다르다. 실행 시간에 여러 구체 객체를 공통 타입으로 관리할 때는 가상 함수가 적합하고, 구체 타입이 컴파일 시점에 알려지며 타입별 최적화와 요구 조건 표현이 중요할 때는 템플릿이 적합하다. 실제 C++ 프로그램은 두 방식을 함께 사용하여 런타임 구조와 컴파일 타임 일반화를 구성할 수 있다.

10.10 학습 과제

과제 **1.** 함수 템플릿과 타입 요구 조건
GetMin(), GetMax()와 Clamp() 함수 템플릿을 작성한다.
요구 사항:

- int, double과 사용자 정의 Level 타입에서 동작
- Level은 비교 연산을 제공
- 혼합 타입 호출이 실패하는 이유 설명
- 명시적 템플릿 인수로 혼합 타입을 호출했을 때 발생하는 변환 확인
- const T& 반환형에서 임시 객체를 전달할 때의 수명 문제 분석

함수가 실제로 요구하는 연산과 의미 조건을 표로 정리한다.

과제 **2.** 배열 크기 추론
C 배열을 참조로 받아 원소 수를 반환하는 ArraySize()를 작성한다. 원소 타입과 크기를 각각 T, Size로 추론해야 한다.
추가 기능:

- 배열의 모든 원소를 출력하는 PrintArray()
- 배열의 합계를 반환하는 SumArray()
- 빈 배열을 만들 수 없는 C++ 배열의 특성 설명
- 배열을 포인터 매개변수로 받았을 때 크기 정보가 사라지는 이유 설명

static_assert로 여러 크기의 결과를 검증한다.

과제 **3. FixedArray** 클래스 템플릿
원소 타입과 크기를 템플릿 인수로 받는 FixedArray<T, Size>를 구현한다.
필수 기능:

- 값 초기화된 원소 저장
- At()의 상수·비상수 오버로드
- operator[]의 상수·비상수 오버로드
- Fill()
- Size()
- 범위 오류 시 std::out_of_range
- Size > 0 컴파일 시점 검사

FixedArray<int, 4>와 FixedArray<int, 8>이 서로 다른 타입임을 std::is_same_v로 확인한다. Mermaid 클래스 다이어그램에 타입 매개변수와 비타입 매개변수를 표시한다.

과제 **4.** 클래스 템플릿 인수 추론
값 하나를 저장하는 ValueBox<T>와 두 값을 저장하는 PairBox<First, Second>를 구현한다.
요구 사항:

- 생성자 인수로 CTAD 적용
- 문자열 리터럴을 std::string으로 저장하는 추론 가이드 작성
- 명시적 템플릿 인수와 추론 결과 비교
- 추론 가이드가 일반 함수가 아닌 이유 설명
- 복사 생성에서 어떤 추론 후보가 사용되는지 확인

컴파일러가 추론한 타입을 static_assert로 검증한다.

과제 **5.** 전체 특수화와 부분 특수화
TypeDescription<T> 클래스 템플릿을 작성한다.
분류할 타입:

- 일반 값 타입
- 포인터 타입
- 고정 크기 배열 타입
- bool 전체 특수화
- const T 형태

각 특수화는 타입 범주 이름을 반환해야 한다. 여러 부분 특수화가 동시에 일치할 수 있는 사례를 만들고 모호성이 발생하는 이유를 설명한다.
같은 기능을 타입 특성과 if constexpr로 작성한 방식과 비교한다.

과제 **6.** 가변 인수 로그 함수
가변 인수 템플릿과 폴드 표현식으로 로그 함수를 작성한다.
필수 기능:

- 로그 수준 출력
- 인수 개수 출력
- 여러 타입의 값을 순서대로 출력
- 각 값 사이에 구분자 삽입
- 인수가 없는 호출 처리
- 출력할 수 없는 타입에서 이해하기 쉬운 컴파일 오류 제공

출력 가능성을 검사하는 StreamWritable Concept를 정의한다. 재귀적 전개 버전과 폴드 표현식 버전의 코드 길이와 인스턴스화 구조를 비교한다.

과제 **7.** 게임 객체 **Concept**
상속 관계가 없는 Player, Monster, BreakableBox를 작성한다. 세 타입은 체력 또는 내구도 상태를 자체적으로 관리한다.
Damageable Concept 요구 조건:

- TakeDamage(int) 호출 가능
- 결과를 int로 변환 가능
- GetHp() 또는 동일한 의미의 상태 조회 함수 제공
- IsDead()가 bool 결과 제공

ApplyDamage()와 PrintHealth()를 Concept로 제약한다. 요구 조건을 만족하지 않는 Decoration 타입을 전달했을 때 컴파일 오류를 확인한다.
동적 인터페이스 IDamageable 방식과 Concept 방식의 UML을 각각 작성하고 사용 시점을 비교한다.

과제 **8.** 타입 특성과 **if constexpr**
여러 타입의 값을 문자열로 설명하는 DescribeValue()를 작성한다.
구분할 범주:

- 정수형
- 부동소수점형
- 열거형
- 포인터
- 문자열
- 그 밖의 출력 가능한 객체

std::is_integral_v, std::is_floating_point_v, std::is_enum_v, std::is_pointer_v, std::is_same_v와 std::remove_cvref_t를 사용한다. 널 포인터를 안전하게 처리하고 선택되지 않은 분기가 인스턴스화되지 않는 사례를 포함한다.

과제 **9. SFINAE** 코드를 **Concepts**로 변경
std::enable_if_t를 사용하여 정수형과 부동소수점형의 Normalize() 오버로드를 작성한다. 같은 인터페이스를 C++20 Concepts로 다시 작성한다.
비교 항목:

- 함수 선언의 가독성
- 오류 메시지 위치
- 오버로드 후보 선택
- 조건 재사용 방식
- 구현 내부에서 타입 특성이 여전히 필요한 부분

기존 SFINAE 코드를 유지해야 하는 상황과 Concepts로 전환할 수 있는 상황을 구분한다.

과제 **10.** 완벽 전달 기반 게임 객체 팩토리
GameObject 추상 클래스와 Player, Monster, Projectile 구체 클래스를 작성한다. 생성자 매개변수의 타입과 개수는 서로 달라야 한다.
CreateGameObject<T>(Args&&...) 요구 사항:

- T가 GameObject의 공개 파생 타입인지 검사
- 지정한 인수로 T를 생성할 수 있는지 검사
- 생성자 인수를 완벽 전달
- std::unique_ptr<GameObject> 반환
- 전역 상태 사용 금지
- 잘못된 생성자 인수에서 컴파일 시점 오류 발생

복사 횟수와 이동 횟수를 출력하는 추적 타입을 만들어 좌측값과 우측값 전달 결과를 비교한다. std::move를 잘못 사용한 팩토리와 std::forward를 사용한 팩토리의 차이를 확인한다.

종합 UML에서 관계 표현확인:

- GameObject와 구체 클래스의 상속
- 각 객체가 소유하는 컴포넌트
- 팩토리 함수 템플릿의 생성 의존
- Concept가 요구하는 정적 인터페이스

실행 시간 다형성과 정적 다형성을 어느 위치에 사용했는지 문장으로 설명한다.

11. 표준 라이브러리와 순차 컨테이너

템플릿을 이용하면 타입에 독립적인 함수와 클래스를 작성할 수 있다. 그러나 프로그램에서 필요한 자료 구조와 알고리즘을 매번 직접 구현할 필요는 없다. C++ 표준 라이브러리는 자주 사용하는 자료 구조, 알고리즘, 문자열, 입출력, 메모리 관리, 동시성 기능을 공통 인터페이스로 제공한다.

여러 값을 저장하는 가장 기본적인 방법은 배열이다. 배열은 메모리가 연속되어 있고 인덱스로 빠르게 접근할 수 있지만 크기를 실행 중에 변경하기 어렵다. 프로그램에서는 고정 크기 배열뿐 아니라 원소를 추가하거나 제거할 수 있는 가변 크기 구조, 양쪽 끝을 효율적으로 사용하는 구조, 원소의 위치가 바뀌지 않는 연결 구조도 필요하다.

표준 라이브러리의 컨테이너는 저장 방식이 서로 다르지만 반복자를 통해 원소 범위를 표현한다. 알고리즘은 컨테이너의 구체 타입보다 반복자가 제공하는 기능을 사용한다. std::vector의 원소도, std::list의 원소도 반복자 범위로 전달하면 같은 알고리즘을 적용할 수 있다.

```
#include <algorithm>
#include <iostream>
#include <list>
#include <vector>

int main()
{
    std::vector<int> scores{70, 95, 80, 60};
    std::list<int> levels{3, 1, 4, 2};

    std::sort(scores.begin(), scores.end());
    levels.sort();

    for (int score : scores)
    {
        std::cout << score << ' ';
    }
}
```

std::vector는 임의 접근 반복자를 제공하므로 std::sort()를 사용할 수 있다. std::list의 반복자는 임의 접근을 지원하지 않으므로 리스트가 제공하는 sort() 멤버 함수를 사용한다. 저장 구조와 반복자의 능력이 알고리즘 선택에 영향을 준다.
이 장은 표준 라이브러리의 구성 요소를 살펴보고, 순차 컨테이너가 제공하는 공통 기능과 저장 구조의 차이를 다룬다. std::array, std::vector, std::deque, std::list, std::forward_list를 비교하고, 연속된 문자 데이터를 관리하는 std::string까지 연결한다. 컨테이너를 선택할 때는 이름이나 익숙함보다 크기 변경, 메모리 연속성, 접근 방식, 삽입과 제거 위치, 반복자 안정성을 함께 판단해야 한다.

11.1 표준 라이브러리의 구조

C++ 표준 라이브러리에는 컨테이너와 알고리즘 외에도 문자열, 스트림, 스마트 포인터, 시간, 난수, 파일 시스템, 스레드와 같은 기능이 포함된다. 각 기능은 헤더 단위로 제공되며 대부분 std 네임스페이스에 정의된다.

```
#include <vector>
#include <algorithm>
#include <memory>
#include <string>
```

헤더를 포함하면 해당 헤더가 공개하는 타입과 함수를 사용할 수 있다. 표준 라이브러리 구현은 컴파일러와 함께 제공되지만, 사용자가 의존하는 인터페이스와 동작은 C++ 표준으로 규정된다.

11.1.1 표준 라이브러리와 STL

STL(Standard Template Library)은 컨테이너, 반복자, 알고리즘, 함수 객체를 템플릿으로 연결한 라이브러리 설계에서 출발했다. 현재 C++ 표준 라이브러리에는 STL에서 발전한 구성 요소가 포함되어 있지만 표준 라이브러리 전체가 STL인 것은 아니다.

std::vector, std::list, std::sort, 반복자, 함수 객체는 보통 STL 계열의 기능으로 설명한다. std::iostream, std::thread, std::filesystem도 표준 라이브러리의 일부이지만 일반적으로 STL이라고 부르지는 않는다. 교재에서는 전체를 가리킬 때 표준 라이브러리, 컨테이너와 반복자와 알고리즘이 결합된 구조를 강조할 때 STL 방식이라는 표현을 사용한다.

<표준 라이브러리의 주요 구성 요소>

구성 요소	역할	대표 예
컨테이너	여러 원소를 저장하고 관리	std::vector, std::map
반복자	원소 위치와 범위를 표현	begin(), end()

알고리즘	범위의 원소를 검색, 정렬, 변환	std::find, std::sort
함수 객체	호출 가능한 동작을 객체로 표현	std::less, 람다 표현식
문자열	문자 시퀀스를 저장하고 가공	std::string
메모리 관리	동적 객체의 수명과 소유권 관리	std::unique_ptr
입출력	콘솔, 파일, 문자열 스트림 처리	std::cout, std::ifstream
유틸리티	이동, 튜플, 시간, 난수 등 제공	std::move, std::tuple

11.1.2 컨테이너

컨테이너(container)는 여러 원소를 저장하고 원소의 수명과 배치를 관리하는 객체이다. 컨테이너에 저장되는 타입을 원소 타입이라고 한다.

```
std::vector<int> scores;
std::list<std::string> names;
```

scores의 원소 타입은 int, names의 원소 타입은 std::string이다. 컨테이너는 원소를 값으로 소유한다. 외부 객체를 가리키는 포인터를 저장할 수도 있지만, 그 경우 가리키는 객체의 수명은 포인터 타입과 소유권 정책에 따라 별도로 관리해야 한다.

표준 컨테이너는 저장 구조와 탐색 방식에 따라 여러 범주로 나뉜다.

범주	특징	대표 타입
순차 컨테이너	원소가 위치 순서에 따라 배치됨	array, vector, deque, list, forward_list
연관 컨테이너	정렬된 키를 기준으로 탐색	set, map
비정렬 연관 컨테이너	해시를 이용해 키를 탐색	unordered_set, unordered_map
컨테이너 어댑터	제한된 인터페이스로 기존 컨테이너를 사용	stack, queue, priority_queue

순차 컨테이너에서 원소의 순서는 삽입 위치와 프로그램의 조작에 의해 결정된다. 연관 컨테이너는 키의 비교 또는 해시 규칙이 저장 위치와 탐색 방식을 결정한다.

11.1.3 반복자

반복자(iterator)는 컨테이너의 원소 위치를 나타내는 객체이다. 포인터와 비슷하게 역참조 연산자 *로 원소에 접근하고 ++로 이동할 수 있다.

```
std::vector<int> values{10, 20, 30};
auto iterator = values.begin();
std::cout << *iterator << '\n';

++iterator;
std::cout << *iterator << '\n';
```

begin()은 첫 원소를 가리키는 반복자를 반환한다. end()는 마지막 원소가 아니라 마지막 원소의 바로 뒤 위치를 나타낸다. 원소가 없는 컨테이너에서는 begin() == end()가 성립한다.

```
[10] [20] [30] [끝 위치]
  ^              ^
begin()          end()
```

반복자 범위는 [first, last) 형태의 반열린 구간으로 표현한다. first가 가리키는 원소는 포함하고 last가 가리키는 위치는 포함하지 않는다. 이 규칙을 사용하면 빈 범위를 first == last로 표현할 수 있고, 처리한 원소 수를 반복자 거리로 계산하기 쉽다.

반복자가 제공하는 이동 능력은 컨테이너마다 다르다.

반복자 범주	주요 연산	대표 컨테이너
전진 반복자	++로 한 방향 이동	forward_list

양방향 반복자	++, --로 양방향 이동	list
임의 접근 반복자	+, -, [], 대소 비교	vector, deque, array
연속 반복자	임의 접근과 연속 메모리 보장	array, vector, string

알고리즘은 필요한 반복자 능력을 요구한다. std::find()는 앞 방향 순회만 필요하지만 std::sort()는 원소 사이를 빠르게 이동할 수 있는 임의 접근 반복자를 요구한다.

11.1.4 알고리즘

알고리즘(algorithm)은 반복자 범위를 받아 검색, 정렬, 복사, 변환, 집계 작업을 수행하는 함수 템플릿이다. <algorithm>과 <numeric> 헤더에 많은 알고리즘이 정의되어 있다.

```
#include <algorithm>
#include <numeric>
#include <vector>

std::vector<int> values{5, 2, 8, 2, 1};

auto position = std::find(values.begin(), values.end(), 8);
std::sort(values.begin(), values.end());
int total = std::accumulate(values.begin(), values.end(), 0);
```

std::find()는 값을 찾지 못하면 두 번째 인수로 전달한 끝 반복자를 반환한다.

```
if (position != values.end())
{
    // 값을 찾음
}
```

알고리즘은 컨테이너를 직접 소유하지 않는다. 반복자 범위에 있는 원소를 읽거나 변경할 뿐이다. 같은 알고리즘을 여러 컨테이너와 원시 배열에 적용할 수 있는 이유도 반복자 범위를 사용하기 때문이다.

```
int values[]{30, 10, 20};
std::sort(std::begin(values), std::end(values));
```

11.1.5 함수 객체 (Function Object)

함수 객체(function object)는 함수 호출 연산자 operator()를 제공하는 객체이다. 알고리즘에 비교, 판정, 변환 규칙을 전달할 때 사용한다.

```
#include <algorithm>
#include <functional>
#include <vector>

std::vector<int> values{5, 1, 9, 3};
std::sort(values.begin(), values.end(), std::greater<int>{});
```

std::greater<int>는 두 정수를 큰 값부터 정렬하도록 비교한다. 람다 표현식도 호출 가능한 객체이므로 같은 위치에 사용할 수 있다.

```
std::sort(
    values.begin(),
    values.end(),
    [](int left, int right)
    {
        return left > right;
    });
```

함수 객체는 상태를 가질 수도 있다.

```
class IsGreaterThan
{
public:
```

```
    explicit IsGreaterThan(int limit)
        : limit(limit)
    {
    }

    bool operator()(int value) const
    {
        return value > limit;
    }

private:
    int limit;
};

int count = std::count_if(
    values.begin(),
    values.end(),
    IsGreaterThan(3));
```

11.1.6 자료구조와 알고리즘의 분리

자료구조와 알고리즘을 분리하면 알고리즘을 특정 컨테이너에 종속시키지 않고 재사용할 수 있다. 컨테이너는 원소 저장과 반복자 제공을 담당하고, 알고리즘은 반복자 범위에 적용할 작업을 담당한다.

std::vector<int>와 std::array<int, 5>는 저장 크기 관리 방식이 다르지만 모두 임의 접근 반복자를 제공한다. 두 컨테이너는 std::sort()에 같은 형태로 전달할 수 있다.

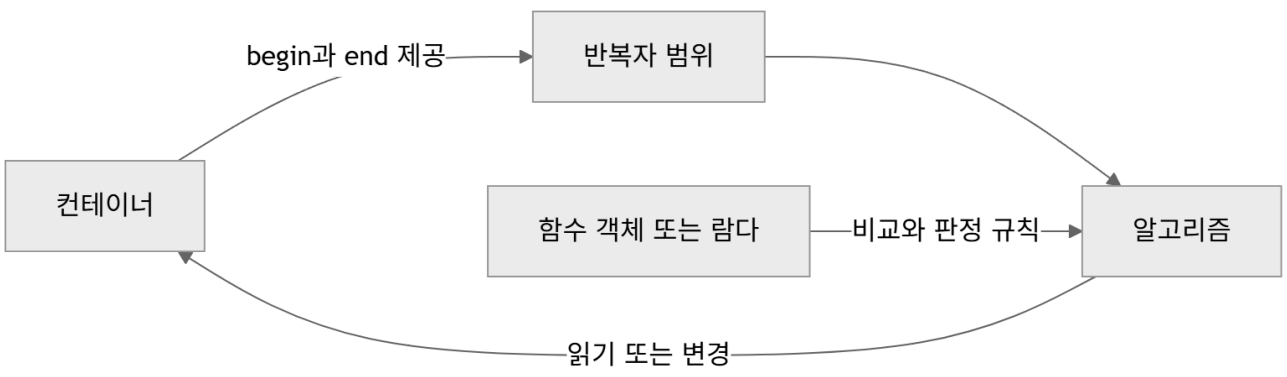
```
std::vector<int> dynamicValues{4, 1, 3, 2};
std::array<int, 4> fixedValues{8, 5, 7, 6};

std::sort(dynamicValues.begin(), dynamicValues.end());
std::sort(fixedValues.begin(), fixedValues.end());
```

반복자 추상화가 모든 성능 차이를 없애는 것은 아니다. 같은 알고리즘을 호출하더라도 컨테이너의 메모리 배치와 반복자 범주에 따라 실행 비용이 달라진다. 인터페이스의 공통성과 저장 구조의 차이를 함께 이해해야 한다.

11.2 컨테이너 공통 기능

표준 컨테이너는 서로 다른 자료 구조를 사용하지만 타입 별칭, 크기 확인, 반복자 접근, 원소 삽입과 제거에서 비슷한 이름을 제공한다. 모든 컨테이너가 완전히 같은 멤버 함수를 제공하는 것은 아니다. 고정 크기 배열에는 원소 추가 함수가 없고, 단방향 리스트에는 뒤쪽 접근과 역방향 반복자가 없다.



11.2.1 원소 타입과 크기

컨테이너는 원소와 반복자에 관한 타입 별칭을 제공한다.

```
using Container = std::vector<int>;
Container::value_type value = 10;
Container::size_type count = 0;
Container::iterator iterator;
Container::const_iterator constIterator;
```

타입 별칭	의미

value_type	원소 타입
size_type	원소 수와 크기를 나타내는 부호 없는 타입
difference_type	반복자 사이 거리를 나타내는 부호 있는 타입
reference	수정 가능한 원소 참조 타입
const_reference	읽기 전용 원소 참조 타입
iterator	수정 가능한 반복자 타입
const_iterator	읽기 전용 반복자 타입

size()는 현재 원소 수를 반환하고 empty()는 원소가 없는지 확인한다.

```
std::vector<int> values{10, 20, 30};
std::cout << values.size() << '\n';
if (!values.empty())
{
    std::cout << values.front() << '\n';
}
```

비어 있는 컨테이너에서 front()나 back()을 호출하면 정의되지 않은 동작이 발생한다. 원소 접근 전에 empty()로 상태를 확인해야 한다.

std::forward_list는 size()를 제공하지 않는다. 단방향 연결 구조에서 원소 수를 즉시 반환하려면 별도의 개수 상태를 유지해야 하고, 표준 forward_list는 작은 노드 구조와 빠른 연결 조작을 우선한다. 원소 수가 필요하면 std::distance()로 순회하여 계산할 수 있지만 선형 시간이 든다.

```
std::forward_list<int> values{10, 20, 30};
auto count = std::distance(values.begin(), values.end());
```

11.2.2 원소 접근

컨테이너가 제공하는 원소 접근 함수는 저장 구조에 따라 달라진다.

함수	의미	주요 지원 컨테이너
front()	첫 원소	모든 순차 컨테이너
back()	마지막 원소	array, vector, deque, list, string
operator[]	범위 검사 없는 인덱스 접근	array, vector, deque, string
at()	범위를 검사하는 인덱스 접근	array, vector, deque, string
data()	연속 저장 영역의 시작 주소	array, vector, string

operator[]는 인덱스 범위를 확인하지 않는다. 잘못된 인덱스를 사용하면 정의되지 않은 동작이 발생한다.

```
std::vector<int> values{10, 20, 30};
int value = values[1];
```

at()은 인덱스가 범위를 벗어나면 std::out_of_range 예외를 던진다.

```
try
{
    int value = values.at(10);
}
catch (const std::out_of_range& error)
{
    std::cout << error.what() << '\n';
}
```

std::list와 std::forward_list는 인덱스 접근을 제공하지 않는다. 특정 위치까지 노드를 차례로 따라가야 하므로 인덱스 연산이 상수 시간이 아니기 때문이다.

11.2.3 삽입과 제거

가변 크기 컨테이너는 위치와 저장 구조에 맞는 삽입과 제거 함수를 제공한다.

```
std::vector<int> values;

values.push_back(10);
values.push_back(20);
values.insert(values.begin() + 1, 15);
values.erase(values.begin());
values.pop_back();
```

push_back()은 끝에 원소를 추가하고 pop_back()은 끝 원소를 제거한다. pop_back()은 제거한 값을 반환하지 않으며 비어 있는 컨테이너에서 호출할 수 없다.

insert()는 삽입 위치를 반복자로 받고 삽입된 원소를 가리키는 반복자를 반환한다. erase()는 제거한 원소 뒤의 위치를 가리키는 반복자를 반환한다. 반환된 반복자를 사용하면 순회 중 안전하게 원소를 제거할 수 있다.

```
for (auto iterator = values.begin(); iterator != values.end();)
{
    if (*iterator < 0)
    {
        iterator = values.erase(iterator);
    }
    else
    {
        ++iterator;
    }
}
```

C++20에서는 조건에 맞는 원소를 제거하는 std::erase_if()도 사용할 수 있다.

```
std::erase_if(
    values,
    [](int value)
    {
        return value < 0;
    });
```

clear()는 모든 원소를 제거한다. std::vector에서 clear()를 호출해도 일반적으로 확보된 용량은 유지된다.

11.2.4 begin과 end

수정 가능한 컨테이너에서 begin()과 end()를 호출하면 보통 iterator를 반환한다. const 컨테이너에서는 const_iterator를 반환한다.

```
std::vector<int> values{10, 20, 30};

for (auto iterator = values.begin(); iterator != values.end();
++iterator)
{
    *iterator *= 2;
}

const std::vector<int>& readOnlyValues = values;

for (auto iterator = readOnlyValues.begin();
     iterator != readOnlyValues.end();
     ++iterator)
{
    std::cout << *iterator << '\n';
}
```

cbegin()과 cend()는 컨테이너가 수정 가능한 객체여도 읽기 전용 반복자를 반환한다.

```
auto iterator = values.cbegin();
```

rbegin()과 rend()는 역방향 순회에 사용한다.

```cpp
for (auto iterator = values.rbegin(); iterator !=
values.rend(); ++iterator)
{
    std::cout << *iterator << ' ';
}
```

std::forward_list는 앞 방향 연결만 가지므로 역방향 반복자를 제공하지 않는다.
범위 기반 for문은 begin()과 end()를 이용해 순회한다.

```cpp
for (int& value : values)
{
    value += 1;
}
```

원소를 복사할 필요가 없으면 참조를 사용한다. 읽기만 수행할 때는 const 참조가 적합하다.

```cpp
for (const std::string& name : names)
{
    std::cout << name << '\n';
}
```

11.2.5 복사와 이동

컨테이너를 복사하면 원소도 복사된다.

```cpp
std::vector<std::string> original{"A", "B", "C"};
std::vector<std::string> copied = original;
```

copied와 original은 서로 다른 저장 공간을 소유한다. 한 컨테이너의 원소를 변경해도 다른 컨테이너에는 영향을 주지 않는다.
컨테이너를 이동하면 내부 저장 자원을 새 객체로 이전할 수 있다.

```cpp
std::vector<std::string> moved = std::move(original);
```

이동 후 original은 유효하지만 원소 상태는 특정 값으로 보장되지 않는다. 비어 있다고 가정하지 말고 clear(), 대입, 소멸처럼 이동 후 객체에 허용된 연산만 사용한다.

원소 타입이 복사 불가능하고 이동만 가능하면 컨테이너도 같은 제약을 받는다.

```cpp
std::vector<std::unique_ptr<int>> values;
values.push_back(std::make_unique<int>(10));

// auto copied = values; // 원소를 복사할 수 없어 오류
```

컨테이너의 이동이 항상 내부 포인터 교환만으로 끝나는 것은 아니다. 할당자 상태와 컨테이너 종류에 따라 원소별 이동이 필요할 수 있다.
일반적인 기본 할당자를 사용하는 컨테이너에서는 이동이 자원 이전으로 처리되는 경우가 많다.

11.2.6 반복자·포인터·참조의 무효화

반복자 무효화(iterator invalidation)는 컨테이너를 변경한 뒤 기존 반복자, 포인터, 참조를 더 이상 사용할 수 없게 되는 상태를 뜻한다.
무효화는 성능 저하가 아니라 프로그램의 유효성과 관련된 규칙이다. 무효화된 반복자, 포인터, 참조가 자동으로 nullptr이 되거나 컴파일 오류로 표시되지는 않는다. 주소나 값이 남아 있는 것처럼 보여도 반복자 연산이나 원소 접근에 다시 사용하면 프로그램의 동작은 보장되지 않는다.

컨테이너가 원소를 다른 저장 공간으로 옮기면 기존 주소를 기반으로 한 반복자, 포인터, 참조가 모두 무효화된다. std::vector의 재할당이 대표적인 경우이다.

```cpp
std::vector<int> values{10, 20, 30};

auto iterator = values.begin();
int* pointer = values.data();
int& reference = values.front();

values.reserve(values.capacity() + 1);
```

```
// reserve()가 저장 공간을 다시 할당했으므로
// iterator, pointer, reference를 더 이상 사용할 수 없다.
```

reserve()에 현재 capacity()보다 큰 값을 전달하면 새 저장 공간이 필요하므로 재할당이 발생한다. 원소의 값은 새 공간으로 이동하거나 복사되지만, 이전 공간을 가리키던 위치 객체는 새 주소를 자동으로 따라가지 않는다.

재할당이 없어도 원소가 이동하면 일부 위치가 무효화될 수 있다. std::vector의 중간 삽입은 삽입 위치부터 끝까지 원소를 뒤로 이동시키고, 제거는 제거 위치 뒤의 원소를 앞으로 이동시킨다. 삽입이나 제거 위치부터 끝까지의 반복자, 포인터, 참조를 다시 얻어야 한다.

```
std::vector<int> values{10, 20, 30, 40};
values.reserve(10);

auto first = values.begin();
auto third = values.begin() + 2;

values.insert(values.begin() + 1, 15);

// first는 삽입 위치보다 앞이므로 유효하다.
// third는 삽입 위치 뒤의 원소를 가리켰으므로 무효화되었다.
```

과거의 end()도 반복자이다. std::vector의 뒤쪽 삽입에서 재할당이 발생하지 않으면 기존 원소를 가리키는 반복자와 참조는 유지되지만, 삽입 전의 end()는 더 이상 끝 위치가 아니므로 무효화된다.

```
std::vector<int> values{10, 20, 30};
values.reserve(10);

auto first = values.begin();
auto oldEnd = values.end();

values.push_back(40);

std::cout << *first << '\n'; // 유효
// oldEnd는 무효화되어 사용할 수 없다.
```

원소 제거 함수가 반환하는 반복자는 제거된 범위 뒤의 유효한 위치를 가리킨다. 순회 중 원소를 제거할 때는 기존 반복자를 증가시키지 말고 반환값을 저장한다.

```
for (auto itr = values.begin(); itr != values.end();)
{
    if (*itr < 0)
    {
        itr = values.erase(itr);
    }
    else
    {
        ++itr;
    }
}
```

컨테이너와 연산	무효화 범위
std::array의 원소 값 변경	반복자, 포인터, 참조 유지
std::vector의 재할당	모든 반복자, 포인터, 참조와 기존 end()
std::vector의 중간 삽입·제거	삽입·제거 위치부터 끝까지
std::deque의 양 끝 삽입	모든 반복자 무효, 기존 원소의 참조와 포인터 유지
std::deque의 중간 삽입	모든 반복자, 포인터, 참조 무효
std::list, std::forward_list의 삽입	기존 반복자, 포인터, 참조 유지
std::list, std::forward_list의 제거	제거된 원소를 가리키는 위치만 무효
std::string의 재할당	모든 반복자, 포인터, 참조와 기존 end()

무효화 여부는 컨테이너 이름만으로 결정되지 않는다. 사용한 연산, 재할당 발생 여부, 삽입·제거 위치를 함께 확인해야 한다. reserve()는 std::vector의 재할당 횟수를 줄이지만 중간 삽입과 제거에 따른 무효화까지 막지는 않는다. 인덱스는 재할당 후에도 숫자 값이 유지되지만, 중간 삽입이나 제거 뒤에는 같은 원소를 나타내지 않을 수 있다.

11.3 std::array

std::array<T, N>은 원소 타입과 크기를 타입에 포함하는 고정 크기 순차 컨테이너이다. 내부 원소는 C 배열처럼 연속된 메모리에 저장되며 반복자, 대입, 비교, 크기 확인과 같은 컨테이너 인터페이스를 제공한다.

```
#include <array>
std::array<int, 4> values{10, 20, 30, 40};
```

11.3.1 C 배열과 std::array

C 배열은 함수 인수로 전달할 때 포인터로 변환되어 배열 크기 정보가 사라질 수 있다.

```
void PrintValues(const int values[], std::size_t size)
{
    for (std::size_t index = 0; index < size; ++index)
    {
        std::cout << values[index] << ' ';
    }
}
```

std::array는 하나의 객체이므로 크기를 스스로 알고 있고 대입과 반환이 가능하다.

```
void PrintValues(const std::array<int, 4>& values)
{
    for (int value : values)
    {
        std::cout << value << ' ';
    }
}

std::array<int, 4> first{1, 2, 3, 4};
std::array<int, 4> second = first;
```

C 배열은 배열 전체를 대입할 수 없지만 std::array는 같은 타입끼리 원소 전체를 복사한다.

11.3.2 고정 크기

std::array의 원소 수 N은 컴파일 시점에 정해지고 객체 수명 동안 바뀌지 않는다.

```
std::array<int, 3> first{};
std::array<int, 5> second{};
```

std::array<int, 3>과 std::array<int, 5>는 서로 다른 타입이다. 원소 수가 실행 중 결정되거나 계속 변해야 한다면 std::vector가 적합하다.
기본형 원소를 모두 0으로 초기화하려면 빈 중괄호를 사용한다.

```
std::array<int, 5> values{};

std::array<int, 5> values;
```

두 번째 선언은 기본 초기화를 수행하므로 int 원소의 값이 결정되지 않는다. 읽기 전에 값을 대입해야 한다.
std::array<T, 0>도 유효한 타입이다. size()는 0이고 begin() == end()가 성립한다. 원소가 없으므로 front(), back(), operator[]를 사용할 수 없다.

11.3.3 반복자 지원

std::array는 연속 반복자를 제공하므로 표준 알고리즘을 적용할 수 있다.

```
#include <algorithm>
#include <array>

std::array<int, 5> values{30, 10, 50, 20, 40};
```

```
std::sort(values.begin(), values.end());
auto position = std::find(values.begin(), values.end(), 30);
```

data()는 첫 원소의 주소를 반환한다.

```
int* data = values.data();
```

C 인터페이스에 배열 주소와 크기를 전달해야 할 때 사용할 수 있다.

```
ProcessData(values.data(), values.size());
```

11.3.4 배열 대입과 비교

같은 원소 타입과 크기를 가진 std::array는 대입할 수 있다.

```
std::array<int, 3> source{10, 20, 30};
std::array<int, 3> destination{};

destination = source;
```

비교 연산은 원소를 앞에서부터 사전식으로 비교한다.

```
std::array<int, 3> left{1, 2, 3};
std::array<int, 3> right{1, 2, 4};

bool equal = left == right;
bool less = left < right;
```

첫 번째로 다른 원소 3과 4의 비교 결과에 따라 left < right가 참이 된다. C++20에서는 비교 연산이 원소 타입의 비교 연산을 이용해 제공된다.
fill()은 모든 원소에 같은 값을 대입한다.

```
std::array<int, 8> cells{};
cells.fill(-1);
```

11.3.5 함수 인수로 전달하기

크기가 고정된 하나의 타입만 받는 함수는 구체적인 std::array 참조를 사용할 수 있다.

```
int Sum(const std::array<int, 4>& values)
{
    int total = 0;

    for (int value : values)
    {
        total += value;
    }

    return total;
}
```

여러 크기를 허용하려면 크기를 템플릿 매개변수로 받는다.

```
#include <array>
#include <cstddef>

template <std::size_t N>
int Sum(const std::array<int, N>& values)
{
    int total = 0;

    for (int value : values)
    {
        total += value;
```

```
    }

    return total;
}

std::array<int, 3> first{1, 2, 3};
std::array<int, 5> second{1, 2, 3, 4, 5};

int firstTotal = Sum(first);
int secondTotal = Sum(second);
```

함수가 배열을 소유하지 않고 연속된 원소 범위만 읽는다면 std::span으로 std::array, std::vector, C 배열을 공통으로 받을 수도 있다.

```
#include <span>

int Sum(std::span<const int> values)
{
    int total = 0;

    for (int value : values)
    {
        total += value;
    }

    return total;
}
```

std::span은 원소를 소유하지 않으므로 전달받은 저장 공간보다 오래 살아서는 안 된다.

11.4 std::vector

std::vector<T>는 원소를 연속된 메모리에 저장하는 가변 크기 순차 컨테이너이다. 인덱스 접근이 빠르고 반복 순회에서 캐시 지역성이 좋다.
특별한 요구가 없다면 가변 크기 순차 컨테이너의 기본 선택으로 적합하다.

11.4.1 연속 메모리

std::vector의 원소는 메모리에 연속해서 배치된다.

```
std::vector<int> values{10, 20, 30};

int* first = &values[0];
int* data = values.data();
```

원소가 존재하면 first와 data는 같은 주소를 나타낸다. data()[index]와 values[index]는 같은 원소에 접근한다.
연속 메모리는 인덱스 접근과 순차 처리에 유리하다. CPU는 가까운 주소의 데이터를 함께 캐시에 가져오는 경향이 있으므로 원소를 차례로 읽는 작업에서 좋은 성능을 얻기 쉽다.

```
long long Sum(const std::vector<int>& values)
{
    long long total = 0;

    for (int value : values)
    {
        total += value;
    }

    return total;
}
```

원소가 연속되어 있다는 보장은 C 함수나 외부 라이브러리에 버퍼를 전달할 때도 유용하다.
WriteValues(values.data(), values.size());

벡터 크기가 0이면 data()의 반환값을 역참조할 수 없다.

11.4.2 원소 추가와 제거

끝에 원소를 추가할 때는 push_back()이나 emplace_back()을 사용한다.

```
std::vector<std::string> names;

names.push_back("Alice");
names.push_back("Bob");
```

끝 원소는 pop_back()으로 제거한다.

```
if (!names.empty())
{
    names.pop_back();
}
```

중간 위치에는 insert()로 추가하고 erase()로 제거한다.

```
std::vector<int> values{10, 30};

values.insert(values.begin() + 1, 20);
values.erase(values.begin());
```

벡터 중간에 삽입하면 삽입 위치 이후의 원소를 뒤로 이동해야 한다. 제거할 때도 뒤쪽 원소를 앞으로 이동한다. 앞이나 중간에서 삽입과 제거가 자주 발생하면 이동 비용이 커질 수 있다.

여러 원소를 한 번에 삽입할 수도 있다.

```
std::vector<int> values{10, 40};
std::array<int, 2> middle{20, 30};

values.insert(
    values.begin() + 1,
    middle.begin(),
    middle.end());
```

11.4.3 size와 capacity

size()는 현재 생성되어 있는 원소 수이고 capacity()는 재할당 없이 저장할 수 있는 원소 수이다.

```
std::vector<int> values;

std::cout << values.size() << '\n';
std::cout << values.capacity() << '\n';
```

원소를 추가하여 size()가 capacity()를 초과해야 하면 벡터는 더 큰 저장 공간을 확보한다. 기존 원소를 새 공간으로 이동하거나 복사한 뒤 이전 공간을 해제한다.

```
size = 3, capacity = 5

[10] [20] [30] [빈 공간] [빈 공간]
```

capacity()가 증가하는 비율은 표준에서 고정하지 않는다. 구현은 여러 번의 작은 재할당을 줄이기 위해 보통 현재 용량보다 크게 확장한다. 특정 증가 배율을 전제로 코드를 작성해서는 안 된다.
capacity()는 원소 수가 아니므로 values[capacity() - 1]처럼 접근할 수 없다. 생성된 원소의 유효 인덱스는 0부터 size() - 1까지이다.

11.4.4 재할당

재할당(reallocation)이 발생하면 모든 원소가 새 주소로 이동한다. 기존 반복자, 포인터, 참조는 모두 무효화된다.

```
std::vector<int> values{10, 20, 30};
int& first = values.front();

values.push_back(40);
```

```
// 재할당이 발생했다면 first를 사용할 수 없음
```

원소 타입이 예외 없이 이동 생성 가능하면 벡터는 재할당 과정에서 이동을 사용할 수 있다. 이동이 예외를 던질 수 있고 복사가 가능하면 강한 예외 보장을 유지하기 위해 복사를 선택할 수 있다.

```
class Data
{
public:
    Data(const Data&) = default;
    Data(Data&&) noexcept = default;
};
```

이동 생성자에 noexcept를 지정하는 것은 컨테이너가 이동을 선택하는 데 영향을 줄 수 있다.
재할당은 모든 push_back()에서 발생하지 않는다. size() < capacity()이면 기존 저장 공간의 빈 위치에 원소를 생성한다.

11.4.5 reserve와 resize

reserve()는 저장 용량을 확보하지만 원소 수를 늘리지 않는다.

```
std::vector<int> values;
values.reserve(100);

std::cout << values.size() << '\n';     // 0
std::cout << values.capacity() << '\n'; // 100 이상
```

추가할 원소 수를 미리 알 수 있다면 reserve()로 재할당 횟수를 줄일 수 있다.

```
std::vector<int> values;
values.reserve(sourceCount);

for (int value : source)
{
    values.push_back(value);
}
```

resize()는 실제 원소 수를 변경한다.

```
std::vector<int> values{10, 20};
values.resize(5);
```

새로 추가된 int 원소는 값 초기화되어 0이 된다.

```
10 20 0 0 0
```

초기값을 지정할 수도 있다.

```
values.resize(8, -1);
```

현재 크기보다 작게 줄이면 뒤쪽 원소가 소멸한다.

```
values.resize(2);
```

resize()로 크기를 줄여도 용량은 보통 줄지 않는다. shrink_to_fit()은 용량 축소를 요청하지만 구현이 반드시 수행해야 하는 명령은 아니다.
reserve()와 resize()의 차이를 혼동하면 생성되지 않은 원소에 접근하거나 불필요한 기본 생성을 수행할 수 있다.

함수	size() 변경	capacity() 확보	새 원소 생성
reserve(n)	아니요	필요하면 증가	아니요
resize(n)	예	필요하면 증가	크기가 늘면 생성

11.4.6 반복자·포인터·참조의 무효화

std::vector의 무효화 범위는 재할당 발생 여부와 변경 위치로 판단한다. 재할당이 발생하면 저장 주소가 바뀌므로 모든 반복자, 포인터, 참조와 기존 end()가 무효화된다. 재할당이 없으면 원소가 이동한 범위만 영향을 받는다.

연산	조건	무효화 범위
reserve(n)	n > capacity()	전체
reserve(n)	n <= capacity()	없음
push_back(), emplace_back()	재할당 발생	전체
push_back(), emplace_back()	재할당 없음	기존 end()
insert(), emplace()	재할당 발생	전체
insert(), emplace()	재할당 없음	삽입 위치부터 끝까지
erase()	항상	제거 위치부터 끝까지
pop_back()	항상	제거된 마지막 원소와 기존 end()
clear()	항상	모든 원소 위치와 기존 end()
resize()로 확대	재할당 발생	전체
resize()로 확대	재할당 없음	기존 end()
resize()로 축소	항상	제거된 원소와 기존 end()
shrink_to_fit()	재할당 발생	전체
shrink_to_fit()	재할당 없음	없음

중간 삽입과 제거에서는 재할당 여부만 확인해서는 안 된다. 재할당이 없더라도 원소 이동이 발생하므로 변경 위치 이후의 반복자, 포인터, 참조가 무효화된다.

벡터를 변경한 뒤에도 같은 논리적 위치가 필요하다면 인덱스를 저장하고 연산 후 다시 접근할 수 있다.

```
std::size_t index = 1;
values.push_back(40);

int& value = values[index];
```

인덱스는 재할당의 영향을 받지 않지만 중간 삽입이나 제거 뒤에는 다른 원소를 가리킬 수 있다. 객체의 주소 유지와 순서상의 위치 유지는 서로 다른 조건이다.

11.4.7 push_back과 emplace_back

push_back()은 원소 타입의 객체 하나를 받아 벡터의 끝에 추가한다. 중괄호를 사용하면 원소 타입을 직접 쓰지 않고도 객체를 초기화할 수 있다.
사용자 정의 생성자가 없는 단순 구조체는 집합체 초기화를 사용한다.

```
struct Item
{
    int id;
    std::string name;
    int count;
};

std::vector<Item> items;
items.push_back({1, "Potion", 10});
```

{1, "Potion", 10}은 push_back()에 세 인수를 전달하는 표현이 아니다. 중괄호로 Item 객체 하나를 만들며, id, name, count가 멤버 선언 순서대로 초기화된다. Item에 작성된 생성자 로직 없이 각 멤버를 직접 초기화한다.

생성자를 추가한 버전으로 Item 정의를 바꾸면 같은 중괄호 표현이 생성자를 호출한다.

```cpp
struct Item
{
    int id;
    std::string name;
    int count;

    Item(int itemId, std::string itemName, int itemCount)
        : id(itemId),
          name(std::move(itemName)),
          count(itemCount < 0 ? 0 : itemCount)
    {
    }
};

std::vector<Item> items;
items.push_back({1, "Potion", -10});
```

중괄호의 값에 맞는 Item 생성자가 선택되며, 생성자에서 음수 수량을 0으로 보정한다. 집합체 초기화는 멤버를 직접 초기화하고, 생성자를 가진 구조체의 리스트 초기화는 생성자에 정의된 규칙을 적용한다.

원소 타입을 명시하면 생성되는 객체가 더 분명하게 드러난다.

```cpp
items.push_back(Item{2, "Ether", 5});
```

push_back({ ... })과 push_back(Item{ ... })은 임시 Item 객체를 만든 뒤 벡터로 이동하거나 복사한다. emplace_back()은 생성자 인수를 받아 벡터의 저장 위치에 원소를 직접 생성한다.

```cpp
items.emplace_back(3, "Elixir", 1);
```

이미 같은 타입의 객체가 있다면 push_back()이 의도를 분명하게 표현한다.

```cpp
Item item{4, "Antidote", 2};
items.push_back(std::move(item));
```

emplace_back()이 항상 더 빠른 것은 아니다. 컴파일러 최적화와 이동 생성으로 차이가 작을 수 있으며, 전달된 인수가 어떤 생성자를 선택하는지 확인해야 한다.

생성자를 explicit로 선언한 별도 예에서는 push_back({ ... })으로 암시적인 리스트 초기화를 할 수 없다. 원소 타입을 명시하거나 emplace_back()을 사용한다.

```cpp
struct Item
{
    int id;
    std::string name;
    int count;

    explicit Item(int itemId, std::string itemName, int
itemCount)
        : id(itemId),
          name(std::move(itemName)),
          count(itemCount)
    {
    }
};

std::vector<Item> items;

// items.push_back({1, "Potion", 10}); // 오류
items.push_back(Item{1, "Potion", 10});
items.emplace_back(2, "Ether", 5);
```

C++17 이후 emplace_back()은 생성된 원소의 참조를 반환한다.

```cpp
Item& item = items.emplace_back(3, "Elixir", 1);
```

반환된 참조도 이후 재할당이 발생하면 무효화될 수 있다.

11.5 std::deque

std::deque<T>는 앞과 뒤에서 원소를 효율적으로 추가하고 제거할 수 있는 가변 크기 순차 컨테이너이다. 이름은 double-ended queue에서 왔다. 인덱스를 이용한 임의 접근도 지원한다.

11.5.1 양쪽 끝 삽입과 제거

std::deque는 push_front()와 push_back()을 모두 제공한다.

```
#include <deque>

std::deque<int> values;

values.push_back(20);
values.push_front(10);
values.push_back(30);

// 10 20 30
```

양쪽 끝 원소는 pop_front()와 pop_back()으로 제거한다.

```
if (!values.empty())
{
    values.pop_front();
}
```

작업 대기열, 최근 기록, 양방향 확장이 필요한 버퍼에서 사용할 수 있다. 한쪽 끝에 추가하고 반대쪽 끝에서 제거하는 단순 큐 인터페이스만 필요하면 std::queue 어댑터를 사용할 수 있다.

11.5.2 메모리 구조

std::deque는 전체 원소를 하나의 연속된 메모리 블록에 저장하지 않는다. 여러 고정 크기 블록을 관리하고 블록 위치를 가리키는 구조를 사용한다. 구체적인 블록 크기와 내부 배치는 구현에 따라 다르다.

```
블록 0        블록 1        블록 2
[10][20]      [30][40]      [50][60]
```

원소가 여러 블록에 나뉘어 있으므로 std::vector처럼 전체 원소를 하나의 T*와 크기로 표현할 수 없다. std::deque는 data()를 제공하지 않는다. 앞에 원소를 추가할 때 모든 기존 원소를 뒤로 이동할 필요가 없다. 필요한 블록을 앞쪽에 연결하고 새 원소를 생성할 수 있다. 뒤쪽에도 필요한 블록을 연결하여 저장 공간을 확장한다.

11.5.3 임의 접근

메모리가 하나의 블록으로 연속되어 있지 않아도 operator[]와 at()를 이용한 임의 접근을 지원한다.

```
std::deque<int> values{10, 20, 30};

std::cout << values[1] << '\n';
std::cout << values.at(2) << '\n';
```

반복자는 임의 접근 반복자이므로 std::sort()도 사용할 수 있다.

```
std::sort(values.begin(), values.end());
```

인덱스 접근은 상수 시간이지만 벡터보다 내부 주소 계산 단계가 더 필요할 수 있다. 순차 순회의 캐시 지역성도 벡터보다 낮을 수 있다.

11.5.4 vector와의 차이와 무효화

std::vector와 std::deque는 모두 가변 크기와 임의 접근을 지원하지만 저장 구조와 끝 연산 특성이 다르다.

기준	std::vector	std::deque

전체 메모리 연속성	보장	보장하지 않음
뒤쪽 추가	효율적	효율적
앞쪽 추가	원소 이동 필요	효율적
data()	제공	제공하지 않음
캐시 지역성	대체로 좋음	벡터보다 낮을 수 있음
기존 원소의 참조·포인터	재할당 시 무효	양 끝 삽입에서 유지
반복자 안정성	재할당과 원소 이동에 영향	양 끝 삽입에서도 모든 반복자 무효

std::deque의 양 끝 삽입은 기존 원소에 대한 참조와 포인터를 유지하지만 모든 반복자를 무효화한다. 중간 삽입은 모든 반복자, 포인터, 참조를 무효화한다. 제거는 위치에 따라 무효화 범위가 달라지므로 제거 연산의 규칙을 확인해야 한다.

연속 메모리와 외부 API 연동이 필요하거나 뒤쪽에서만 확장한다면 std::vector가 적합하다. 앞과 뒤 양쪽에서 삽입과 제거가 빈번하고 전체 연속 저장이 필요하지 않다면 std::deque를 검토한다.

11.6 std::list와 std::forward_list

std::list와 std::forward_list는 원소를 독립된 노드에 저장하고 포인터로 연결하는 순차 컨테이너이다. 원소가 메모리에 연속해서 배치되지 않으며 인덱스 접근을 제공하지 않는다.

11.6.1 연결 리스트

연결 리스트의 각 노드는 원소와 다른 노드를 가리키는 연결 정보를 가진다.

```
[값 | 연결] -> [값 | 연결] -> [값 | 연결]
```

노드 위치를 알고 있으면 주변 원소를 이동하지 않고 연결만 바꾸어 삽입과 제거를 수행할 수 있다. 반면 특정 인덱스의 원소에 접근하려면 처음부터 노드를 따라가야 한다.

```
std::list<int> values{10, 20, 30};

auto iterator = values.begin();
std::advance(iterator, 2);
std::cout << *iterator << '\n';
```

std::advance()는 리스트 반복자를 두 번 증가시킨다. 이동 거리에 비례하는 시간이 든다.
각 원소가 별도 노드에 저장되므로 값 외에도 연결 포인터와 메모리 할당 비용이 필요하다. 작은 기본형을 많이 저장할 때는 부가 비용이 커질 수 있다.

11.6.2 양방향 리스트와 단방향 리스트

std::list는 이전 노드와 이후 노드를 모두 연결하는 양방향 리스트이다.

```
null <- [10] <-> [20] <-> [30] -> null
```

양방향 반복자를 제공하므로 앞뒤로 이동할 수 있다.

```
std::list<int> values{10, 20, 30};
auto iterator = values.end();

--iterator;
std::cout << *iterator << '\n';
```

std::forward_list는 이후 노드만 연결하는 단방향 리스트이다.

```
[10] -> [20] -> [30] -> null

std::forward_list<int> values{10, 20, 30};
```

단방향 구조는 노드당 연결 정보가 적지만 뒤로 이동할 수 없다. back(), push_back(), size(), 역방향 반복자를 제공하지 않는다.

11.6.3 삽입과 제거

std::list는 삽입할 위치를 가리키는 반복자를 사용한다.

```
std::list<int> values{10, 30};
auto position = std::next(values.begin());

values.insert(position, 20);
```

std::forward_list는 현재 노드의 앞 노드를 바로 찾을 수 없으므로 위치 뒤에 삽입하고 제거하는 함수를 제공한다.

```
std::forward_list<int> values{20, 30};

values.push_front(10);
values.insert_after(values.begin(), 15);
```

첫 원소 앞의 가상 위치는 before_begin()으로 얻는다.

```
values.insert_after(values.before_begin(), 5);
```

erase_after()는 지정한 반복자 뒤의 원소를 제거한다.

```
auto before = values.before_begin();
values.erase_after(before);
```

리스트는 노드 연결을 다른 리스트로 옮기는 splice() 계열 함수를 제공한다. 원소를 복사하거나 이동하지 않고 노드 소유 관계를 변경할 수 있다.

```
std::list<int> first{10, 20};
std::list<int> second{30, 40};

first.splice(first.end(), second);
```

작업 후 first는 네 원소를 가지고 second는 비게 된다. 기존 노드에 대한 반복자와 참조는 옮겨진 노드를 계속 가리킨다.

11.6.4 순회 비용

연결 리스트는 각 노드가 메모리의 서로 다른 위치에 있을 수 있다. 순회할 때 포인터를 따라가며 여러 메모리 위치를 읽으므로 캐시 지역성이 낮을 수 있다.

벡터의 중간 삽입은 원소 이동이 필요하지만, 실제 프로그램에서는 벡터 순회 속도와 낮은 메모리 부가 비용이 더 유리한 경우가 많다. 연결 리스트가 이론적으로 상수 시간 삽입을 제공해도 삽입 위치를 찾는 시간이 선형이고 노드 할당 비용도 발생한다.

```
std::list<int> values{4, 1, 3, 2};
values.sort();
```

std::sort()는 임의 접근 반복자를 요구하므로 리스트에는 사용할 수 없다. list::sort()는 노드 연결을 이용해 정렬한다.

```
values.remove_if(
    [](int value)
    {
        return value < 0;
    });
```

리스트의 멤버 함수 remove_if(), unique(), merge(), sort()는 노드 구조를 활용한다.

11.6.5 연결 리스트 선택 기준

연결 리스트는 중간 삽입이 있다는 이유만으로 선택하지 않는다. 삽입 위치를 이미 반복자로 알고 있는지, 기존 원소의 주소가 유지되어야 하는지, 노드 이동이 필요한지 확인해야 한다.

std::list는 삽입 위치와 원소 위치의 안정성이 중요한 조건에서 적합할 수 있다.

- 삽입과 제거 위치를 반복자로 이미 보유한다.
- 삽입과 제거 후에도 다른 원소의 반복자와 참조가 유지되어야 한다.
- 여러 리스트 사이에서 원소를 복사하지 않고 옮겨야 한다.
- 임의 접근과 연속 메모리가 필요하지 않다.

std::forward_list는 앞 방향 순회만 필요하고 노드당 연결 정보를 줄이는 것이 중요할 때 검토한다. 일반 응용 프로그램에서는 기능 제약과 원소 수 계산 비용 때문에 사용 빈도가 낮다.

연결 리스트의 반복자와 참조는 삽입으로 무효화되지 않는다. 제거된 원소를 가리키는 반복자와 참조만 무효화된다. 컨테이너 자체가 소멸하거나 전체 대입이 수행되는 경우는 별도로 판단해야 한다.

11.7 std::string

std::string은 char 원소로 이루어진 가변 길이 문자열을 관리한다. 문자 데이터는 연속된 메모리에 저장되고 끝에는 C 문자열과 호환되는 널 문자가 관리된다. std::string은 표준의 순차 컨테이너 범주에 직접 포함되지는 않지만 반복자, size(), capacity(), insert(), erase()처럼 컨테이너와 유사한 인터페이스를 제공한다.

11.7.1 문자열 생성과 대입

문자열 리터럴, 반복 문자, 다른 문자열로 객체를 생성할 수 있다.

```
#include <string>

std::string first = "Hello";
std::string second(5, '*');
std::string third = first;

// 출력:
// first  : Hello
// second : *****
// third  : Hello
```

대입하면 기존 내용이 새 문자열로 바뀐다.

```
std::string name;
name = "Alice";
```

size()와 length()는 같은 값을 반환한다.

```
std::cout << name.size() << '\n';
std::cout << name.length() << '\n';
```

문자열은 널 문자를 내용 길이에 포함하지 않는다. 문자열 안에 \0 문자를 원소로 저장할 수도 있으므로 size()를 기준으로 처리하는 것이 안전하다.

11.7.2 연결과 비교

operator+=와 append()로 문자열 뒤에 내용을 추가할 수 있다.

```
std::string message = "Hello";
message += ", ";
message += "C++";
```

operator+는 새 문자열을 만든다.

```
std::string firstName = "Ada";
std::string lastName = "Lovelace";
std::string fullName = firstName + " " + lastName;
```

짧은 문자열 연결이 반복되는 루프에서는 여러 임시 객체와 재할당이 생길 수 있다. 최종 길이를 예상할 수 있으면 reserve()로 용량을 확보하고 +=를 사용할 수 있다.

```
std::string result;
```

```
result.reserve(128);

for (const std::string& word : words)
{
    result += word;
    result += ' ';
}
```

비교 연산은 문자 코드를 기준으로 사전식 비교를 수행한다.

```
std::string left = "apple";
std::string right = "banana";

bool less = left < right;
bool equal = left == right;
```

대소문자를 무시하는 비교나 자연어 정렬은 별도의 규칙이 필요하다. 기본 비교는 언어별 사전 규칙을 적용하지 않는다.

11.7.3 검색과 부분 문자열

find()는 문자열이나 문자를 처음 찾은 위치를 반환한다.

```
std::string path = "assets/player.png";
std::size_t position = path.find('/');
```

찾지 못하면 std::string::npos를 반환한다.

```
if (position != std::string::npos)
{
    std::cout << position << '\n';
}
```

검색 시작 위치를 지정할 수 있다.

```
std::size_t secondPosition = path.find('/', position + 1);
```

rfind()는 뒤쪽부터 검색한다.

```
std::size_t extensionPosition = path.rfind('.');
```

substr()은 지정한 위치부터 부분 문자열을 새 객체로 복사한다.

```
std::string extension = path.substr(extensionPosition + 1);
```

두 번째 인수를 생략하면 끝까지 복사한다. 시작 위치가 문자열 범위를 벗어나면 std::out_of_range 예외가 발생한다.
C++20에서는 접두사와 접미사를 확인하는 starts_with()와 ends_with()를 사용할 수 있다.

```
bool isAsset = path.starts_with("assets/");
bool isPng = path.ends_with(".png");
```

11.7.4 삽입과 제거

문자열 중간에 내용을 삽입할 수 있다.

```
std::string text = "Hello World";
text.insert(5, ",");

// 출력:
// Hello, World
```

erase()는 시작 위치와 문자 수를 받아 내용을 제거한다.

```
text.erase(5, 1);
```

replace()는 범위를 다른 문자열로 교체한다.

```
std::string text = "red potion";
text.replace(0, 3, "blue");

blue potion
```

한 문자는 push_back()과 pop_back()으로 추가하거나 제거할 수 있다.

```
std::string text = "ABC";
text.push_back('D');
text.pop_back();
```

문자열 중간을 변경하면 뒤쪽 문자 이동이 발생한다. 저장 용량이 부족하면 재할당도 발생할 수 있다.

11.7.5 숫자와 문자열 변환

std::to_string()은 기본 숫자 타입을 문자열로 변환한다.

```
int score = 1200;
std::string text = std::to_string(score);
```

문자열을 숫자로 변환할 때는 std::stoi(), std::stol(), std::stof(), std::stod() 등을 사용할 수 있다.

```
std::string text = "1200";
int score = std::stoi(text);
```

변환할 수 없는 문자열은 std::invalid_argument, 표현 범위를 벗어난 값은 std::out_of_range 예외를 발생시킨다.

```
try
{
    int value = std::stoi("12A");
}
catch (const std::invalid_argument&)
{
    std::cout << "invalid number\n";
}
```

std::stoi("12A")는 앞부분의 12를 변환하고 남은 문자를 허용한다. 문자열 전체가 숫자인지 확인하려면 변환이 끝난 위치를 받는다.

```
std::size_t parsed = 0;
int value = std::stoi("12A", &parsed);

if (parsed != 3)
{
    std::cout << "remaining characters\n";
}
```

예외 없이 낮은 수준에서 변환 결과를 확인하려면 <charconv>의 std::from_chars()와 std::to_chars()를 사용할 수 있다.

```
#include <charconv>
#include <string_view>

std::string_view text = "1200";
int value = 0;

auto result = std::from_chars(
    text.data(),
    text.data() + text.size(),
    value);

if (result.ec == std::errc{} &&
```

```
    result.ptr == text.data() + text.size())
{
    // 전체 문자열 변환 성공
}
```

std::from_chars()는 지역화 설정에 의존하지 않고 동적 메모리를 할당하지 않는다. 사용법은 조금 더 복잡하지만 반복적인 파싱이나 성능이 중요한 코드에 적합하다.

11.7.6 문자열의 저장 공간

std::string은 std::vector와 비슷하게 size()와 capacity()를 가진다.

```
std::string text = "Hello";

std::cout << text.size() << '\n';
std::cout << text.capacity() << '\n';
```

reserve()는 재할당 없이 저장할 수 있는 문자 수를 확보한다.

```
std::string text;
text.reserve(256);
```

data()와 c_str()은 연속된 문자 배열의 주소를 반환한다.

```
const char* cText = text.c_str();
```

C++17 이후 수정 가능한 std::string의 비상수 data()는 char*를 반환한다. 문자 내용을 바꿀 수 있지만 문자열 크기를 넘어서 쓰거나 널 종료 규칙을 깨뜨려서는 안 된다.

```
std::string text = "abc";
text.data()[0] = 'A';
```

문자열을 변경하여 재할당이 발생하면 기존 data() 포인터, 반복자, 참조가 무효화된다. 외부 함수에 c_str() 포인터를 전달한 뒤 문자열을 수정하면 저장해 둔 포인터를 다시 얻어야 한다.
많은 구현은 짧은 문자열을 객체 내부에 직접 저장하는 짧은 문자열 최적화(SSO)를 사용한다. SSO 적용 여부와 저장 가능한 문자 수는 표준이 보장하지 않는다. 특정 길이까지 동적 할당이 없다고 가정해서는 안 된다.

11.8 순차 컨테이너 선택

컨테이너 선택은 단일 연산의 이론적 복잡도만으로 결정하지 않는다. 원소 수, 접근 패턴, 메모리 배치, 반복자 안정성, 외부 API와의 연결, 구현의 단순성을 함께 고려한다.

11.8.1 고정 크기와 가변 크기

크기가 컴파일 시점에 정해지고 바뀌지 않으면 std::array가 적합하다.

```
std::array<float, 3> position{};
```

원소 수가 실행 중 결정되거나 증가하고 감소하면 std::vector나 다른 가변 크기 컨테이너를 사용한다.

```
std::vector<int> scores;
```

최대 크기만 정해져 있다고 해서 항상 std::array를 사용할 필요는 없다. 실제 원소 수와 최대 저장 공간을 별도로 관리해야 하면 벡터에 reserve()를 적용하는 편이 간단할 수 있다.

11.8.2 연속 메모리와 캐시 지역성

연속 메모리가 필요하면 std::array, std::vector, std::string을 선택한다. 그래픽, 오디오, 네트워크, 파일 API에 원소 주소와 개수를 전달할 때 연속성이 중요하다.
순차 처리에서도 연속 저장은 캐시 지역성에 유리하다. 연결 리스트는 원소 이동이 없다는 장점이 있지만 노드 포인터 추적과 개별 할당 비용이 발생한다. 실제 성능은 자료 크기와 접근 패턴으로 측정해야 한다.

11.8.3 삽입과 제거 위치

끝에서만 추가하고 제거하면 std::vector가 적합하다. 앞과 뒤 양쪽에서 작업하면 std::deque를 검토한다.
중간 삽입과 제거가 많더라도 삽입 위치를 찾는 비용과 순회 성능을 함께 판단해야 한다. 위치 반복자를 이미 가지고 있고 기존 원소의 참조 안정성이 중요하면 std::list가 유리할 수 있다.

원소 순서를 유지할 필요가 없다면 벡터에서 제거할 원소를 마지막 원소와 교환한 뒤 pop_back()하여 중간 이동을 피할 수도 있다.

```
void UnorderedErase(std::vector<int>& values, std::size_t
index)
{
    if (index >= values.size())
    {
        return;
    }

    values[index] = std::move(values.back());
    values.pop_back();
}
```

UnorderedErase()는 원소 순서를 바꾸므로 순서가 의미 있는 데이터에는 사용할 수 없다.

11.8.4 임의 접근

인덱스로 자주 접근해야 하면 std::array, std::vector, std::deque가 적합하다. 세 컨테이너는 임의 접근 반복자를 제공한다.
std::list와 std::forward_list에서 n번째 원소에 접근하는 비용은 n에 비례한다. 인덱스 접근이 필요한 구조를 연결 리스트로 구현하면 코드가 단순해지지 않고 성능도 나빠질 수 있다.

정렬 알고리즘 선택도 반복자 범주에 영향을 받는다. std::sort()는 벡터, 덱, 배열에 사용할 수 있고 리스트는 자체 sort()를 사용한다.

11.8.5 컨테이너 선택 기준

<순차 컨테이너 선택 기준>

요구 사항	우선 검토할 타입
컴파일 시점에 고정된 작은 크기	std::array
가변 크기, 빠른 순회, 임의 접근	std::vector
앞과 뒤의 빈번한 삽입과 제거	std::deque
위치를 알고 있는 중간 삽입과 참조 안정성	std::list
앞 방향 순회만 필요한 최소 연결 구조	std::forward_list
가변 길이 문자 시퀀스	std::string

가변 크기 순차 데이터에는 먼저 std::vector를 검토한다. 다른 컨테이너는 벡터가 충족하지 못하는 구체적인 요구가 있을 때 선택한다.
컨테이너 선택에서는 원소 수, 메모리 배치, 접근 방식, 위치 안정성을 확인한다.

- 원소 수가 고정인지 가변인지 확인한다.
- 연속된 메모리가 필요한지 확인한다.
- 인덱스 접근 빈도를 확인한다.
- 삽입과 제거가 발생하는 위치를 확인한다.
- 반복자, 포인터, 참조의 안정성이 필요한지 확인한다.
- 원소당 메모리 부가 비용을 확인한다.
- 컨테이너를 외부 API에 전달해야 하는지 확인한다.
- 예상이 아닌 실제 데이터로 성능을 측정한다.

실제 설계에서는 모든 항목을 같은 비중으로 적용하지 않고 프로그램의 핵심 요구에 맞춰 우선순위를 정한다.

11.9 학습 정리

C++ 표준 라이브러리는 자료 구조와 알고리즘을 템플릿과 반복자로 연결한다. 컨테이너는 원소 저장과 수명을 관리하고, 반복자는 원소 위치와 범위를 표현하며, 알고리즘은 반복자 범위에 검색, 정렬, 변환 작업을 적용한다. 함수 객체와 람다 표현식은 비교와 판정 규칙을 알고리즘에 전달한다.

STL은 컨테이너, 반복자, 알고리즘, 함수 객체를 중심으로 발전한 설계와 구성 요소를 가리킨다. 표준 라이브러리는 STL 계열 기능뿐 아니라 문자열, 입출력, 메모리, 시간, 파일 시스템, 동시성 기능까지 포함한다.

표준 컨테이너는 공통된 이름을 많이 제공하지만 저장 구조에 따라 지원 기능과 비용이 다르다. size(), empty(), begin(), end()는 여러 컨테이너에서 사용할 수 있다. 인덱스 접근, 뒤쪽 접근, 역방향 반복, data()는 모든 순차 컨테이너에 공통으로 제공되지 않는다.

반복자 무효화는 컨테이너 변경 후 기존 위치 객체를 사용할 수 있는지를 결정한다. 벡터는 재할당 시 모든 반복자와 참조가 무효화되고, 리스트는 제거된 노드 외의 반복자와 참조를 유지한다. 덱은 양 끝 삽입에서 기존 원소의 참조를 유지할 수 있지만 반복자는 무효화될 수 있다. 컨테이너를 변경하는 코드에서는 무효화 규칙을 확인하고 반환된 반복자나 새로 얻은 반복자를 사용해야 한다.

std::array는 고정 크기와 연속 메모리를 제공하며 C 배열보다 대입, 비교, 반복자 사용이 편리하다. 크기는 타입의 일부이므로 서로 다른 크기의 배열은 서로 다른 타입이다.

std::vector는 연속 메모리, 가변 크기, 빠른 임의 접근을 제공한다. size()는 실제 원소 수이고 capacity()는 재할당 없이 저장할 수 있는 원소 수이다. reserve()는 용량만 확보하고 resize()는 원소 수를 변경한다. 재할당이 발생하면 기존 반복자, 포인터, 참조가 모두 무효화된다.

std::deque는 양쪽 끝 삽입과 제거가 효율적이며 임의 접근을 지원한다. 원소는 여러 블록에 나뉘어 저장되므로 전체 연속 메모리와 data()를 제공하지 않는다.

std::list와 std::forward_list는 노드 기반 연결 구조를 사용한다. 삽입과 제거가 다른 원소의 위치를 바꾸지 않지만 인덱스 접근이 없고 캐시 지역성과 메모리 부가 비용에서 불리할 수 있다. 연결 리스트는 삽입 위치를 이미 알고 있고 참조 안정성이나 노드 이동이 필요한 경우에 선택한다.

std::string은 연속된 문자 데이터를 소유하고 검색, 연결, 부분 문자열, 삽입, 제거 기능을 제공한다. 숫자 변환에는 간단한 std::stoi() 계열과 결과를 직접 검사하는 std::from_chars()를 사용할 수 있다. 문자열 수정으로 저장 공간이 바뀌면 기존 문자 포인터와 반복자가 무효화될 수 있다.

가변 크기 순차 컨테이너의 기본 선택은 std::vector이다. 고정 크기, 양 끝 조작, 노드 안정성, 문자 처리처럼 벡터가 충족하지 못하는 요구가 분명할 때 다른 컨테이너를 선택한다.

11.10 학습 과제

과제 **1.** 표준 라이브러리 구성 요소 구분
std::vector, std::sort, std::greater, std::string, std::unique_ptr, std::ifstream, std::thread를 컨테이너, 알고리즘, 함수 객체, 문자열, 메모리 관리, 입출력, 동시성으로 분류한다.
각 타입이나 함수가 정의된 헤더와 std 네임스페이스 사용 여부를 기록한다. 표준 라이브러리 전체와 STL이라는 표현의 범위 차이도 서술한다.
과제 2. 반복자 범위와 알고리즘
정수 원소 20개를 가진 std::vector를 작성하고 표준 알고리즘으로 작업을 수행한다.

- 값 50 검색
- 오름차순 정렬
- 짝수 개수 계산
- 모든 원소의 합 계산
- 30 미만 원소 제거

인덱스 기반 반복문을 사용하지 않고 반복자, 범위 기반 for, 표준 알고리즘을 활용한다. erase()와 반복자 반환값을 이용한 제거 방식과 C++20 std::erase_if() 방식도 비교한다.

과제 **3. std::array**와 **C** 배열 비교
정수 5개를 저장하는 C 배열과 std::array<int, 5>를 작성한다. 두 배열에 대해 크기 확인, 함수 인수 전달, 전체 복사, 비교, 정렬 코드를 작성한다.
C 배열이 함수 인수에서 포인터로 변환될 때 크기 정보가 어떻게 처리되는지 설명한다. 여러 크기의 std::array를 받을 수 있는 함수 템플릿도 구현한다.

과제 **4. vector**의 크기와 용량 관찰
빈 std::vector<int>에 원소 100개를 하나씩 추가하면서 size(), capacity(), data() 주소가 변한 시점을 출력한다.
reserve(100)을 호출한 경우와 호출하지 않은 경우를 비교한다. 용량 증가 배율이 표준으로 고정되어 있지 않다는 점을 기록하고, 재할당 횟수와 주소 변경 횟수를 측정한다.

과제 **5. reserve**와 **resize** 구분
std::vector<int>에 reserve(10)과 resize(10)을 각각 적용하고 size(), capacity(), 반복 가능한 원소 수를 비교한다.
생성자와 소멸자 호출 횟수를 출력하는 Trace 클래스를 원소 타입으로 사용하여 두 함수가 객체 생성에 미치는 영향을 확인한다.

과제 **6.** 반복자 무효화 확인
std::vector<int>의 첫 원소를 가리키는 반복자, 포인터, 참조를 저장한다. 원소 추가 전후의 capacity()와 data()를 출력하여 재할당 여부를 확인한다.
무효화된 반복자를 실제로 역참조하지 않고 주소와 용량 변화만으로 사용 가능 여부를 판단한다. reserve()를 적용한 경우와 적용하지 않은 경우를 비교한다.
같은 실험을 std::list<int>에서도 수행하여 삽입 후 기존 반복자가 유지되는지 확인한다.

과제 **7. vector**와 **deque** 비교

앞과 뒤에 정수를 추가하고 제거하는 작업을 std::vector와 std::deque로 각각 구현한다.
벡터에서 앞쪽 삽입이 원소 이동을 발생시키는 이유와 덱이 양쪽 끝 작업을 효율적으로 처리하는 이유를 저장 구조와 함께 설명한다. 두 컨테이너가 임의 접근을 지원하지만 data() 제공 여부가 다른 이유도 기술한다.

과제 **8. list**와 **forward_list** 조작
std::list<std::string> 두 개를 만들고 splice()로 한 리스트의 일부 원소를 다른 리스트로 옮긴다. 원소 복사 없이 노드 연결이 변경되는 것을 반복자와 주소 출력으로 확인한다.
std::forward_list<int>에서는 before_begin(), insert_after(), erase_after()를 사용하여 첫 위치 삽입과 제거를 구현한다. size()와 push_back()이 없는 구조적 이유를 설명한다.

과제 **9.** 문자열 파싱
쉼표로 구분된 문자열을 입력받아 이름, 레벨, 체력으로 분리한다.
Knight,12,150

find()와 substr()로 각 필드를 추출하고 숫자 필드는 std::stoi()로 변환한다. 구분자 누락, 숫자 변환 실패, 남은 문자가 있는 숫자 필드를 검사한다.
같은 숫자 변환을 std::from_chars()로 구현하고 예외 처리 방식과 오류 코드 검사 방식을 비교한다.

과제 **10.** 순차 컨테이너 선택 보고서
제시된 기능마다 적합한 순차 컨테이너를 선택하고 근거를 작성한다.

- 항상 세 좌표를 저장하는 위치 데이터
- 실행 중 계속 추가되는 로그 목록
- 앞과 뒤에서 작업을 넣고 빼는 작업 대기열
- 원소 주소가 유지되어야 하고 리스트 사이 이동이 잦은 객체 목록
- 고정 길이가 아닌 사용자 입력 문자열
- 대량 숫자를 외부 C 함수에 전달하는 버퍼

각 선택에서 크기 변경, 연속 메모리, 임의 접근, 삽입과 제거 위치, 반복자 안정성, 메모리 부가 비용을 평가한다. std::vector가 부적합하다고 판단한 항목은 벡터가 충족하지 못하는 요구를 구체적으로 밝힌다.

12. 연관 컨테이너와 컨테이너 어댑터

순차 컨테이너는 원소의 위치와 순서를 중심으로 데이터를 관리한다. std::vector에서 특정 원소를 찾으려면 처음부터 비교하거나 정렬 후 이진 탐색을 적용해야 한다. 프로그램에서 이름, 식별자, 코드처럼 원소를 구분하는 기준이 분명하다면 키를 중심으로 저장하고 탐색하는 구조가 필요하다.

연관 컨테이너(associative container)는 키를 이용해 원소를 저장하고 찾는다. std::set과 std::map은 비교 규칙에 따라 키를 정렬하며, std::unordered_set과 std::unordered_map은 해시값을 이용해 키가 들어갈 버킷을 결정한다.

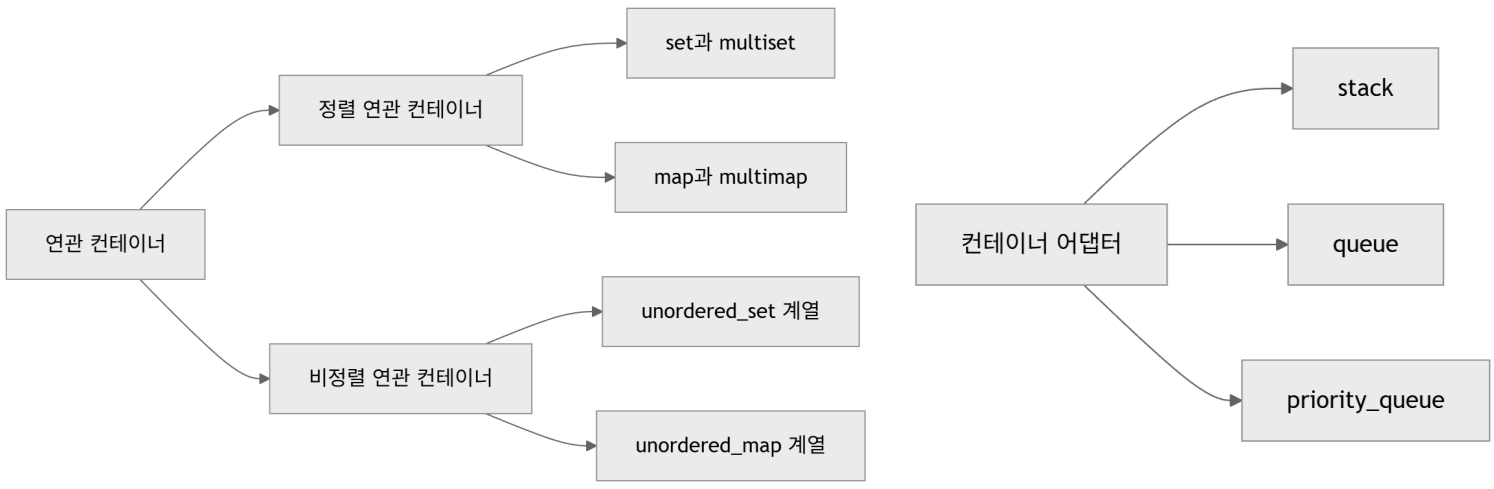
```
#include <map>
#include <string>

std::map<int, std::string> itemNames{
    {1001, "Potion"},
    {1002, "Ether"},
    {1003, "Elixir"}
};

std::string name = itemNames.at(1002);
```

키 1002를 이용해 문자열 값을 찾는다. 순차 컨테이너처럼 몇 번째 원소인지를 지정하지 않아도 된다.

컨테이너 어댑터(container adaptor)는 기존 컨테이너의 기능을 제한하여 특정 자료 처리 규칙을 제공한다. std::stack은 마지막에 넣은 원소를 먼저 꺼내고, std::queue는 먼저 넣은 원소를 먼저 꺼낸다. std::priority_queue는 삽입 순서보다 우선순위를 기준으로 처리한다.



연관 컨테이너를 선택할 때는 정렬된 순회가 필요한지, 중복 키를 허용하는지, 평균 탐색 속도와 최악의 경우를 어떻게 판단할지 확인해야 한다. 사용자 정의 타입을 키로 사용하려면 비교 규칙이나 해시 규칙도 타입의 의미에 맞게 정의해야 한다.

12.1 std::set과 std::multiset

std::set<Key>은 키 자체를 원소로 저장하는 정렬 연관 컨테이너이다. 같은 키를 하나만 저장한다. std::multiset<Key>은 정렬 구조를 유지하면서 동등한 키를 여러 개 저장한다.

```
#include <set>
std::set<int> uniqueIds{30, 10, 20, 10};
std::multiset<int> scores{80, 90, 80, 70};
```

uniqueIds에는 10, 20, 30만 저장된다. scores에는 두 개의 80이 모두 저장된다.

12.1.1 정렬된 키와 키 동등성

std::set은 기본적으로 std::less<Key>를 사용해 키를 오름차순으로 정렬한다.

```
std::set<int> values{30, 10, 20};
for (int value : values)
{
    std::cout << value << ' ';
}
// 출력: 10 20 30
```

표준은 내부 자료 구조의 종류를 규정하지 않지만, 구현은 보통 균형 이진 검색 트리를 사용한다. 사용자가 의존할 수 있는 부분은 키가 비교 규칙에 따라 정렬되고 삽입, 탐색, 제거가 로그 시간 복잡도를 가진다는 점이다.

정렬 연관 컨테이너는 operator==가 아니라 비교 함수로 키의 동등성을 판단한다. 비교 함수를 comp라고 하면 두 키 left, right에 대해 두 방향의 비교가 모두 거짓일 때 동등한 키로 본다.

```
!comp(left, right) && !comp(right, left)
```

값이 완전히 같지 않아도 비교 기준에서 순서가 구분되지 않으면 같은 키로 취급될 수 있다. 사용자 정의 비교 함수에서 동점 조건을 빠뜨리지 않아야 하는 이유이다.

12.1.2 중복 허용 여부

std::set::insert()는 삽입된 위치와 삽입 성공 여부를 함께 반환한다.

```
std::set<int> ids;

auto [position, inserted] = ids.insert(1001);

if (inserted)
{
    std::cout << "inserted: " << *position << '\n';
}
```

같은 키를 다시 삽입하면 inserted가 false가 되고 position은 이미 저장된 원소를 가리킨다.

```
auto [samePosition, insertedAgain] = ids.insert(1001);
```

std::multiset::insert()는 중복 키를 허용하므로 삽입된 원소를 가리키는 반복자만 반환한다.

```
std::multiset<int> scores;

scores.insert(80);
scores.insert(80);
scores.insert(90);
```

키의 존재 여부만 중요하고 중복을 제거해야 한다면 std::set이 적합하다. 같은 키의 발생 횟수나 여러 항목을 모두 보관해야 한다면 std::multiset을 사용할 수 있다.

12.1.3 삽입, 탐색, 범위 검색과 제거

insert()와 emplace()로 원소를 추가할 수 있다.

```
std::set<std::string> names;

names.insert("Alice");
names.emplace("Bob");
```

find()는 키를 찾으면 해당 원소를 가리키는 반복자를 반환하고, 찾지 못하면 end()를 반환한다.

```
auto position = names.find("Alice");

if (position != names.end())
{
    std::cout << *position << '\n';
}
```

C++20의 contains()는 존재 여부만 확인할 때 사용할 수 있다.

```
if (names.contains("Bob"))
{
    std::cout << "found\n";
}
```

count()는 지정한 키와 동등한 원소 수를 반환한다. std::set에서는 결과가 0 또는 1이고, std::multiset에서는 여러 개가 될 수 있다.

```
std::multiset<int> scores{70, 80, 80, 80, 90};
std::size_t count = scores.count(80);
```

정렬 상태를 이용하면 키 범위를 찾을 수 있다.

```
std::set<int> values{10, 20, 30, 40, 50};

auto first = values.lower_bound(20);
auto last = values.upper_bound(40);

for (auto iterator = first; iterator != last; ++iterator)
{
    std::cout << *iterator << ' ';
}
```

lower_bound(key)는 key보다 작지 않은 첫 원소를 가리킨다. upper_bound(key)는 key보다 큰 첫 원소를 가리킨다. 위 범위에는 20, 30, 40이 포함된다.
equal_range(key)는 lower_bound()와 upper_bound()를 한 번에 얻는다. 중복 키가 있는 std::multiset에서 유용하다.

```
std::multiset<int> scores{70, 80, 80, 80, 90};

auto [first, last] = scores.equal_range(80);

for (auto iterator = first; iterator != last; ++iterator)
{
    std::cout << *iterator << ' ';
}
```

키를 전달한 erase()는 해당 키와 동등한 원소를 제거하고 제거한 개수를 반환한다.

```
std::size_t removed = scores.erase(80);
```

std::multiset에서는 같은 키를 가진 원소가 모두 제거된다. 한 원소만 제거하려면 반복자를 전달한다.

```
auto position = scores.find(80);

if (position != scores.end())
{
    scores.erase(position);
}
```

범위를 전달해 여러 원소를 제거할 수도 있다.

```
auto first = scores.lower_bound(70);
auto last = scores.upper_bound(80);
scores.erase(first, last);
```

12.1.4 비교 함수와 정렬 기준

기본 오름차순 대신 std::greater<Key>를 사용하면 내림차순으로 저장한다.

```
#include <functional>
#include <set>

std::set<int, std::greater<int>> values{10, 30, 20};
```

사용자 정의 비교 함수 객체도 지정할 수 있다.

```
struct Player
{
    int id;
    int score;
};
```

```cpp
struct ComparePlayer
{
    bool operator()(const Player& left, const Player& right) const
    {
        if (left.score != right.score)
        {
            return left.score > right.score;
        }

        return left.id < right.id;
    }
};

std::set<Player, ComparePlayer> ranking;
```

점수가 높은 플레이어가 앞에 오고, 점수가 같으면 id가 작은 플레이어가 앞에 온다. id 비교를 생략하면 점수가 같은 모든 플레이어가 동등한 키로 판단되어 std::set에 하나만 저장될 수 있다.

비교 함수는 엄격 약순서(strict weak ordering)를 만족해야 한다.

- 자기 자신보다 앞선다고 판단하지 않는다.
- a가 b보다 앞서고 b가 c보다 앞서면 a는 c보다 앞선다.
- 비교 결과가 동등한 관계도 일관되어야 한다.

비교 결과가 호출 시점마다 달라지거나 외부 상태에 따라 정렬 기준이 바뀌면 컨테이너의 내부 순서가 깨질 수 있다.

12.1.5 원소 수정 제한과 노드 핸들

std::set의 반복자로 얻은 원소는 직접 수정할 수 없다.

```cpp
std::set<int> values{10, 20, 30};
auto position = values.find(20);

// *position = 25; // 오류
```

키를 직접 바꾸면 정렬 위치도 달라질 수 있다. 컨테이너는 원소가 저장된 동안 정렬 기준이 유지된다고 가정하므로 수정 인터페이스를 제공하지 않는다.
일반적인 변경은 기존 원소를 제거하고 새 값을 삽입하는 방식으로 처리한다.

```cpp
values.erase(20);
values.insert(25);
```

C++17의 노드 핸들(node handle)을 이용하면 원소를 노드째 분리한 뒤 값을 바꾸고 다시 삽입할 수 있다.

```cpp
std::set<std::string> names{"Alice", "Bob", "Carol"};
auto node = names.extract("Bob");
if (!node.empty())
{
    node.value() = "Bobby";
    names.insert(std::move(node));
}
```

extract()는 원소 객체를 복사하거나 이동하지 않고 컨테이너에서 노드를 분리한다. 수정한 키가 이미 존재하면 std::set에 다시 들어가지 못할 수 있으므로 삽입 결과를 확인해야 한다.

12.2 std::map과 std::multimap

std::map<Key, T>는 키와 값을 한 쌍으로 저장한다. 키는 정렬과 탐색에 사용하고 값은 키에 연결된 정보를 나타낸다. 같은 키는 하나만 저장한다.

```cpp
#include <map>
#include <string>

std::map<int, std::string> itemNames{
    {1001, "Potion"},
    {1002, "Ether"}
```

```
};
```

std::multimap<Key, T>은 같은 키에 여러 값을 연결할 수 있다.

```
std::multimap<std::string, std::string> categoryItems{
    {"Potion", "Small Potion"},
    {"Potion", "Large Potion"},
    {"Weapon", "Sword"}
};
```

12.2.1 키와 값

std::map<Key, T>::value_type은 std::pair<const Key, T>이다. 키는 first, 값은 second에 저장된다.

```
for (const auto& entry : itemNames)
{
    std::cout << entry.first << ": " << entry.second << '\n';
}
```

키 타입에 const가 포함되므로 저장된 키를 직접 바꿀 수 없다. 값은 수정할 수 있다.

```
auto position = itemNames.find(1001);

if (position != itemNames.end())
{
    position->second = "High Potion";
}
```

키를 바꾸려면 제거 후 재삽입하거나 노드 핸들을 사용한다.

```
auto node = itemNames.extract(1001);

if (!node.empty())
{
    node.key() = 2001;
    itemNames.insert(std::move(node));
}
```

12.2.2 operator[]와 at

operator[]는 키에 연결된 값에 접근한다.

```
std::map<int, std::string> names;

names[1001] = "Potion";
std::cout << names[1001] << '\n';
```

키가 없으면 새 원소를 삽입하고 값 타입을 값 초기화한다.

```
std::map<std::string, int> counts;

int potionCount = counts["Potion"];
```

"Potion"이 없던 상태라면 값 0을 가진 원소가 새로 생긴다. 존재 여부만 확인하려고 operator[]를 사용하면 컨테이너가 변경될 수 있다.

```
if (counts.contains("Potion"))
{
    std::cout << counts.at("Potion") << '\n';
}
```

at()은 키가 없을 때 원소를 삽입하지 않고 std::out_of_range 예외를 던진다.

```
try
```

```
{
    int count = counts.at("Ether");
}
catch (const std::out_of_range&)
{
    std::cout << "key not found\n";
}
```

읽기 전용 const std::map에는 operator[]가 없다. 키가 없을 때 삽입해야 하는 함수이기 때문이다. at(), find(), contains()는 읽기 전용 객체에도 사용할 수 있다.

std::multimap은 operator[]와 at()를 제공하지 않는다. 하나의 키에 여러 값이 연결될 수 있어 하나의 값 참조를 결정할 수 없기 때문이다.

12.2.3 insert, emplace와 try_emplace

insert()는 키와 값으로 구성된 객체를 삽입한다.

```
std::map<int, std::string> names;
auto [position, inserted] = names.insert({1001, "Potion"});
```

같은 키가 있으면 삽입하지 않고 기존 원소를 가리키는 반복자를 반환한다.

```
auto [samePosition, insertedAgain] = names.insert({1001,
"Ether"});
```

insertedAgain은 false이고 기존 값 "Potion"은 유지된다.
emplace()는 전달한 인수로 원소 객체를 생성한다.

```
names.emplace(1002, "Ether");
```

같은 키가 이미 있어도 인수로부터 임시 객체가 생성될 가능성이 있다. 값 객체의 생성 비용이 크고 키 중복 가능성이 있다면 try_emplace()가 적합하다.

```
std::map<int, std::string> names;
names.try_emplace(1001, "Potion");
names.try_emplace(1001, "Ether");
```

두 번째 호출은 키 1001이 이미 있으므로 값 객체를 새로 구성하지 않는다. 기존 값도 변경하지 않는다.
값 타입의 생성자 인수를 직접 전달할 수 있다.

```
struct Item
{
    std::string name;
    int price;

    Item(std::string itemName, int itemPrice)
        : name(std::move(itemName)), price(itemPrice)
    {
    }
};

std::map<int, Item> items;
items.try_emplace(1001, "Potion", 50);
```

try_emplace()는 키와 값 생성자 인수를 분리해 받는다. std::unique_ptr처럼 이동 전용 값을 조건부로 삽입할 때도 유용하다.

```
std::map<int, std::unique_ptr<Item>> items;

auto item = std::make_unique<Item>("Potion", 50);
auto [position, inserted] = items.try_emplace(1001,
std::move(item));
```

삽입에 실패하면 item이 이동되지 않고 그대로 남는다.

12.2.4 insert_or_assign과 값 갱신

키가 없으면 삽입하고 키가 있으면 값을 교체하려면 insert_or_assign()을 사용한다.

```
std::map<int, std::string> names;

names.insert_or_assign(1001, "Potion");
names.insert_or_assign(1001, "High Potion");
```

두 번째 호출은 키 1001의 값을 "High Potion"으로 바꾼다.
함수별 동작 차이는 키가 이미 존재할 때 분명해진다.

함수	키가 없을 때	키가 있을 때
insert()	삽입	기존 원소 유지
emplace()	삽입	기존 원소 유지
try_emplace()	값 생성 후 삽입	값 생성 없이 기존 원소 유지
insert_or_assign()	삽입	기존 값에 대입
operator[]	기본값 삽입	값 참조 반환
at()	예외 발생	값 참조 반환

단순 개수 누적에는 operator[]가 간결하다.

```
std::map<std::string, int> counts;

for (const std::string& name : itemNames)
{
    ++counts[name];
}
```

값 타입을 기본 생성할 수 없거나 불필요한 기본 삽입을 피해야 한다면 try_emplace()와 insert_or_assign()을 구분해 사용한다.

12.2.5 구조적 바인딩을 이용한 순회

구조적 바인딩을 사용하면 키와 값을 이름으로 분리할 수 있다.

```
for (const auto& [id, name] : itemNames)
{
    std::cout << id << ": " << name << '\n';
}
```

값을 수정하려면 auto&를 사용한다.

```
std::map<int, int> prices {
    {1001, 50},
    {1002, 80}
};

for (auto& [id, price] : prices)
{
    price += 10;
}
```

id는 const int&로 바인딩되므로 수정할 수 없다. price는 int&로 바인딩되어 변경할 수 있다.

std::multimap에서 같은 키에 연결된 값을 처리하려면 equal_range()를 사용할 수 있다.

```
std::multimap<std::string, std::string> items{
    {"Potion", "Small Potion"},
    {"Potion", "Large Potion"},
    {"Weapon", "Sword"}
```

```
};

auto [first, last] = items.equal_range("Potion");

for (auto iterator = first; iterator != last; ++iterator)
{
    std::cout << iterator->second << '\n';
}
```

12.3 비 정렬 연관 컨테이너

비 정렬 연관 컨테이너(unordered associative container)는 키의 해시값을 이용해 원소가 속할 버킷을 선택한다. 원소가 키 순서대로 배치되지 않으며 순회 순서도 정렬 의미를 가지지 않는다.

12.3.1 std::unordered_set과 std::unordered_multiset

std::unordered_set<Key>은 중복되지 않는 키를 저장하고, std::unordered_multiset<Key>은 동등한 키를 여러 개 저장한다.

```
#include <unordered_set>

std::unordered_set<int> ids{30, 10, 20, 10};
std::unordered_multiset<int> scores{80, 90, 80, 70};
```

std::unordered_set은 10을 하나만 저장한다. 순회 결과가 10, 20, 30 순서라는 보장은 없다.

```
for (int id : ids)
{
    std::cout << id << ' ';
}
```

같은 실행에서 특정 순서로 보이더라도 정렬이나 삽입 순서에 의존해서는 안 된다. 원소 추가, 제거, 재 해시 또는 라이브러리 구현 변경으로 순회 순서가 달라질 수 있다.

insert(), emplace(), find(), contains(), count(), erase()의 기본 사용 형태는 정렬 연관 컨테이너와 비슷하다.

```
ids.insert(1001);

if (ids.contains(1001))
{
    ids.erase(1001);
}
```

12.3.2 std::unordered_map과 std::unordered_multimap

std::unordered_map<Key, T>은 중복되지 않는 키와 값을 저장한다.

```
#include <string>
#include <unordered_map>

std::unordered_map<int, std::string> itemNames{
    {1001, "Potion"},
    {1002, "Ether"}
};
```

operator[], at(), insert(), try_emplace(), insert_or_assign()을 사용할 수 있다.

```
itemNames[1003] = "Elixir";
itemNames.try_emplace(1004, "Antidote");
itemNames.insert_or_assign(1002, "High Ether");
```

std::unordered_multimap<Key, T>은 같은 키에 여러 값을 연결한다. operator[]와 at()는 제공하지 않는다.

```
std::unordered_multimap<std::string, std::string> categoryItems;
```

```
categoryItems.emplace("Potion", "Small Potion");
categoryItems.emplace("Potion", "Large Potion");

// 출력: 10 20 30 동등한 키의 범위는 equal_range()로 얻는다.
auto [first, last] = categoryItems.equal_range("Potion");

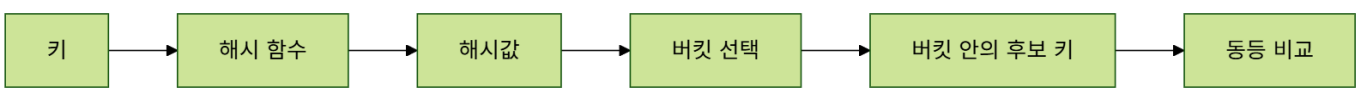
for (auto iterator = first; iterator != last; ++iterator)
{
    std::cout << iterator->second << '\n';
}
```

12.3.3 해시 함수, 동등 비교와 버킷

해시 함수(hash function)는 키를 std::size_t 값으로 변환한다.

```
std::size_t hashValue = std::hash<std::string>{}("Potion");
```

컨테이너는 해시값을 이용해 버킷을 선택하고, 해당 버킷 안에서 동등 비교를 수행해 정확한 키를 찾는다.



std::unordered_map<Key, T>의 기본 해시 함수는 std::hash<Key>, 기본 동등 비교는 std::equal_to<Key>이다.

```
using Table = std::unordered_map<
    std::string,
    int,
    std::hash<std::string>,
    std::equal_to<std::string>>;
```

버킷 수와 원소가 속한 버킷을 확인할 수 있다.

```
std::unordered_set<std::string> names{"Alice", "Bob", "Carol"};
std::cout << names.bucket_count() << '\n';
std::cout << names.bucket("Alice") << '\n';
```

버킷 인터페이스는 해시 구조를 이해하거나 상태를 분석할 때 사용할 수 있다. 일반적인 원소 처리에서는 버킷 번호를 직접 관리하지 않는다.

12.3.4 충돌, 적재율과 재 해시

서로 다른 키가 같은 버킷에 배치되는 현상을 해시 충돌(hash collision)이라고 한다. 충돌은 오류가 아니다. 컨테이너는 버킷 안에서 동등 비교를 수행해 키를 구분한다.

```
버킷 0 :
버킷 1 : Alice -> Carol
버킷 2 : Bob
버킷 3 :
```

모든 키가 같은 버킷에 몰리면 탐색이 선형 시간에 가까워질 수 있다. 해시 함수는 키를 버킷에 고르게 분산하도록 작성해야 한다.
적재율(load factor)은 원소 수를 버킷 수로 나눈 값이다.

```
std::unordered_set<int> values;

std::cout << values.load_factor() << '\n';
std::cout << values.max_load_factor() << '\n';
```

원소가 늘어 적재율이 기준을 넘으면 컨테이너는 버킷 배열을 확장하고 원소를 새 버킷에 다시 배치한다. 이 과정을 재 해시(rehash)라고 한다.

reserve(n)은 약 n개의 원소를 저장할 수 있도록 버킷 수를 준비한다.

```
std::unordered_map<int, std::string> items;
items.reserve(1000);
```

많은 원소를 삽입할 예정이라면 미리 reserve()를 호출해 반복적인 재 해시를 줄일 수 있다.
rehash(n)은 버킷 수가 최소한 지정한 값 이상이 되도록 요청한다.

```
items.rehash(128);
```

재 해시가 발생하면 모든 반복자가 무효화된다. 기존 원소를 가리키는 포인터와 참조는 유지된다. 반복자는 버킷 순회 구조와 연결되어 있지만 원소 객체 자체는 이동하지 않도록 규정되어 있기 때문이다.

```
std::unordered_map<int, std::string> items{
    {1001, "Potion"},
    {1002, "Ether"}
};

auto iterator = items.find(1001);
std::string& name = iterator->second;

items.reserve(1000);

// iterator는 재 해시로 무효화될 수 있다.
std::cout << name << '\n'; // 원소가 제거되지 않았다면 참조는 유효
```

12.3.5 사용자 정의 키

사용자 정의 타입을 키로 사용하려면 해시 함수와 동등 비교를 제공해야 한다.

```
#include <functional>
#include <string>
#include <unordered_map>

struct ItemKey
{
    int category;
    int id;

    bool operator==(const ItemKey&) const = default;
};

struct ItemKeyHash
{
    std::size_t operator()(const ItemKey& key) const noexcept
    {
        std::size_t first = std::hash<int>{}(key.category);
        std::size_t second = std::hash<int>{}(key.id);
        return first ^ (second << 1);
    }
};

std::unordered_map<ItemKey, std::string, ItemKeyHash>
itemNames;

itemNames.insert({ItemKey{1, 1001}, "Potion"});
```

동등 비교가 참인 두 키는 반드시 같은 해시값을 만들어야 한다.

```
left == right  →  hash(left) == hash(right)
```

같은 해시값이 나왔다고 두 키가 같은 것은 아니다. 충돌은 허용되며 동등 비교가 최종 판단을 담당한다.

해시 조합식은 예제 수준의 단순한 방식이다. 키 분포와 데이터 규모에 따라 더 적절한 조합 방식을 선택할 수 있다. 해시 함수는 키가 컨테이너에 저장된 동안 결과가 바뀌지 않아야 한다.

12.4 정렬 연관 컨테이너와 비 정렬 연관 컨테이너

std::map과 std::unordered_map은 모두 키로 값을 찾지만 성능 특성과 제공 기능이 다르다. 평균 탐색 시간만 보고 선택하면 범위 검색, 순회 순서, 최악의 경우, 메모리 구조를 놓칠 수 있다.

12.4.1 정렬 여부와 순회 순서

정렬 연관 컨테이너는 비교 함수가 정한 키 순서로 순회한다.

```
std::map<int, std::string> ordered {
    {30, "C"},
    {10, "A"},
    {20, "B"}
};
```

순회 결과의 키는 10, 20, 30 순서이다.

비 정렬 연관 컨테이너의 순회 순서는 지정되지 않는다.

```
std::unordered_map<int, std::string> unordered {
    {30, "C"},
    {10, "A"},
    {20, "B"}
};
```

정렬된 출력이 필요하면 std::map을 사용하거나 키를 별도 컨테이너에 복사해 정렬해야 한다.

12.4.2 평균 탐색 비용과 최악 탐색 비용

정렬 연관 컨테이너의 삽입, 탐색, 제거는 일반적으로 로그 시간이다.

```
std::map, std::set: O(log n)
```

비 정렬 연관 컨테이너는 해시 분포가 양호할 때 평균 상수 시간이다.

```
std::unordered_map, std::unordered_set: 평균 O(1), 최악 O(n)
```

최악의 경우 모든 키가 같은 버킷에 모이면 선형 탐색이 필요하다. 외부 입력이 해시 키를 통제할 수 있는 환경에서는 의도적인 충돌 공격도 고려할 수 있다.
원소 수가 적으면 이론적 복잡도보다 메모리 접근과 할당 비용이 더 크게 작용할 수 있다. 실제 성능이 중요하면 대상 데이터와 연산 비율로 측정해야 한다.

12.4.3 메모리 구조와 캐시 지역성

정렬 연관 컨테이너는 보통 각 원소를 트리 노드에 저장한다. 노드에는 값 외에도 부모와 자식 연결, 균형 정보가 필요하다.

비 정렬 연관 컨테이너는 버킷 배열과 원소 저장 구조를 함께 관리한다. 적재율을 낮게 유지하면 탐색 후보 수는 줄지만 버킷 메모리가 늘어난다.

두 범주 모두 std::vector처럼 원소가 하나의 연속 메모리 영역에 배치된다고 가정할 수 없다. 많은 작은 노드 할당과 불연속 메모리 접근은 캐시 효율에 영향을 줄 수 있다.

12.4.4 범위 검색과 선택 기준

정렬 연관 컨테이너는 순서 관계를 이용한 연산을 제공한다.

```
std::map<int, std::string> players;
auto first = players.lower_bound(1000);
auto last = players.upper_bound(1999);
```

특정 구간의 키를 순회하거나 최소 키, 최대 키를 얻는 작업에 적합하다.
비 정렬 연관 컨테이너는 키의 정확한 일치 검색에 초점을 둔다. lower_bound()와 upper_bound()를 제공하지 않는다.

<정렬 연관 컨테이너와 비 정렬 연관 컨테이너 비교>

기준	set, map 계열	unordered_set, unordered_map 계열
키 배치	비교 함수 순서	해시 버킷

순회 순서	정렬됨	지정되지 않음
평균 탐색	O(log n)	O(1)
최악 탐색	O(log n)	O(n)
범위 검색	지원	지원하지 않음
사용자 정의 키	비교 규칙 필요	해시와 동등 비교 필요
삽입 시 반복자	기존 위치 유지	재 해시 발생 시 전체 무효
대표 용도	정렬 순회, 구간 검색	정확한 키의 빠른 검색

12.5 std::stack

std::stack<T>은 후입선출(LIFO, Last In First Out) 방식의 컨테이너 어댑터이다. 마지막에 넣은 원소를 가장 먼저 꺼낸다.

12.5.1 후입선출

원소를 넣을 때는 push(), 가장 위 원소를 확인할 때는 top(), 제거할 때는 pop()을 사용한다.

```
#include <stack>

std::stack<int> values;

values.push(10);
values.push(20);
values.push(30);

std::cout << values.top() << '\n';
values.pop();
std::cout << values.top() << '\n';

// 출력: 30
// 20
```

pop()은 제거한 값을 반환하지 않는다. 값을 사용해야 한다면 top()으로 읽은 뒤 pop()을 호출한다.

```
if (!values.empty())
{
    int value = values.top();
    values.pop();
}
```

비어 있는 스택에서 top()이나 pop()을 호출하면 동작이 보장되지 않는다. empty()로 확인해야 한다.

12.5.2 기반 컨테이너와 제한된 인터페이스

std::stack은 원소를 직접 저장하는 독립 컨테이너가 아니라 기반 컨테이너를 감싸는 어댑터이다. 기본 기반 컨테이너는 std::deque<T>이다.

```
std::stack<int> defaultStack;
std::stack<int, std::vector<int>> vectorStack;
std::stack<int, std::list<int>> listStack;
```

기반 컨테이너는 back(), push_back(), pop_back()을 제공해야 한다.

std::stack은 반복자를 제공하지 않는다. 중간 원소를 순회하거나 수정할 수 있게 하면 후입선출 규칙을 우회할 수 있기 때문이다. 전체 원소를 임의로 다뤄야 한다면 기반 컨테이너를 직접 사용한다.

emplace()는 스택의 위쪽에 원소를 직접 생성한다.

```
struct Command
{
```

```
    std::string name;
    int value;

    Command(std::string commandName, int commandValue)
        : name(std::move(commandName)), value(commandValue)
    {
    }
};

std::stack<Command> commands;
commands.emplace("Move", 10);
```

12.5.3 스택 활용

스택은 가장 최근 상태를 먼저 되돌리는 작업에 적합하다.

```
std::stack<std::string> undoHistory;
undoHistory.push("Create object");
undoHistory.push("Move object");
undoHistory.push("Delete object");
```

실행 취소를 수행하면 가장 최근 명령부터 꺼낸다.

```
while (!undoHistory.empty())
{
    std::cout << "undo: " << undoHistory.top() << '\n';
    undoHistory.pop();
}
```

괄호 검사에도 사용할 수 있다.

```
bool IsBalanced(const std::string& text)
{
    std::stack<char> brackets;

    for (char character : text)
    {
        if (character == '(')
        {
            brackets.push(character);
        }
        else if (character == ')')
        {
            if (brackets.empty())
            {
                return false;
            }

            brackets.pop();
        }
    }

    return brackets.empty();
}
```

재귀 호출을 반복문으로 바꾸는 과정에서도 처리할 상태를 명시적인 스택에 저장할 수 있다.

12.6 std::queue

std::queue<T>는 선입선출(FIFO, First In First Out) 방식의 컨테이너 어댑터이다. 먼저 넣은 원소를 먼저 꺼낸다.

12.6.1 선입선출

원소는 push()로 뒤에 추가하고 front()로 앞 원소를 확인한다. pop()은 앞 원소를 제거한다.

```cpp
#include <queue>

std::queue<std::string> messages;

messages.push("Connect");
messages.push("Load");
messages.push("Start");

std::cout << messages.front() << '\n';
messages.pop();
std::cout << messages.front() << '\n';

// 출력:
// Connect
// Load
```

마지막 원소는 back()으로 확인할 수 있다.

```cpp
std::cout << messages.back() << '\n';
```

비어 있는 큐에서 front(), back(), pop()을 호출하면 동작이 보장되지 않는다.

12.6.2 기반 컨테이너와 제한된 인터페이스

기본 기반 컨테이너는 std::deque<T>이다.

```cpp
std::queue<int> defaultQueue;
std::queue<int, std::list<int>> listQueue;
```

기반 컨테이너는 front(), back(), push_back(), pop_front()을 제공해야 한다. std::vector는 앞 원소를 제거하는 pop_front()가 없으므로 std::queue의 기반 컨테이너로 사용할 수 없다.

std::queue도 반복자를 제공하지 않는다. 중간 원소를 직접 꺼내면 선입선출 규칙이 깨지기 때문이다.
emplace()는 큐의 뒤쪽에 원소를 직접 생성한다.

```cpp
std::queue<Command> commands;
commands.emplace("Attack", 30);
```

12.6.3 작업 대기열과 메시지 처리

작업 요청을 받은 순서대로 처리할 때 큐를 사용할 수 있다.

```cpp
struct Job
{
    int id;
    std::string description;
};

std::queue<Job> jobs;

jobs.push({1, "Load texture"});
jobs.push({2, "Create mesh"});
jobs.push({3, "Save scene"});

while (!jobs.empty())
{
    Job job = std::move(jobs.front());
    jobs.pop();

    std::cout << job.id << ": " << job.description << '\n';
}
```

front()이 반환하는 참조는 pop()을 호출하면 제거된 원소와 함께 무효화된다. 값을 처리한 후 제거하거나, 이동이 필요하면 제거 전에 별도 객체로 옮긴다.
너비 우선 탐색(BFS)도 큐의 대표적인 활용이다. 시작 위치를 넣고, 방문한 위치에서 인접 위치를 큐 뒤에 추가하면 가까운 위치부터 처리할 수 있다.

12.7 std::priority_queue

std::priority_queue<T>는 삽입 순서가 아니라 우선순위가 가장 높은 원소를 먼저 제공하는 컨테이너 어댑터이다.

12.7.1 우선순위 기반 처리

기본 std::priority_queue<int>에서는 가장 큰 값이 top()에 놓인다.

```
#include <queue>

std::priority_queue<int> values;

values.push(30);
values.push(10);
values.push(50);
values.push(20);

std::cout << values.top() << '\n';

// 출력: 50
```

pop()으로 가장 높은 우선순위의 원소를 제거하면 남은 원소 중 우선순위가 가장 높은 값이 top()에 놓인다.

```
values.pop();
std::cout << values.top() << '\n';

// 출력: 30
```

12.7.2 힙 구조와 top

std::priority_queue는 기반 컨테이너를 힙(heap) 조건에 맞게 유지한다. 기본 기반 컨테이너는 std::vector<T>이다.

```
std::priority_queue<int> values;
```

힙은 전체 원소가 정렬된 구조가 아니다. 가장 높은 우선순위의 원소가 앞에 있고 부모와 자식 사이의 우선순위 관계만 유지한다.

```
       50
      /    \
    30      20
   /
 10
```

top()은 상수 시간에 가장 높은 우선순위 원소를 반환한다. push()와 pop()은 힙 조건을 복원하므로 로그 시간이 든다.

연산	시간 복잡도
top()	O(1)
push(), emplace()	O(log n)
pop()	O(log n)

기반 컨테이너는 임의 접근 반복자와 front(), push_back(), pop_back()을 제공해야 한다. std::vector와 std::deque를 사용할 수 있지만 std::list는 사용할 수 없다.

```
std::priority_queue<int, std::deque<int>> values;
```

12.7.3 비교 함수

사용자 정의 타입의 우선순위를 비교 함수 객체로 지정할 수 있다.

```
struct Task
```

```
{
    int id;
    int priority;
};

struct CompareTask
{
    bool operator()(const Task& left, const Task& right) const
    {
        return left.priority < right.priority;
    }
};

std::priority_queue<
    Task,
    std::vector<Task>,
    CompareTask> tasks;
```

비교 함수가 **true**를 반환하면 **left**의 우선순위가 **right**보다 낮다고 판단한다. 위 비교에서는 **priority**가 큰 작업이 **top()**에 놓인다.

```
tasks.push({1, 10});
tasks.push({2, 30});
tasks.push({3, 20});

std::cout << tasks.top().id << '\n';
```

작업 **2**가 가장 먼저 선택된다.
우선순위가 같을 때 처리 순서가 필요하면 보조 기준을 추가한다.

```
struct Task
{
    int id;
    int priority;
    std::uint64_t sequence;
};

struct CompareTask
{
    bool operator()(const Task& left, const Task& right) const
    {
        if (left.priority != right.priority)
        {
            return left.priority < right.priority;
        }

        return left.sequence > right.sequence;
    }
};
```

우선순위가 같으면 **sequence**가 작은 작업이 먼저 처리된다.

12.7.4 최대값과 최소값 우선순위

기본 비교 함수는 **std::less<T>**이므로 최대값이 **top()**에 놓인다.

```
std::priority_queue<int> maxQueue;
```

최소값을 먼저 처리하려면 **std::greater<T>**를 지정한다.

```
#include <functional>
#include <vector>

std::priority_queue<
    int,
    std::vector<int>,
    std::greater<int>> minQueue;
```

```
minQueue.push(30);
minQueue.push(10);
minQueue.push(20);

std::cout << minQueue.top() << '\n';

// 출력: 10
```

top()은 읽기 전용 참조를 반환한다. 저장된 원소의 우선순위 값을 직접 바꾸면 힙 조건이 깨질 수 있기 때문이다. 우선순위를 변경하려면 기존 원소를 제거하고 수정된 원소를 다시 삽입하는 방식으로 처리한다.

12.8 사용자 정의 타입과 연관 컨테이너

사용자 정의 타입을 연관 컨테이너의 키로 사용하려면 키의 동일성과 순서를 프로그램의 의미에 맞게 정의해야 한다. 정렬 연관 컨테이너는 비교 규칙을 사용하고 비 정렬 연관 컨테이너는 해시 함수와 동등 비교를 사용한다.

12.8.1 비교 연산자

키 타입에 비교 연산을 정의하면 기본 std::less<Key>를 사용할 수 있다.

```
#include <compare>

struct ItemKey
{
    int category;
    int id;

    auto operator<=>(const ItemKey&) const = default;
};

std::set<ItemKey> keys;
```

기본 비교는 멤버 선언 순서대로 category, id를 비교한다.

```
keys.insert({1, 1002});
keys.insert({1, 1001});
keys.insert({2, 1001});
```

타입 전체에서 자연스럽게 사용할 하나의 순서가 있다면 비교 연산자를 정의할 수 있다. 컨테이너마다 서로 다른 정렬 기준이 필요하다면 비교 함수 객체가 더 적합하다.

12.8.2 비교 함수 객체

플레이어를 점수 순위와 이름 순서로 각각 저장하려면 서로 다른 비교 함수를 지정할 수 있다.

```
struct Player
{
    int id;
    std::string name;
    int score;
};

struct CompareByScore
{
    bool operator()(const Player& left, const Player& right) const
    {
        if (left.score != right.score)
        {
            return left.score > right.score;
        }

        return left.id < right.id;
    }
};
```

```
struct CompareByName
{
    bool operator()(const Player& left, const Player& right) const
    {
        if (left.name != right.name)
        {
            return left.name < right.name;
        }

        return left.id < right.id;
    }
};

std::set<Player, CompareByScore> scoreRanking;
std::set<Player, CompareByName> nameIndex;
```

id를 보조 기준으로 사용하면 점수나 이름이 같은 플레이어도 서로 다른 키로 구분된다.

비교 함수가 객체의 변경 가능한 멤버를 기준으로 삼을 때는 주의해야 한다. 컨테이너에 저장된 키는 직접 수정할 수 없지만 외부 객체를 가리키는 포인터를 키로 저장하면 가리키는 객체의 값이 바뀔 수 있다. 비교 결과가 바뀌어도 컨테이너가 자동으로 재정렬되지는 않는다.

```
std::set<Player*, ComparePlayerPointer> players;
```

포인터가 가리키는 score를 변경하면 저장 순서와 비교 규칙이 불일치할 수 있다. 변경 가능한 값을 정렬 키로 사용하려면 제거 후 수정하고 다시 삽입하는 절차가 필요하다.

12.8.3 해시 함수

비 정렬 연관 컨테이너용 해시 함수는 키의 동등성을 구성하는 모든 멤버를 반영해야 한다.

```
struct PlayerKey
{
    int serverId;
    int playerId;

    bool operator==(const PlayerKey&) const = default;
};

struct PlayerKeyHash
{
    std::size_t operator()(const PlayerKey& key) const noexcept
    {
        std::size_t first = std::hash<int>{}(key.serverId);
        std::size_t second = std::hash<int>{}(key.playerId);
        return first ^ (second << 1);
    }
};

std::unordered_map<PlayerKey, std::string, PlayerKeyHash> playerNames;
```

noexcept는 해시 계산이 예외를 던지지 않는다는 의도를 표현한다. 해시 함수는 같은 키에 대해 일관된 값을 반환해야 한다.

std::hash를 사용자 정의 타입에 특수화하는 방법도 있지만, 별도 함수 객체를 템플릿 인수로 전달하면 컨테이너마다 해시 정책을 구분하기 쉽다.

12.8.4 동등 비교와 해시 계약

비 정렬 연관 컨테이너는 해시 함수로 후보 버킷을 고르고 동등 비교로 키를 최종 확인한다. 두 규칙은 같은 키 정의를 공유해야 한다.

```
struct CaseInsensitiveEqual
{
    bool operator()(std::string_view left,
                    std::string_view right) const
    {
        if (left.size() != right.size())
```

```
        {
            return false;
        }

        for (std::size_t index = 0; index < left.size(); ++index)
        {
            unsigned char leftCharacter =
                static_cast<unsigned char>(left[index]);
            unsigned char rightCharacter =
                static_cast<unsigned char>(right[index]);

            if (std::tolower(leftCharacter) !=
                std::tolower(rightCharacter))
            {
                return false;
            }
        }

        return true;
    }
};
```

대소문자를 무시하는 동등 비교를 사용한다면 해시 함수도 대소문자를 무시해야 한다. 동등 비교는 "Potion"과 "potion"을 같은 키로 판단하면서 해시 함수가 서로 다른 값을 반환하면 컨테이너의 요구 조건을 위반한다.

```
struct CaseInsensitiveHash
{
    std::size_t operator()(std::string_view text) const noexcept
    {
        std::size_t result = 0;

        for (unsigned char character : text)
        {
            unsigned char lower = static_cast<unsigned char>(
                std::tolower(character));
            result = result * 131 + lower;
        }

        return result;
    }
};

using NameSet = std::unordered_set<
    std::string,
    CaseInsensitiveHash,
    CaseInsensitiveEqual>;
```

동등한 키가 같은 해시값을 가져야 한다는 조건은 필수이다. 서로 다른 키가 같은 해시값을 갖는 충돌은 허용된다.

12.9 컨테이너와 객체 수명

컨테이너가 원소를 값으로 저장하는지 포인터로 저장하는지에 따라 객체의 수명과 소유권이 달라진다. 키 탐색 방식과 별개로 누가 객체를 소멸시키는지 분명해야 한다.

12.9.1 값 객체 저장

값 객체를 저장하면 컨테이너가 원소 객체의 수명을 관리한다.

```
struct CaseInsensitiveHash
{
    std::size_t operator()(std::string_view text) const noexcept
    {
        std::size_t result = 0;

        for (unsigned char character : text)
        {
            unsigned char lower = static_cast<unsigned char>(
```

```
                std::tolower(character));
            result = result * 131 + lower;
        }

        return result;
    }
};

using NameSet = std::unordered_set<
    std::string,
    CaseInsensitiveHash,
    CaseInsensitiveEqual>;
```

원소를 제거하면 해당 객체의 소멸자가 호출된다. 컨테이너가 소멸할 때 남아 있는 원소도 함께 소멸한다.

```
items.erase(1001);
```

값 저장은 소유권이 분명하고 메모리 관리가 단순하다. 객체가 크거나 복사 비용이 높아도 이동 연산과 emplace() 계열 함수를 활용할 수 있다.

연관 컨테이너의 원소는 노드에 저장되는 경우가 많아 객체 주소가 삽입으로 바뀌지 않는다. 제거된 객체의 주소는 즉시 유효성을 잃는다.

12.9.2 원시 포인터 저장

원시 포인터를 저장하면 컨테이너는 포인터 값만 보관한다. 가리키는 객체를 자동으로 삭제하지 않는다.

```
std::map<int, Item*> items;
Item potion{"Potion", 50};
items.insert({1001, &potion});
```

items.erase(1001)은 포인터 원소만 제거하고 potion을 소멸시키지 않는다.
동적 할당한 객체를 원시 포인터로 저장하면 삭제 책임을 별도로 관리해야 한다.

```
std::map<int, Item*> items;
items.insert({1001, new Item{"Potion", 50}});
```

컨테이너를 비우거나 소멸시켜도 Item 객체는 자동으로 삭제되지 않는다. 예외나 조기 반환이 발생하면 메모리 누수가 생기기 쉽다. 소유권이 필요한 경우 스마트 포인터를 사용한다.

포인터를 키로 사용하면 기본 비교와 해시는 객체 내용이 아니라 주소를 기준으로 한다.

```
std::set<Item*> itemPointers;
std::unordered_set<Item*> itemTable;
```

객체의 id나 name을 기준으로 찾으려면 해당 의미를 반영하는 비교 함수 또는 해시 함수가 필요하다.

12.9.3 unique_ptr 저장

std::unique_ptr를 저장하면 컨테이너가 객체의 단독 소유권을 가진다.

```
#include <memory>

std::map<int, std::unique_ptr<Item>> items;

items.try_emplace(
    1001,
    std::make_unique<Item>(Item{"Potion", 50}));
```

원소를 제거하거나 컨테이너가 소멸하면 unique_ptr가 객체를 자동으로 삭제한다.

```
items.erase(1001);
```

unique_ptr는 복사할 수 없으므로 컨테이너에 이동하여 넣는다.

```
auto item = std::make_unique<Item>(Item{"Ether", 80});
items.insert({1002, std::move(item)});
```

try_emplace()는 키가 이미 있을 때 이동 인수를 소비하지 않는다.

```
auto item = std::make_unique<Item>(Item{"Elixir", 100});
auto [position, inserted] = items.try_emplace(1002, std::move(item));

if (!inserted)
{
    // item은 이동되지 않고 소유권을 유지한다.
}
```

12.9.4 shared_ptr 저장

여러 위치가 객체를 공동 소유해야 한다면 std::shared_ptr를 저장할 수 있다.

```
std::map<int, std::shared_ptr<Item>> items;
auto potion = std::make_shared<Item>(Item{"Potion", 50});
items.insert({1001, potion});
```

컨테이너에서 원소를 제거해도 다른 shared_ptr가 객체를 소유하고 있으면 객체는 유지된다.

```
items.erase(1001);
std::cout << potion->name << '\n';
```

shared_ptr 복사는 참조 계수 갱신 비용이 있고 순환 소유가 생기면 자동으로 해제되지 않는다. 단독 소유가 가능하면 unique_ptr가 더 분명하다.

12.9.5 다형 객체 저장

서로 다른 파생 객체를 하나의 컨테이너에 저장하려면 기반 클래스의 스마트 포인터를 사용할 수 있다.

```
class GameObject
{
public:
    virtual ~GameObject() = default;
    virtual void Update() = 0;
};

class Player : public GameObject
{
public:
    void Update() override
    {
        std::cout << "Player update\n";
    }
};

class Monster : public GameObject
{
public:
    void Update() override
    {
        std::cout << "Monster update\n";
    }
};

std::map<int, std::unique_ptr<GameObject>> objects;

objects.try_emplace(1, std::make_unique<Player>());
objects.try_emplace(2, std::make_unique<Monster>());

for (auto& [id, object] : objects)
{
    object->Update();
```

```
}
```

기반 클래스 객체를 값으로 저장하면 객체 잘림이 발생할 수 있다.

```
// 추상 클래스이므로 저장 불가
// std::map<int, GameObject> objects;
```

추상 클래스가 아니더라도 기반 타입 값으로 파생 객체를 복사하면 파생 부분이 사라진다. 다형성에는 기반 클래스 포인터와 가상 함수가 필요하다.

12.9.6 반복자·포인터·참조의 유효성과 무효화

연관 컨테이너의 무효화 규칙은 자료 구조 선택과 안전한 코드 작성에 직접 영향을 준다. 무효화된 반복자, 포인터, 참조는 자동으로 초기화되지 않는다. 주소가 남아 있는 것처럼 보여도 다시 사용하면 프로그램의 동작은 보장되지 않는다.

std::set, std::map, std::multiset, std::multimap의 삽입은 기존 원소를 가리키는 반복자, 포인터, 참조를 무효화하지 않는다.

```
std::map<int, std::string> items{
    {1001, "Potion"},
    {1002, "Ether"}
};

auto iterator = items.find(1001);
std::string& name = iterator->second;

items.insert({1003, "Elixir"});

std::cout << iterator->second << '\n';
std::cout << name << '\n';
```

새 노드를 연결해도 기존 노드의 위치는 유지된다.
제거는 제거된 원소를 가리키는 반복자, 포인터, 참조만 무효화한다.

```
auto first = items.find(1001);
auto second = items.find(1002);

items.erase(first);

// first는 무효화됨
std::cout << second->second << '\n'; // 유효
```

std::unordered_set, std::unordered_map 계열은 재 해시 여부를 함께 판단해야 한다. 재 해시가 발생하지 않은 삽입은 기존 반복자를 유지한다. 재 해시가 발생하면 모든 반복자가 무효화된다.

```
std::unordered_map<int, std::string> items{
    {1001, "Potion"},
    {1002, "Ether"}
};

auto iterator = items.find(1001);

items.reserve(1000);

// iterator는 재 해시로 무효화될 수 있다.
```

재 해시는 기존 원소에 대한 포인터와 참조를 무효화하지 않는다.

```
std::string& name = items.at(1001);
items.reserve(2000);
std::cout << name << '\n';
```

원소가 제거되지 않았다면 name은 유효하다. 반복자와 참조의 무효화 규칙이 서로 다르다는 점이 중요하다.

비 정렬 연관 컨테이너의 제거도 제거된 원소를 가리키는 위치만 무효화한다. 다른 원소의 반복자와 참조는 유지된다.

순회 중 제거할 때는 erase()가 반환한 반복자를 사용한다.

```
for (auto iterator = items.begin(); iterator != items.end();)
{
    if (iterator->second.empty())
    {
        iterator = items.erase(iterator);
    }
    else
    {
        ++iterator;
    }
}
```

<연관 컨테이너의 무효화 규칙>

컨테이너와 연산	반복자	포인터와 참조
set, map 계열 삽입	유지	유지
set, map 계열 제거	제거된 원소만 무효	제거된 원소만 무효
unordered 계열 삽입, 재 해시 없음	유지	유지
unordered 계열 삽입, 재 해시 발생	전체 무효	유지
unordered 계열 reserve(), rehash()로 재 해시	전체 무효	유지
unordered 계열 제거	제거된 원소만 무효	제거된 원소만 무효
컨테이너 clear()	전체 무효	전체 무효
컨테이너 소멸	전체 무효	전체 무효

컨테이너 어댑터는 반복자를 제공하지 않는다. top(), front(), back()이 반환한 참조는 기반 컨테이너의 변경과 제거에 영향을 받는다. stack::pop(), queue::pop(), priority_queue::pop()을 호출한 뒤 제거된 원소의 참조를 사용해서는 안 된다.

12.10 컨테이너 선택 기준

컨테이너 이름보다 데이터의 키 관계와 처리 규칙을 먼저 판단한다.
키만 저장하고 중복을 제거해야 하면 std::set 또는 std::unordered_set을 검토한다. 키에 값을 연결해야 하면 std::map 또는 std::unordered_map이 적합하다. 같은 키를 여러 개 허용해야 하면 multi 계열을 사용할 수 있지만, 하나의 키에 값 목록을 연결하는 구조가 더 분명한 경우도 있다.

```
std::map<std::string, std::vector<Item>> itemsByCategory;
```

std::multimap<std::string, Item>과 비교해 한 키의 모든 값을 하나의 벡터로 관리할 수 있다. 값 목록의 수정 방식과 소유권에 따라 선택한다.

정렬된 순회, 최소값과 최대값, 구간 검색이 필요하면 정렬 연관 컨테이너가 적합하다. 정확한 키 검색이 대부분이고 정렬 순서가 필요하지 않으면 비 정렬 연관 컨테이너의 평균 상수 시간 탐색을 활용할 수 있다.

처리 규칙이 후입선출이면 std::stack, 선입선출이면 std::queue, 우선순위 기반이면 std::priority_queue를 사용한다. 중간 원소 순회와 임의 수정이 필요하다면 어댑터가 아니라 기반 컨테이너를 직접 사용한다.

<연관 컨테이너와 컨테이너 어댑터 선택 비교>

요구 사항	적합한 타입
중복 없는 정렬 키	std::set
중복을 허용하는 정렬 키	std::multiset
정렬된 키와 값	std::map
같은 정렬 키에 여러 값	std::multimap
중복 없는 빠른 키 검색	std::unordered_set
빠른 키와 값 검색	std::unordered_map

같은 해시 키에 여러 원소	std::unordered_multiset, std::unordered_multimap
최근 원소부터 처리	std::stack
도착 순서대로 처리	std::queue
우선순위가 높은 원소부터 처리	std::priority_queue

선택 과정에서 확인할 항목은 키 정렬 여부, 중복 허용 여부, 범위 검색, 평균과 최악 탐색 비용, 메모리 사용, 반복자 안정성, 객체 소유권이다. 실제 성능이 중요하면 예상 데이터 크기와 연산 패턴으로 측정한다.

12.11 학습 정리

연관 컨테이너는 키를 기준으로 원소를 저장하고 찾는다. std::set은 키만 저장하고 std::map은 키와 값을 저장한다. multi 계열은 동등한 키를 여러 개 허용한다.

정렬 연관 컨테이너는 비교 함수에 따라 키를 정렬하며 삽입, 탐색, 제거에 로그 시간이 든다. 비교 함수는 엄격 약순서를 만족해야 하며, 두 방향 비교가 모두 거짓이면 동등한 키로 판단한다. 저장된 키는 정렬 규칙을 보호하기 위해 직접 수정할 수 없다.

비 정렬 연관 컨테이너는 해시값으로 버킷을 선택하고 동등 비교로 키를 확인한다. 평균 탐색은 상수 시간이지만 충돌이 집중되면 최악의 경우 선형 시간이 될 수 있다. 동등한 키는 반드시 같은 해시값을 가져야 한다.

std::map::operator[]는 키가 없으면 기본값을 삽입한다. 삽입 여부만 확인하려면 contains()나 find()를 사용하고, 값 생성 비용을 피하려면 try_emplace(), 기존 값까지 교체하려면 insert_or_assign()을 사용할 수 있다.

std::stack, std::queue, std::priority_queue는 기반 컨테이너의 인터페이스를 제한한 어댑터이다. 각각 후입선출, 선입선출, 우선순위 기반 처리를 제공하며 중간 원소를 순회하는 반복자는 제공하지 않는다.

컨테이너에 값 객체를 저장하면 컨테이너가 수명을 관리한다. 원시 포인터는 객체를 소유하지 않으며, unique_ptr는 단독 소유, shared_ptr는 공동 소유를 표현한다. 다형 객체는 기반 클래스의 스마트 포인터로 저장할 수 있다.

정렬 연관 컨테이너는 삽입으로 기존 반복자와 참조가 무효화되지 않는다. 비 정렬 연관 컨테이너는 재 해시가 발생하면 반복자가 모두 무효화되지만 기존 원소의 포인터와 참조는 유지된다. 제거된 원소를 가리키는 위치는 모든 컨테이너에서 사용할 수 없다.

12.12 학습 과제

과제 **1.** 중복 없는 사용자 이름 관리
사용자 이름을 입력받아 std::set<std::string>에 저장한다. 이미 등록된 이름이면 중복 메시지를 출력하고, 입력이 끝나면 이름을 정렬된 순서로 출력한다.

과제 **2.** 아이템 수량 집계
아이템 이름 목록을 std::vector<std::string>로 제공받아 std::map<std::string, int>로 각 이름의 개수를 집계한다. 구조적 바인딩을 사용해 이름과 수량을 출력한다.

과제 **3. map** 삽입 함수 비교
같은 키를 대상으로 insert(), try_emplace(), insert_or_assign(), operator[]를 호출한다. 키가 없을 때와 있을 때 값이 어떻게 달라지는지 출력한다.

과제 **4.** 점수 범위 검색
학생 점수를 std::multiset<int>에 저장하고 lower_bound()와 upper_bound()를 이용해 80점 이상 90점 이하의 점수만 출력한다. 해당 범위의 원소 수도 계산한다.

과제 **5.** 사용자 정의 키 해시
서버 번호와 사용자 번호를 멤버로 가진 PlayerKey 구조체를 정의한다. operator==와 해시 함수 객체를 작성하고 std::unordered_map<PlayerKey, std::string, PlayerKeyHash>에 사용자 이름을 저장한다.

과제 **6.** 해시 테이블 상태 확인
std::unordered_set<int>에 원소를 단계적으로 삽입하면서 size(), bucket_count(), load_factor()를 출력한다. reserve() 호출 전후의 버킷 수와 반복자 유효성을 비교한다.

과제 **7.** 명령 실행 취소
문자열 명령을 std::stack<std::string>에 저장하고 사용자가 실행 취소를 요청할 때 가장 최근 명령을 제거한다. 스택이 비어 있을 때는 실행 취소할 명령이 없다는 메시지를 출력한다.

과제 **8.** 작업 대기열
작업 번호와 설명을 가진 Job 구조체를 정의하고 std::queue<Job>에 저장한다. 입력 순서대로 작업을 꺼내 처리 결과를 출력한다.

과제 **9.** 우선순위 작업 처리

작업 번호, 우선순위, 등록 순서를 가진 Task 구조체를 정의한다. 우선순위가 높은 작업을 먼저 처리하고 우선순위가 같으면 먼저 등록된 작업을 처리하도록 std::priority_queue의 비교 함수 객체를 작성한다.

과제 **10.** 다형 객체 관리
GameObject 추상 클래스와 Player, Monster 파생 클래스를 정의한다. std::map<int, std::unique_ptr<GameObject>>에 객체를 저장하고 식별자 순서로 Update()를 호출한다.

11. 반복자 무효화 확인
std::map과 std::unordered_map에서 원소 반복자와 값 참조를 저장한다. 원소 삽입과 reserve()를 수행한 뒤 어떤 위치가 유지되고 어떤 위치가 무효화되는지 주석으로 설명한다. 무효화된 반복자를 실제로 역참조하지 않는다.

12. 컨테이너 선택 설명
제시한 상황마다 적절한 컨테이너를 선택하고 이유를 작성한다.

- 회원 번호로 회원 정보를 빠르게 검색한다.
- 점수를 내림차순으로 유지하며 같은 점수를 허용한다.
- 네트워크 메시지를 도착 순서대로 처리한다.
- 예약 작업을 우선순위와 등록 순서에 따라 처리한다.
- 객체 이름을 중복 없이 저장하고 사전순으로 출력한다.

13. 반복자, 함수 객체, 람다와 알고리즘

컨테이너는 원소를 저장하고 수명을 관리한다. 저장된 원소를 검색하거나 정렬하고, 조건에 맞는 원소를 세거나 변환하는 작업은 알고리즘이 담당한다. 표준 알고리즘은 특정 컨테이너 타입을 직접 받지 않고 반복자 범위를 받는다.

```
#include <algorithm>
#include <iostream>
#include <vector>

int main()
{
    std::vector<int> values{40, 10, 30, 20};

    std::sort(values.begin(), values.end());

    for (int value : values)
    {
        std::cout << value << ' ';
    }
}
```

std::sort()는 std::vector의 내부 저장 방식을 알 필요가 없다. 첫 원소와 끝 위치를 나타내는 반복자를 받아 범위를 정렬한다. 알고리즘에 비교 함수나 판정 함수를 전달하면 정렬 기준과 검색 조건도 프로그램의 목적에 맞게 바꿀 수 있다.

```
std::sort(
    values.begin(),
    values.end(),
    [](int left, int right)
    {
        return left > right;
    });
```

람다 표현식은 알고리즘에 짧은 동작을 전달하는 데 적합하지만, 람다만 알고서는 표준 알고리즘을 안전하게 사용하기 어렵다. 반복자 범주의 차이, 반 개방 구간 (Half-open intervals), 정렬 사전 조건, 반복자 무효화, 람다 캡처의 수명을 함께 이해해야 한다.
반복자가 범위와 위치를 표현하는 방식을 정리하고, 함수 포인터와 함수 객체, 람다 표현식을 호출 가능한 객체라는 관점에서 연결한다. 검색, 정렬, 복사, 변환, 제거, 수치 알고리즘을 적용한 뒤 C++20 Ranges가 범위와 알고리즘을 결합하는 방식까지 다룬다.

13.1 반복자의 개념

반복자(iterator)는 범위 안의 원소 위치를 나타내는 객체이다. 반복자는 포인터와 비슷한 연산을 제공하지만 반드시 원시 포인터일 필요는 없다. 컨테이너는 저장 구조에 맞는 반복자 타입을 제공하고, 알고리즘은 반복자가 지원하는 연산만 사용한다.

13.1.1 반복자와 포인터

배열의 원소는 포인터로 순회할 수 있다.

```
int values[]{10, 20, 30};

int* first = values;
int* last = values + 3;

for (int* pointer = first; pointer != last; ++pointer)
{
    std::cout << *pointer << ' ';
}
```

포인터는 현재 위치를 가리키고, *로 원소를 읽으며, ++로 뒤 원소로 이동한다. 반복자도 같은 형태의 기본 연산을 제공한다.

```
std::vector<int> values{10, 20, 30};

auto first = values.begin();
auto last = values.end();

for (auto iterator = first; iterator != last; ++iterator)
{
    std::cout << *iterator << ' ';
```

```
}
```

std::vector<int>::iterator는 벡터 원소를 가리키는 반복자 타입이다. 구현에서는 포인터와 비슷한 형태일 수 있지만 포인터라고 가정해서는 안 된다. std::list<int>::iterator는 연결 리스트의 노드 사이를 이동해야 하므로 원시 포인터 산술을 제공하지 않는다.

반복자는 원소를 직접 소유하지 않는다. 컨테이너가 소멸하거나 컨테이너 변경으로 반복자가 무효화되면 반복자만 남아 있어도 원소에 접근할 수 없다.

```
std::vector<int>::iterator position;

{
    std::vector<int> values{10, 20, 30};
    position = values.begin();
}

// values가 소멸했으므로 position은 사용할 수 없다.
```

반복자 타입은 보통 길고 컨테이너에 종속되므로 auto를 사용하면 표현이 간결해진다.

```
auto position = values.begin();
```

13.1.2 반복 구간과 반 개방 (Half-open intervals) 구간

표준 알고리즘은 범위를 [first, last) 형태의 반 개방 구간으로 표현한다. first가 가리키는 원소는 포함하고 last가 가리키는 위치는 포함하지 않는다.

```
[10] [20] [30] [끝 위치]
  ↑              ↑
first            last
```

end()는 마지막 원소를 가리키지 않는다. 마지막 원소 바로 뒤의 위치를 나타내는 센티널 역할을 한다. end()를 역참조하면 유효한 원소가 없으므로 프로그램의 동작은 보장되지 않는다.

```
std::vector<int> values{10, 20, 30};

auto last = values.end();
// std::cout << *last; // 잘못된 접근
```

반 개방 구간은 빈 범위를 하나의 조건으로 표현할 수 있다.

```
if (values.begin() == values.end())
{
    std::cout << "empty\n";
}
```

부분 범위도 같은 규칙으로 표현한다.

```
std::vector<int> values{10, 20, 30, 40, 50};

auto first = values.begin() + 1;
auto last = values.begin() + 4;

std::sort(first, last);
```

정렬 대상은 인덱스 1, 2, 3에 있는 세 원소이다. last가 가리키는 인덱스 4의 원소는 범위에 포함되지 않는다.
반 개방 구간에서는 인접한 두 범위를 겹치지 않게 나누기 쉽다.
[first, middle) [middle, last)

첫 범위의 끝과 둘째 범위의 시작이 같은 반복자를 사용하므로 경계 원소를 중복 처리하지 않는다.

13.1.3 begin, end와 범위

컨테이너의 begin()과 end()는 수정 가능한 반복자를 반환한다.

```
std::vector<int> values{10, 20, 30};

for (auto iterator = values.begin(); iterator != values.end();
++iterator)
{
    *iterator *= 2;
}
```

상수 컨테이너의 begin()과 end()는 원소를 수정할 수 없는 반복자를 반환한다.

```
const std::vector<int> values{10, 20, 30};

auto iterator = values.begin();
// *iterator = 100; // 오류
```

cbegin()과 cend()는 컨테이너 자체가 상수가 아니어도 상수 반복자를 반환한다.

```
std::vector<int> values{10, 20, 30};
auto iterator = values.cbegin();
// *iterator = 100; // 오류
```

배열과 사용자 정의 범위에는 std::begin()과 std::end()를 사용할 수 있다.

```
#include <iterator>
int values[]{30, 10, 20};
std::sort(std::begin(values), std::end(values));
```

범위 기반 for문도 시작 위치와 끝 위치를 얻어 반복하는 구조로 처리된다.

```
for (int value : values)
{
    std::cout << value << ' ';
}
```

배열과 표준 컨테이너는 범위 기반 for문에 바로 사용할 수 있다. 사용자 정의 타입도 begin()과 end()를 제공하면 범위로 사용할 수 있다.

```
class NumberRange
{
public:
    NumberRange(int* first, int* last)
        : first(first), last(last)
    {
    }

    int* begin()
    {
        return first;
    }

    int* end()
    {
        return last;
    }

private:
    int* first;
    int* last;
};
```

```
int values[]{10, 20, 30};
NumberRange range{values, values + 3};

for (int value : range)
{
    std::cout << value << ' ';
```

```
}
```

13.1.4 const_iterator와 반복자의 const

const_iterator는 반복자가 가리키는 원소를 수정하지 못하게 한다.

```
std::vector<int> values{10, 20, 30};
std::vector<int>::const_iterator iterator = values.cbegin();

++iterator;
// *iterator = 100; // 오류
```

반복자 자체는 이동할 수 있지만 역참조한 원소는 수정할 수 없다.
반복자 객체에 const를 붙이면 의미가 달라진다.

```
std::vector<int>::iterator const iterator = values.begin();

*iterator = 100;
// ++iterator; // 오류
```

반복자 자체의 위치는 바꿀 수 없지만 가리키는 원소는 수정할 수 있다. 포인터의 상수 한정과 같은 구분이다.

선언	반복자 이동	원소 수정
iterator	가능	가능
const_iterator	가능	불가
iterator const	불가	가능
const_iterator const	불가	불가

읽기 전용 범위를 순회할 때는 cbegin()과 cend()를 사용하거나 컨테이너를 const 참조로 받는 방법이 분명하다.

```
void PrintValues(const std::vector<int>& values)
{
    for (auto iterator = values.begin(); iterator != 
values.end(); ++iterator)
    {
        std::cout << *iterator << ' ';
    }
}
```

13.2 반복자 범주

반복자는 모두 같은 능력을 제공하지 않는다. 알고리즘은 필요한 이동과 접근 연산을 반복자 범주로 표현한다. 요구보다 약한 반복자를 전달하면 컴파일 오류가 발생하거나 해당 알고리즘을 사용할 수 없다.

13.2.1 입력 반복자와 단일 순회

입력 반복자(input iterator)는 원소를 읽으며 앞쪽으로 한 번 순회하는 데 사용한다. 대표적인 예는 입력 스트림 반복자이다.

```
#include <iterator>
std::istream_iterator<int> first{std::cin};
std::istream_iterator<int> last;

for (; first != last; ++first)
{
    std::cout << *first << '\n';
}
```

스트림에서 값을 읽으면 입력 위치가 앞으로 진행된다. 이미 소비한 입력을 반복자 복사본으로 다시 읽을 수 있다고 가정해서는 안 된다. 입력 반복자는 단일 순회(single-pass) 성질을 가진다.

std::find()처럼 앞쪽으로 읽으며 한 번 검사하는 알고리즘은 입력 반복자로 수행할 수 있다.

```
std::istream_iterator<int> found = std::find(first, last, 100);
```

13.2.2 출력 반복자와 단일 순회

출력 반복자(output iterator)는 값을 기록하며 앞쪽으로 진행한다. 역참조 결과는 일반적인 원소 참조와 다를 수 있으며 값을 읽는 용도로 사용할 수 없다.

```
std::ostream_iterator<int> output{std::cout, " "};

*output = 10;
++output;
*output = 20;
```

출력 반복자도 단일 순회를 전제로 한다. std::copy()의 목적지에 출력 반복자를 전달하면 원소를 스트림이나 컨테이너에 기록할 수 있다.

```
std::vector<int> values{10, 20, 30};
std::copy(values.begin(), values.end(), output);
```

13.2.3 순방향 반복자와 다중 순회

순방향 반복자(forward iterator)는 앞쪽으로 이동하며 같은 범위를 여러 번 순회할 수 있다. 여러 반복자 복사본이 같은 위치를 안정적으로 나타내는 다중 순회(multi-pass) 보장을 제공한다.

std::forward_list와 비정렬 연관 컨테이너의 반복자가 대표적인 순방향 반복자이다.

```
std::forward_list<int> values{10, 20, 30};

auto first = values.begin();
auto copy = first;

++first;
std::cout << *copy << '\n';
```

copy는 여전히 첫 원소를 가리킨다. 컨테이너 변경으로 무효화되지 않는 한 같은 범위를 반복해서 순회할 수 있다.

13.2.4 양방향 반복자

양방향 반복자(bidirectional iterator)는 ++와 --를 모두 지원한다. std::list, std::set, std::map 계열이 양방향 반복자를 제공한다.

```
std::list<int> values{10, 20, 30};

auto iterator = values.end();
--iterator;

std::cout << *iterator << '\n';
```

end() 자체는 역참조할 수 없지만 비어 있지 않은 범위에서 한 번 감소시키면 마지막 원소를 가리킨다.

양방향 반복자는 역방향 반복자와 std::reverse() 같은 알고리즘에 사용할 수 있다.

```
std::reverse(values.begin(), values.end());
```

13.2.5 임의 접근 반복자

임의 접근 반복자(random access iterator)는 일정 거리만큼 상수 시간에 이동하고, 반복자 사이의 거리를 상수 시간에 계산할 수 있다.

```
std::vector<int> values{10, 20, 30, 40};

auto iterator = values.begin();
iterator += 2;
```

```
std::cout << iterator[1] << '\n';
```

지원하는 대표 연산은 +, -, +=, -=, [], <, <=, >, >=이다. std::vector, std::deque, std::array의 반복자가 이 범주에 속한다.
std::sort()는 임의 접근 반복자를 요구한다. 정렬 과정에서 떨어진 원소 사이를 반복해서 이동해야 하기 때문이다.

```
std::sort(values.begin(), values.end());
```

std::list는 양방향 반복자만 제공하므로 std::sort()에 전달할 수 없고 list::sort()를 사용한다.

13.2.6 연속 반복자

연속 반복자(contiguous iterator)는 임의 접근 반복자의 기능에 더해 원소가 메모리에 연속되어 있음을 보장한다. std::array, std::vector, std::string의 반복자가 대표적이다.

```
std::vector<int> values{10, 20, 30};

auto iterator = values.begin();
int* pointer = std::to_address(iterator);

std::cout << pointer[1] << '\n';
```

연속 반복자에서는 반복자 i와 정수 n에 대해 std::to_address(i + n)이 std::to_address(i) + n과 같은 원소를 가리킨다.

std::deque는 임의 접근 반복자를 제공하지만 원소 전체가 하나의 연속된 메모리 블록에 저장되지는 않는다. 임의 접근 가능 여부와 연속 저장 여부는 서로 다른 성질이다.

<반복자 범주와 대표 기능>

범주	읽기	쓰기	뒤로 이동	임의 이동	연속 메모리	대표 예
입력	가능	제한적	불가	불가	보장 없음	istream_iterator
출력	불가	가능	불가	불가	보장 없음	ostream_iterator
순방향	가능	가능	불가	불가	보장 없음	forward_list
양방향	가능	가능	가능	불가	보장 없음	list, map
임의 접근	가능	가능	가능	가능	보장 없음	deque
연속	가능	가능	가능	가능	보장	vector, array

13.3 반복자 도구

<iterator> 헤더는 반복자의 이동, 거리 계산, 역방향 순회, 삽입, 스트림 연결을 위한 도구를 제공한다. 반복자 범주에 따라 같은 함수의 비용이 달라질 수 있다.

13.3.1 advance, next와 prev

std::advance()는 전달한 반복자를 지정한 거리만큼 이동시킨다.

```
#include <iterator>
#include <list>

std::list<int> values{10, 20, 30, 40};

auto iterator = values.begin();
std::advance(iterator, 2);

std::cout << *iterator << '\n';
```

std::list 반복자는 iterator + 2를 지원하지 않지만 std::advance()는 반복자 범주에 맞게 ++를 반복하여 이동한다.
std::next()는 원래 반복자를 바꾸지 않고 앞쪽으로 이동한 반복자를 반환한다.

```
auto first = values.begin();
auto third = std::next(first, 2);

std::cout << *first << '\n';
std::cout << *third << '\n';
```

std::prev()는 뒤쪽으로 이동한 반복자를 반환한다. 양방향 반복자 이상에서 사용할 수 있다.

```
auto last = values.end();
auto finalElement = std::prev(last);

std::cout << *finalElement << '\n';
```

임의 접근 반복자에서는 거리와 관계없이 상수 시간에 이동할 수 있다. 리스트 반복자는 이동 거리만큼 순회하므로 선형 시간이 든다.

13.3.2 distance

std::distance(first, last)는 두 반복자 사이의 원소 수를 반환한다.

```
std::list<int> values{10, 20, 30, 40};

auto count = std::distance(values.begin(), values.end());
std::cout << count << '\n';
```

임의 접근 반복자에서는 반복자 뺄셈으로 상수 시간에 계산한다. 입력, 순방향, 양방향 반복자에서는 first에서 last까지 이동해야 하므로 거리에 비례한 시간이 든다.

```
std::vector<int> vectorValues(1'000'000);
std::list<int> listValues(1'000'000);

std::distance(vectorValues.begin(), vectorValues.end()); // 상수 시간
std::distance(listValues.begin(), listValues.end());     // 선형 시간
```

std::distance()를 반복문 안에서 리스트에 계속 호출하면 불필요한 반복 순회가 생길 수 있다. 컨테이너가 size()를 제공한다면 전체 원소 수는 size()로 확인하는 편이 적합하다.

13.3.3 역방향 반복자

역방향 반복자는 원소를 뒤에서 앞으로 순회한다. rbegin()은 마지막 원소를 가리키는 역방향 반복자를 반환하고 rend()는 첫 원소 앞의 끝 위치를 나타낸다.

```
std::vector<int> values{10, 20, 30};

for (auto iterator = values.rbegin();
     iterator != values.rend(); ++iterator)
{
    std::cout << *iterator << ' ';
}
```

역방향 반복자에서 ++는 원래 범위의 앞쪽으로 이동한다.

역방향 반복자의 base()는 대응하는 정방향 반복자를 반환한다. 역방향 반복자가 가리키는 원소와 base()가 가리키는 위치는 하나 차이가 난다.

```
[10] [20] [30] [끝 위치]
          ↑      ↑
       rbegin  rbegin.base()
```

마지막으로 발견한 원소를 지울 때 이 차이를 고려해야 한다.

```
std::vector<int> values{10, 20, 30, 20};

auto reversePosition = std::find(values.rbegin(), values.rend(), 20);
```

```
if (reversePosition != values.rend())
{
    values.erase(std::prev(reversePosition.base()));
}
```

reversePosition.base()는 발견한 원소 바로 뒤를 가리키므로 std::prev()로 한 칸 앞의 정방향 반복자를 얻는다.

13.3.4 삽입 반복자

std::copy()는 목적지에 충분한 저장 공간이 있다고 가정한다.

```
std::vector<int> source{10, 20, 30};
std::vector<int> destination(3);

std::copy(source.begin(), source.end(), destination.begin());
```

빈 벡터의 begin()을 목적지로 전달하면 쓸 원소가 없으므로 올바르지 않다.

```
std::vector<int> destination;
// std::copy(source.begin(), source.end(), destination.begin()); // 오류 가능
```

삽입 반복자는 대입 연산을 컨테이너 삽입 함수 호출로 바꾼다. std::back_inserter()는 push_back()을 사용한다.

```
std::vector<int> destination;

std::copy(
    source.begin(),
    source.end(),
    std::back_inserter(destination));
```

std::front_inserter()는 push_front()를 사용하고, std::inserter()는 지정한 위치에 insert()를 호출한다.

```
std::deque<int> frontValues;
std::copy(source.begin(), source.end(),
std::front_inserter(frontValues));

std::set<int> uniqueValues;
std::copy(source.begin(), source.end(),
std::inserter(uniqueValues, uniqueValues.end()));
```

앞쪽 삽입은 입력 순서를 뒤집을 수 있다. source가 10, 20, 30이면 frontValues에는 30, 20, 10 순서로 저장된다.

13.3.5 스트림 반복자

std::istream_iterator<T>는 입력 스트림에서 T형 값을 읽는다.

```
#include <iterator>
#include <vector>

std::istream_iterator<int> first{std::cin};
std::istream_iterator<int> last;

std::vector<int> values(first, last);
```

기본 생성한 입력 스트림 반복자는 입력 종료 위치를 나타낸다. 입력 실패나 파일 끝에 도달하면 현재 반복자가 끝 반복자와 같아진다.
std::ostream_iterator<T>는 값을 출력 스트림에 기록한다.

```
std::ostream_iterator<int> output{std::cout, ", "};
std::copy(values.begin(), values.end(), output);
```

두 번째 인수는 각 값 뒤에 출력할 구분 문자열이다. 마지막 값 뒤에도 구분 문자열이 출력된다.
스트림 반복자를 사용하면 입력, 알고리즘, 출력을 하나의 범위 처리 흐름으로 연결할 수 있다.

```
std::transform(
```

```
    std::istream_iterator<int>{std::cin},
    std::istream_iterator<int>{},
    std::ostream_iterator<int>{std::cout, " "},
    [](int value)
    {
        return value * value;
    });
```

입력된 정수를 읽어 제곱한 값을 바로 출력한다. 입력 반복자는 단일 순회이므로 이미 읽은 값을 다시 사용할 수 없다.

13.3.6 반복 중 변경과 반복자 무효화

반복자 무효화(iterator invalidation)는 반복자가 더 이상 유효한 원소 위치를 나타내지 못하게 되는 상태이다. 무효화된 반복자를 비교하거나 이동하거나 역참조하면 프로그램의 동작은 보장되지 않는다. 포인터나 참조가 남아 있는 것처럼 보여도 안전하다는 뜻은 아니다.

범위 기반 for문에서 벡터에 원소를 추가하면 내부에서 사용 중인 반복자가 재할당으로 무효화될 수 있다.

```
std::vector<int> values{1, 2, 3};

// 잘못된 코드
for (int value : values)
{
    if (value == 2)
    {
        values.push_back(20);
    }
}
```

push_back()으로 재할당이 발생하면 현재 반복자와 끝 반복자가 모두 무효화된다. 재할당이 없더라도 반복 도중 범위의 끝이 바뀌어 의도한 순회 규칙이 깨질 수 있다.

순회 중 제거는 erase()가 반환한 반복자를 사용한다.

```
std::vector<int> values{1, 2, 3, 4, 5, 6};

for (auto iterator = values.begin(); iterator != values.end();)
{
    if (*iterator % 2 == 0)
    {
        iterator = values.erase(iterator);
    }
    else
    {
        ++iterator;
    }
}
```

vector::erase()는 제거된 원소 뒤의 원소를 앞으로 이동시키고, 제거 위치에 새로 놓인 원소를 가리키는 반복자를 반환한다. 제거 후 기존 반복자를 증가시키면 원소를 건너뛰거나 무효화된 반복자를 사용할 수 있다.

리스트에서도 제거한 원소를 가리키는 반복자는 무효화된다. 다른 원소의 반복자는 유지되지만 반환값을 사용하는 형태가 일관되고 안전하다.

```
std::list<int> values{1, 2, 3, 4};

for (auto iterator = values.begin(); iterator != values.end();)
{
    if (*iterator % 2 == 0)
    {
        iterator = values.erase(iterator);
    }
    else
    {
        ++iterator;
    }
}
```

알고리즘을 실행하는 동안 전달한 범위도 유효해야 한다. 판정 함수 안에서 같은 컨테이너를 변경하면 알고리즘이 보관한 반복자가 무효화될 수 있다.
std::vector<int> values{1, 2, 3};

```
// 판정 함수에서 values를 변경해서는 안 된다.
std::find_if(
    values.begin(),
    values.end(),
    [&values](int value)
    {
        // values.push_back(value); // 반복자 무효화 가능
        return value == 2;
    });
```

컨테이너 변경이 필요한 작업은 탐색과 변경 단계를 분리하거나, 변경을 위해 설계된 알고리즘과 컨테이너 멤버 함수를 사용한다.

반복자 무효화 규칙은 컨테이너와 연산마다 다르다. 벡터 재할당은 모든 반복자·포인터·참조를 무효화하고, 정렬 연관 컨테이너 삽입은 기존 위치를 유지한다. 비정렬 연관 컨테이너는 재해시가 발생하면 반복자가 모두 무효화된다. 코드를 작성할 때는 컨테이너 이름만 보지 말고 수행하는 연산까지 확인해야 한다.

13.4 Callable과 함수 유틸리티

앞에서 operator() 연산자를 정의한 객체는 함수처럼 호출할 수 있으며, 이를 함수 객체(function object)라고 했다. C++에서는 함수 객체뿐 아니라 일반 함수, 함수 포인터, 람다 표현식과 멤버 함수 포인터 등 함수 호출에 사용할 수 있는 여러 형태의 대상을 더 포괄적으로 다룬다. 이들을 **Callable(**호출 가능한 대상**)** 이라고 한다.

Callable은 간단히 말해 적절한 객체와 인수를 제공했을 때 호출할 수 있는 대상이다. 형태와 타입은 서로 다르지만 표준 알고리즘의 비교와 판정, 콜백, 스레드의 실행 함수처럼 함수를 전달받는 여러 기능에 사용할 수 있다.

<functional> 헤더에는 Callable을 저장하거나 호출하고, 기존 호출 형태를 다른 형태로 바꾸기 위한 기능이 정의되어 있다. std::function, std::invoke, std::mem_fn, std::bind가 대표적이다.

```
일반 함수 ──────┐
함수 포인터 ────┤
함수 객체 ──────┤
람다 표현식 ────┼──── Callable
멤버 함수 포인터 ┘
                │
                ├── 저장 : std::function
                ├── 호출 : std::invoke
                └── 변환 : std::mem_fn, std::bind
```

13.4.1 함수 포인터

일반 함수의 이름은 호환되는 함수 포인터로 변환할 수 있다. 함수 포인터 타입에는 반환 타입과 매개변수 타입이 포함된다.

```
#include <algorithm>
#include <iostream>
#include <vector>

bool IsEven(int value)
{
    return value % 2 == 0;
}

int main()
{
    // 함수 포인터.
    using Predicate = bool (*)(int);
    Predicate predicate = IsEven;
    std::cout << std::boolalpha
              << predicate(10) << '\n';

    std::vector<int> values{1, 2, 3, 4, 5};

    // 표준 알고리즘에 함수 포인터 전달.
    auto position = std::find_if(
```

```
        values.begin(),
        values.end(),
        predicate);
    if (position != values.end())
    {
        std::cout << *position << '\n';
    }
}
```

함수 포인터는 함수의 주소만 저장하므로 구조가 단순하고 C 인터페이스와 연결하기 쉽다. 그러나 함수 포인터 자체에는 호출에 필요한 상태를 저장할 수 없다.

13.4.2 함수 객체와 상태

함수 객체는 구조체나 클래스를 기반으로 생성된 일반 객체이므로, 단순히 함수의 주소만 저장하는 함수 포인터와 달리 내부에 멤버 변수를 선언하여 자신만의 상태(state)를 저장하고 유지할 수 있다.

```
// IsGreaterThan 클래스 정의.
class IsGreaterThan
{
public:
    explicit IsGreaterThan(int limit) : limit(limit)
    {
    }
    // 함수 호출 연산자.
    bool operator()(int value) const
    {
        return value > limit;
    }
private:
    int limit;
};
```

```
#include <algorithm>
#include <iostream>
#include <vector>

// <IsGreaterThan 클래스 정의...>

int main()
{
  std::vector<int> values{10, 25, 40, 5, 66, 75};
  // 함수 객체 생성.
  auto gt = IsGreaterThan{20};
  // 함수 객체 전달.
  auto count = std::count_if(values.begin(), values.end(), gt);
  std::cout << count << '\n';

  // 임시 함수 객체 직접 전달.
  count = std::count_if(values.begin(), values.end(), IsGreaterThan{40});
  std::cout << count << '\n';
}
```

IsGreaterThan{20}은 비교에 사용할 기준값 20을 객체 내부에 저장한다. std::count_if()는 각 원소를 이 함수 객체에 전달하고, operator()가 true를 반환하는 원소의 개수를 센다. 이처럼 함수 객체는 호출에 필요한 상태와 동작을 하나의 객체로 전달할 수 있다.

13.4.3 표준 함수 객체

표준 라이브러리도 std::less<>, std::greater<>, std::plus<>처럼 자주 사용하는 연산을 함수 객체로 제공한다.
<functional> 헤더는 자주 사용하는 연산을 함수 객체로 제공한다.

함수 객체	연산	함수 객체	연산
std::plus<T>	left + right	std::less<T>	left < right
std::minus<T>	left - right	std::greater<T>	left > right

std::multiplies<T>	left * right		std::logical_and<T>	논리곱
std::divides<T>	left / right		std::logical_or<T>	논리합
std::equal_to<T>	left == right		std::negate<T>	단항 부호 반전

내림차순 정렬에는 std::greater<>를 사용할 수 있다.

```
std::vector<int> values{30, 10, 20};
std::sort(values.begin(), values.end(), std::greater<>{});
```

빈 꺾쇠의 투명 함수 객체는 호출 인수 타입을 직접 사용한다. std::greater<int>처럼 타입을 고정하지 않아도 된다.

```
std::plus<> add;

std::cout << add(10, 20) << '\n';
std::cout << add(1.5, 2.5) << '\n';
```

연산 의미가 짧고 표준 함수 객체 이름으로 충분히 드러난다면 람다보다 간결할 수 있다.

표준 라이브러리도 std::less<>, std::greater<>, std::plus<>처럼 자주 사용하는 연산을 함수 객체로 제공한다. 예를 들어 std::greater<>는 내림차순 정렬의 비교 함수로 사용할 수 있다.

```
std::sort(values.begin(), values.end(), std::greater<>{});
```

함수 포인터와 함수 객체는 모두 Callable이지만 저장되는 정보와 타입의 성격이 다르다. 함수 포인터는 하나의 함수 주소를 저장하고, 함수 객체는 자신의 타입과 상태를 가진다. 뒤에서 배우는 람다 표현식도 고유한 함수 객체를 생성하므로 같은 Callable의 범주에서 사용할 수 있다.

13.4.4 std::function과 Callable 저장

함수 포인터와 함수 객체는 같은 형태로 호출할 수 있어도 타입은 서로 다르다.

```
bool (*first)(int) = IsEven;
IsGreaterThan second{20};
```

first(10)과 second(10)은 모두 bool 결과를 반환하지만 first는 함수 포인터이고 second는 IsGreaterThan 타입의 객체이다. 서로 다른 타입의 Callable을 하나의 변수나 컨테이너에 저장해야 한다면 공통된 저장 타입이 필요하다.

std::function<Return(Args...)>은 지정한 호출 형식과 호환되는 Callable을 저장하는 함수 래퍼(function wrapper)이다.

```
#include <functional>
#include <iostream>

// <IsEven 함수 정의...>

// <IsGreaterThan 클래스 정의...>

int main()
{
    std::function<bool(int)> predicate = IsEven;
    std::cout << std::boolalpha
              << predicate(10) << '\n';

    predicate = IsGreaterThan{20};
    std::cout << predicate(30) << '\n';
}
```

std::function<bool(int)>은 bool(int) 형식으로 호출할 수 있는 대상을 저장한다. 일반 함수는 함수 포인터 형태로 저장할 수 있고, 함수 객체도 같은 호출 형식에 맞으면 저장할 수 있다. 13.5에서 배우는 람다 표현식도 호출 형식이 맞으면 같은 std::function에 저장할 수 있다.

```
함수 포인터 ──────┐
함수 객체   ──────┼
람다 표현식 ──────┘
```

```
                    ↓
        std::function<bool(int)>
                    ↓
          predicate(value)
```

저장되는 Callable의 실제 타입은 서로 다르지만 std::function을 사용하는 쪽에서는 구체 타입을 알 필요가 없다. std::function은 호출에 필요한 공통 형식만 외부에 제공하고 저장된 대상의 구체 타입을 감춘다. 이를 **type erasure(**타입 소거**)** 라고 한다.

호출 대상이 없는 std::function은 빈 상태이다. 조건식에서 호출 대상이 저장되어 있는지 확인할 수 있다.

```cpp
#include <functional>
#include <iostream>

int main()
{
    std::function<void()> callback;

    if (callback)
    {
        callback();
    }
    else
    {
        std::cout << "empty callback\n";
    }
}
```

빈 std::function을 직접 호출하면 std::bad_function_call 예외가 발생한다. 선택적인 콜백이라면 호출 전에 상태를 먼저 확인한다.

std::function은 서로 다른 Callable을 하나의 타입으로 관리할 수 있는 대신, 구체 타입을 직접 호출하는 방식보다 비용이 추가될 수 있다. 호출에는 타입 소거된 대상을 통한 간접 호출이 필요하고, 저장 대상의 크기와 구현에 따라 동적 메모리 할당이 발생할 수도 있다. 복사하면 내부에 저장된 호출 대상도 함께 복사된다.

C++20의 std::function에 저장하는 호출 대상은 복사 가능해야 한다. 호출 대상의 구체 타입을 컴파일 시점에 유지할 수 있고 하나의 공통 저장 타입이 필요하지 않다면 auto나 함수 템플릿을 사용하는 방법도 있다.

```cpp
template <typename Callable>
void Execute(int value, Callable callable)
{
    callable(value);
}
```

std::function은 콜백을 멤버 변수나 컨테이너에 저장하거나 서로 다른 Callable을 하나의 인터페이스로 관리해야 할 때 특히 유용하다. 콜백을 등록하고 관리하는 구조는 13.10에서 다룬다.

13.4.5 std::invoke와 멤버 포인터

std::invoke()는 일반 함수, 함수 객체, 람다 표현식과 멤버 포인터처럼 여러 종류의 Callable을 하나의 형식으로 호출한다. <functional> 헤더에 정의되어 있다.

```cpp
#include <functional>
int Add(int left, int right)
{
    return left + right;
}
int result = std::invoke(Add, 10, 20);
```

함수 객체와 람다도 같은 방식으로 호출한다.

```cpp
std::greater<int> greater;
bool first = std::invoke(greater, 20, 10);

bool second = std::invoke([](int value) {
        return value > 0;
    }, 10);
```

Callable은 종류에 따라 직접 호출하는 문법이 다를 수 있다. 일반 함수와 함수 객체는 callable(args...) 형태로 호출하지만 멤버 함수 포인터는 어느 객체의 멤버 함수를 호출할 것인지 함께 지정해야 한다.

```
// Player 클래스 정의.
class Player
{
public:
    explicit Player(int hp) : hp(hp) { }
    void TakeDamage(int damage)
    {
        hp -= damage;
    }
    int GetHp() const
    {
        return hp;
    }
private:
    int hp;
};
```

```
#include <iostream>

// <Player 클래스 정의...>

int main()
{
    Player player{100};

    // 멤버 함수 포인터 저장.
    auto takeDamage = &Player::TakeDamage;
    auto getHp = &Player::GetHp;

    // 객체에는 .* 연산자를 사용.
    (player.*takeDamage)(30);
    std::cout << (player.*getHp)() << '\n';

    // 객체 포인터에는 ->* 연산자를 사용.
    Player* playerPointer = &player;
    (playerPointer->*takeDamage)(10);
}
```

멤버 함수 포인터를 직접 호출할 때는 대상이 객체인지 객체 포인터인지에 따라 사용하는 연산자가 다르다. 객체에는 .*, 객체 포인터에는 ->* 연산자를 사용한다.

std::invoke()를 사용하면 멤버 함수 포인터도 같은 형식으로 호출할 수 있다. 멤버 함수 포인터에는 호출 대상 객체를 첫 번째 인수로 전달한다.

```
Player player{100};
auto takeDamage = &Player::TakeDamage;
auto getHp = &Player::GetHp;
std::invoke(takeDamage, player, 30);
std::cout << std::invoke(getHp, player) << '\n';
```

객체 포인터와 std::reference_wrapper도 호출 대상으로 사용할 수 있다.

```
std::invoke(takeDamage, &player, 10);
std::invoke(takeDamage, std::ref(player), 10);
```

멤버 데이터 포인터를 전달하면 해당 멤버에 대한 참조를 얻을 수 있다.

```
struct Item
{
    int id;
    int price;
};
Item item{1001, 500};
auto price = &Item::price;
std::invoke(price, item) = 700;
```

std::invoke()는 호출 대상의 종류에 따라 직접 호출 문법을 구분하지 않아도 된다는 장점이 있다. 특히 템플릿에서 전달받은 Callable이 일반 함수인지 함수 객체인지 멤버 포인터인지 알 수 없을 때 하나의 호출 형식으로 처리할 수 있다.

13.4.6 std::mem_fn

std::mem_fn()은 멤버 포인터를 함수 객체로 감싸서 일반 Callable처럼 저장하거나 전달할 수 있게 한다.

```
#include <functional>
#include <iostream>

// <Player 클래스 정의...>

int main()
{
    Player player{100};
    auto takeDamage = std::mem_fn(&Player::TakeDamage);
    auto getHp = std::mem_fn(&Player::GetHp);
    takeDamage(player, 30);
    std::cout << getHp(player) << '\n';
}
```

std::mem_fn()으로 만든 함수 객체는 대상 객체를 첫 번째 인수로 받고 그 뒤에 멤버 함수의 인수를 받는다. 객체뿐 아니라 객체 포인터나 std::reference_wrapper도 사용할 수 있다.

std::invoke()와 std::mem_fn()은 멤버 포인터를 다룬다는 점에서는 비슷하지만 목적이 다르다.

```
std::invoke
Callable + 호출 대상 + 인수
         ↓
        즉시 호출
```

```
std::mem_fn
멤버 포인터
    ↓
함수 객체 생성
    ↓
저장하거나 전달한 뒤 호출
```

std::invoke()는 전달받은 Callable을 바로 실행하고, std::mem_fn()은 멤버 포인터를 새로운 Callable로 만들어 이후에 사용할 수 있게 한다.

13.4.7 std::bind와 std::placeholders

함수나 함수 객체를 다른 인터페이스에 전달할 때 일부 인수를 미리 정해 두어야 하는 경우가 있다. std::bind()는 Callable과 인수를 결합하여 새로운 Callable을 만든다.

```
#include <algorithm>
#include <functional>
#include <iostream>
#include <vector>

bool IsGreaterThan(int value, int limit)
{
    return value > limit;
}

int main()
{
  std::vector<int> values{10, 25, 40, 5, 7, 30, 60};
  auto predicate = std::bind(IsGreaterThan, std::placeholders::_1, 20);
  auto count = std::count_if(values.begin(), values.end(), predicate);
  std::cout << count << '\n';
}
```

IsGreaterThan()은 두 인수를 받지만 20을 두 번째 인수에 미리 저장했기 때문에 predicate는 하나의 인수만 받아 호출할 수 있다.

```
predicate(25)
     ↓
IsGreaterThan(25, 20)
```

std::placeholders::_1, _2, _3 등은 std::bind()가 만든 Callable을 호출할 때 전달되는 인수를 나타낸다. placeholder의 위치를 바꾸면 원래 Callable에 전달되는 인수의 순서도 바꿀 수 있다.

```cpp
#include <functional>
#include <iostream>

int Subtract(int left, int right)
{
    return left - right;
}

int main()
{
    auto reverse = std::bind(Subtract
                           , std::placeholders::_2
                           , std::placeholders::_1);
    std::cout << reverse(10, 30) << '\n';
}
```

reverse(10, 30)을 호출하면 _2가 첫 번째 인수로, _1이 두 번째 인수로 사용되므로 실제 호출은 Subtract(30, 10)이 된다.

13.4.8 참조 바인딩과 std::ref

std::bind()에 일반 인수를 전달하면 바인드 객체 내부에 decay된 값으로 저장된다. 원본 객체를 참조해야 한다면 std::ref()나 std::cref()를 사용한다.

```cpp
#include <functional>
#include <iostream>

void AddValue(int& target, int value)
{
    target += value;
}

int main()
{
    int total = 10;
    auto add = std::bind(AddValue
                       , std::ref(total)
                       , std::placeholders::_1);
    add(5);
    std::cout << total << '\n';
}
```

std::ref(total)은 원본 total을 참조하는 std::reference_wrapper를 만든다. 상수 참조가 필요하면 std::cref()를 사용할 수 있다. 참조 래퍼는 대상 객체를 소유하지 않으므로 바인드 객체가 사용하는 동안 원본 객체가 살아 있어야 한다.

멤버 함수 포인터도 객체와 함께 바인딩할 수 있다.

```cpp
Player player{100};
auto damage = std::bind(&Player::TakeDamage
                        , &player, std::placeholders::_1);
damage(30);
```

damage(30)은 player.TakeDamage(30)과 같은 동작을 한다.

13.4.9 std::bind_front

앞쪽 인수만 순서대로 고정하는 경우에는 C++20의 std::bind_front()를 사용할 수 있다.

```cpp
#include <functional>
#include <iostream>
```

```cpp
int Add(int left, int right)
{
    return left + right;
}

int main()
{
    auto add10 = std::bind_front(Add, 10);
    std::cout << add10(5) << '\n';
}
```

같은 동작을 std::bind()로 작성하면 placeholder가 필요하다.

```cpp
auto first = std::bind(Add, 10, std::placeholders::_1);
auto second = std::bind_front(Add, 10);
```

second(5)를 호출하면 Add(10, 5)로 변환되어 실행된다.

std::bind()는 일부 인수를 고정하거나 인수의 순서를 재배치할 수 있고, std::bind_front()는 앞쪽 인수를 순서대로 고정하는 형태를 간결하게 표현한다.

13.5에서 배우는 람다 표현식으로도 같은 호출 형태를 직접 구성할 수 있다.

13.4.10 Callable의 선택

Callable은 하나의 기능을 여러 방식으로 표현할 수 있으므로 사용 목적에 따라 적절한 형태를 선택해야 한다.

목적	사용할 수 있는 방법
일반 함수의 주소를 저장하거나 C 인터페이스와 연결	함수 포인터
상태와 동작을 하나의 타입으로 구성	함수 객체
짧은 동작이나 주변 상태를 이용	람다 표현식
서로 다른 Callable을 하나의 타입으로 저장	std::function
종류가 다른 Callable을 같은 형식으로 호출	std::invoke
멤버 포인터를 Callable로 변환	std::mem_fn
일부 인수를 고정하거나 재배치	std::bind
앞쪽 인수를 순서대로 고정	std::bind_front
원본 객체의 참조를 복사 가능한 형태로 전달	std::ref, std::cref

함수 포인터는 구조가 단순하지만 자체 상태를 저장할 수 없다. 함수 객체와 람다는 상태를 가질 수 있으며 구체 타입이 컴파일 시점에 유지된다. std::function은 서로 다른 Callable을 하나의 호출 인터페이스로 저장할 수 있지만 타입 소거와 간접 호출에 따른 비용이 추가될 수 있다.

std::invoke()는 호출 방식을 통일하고, std::mem_fn(), std::bind()와 std::bind_front()는 기존 Callable을 다른 호출 형태로 바꾼다. 모던 C++에서는 단순한 호출 조합을 람다로 표현하는 경우도 많지만, 각 함수 유틸리티는 표준 라이브러리와 기존 코드에서 실제로 사용되므로 역할과 동작을 이해할 필요가 있다.

13.5 람다 표현식

람다 표현식(lambda expression)은 호출 가능한 객체를 표현하는 문법이다. 람다 표현식이 평가되면 고유한 클로저 타입의 객체가 생성된다. 본문은 그 객체의 함수 호출 연산자에 대응한다.

13.5.1 람다의 기본 구조

람다 표현식은 캡처 목록, 매개변수 목록, 지정자, 반환 타입, 함수 본문으로 구성된다.

```cpp
[capture](parameters) specifiers -> return_type
{
    body
```

```
}
```

매개변수가 없고 추가 지정자가 없다면 빈 괄호를 생략할 수 있다.

```
[ ] {
    std::cout << "hello\n";
}();
```

람다 객체를 변수에 저장하면 여러 번 호출할 수 있다.

```
auto add = [](int left, int right)
{
    return left + right;
};

std::cout << add(10, 20) << '\n';
```

반환 타입은 본문의 return 표현식에서 추론된다.
표준 알고리즘의 판정 함수로 람다를 전달할 수 있다.

```
std::vector<int> values{10, 15, 20, 25};

auto count = std::count_if(
    values.begin(),
    values.end(),
    [](int value)
    {
        return value % 2 == 0;
    });
```

13.5.2 값 캡처와 참조 캡처

람다 본문에서 바깥 지역 변수를 사용하려면 캡처 목록에 포함해야 한다.

```
int limit = 20;

auto isGreater = [limit](int value)
{
    return value > limit;
};
```

값 캡처는 람다 객체를 만들 때 변수의 값을 복사한다.

```
int limit = 20;
auto predicate = [limit](int value)
{
    return value > limit;
};

limit = 100;
std::cout << std::boolalpha << predicate(50) << '\n';
```

람다에는 캡처 당시의 20이 저장되어 있으므로 결과는 true이다.
참조 캡처는 바깥 변수를 참조한다.

```
int total = 0;

auto add = [&total](int value)
{
    total += value;
};

add(10);
add(20);
```

total의 실제 값이 변경된다. 참조 캡처를 가진 람다가 참조 대상보다 오래 살아남으면 댕글링 참조가 생긴다.
캡처 기본값을 사용할 수도 있다.

```
[=](int value)
{
    return value + offset + bonus;
};
```

[=]는 사용한 지역 변수를 값으로 캡처한다. [&]는 사용한 지역 변수를 참조로 캡처한다. 캡처 범위가 넓어지면 람다가 어떤 상태에 의존하는지 파악하기 어려워질 수 있으므로 필요한 변수만 명시하는 방식이 안전하다.

혼합 캡처도 가능하다.

```
[=, &total](int value)
{
    total += value * multiplier;
};
```

total은 참조로, 나머지 사용 변수는 값으로 캡처한다.

13.5.3 초기화 캡처

초기화 캡처(init capture)는 캡처 목록에서 새 멤버를 초기화한다.

```
int base = 10;
auto add = [offset = base * 2](int value)
{
    return value + offset;
};
```

람다 안에서는 offset이라는 이름을 사용하고, 바깥의 base와는 별도의 값이 저장된다.
이동 전용 객체도 초기화 캡처로 옮길 수 있다.

```
#include <memory>

auto value = std::make_unique<int>(100);

auto print = [stored = std::move(value)]
{
    std::cout << *stored << '\n';
};
```

stored는 람다 객체의 멤버가 되고 value는 소유권을 잃는다. 해당 람다의 클로저 객체는 unique_ptr를 포함하므로 복사할 수 없고 이동만 가능하다.
초기화 캡처는 계산 결과에 이름을 붙이거나 수명과 소유권을 람다로 옮길 때 사용한다.

13.5.4 mutable

값 캡처로 저장된 멤버는 기본적으로 람다 호출 안에서 수정할 수 없다. 람다의 함수 호출 연산자가 기본적으로 const이기 때문이다.

```
int count = 0;

auto increment = [count]
{
    // ++count; // 오류
};
```

mutable을 지정하면 람다 객체 내부의 캡처 값을 변경할 수 있다.

```
int count = 0;

auto increment = [count]() mutable
{
    return ++count;
};
```

```
std::cout << increment() << '\n';
std::cout << increment() << '\n';
std::cout << count << '\n';
```

람다 내부의 복사본은 1, 2로 증가하지만 바깥 count는 0이다. mutable은 값 캡처를 참조 캡처로 바꾸지 않는다.

상태가 변경되는 람다를 상수 객체로 호출할 수는 없다.

```
const auto fixed = [count]() mutable
{
    return ++count;
};

// fixed(); // 오류
```

13.5.5 반환 형식

반환 표현식들이 호환되는 타입이면 반환 타입을 생략할 수 있다.

```
auto absolute = [](int value)
{
    if (value < 0)
    {
        return -value;
    }

    return value;
};
```

서로 다른 타입을 반환하면 추론할 수 없다.

```
// 오류
// auto convert = [](bool useInteger)
// {
//     if (useInteger)
//     {
//         return 10;
//     }
//
//     return 3.5;
// };
```

반환 타입을 명시하면 변환 규칙을 적용할 수 있다.

```
auto convert = [](bool useInteger) -> double
{
    if (useInteger)
    {
        return 10;
    }

    return 3.5;
};
```

참조를 반환할 때는 수명과 추론 규칙을 확인해야 한다.

```
std::vector<int> values{10, 20, 30};

auto getFirst = [&values]() -> int&
{
    return values.front();
};

getFirst() = 100;
```

컨테이너가 소멸하거나 첫 원소를 가리키는 참조가 무효화되면 반환된 참조도 사용할 수 없다.

13.5.6 제네릭 람다

매개변수 타입에 auto를 사용하면 여러 타입을 받을 수 있는 제네릭 람다가 된다.

```
auto add = [](auto left, auto right)
{
    return left + right;
};

std::cout << add(10, 20) << '\n';
std::cout << add(1.5, 2.5) << '\n';
```

컴파일러는 호출 인수 타입에 맞는 함수 호출 연산자 템플릿을 생성한다.

참조성과 const를 보존하려면 auto&&와 decltype(auto)를 사용할 수 있다.

```
auto access = [](auto&& object) -> decltype(auto)
{
    return std::forward<decltype(object)>(object).GetValue();
};
```

C++20에서는 람다의 템플릿 매개변수 목록을 직접 작성할 수 있다.

```
auto getSize = []<typename T>(const std::vector<T>& values)
{
    return values.size();
};
```

여러 매개변수가 같은 타입이어야 한다는 조건도 표현하기 쉽다.

```
auto maximum = []<typename T>(T left, T right)
{
    return left < right ? right : left;
};
```

auto 매개변수 두 개를 사용하면 서로 다른 타입도 추론될 수 있지만 하나의 T를 사용하면 두 인수에 같은 타입이 요구된다.

13.5.7 클로저 타입과 클로저 객체

컴파일러는 람다 표현식마다 고유한 이름 없는 클래스 타입을 정의한다. 이 타입을 클로저 타입(closure type), 람다 표현식이 평가되어 만들어진 객체를 클로저 객체(closure object)라고 한다.

```
int offset = 10;

auto add = [offset](int value)
{
    return value + offset;
};
```

add는 함수가 아니라 클로저 객체이다. add(value) 호출은 클로저 객체의 함수 호출 연산자를 호출한다.

동작의 핵심은 캡처 값을 멤버로 저장하고 operator()에서 사용하는 클래스 형태로 이해할 수 있다.

```
class AddOffset
{
public:
    explicit AddOffset(int offset)
        : offset(offset)
    {
    }

    int operator()(int value) const
    {
```

```
        return value + offset;
    }

private:
    int offset;
};
```

이 코드는 컴파일러가 생성하는 실제 클로저 타입의 정의가 아니다. 캡처와 호출이 클래스 객체로 표현된다는 점을 설명하기 위해 대응되는 구조만 작성한 것이다.

캡처가 없는 람다는 호환되는 함수 포인터로 변환될 수 있다.

```
int (*add)(int, int) = [](int left, int right)
{
    return left + right;
};
```

캡처가 있는 람다는 상태를 저장해야 하므로 일반 함수 포인터로 변환되지 않는다.

각 람다 표현식의 클로저 타입은 서로 다르다.

```
auto first = [](int value) { return value * 2; };
auto second = [](int value) { return value * 2; };

static_assert(!std::is_same_v<decltype(first), decltype(second)>);
```

본문이 같아 보여도 서로 다른 표현식에서 만들어진 람다는 다른 타입이다.

13.5.8 람다 캡처와 객체 수명

값 캡처는 값을 람다 객체 안에 저장하므로 바깥 지역 변수가 소멸해도 복사된 값은 유지된다.

```
auto MakeAdder(int offset)
{
    return [offset](int value)
    {
        return value + offset;
    };
}

auto addTen = MakeAdder(10);
std::cout << addTen(5) << '\n';
```

참조 캡처는 참조 대상보다 오래 살아서는 안 된다.

```
auto MakeInvalidAdder()
{
    int offset = 10;

    return [&offset](int value)
    {
        return value + offset;
    };
}
```

함수가 끝나면 offset이 소멸하므로 반환된 람다의 참조는 댕글링 상태가 된다. 해당 람다를 호출하면 프로그램의 동작은 보장되지 않는다.

멤버 함수 안의 [this] 캡처는 현재 객체의 포인터를 저장한다.

```
class Counter
{
public:
    explicit Counter(int value)
        : value(value)
    {
    }
```

```
    auto MakeReader()
    {
        return [this]
        {
            return value;
        };
    }

private:
    int value;
};
```

반환된 람다보다 Counter 객체가 먼저 소멸하면 저장된 this 포인터가 유효하지 않다.

C++17의 [*this]는 현재 객체를 복사해 캡처한다.

```
class Counter
{
public:
    explicit Counter(int value)
        : value(value)
    {
    }

    auto MakeReader() const
    {
        return [*this]
        {
            return value;
        };
    }

private:
    int value;
};
```

람다 안에는 Counter의 복사본이 저장된다. 원본 객체와 독립된 값을 읽으며, 객체가 복사 가능해야 하고 복사 비용이 발생한다.

공동 소유가 필요한 비동기 콜백에서는 shared_ptr를 값으로 캡처할 수 있다.

```
class Task : public std::enable_shared_from_this<Task>
{
public:
    auto MakeCallback()
    {
        std::shared_ptr<Task> self = shared_from_this();

        return [self]
        {
            self->Run();
        };
    }

    void Run()
    {
        std::cout << "run\n";
    }
};
```

shared_from_this()는 객체가 이미 shared_ptr로 소유되고 있을 때만 호출해야 한다. 콜백이 객체를 소유하므로 콜백이 살아 있는 동안 객체도 유지된다.
객체가 콜백을 다시 소유하면 순환 소유가 생길 수 있으므로 weak_ptr 캡처가 필요한 구조도 있다.

```
std::weak_ptr<Task> weak = task;

auto callback = [weak]
{
```

```
    if (std::shared_ptr<Task> locked = weak.lock())
    {
        locked->Run();
    }
};
```

람다를 저장하거나 비동기 실행에 넘길 때는 캡처한 값의 소유권과 참조 대상의 수명을 인터페이스의 일부로 판단해야 한다.

13.6 검색과 판정 알고리즘

검색과 판정 알고리즘은 범위에서 값을 찾거나 조건을 만족하는 원소 수와 범위 전체의 상태를 확인한다. 선형 검색 알고리즘은 보통 입력 반복자만으로 사용할 수 있다.

13.6.1 find와 find_if

std::find()는 지정한 값과 같은 첫 원소를 찾는다.

```
std::vector<int> values{10, 20, 30, 20};

auto position = std::find(values.begin(), values.end(), 20);

if (position != values.end())
{
    std::cout << *position << '\n';
}
```

값을 찾지 못하면 전달한 끝 반복자를 반환한다. 반환 반복자는 검색 범위의 끝과 비교해야 한다.

```
auto first = values.begin() + 1;
auto last = values.end();
auto position = std::find(first, last, 100);

if (position == last)
{
    std::cout << "not found\n";
}
```

부분 범위를 검색했으므로 values.end()보다 last와 비교하는 표현이 범위의 경계를 정확히 드러낸다.
std::find_if()는 판정 함수가 true를 반환하는 첫 원소를 찾는다.

```
struct Player
{
    std::string name;
    int hp;
};

std::vector<Player> players{
    {"Knight", 100},
    {"Mage", 0},
    {"Archer", 70}
};

auto position = std::find_if(
    players.begin(),
    players.end(),
    [](const Player& player)
    {
        return player.hp == 0;
    });
```

std::find_if_not()은 판정 결과가 false인 첫 원소를 찾는다.

```
auto position = std::find_if_not(
    values.begin(),
    values.end(),
    [](int value)
```

```
    {
        return value > 0;
    });
```

정렬된 범위에서 키를 찾는다면 선형 검색보다 lower_bound()나 컨테이너의 find()가 적합할 수 있다. std::map::find()는 트리 구조를 이용하지만 std::find(map.begin(), map.end(), value)는 처음부터 순회한다.

13.6.2 count와 count_if

std::count()는 값과 같은 원소의 수를 센다.

```
std::vector<int> values{10, 20, 10, 30, 10};

auto count = std::count(values.begin(), values.end(), 10);
```

반환 타입은 반복자 거리 타입이다. 보통 auto로 받거나 컨테이너의 difference_type을 사용할 수 있다.
std::count_if()는 조건을 만족하는 원소 수를 센다.

```
auto aliveCount = std::count_if(
    players.begin(),
    players.end(),
    [](const Player& player)
    {
        return player.hp > 0;
    });
```

조건식은 원소를 변경하지 않는 판정으로 작성하는 편이 좋다. 판정 함수가 호출될 때마다 외부 상태를 바꾸면 알고리즘의 의미를 파악하기 어렵고 병렬 실행 정책과도 맞지 않을 수 있다.

13.6.3 all_of, any_of와 none_of

std::all_of()는 모든 원소가 조건을 만족하는지 확인한다.

```
bool allAlive = std::all_of(
    players.begin(),
    players.end(),
    [](const Player& player)
    {
        return player.hp > 0;
    });
```

std::any_of()는 하나 이상의 원소가 조건을 만족하는지 확인한다.

```
bool hasDefeatedPlayer = std::any_of(
    players.begin(),
    players.end(),
    [](const Player& player)
    {
        return player.hp == 0;
    });
```

std::none_of()는 조건을 만족하는 원소가 하나도 없는지 확인한다.

```
bool noInvalidHp = std::none_of(
    players.begin(),
    players.end(),
    [](const Player& player)
    {
        return player.hp < 0;
    });
```

세 알고리즘은 결과가 확정되면 남은 원소를 검사하지 않을 수 있다. any_of()는 참인 원소를 찾는 순간 종료할 수 있고, all_of()는 거짓인 원소를 찾는 순간 종료할 수 있다.

빈 범위에서는 논리적 항등 관계가 적용된다.

알고리즘	빈 범위 결과
all_of()	true
any_of()	false
none_of()	true

빈 범위에 반례가 없으므로 all_of()와 none_of()는 true가 된다.

13.7 정렬과 탐색 알고리즘

정렬 알고리즘은 원소의 순서를 비교 규칙에 맞게 재배치한다. 이진 탐색 알고리즘은 정렬된 범위에서 검색 위치를 빠르게 좁힌다. 정렬에 사용한 비교 규칙과 탐색에 사용한 비교 규칙은 일관되어야 한다.

13.7.1 sort와 stable_sort

std::sort()는 임의 접근 반복자 범위를 정렬한다.

```
std::vector<int> values{40, 10, 30, 20};
std::sort(values.begin(), values.end());
```

기본 비교는 오름차순이다. 사용자 정의 타입은 비교 함수를 전달할 수 있다.

```
struct Player
{
    std::string name;
    int score;
};

std::vector<Player> players{
    {"Knight", 100},
    {"Mage", 80},
    {"Archer", 100}
};

std::sort(
    players.begin(),
    players.end(),
    [](const Player& left, const Player& right)
    {
        return left.score > right.score;
    });
```

비교 함수는 왼쪽 원소가 오른쪽 원소보다 앞에 와야 할 때 true를 반환한다. 같은 객체끼리 비교했을 때는 반드시 false여야 하며 엄격 약순서를 만족해야 한다.

```
// 잘못된 비교 함수
// return left.score >= right.score;
```

>=를 사용하면 같은 점수끼리 양방향 비교가 모두 참이 될 수 있어 정렬 알고리즘의 요구 조건을 위반한다.
std::sort()는 비교 기준이 같은 원소의 기존 상대 순서를 보장하지 않는다. std::stable_sort()는 동등한 원소의 상대 순서를 유지한다.

```
std::stable_sort(
    players.begin(),
    players.end(),
    [](const Player& left, const Player& right)
    {
        return left.score > right.score;
    });
```

Knight와 Archer의 점수가 같다면 입력에서 먼저 있던 Knight가 앞에 유지된다. 동점 원소의 등록 순서나 이전 정렬 결과를 보존해야 할 때 안정 정렬이 필요하다.

여러 기준을 명시적으로 비교하면 std::sort()만으로 전체 순서를 결정할 수 있다.

```
std::sort(
    players.begin(),
    players.end(),
    [](const Player& left, const Player& right)
    {
        if (left.score != right.score)
        {
            return left.score > right.score;
        }
        return left.name < right.name;
    });
```

13.7.2 partial_sort와 nth_element

전체 정렬이 필요하지 않다면 일부 정렬 알고리즘으로 작업량을 줄일 수 있다.
std::partial_sort(first, middle, last)는 [first, middle)에 가장 앞설 원소들을 정렬된 상태로 배치한다.

```
std::vector<int> scores{70, 95, 80, 60, 100, 85};

std::partial_sort(
    scores.begin(),
    scores.begin() + 3,
    scores.end(),
    std::greater<>{});
```

앞의 세 원소에는 가장 높은 점수 세 개가 내림차순으로 배치된다. 뒤쪽 범위의 순서는 지정되지 않는다.

std::nth_element(first, nth, last)는 nth 위치에 전체 정렬 후 그 위치에 올 원소를 배치한다. 앞쪽에는 그 원소보다 앞설 원소들이, 뒤쪽에는 뒤에 올 원소들이 놓이지만 각 구간은 정렬되지 않는다.

```
std::vector<int> values{7, 2, 9, 1, 5, 4};
auto middle = values.begin() + values.size() / 2;
std::nth_element(values.begin(), middle, values.end());

std::cout << *middle << '\n';
```

중앙값이나 상위 n개 후보를 구할 때 전체 정렬이 필요하지 않을 수 있다. 상위 n개를 정렬된 상태로 사용하려면 partial_sort(), 경계 원소와 분할만 필요하면 nth_element()가 적합하다.

13.7.3 lower_bound, upper_bound와 equal_range

std::lower_bound()는 정렬된 범위에서 지정한 값보다 작지 않은 첫 위치를 찾는다.

```
std::vector<int> values{10, 20, 20, 20, 30, 40};

auto lower = std::lower_bound(values.begin(), values.end(),
20);
```

std::upper_bound()는 지정한 값보다 큰 첫 위치를 찾는다.

```
auto upper = std::upper_bound(values.begin(), values.end(), 20);
```

[lower, upper)는 값 20과 동등한 원소 범위이다.

```
std::cout << std::distance(lower, upper) << '\n';
```

std::equal_range()는 두 경계를 한 번에 반환한다.

```
auto [first, last] = std::equal_range(
    values.begin(),
    values.end(),
    20);
```

정렬 상태를 유지하는 벡터에 새 값을 삽입할 위치도 lower_bound()로 구할 수 있다.

```
int value = 25;
auto position = std::lower_bound(values.begin(), values.end(), value);
values.insert(position, value);
```

벡터의 위치 탐색은 로그 시간이지만 중간 삽입은 뒤 원소 이동 때문에 선형 시간이 든다. 알고리즘의 검색 복잡도만으로 전체 연산 비용을 판단해서는 안 된다.

사용자 정의 비교 규칙을 사용했다면 탐색에서도 같은 정렬 관계를 사용해야 한다.

```
std::sort(players.begin(), players.end(),
        [](const Player& left, const Player& right)
{
    return left.score < right.score;
});

int score = 100;

auto position = std::lower_bound(
    players.begin(),
    players.end(),
    score,
    [](const Player& player, int value)
    {
        return player.score < value;
    });
```

13.7.4 binary_search와 정렬 사전 조건

std::binary_search()는 정렬된 범위에 동등한 값이 존재하는지 bool로 반환한다.

```
std::vector<int> values{10, 20, 30, 40};

bool found = std::binary_search(values.begin(), values.end(), 30);
```

원소 위치가 필요하면 lower_bound()를 사용한다.

```
auto position = std::lower_bound(values.begin(), values.end(), 30);

if (position != values.end() && *position == 30)
{
    std::cout << "found\n";
}
```

이진 탐색 알고리즘의 범위는 비교 규칙에 따라 분할되어 있어야 한다. 일반적으로 같은 비교 규칙으로 정렬된 범위를 사용한다.

```
std::vector<int> values{30, 10, 40, 20};

// 정렬되지 않았으므로 결과를 신뢰할 수 없다.
// bool found = std::binary_search(values.begin(),
values.end(), 20);
```

내림차순으로 정렬했다면 탐색에도 std::greater<>를 전달한다.

```
std::sort(values.begin(), values.end(), std::greater<>{});

bool found = std::binary_search(
    values.begin(),
    values.end(),
    20,
    std::greater<>{});
```

정렬 후 원소를 변경하여 순서가 깨지면 이진 탐색의 사전 조건도 깨진다. 정렬 여부는 컨테이너 타입이 자동으로 보장하지 않는다. std::set과 std::map은 컨테이너 자체가 정렬을 유지하므로 멤버 find()와 lower_bound()를 사용하는 편이 적합하다.

13.8 복사, 이동, 변환과 제거

범위 알고리즘은 원소를 다른 범위로 복사하거나 이동하고, 값을 변환하고, 조건에 맞지 않는 원소를 논리적으로 제거한다. 목적지 크기와 범위 겹침, 컨테이너 크기 변경 여부를 확인해야 한다.

13.8.1 copy와 move

std::copy()는 원본 범위의 원소를 목적지에 대입한다.

```
std::vector<int> source{10, 20, 30};
std::vector<int> destination(source.size());

std::copy(
    source.begin(),
    source.end(),
    destination.begin());
```

목적지에 원소가 없으면 삽입 반복자를 사용한다.

```
std::vector<int> destination;
destination.reserve(source.size());
std::copy(
    source.begin(),
    source.end(),
    std::back_inserter(destination));
```

reserve()는 필수는 아니지만 벡터의 재할당 횟수를 줄일 수 있다.
std::copy_if()는 조건을 만족하는 원소만 복사한다.

```
std::vector<int> evenValues;

std::copy_if(
    source.begin(),
    source.end(),
    std::back_inserter(evenValues),
    [](int value)
    {
        return value % 2 == 0;
    });
```

std::move() 알고리즘은 각 원소를 이동 대입한다. 하나의 객체를 우측값으로 변환하는 std::move()와 이름이 같지만 인수 개수와 헤더가 다르다.

```
std::vector<std::string> source{"Sword", "Shield", "Potion"};
std::vector<std::string> destination(source.size());

std::move(
    source.begin(),
    source.end(),
    destination.begin());
```

<std::move의 두가지 형태>

형태	역할
std::move(object)	객체를 우측값으로 변환
std::move(first, last, destination)	범위의 원소들을 이동

이동 후 source의 원소는 유효하지만 값은 지정되지 않은 상태이다. 컨테이너의 크기는 그대로 유지된다.

범위가 겹칠 때 목적지가 원본보다 뒤에서 시작하면 std::copy_backward()나 std::move_backward()가 필요할 수 있다.

```
std::vector<int> values{1, 2, 3, 4, 5, 0, 0};

std::copy_backward(
```

```
    values.begin(),
    values.begin() + 5,
    values.end());
```

겹치는 범위에 일반 copy()를 무조건 사용하면 아직 읽지 않은 원소를 덮어쓸 수 있다.

13.8.2 transform

std::transform()은 원소에 변환 함수를 적용해 목적지에 저장한다.

```
std::vector<int> values{1, 2, 3, 4};
std::vector<int> squares(values.size());

std::transform(
    values.begin(),
    values.end(),
    squares.begin(),
    [](int value)
    {
        return value * value;
    });
```

입력과 출력 범위를 같게 하면 원소를 제자리에서 변환할 수 있다.

```
std::transform(
    values.begin(),
    values.end(),
    values.begin(),
    [](int value)
    {
        return value * 2;
    });
```

두 입력 범위를 결합하는 이항 형태도 제공한다.

```
std::vector<int> left{1, 2, 3};
std::vector<int> right{10, 20, 30};
std::vector<int> sums(left.size());

std::transform(
    left.begin(),
    left.end(),
    right.begin(),
    sums.begin(),
    std::plus<>{});
```

둘째 입력 범위가 첫 범위보다 짧으면 범위를 벗어나므로 호출 전에 크기를 확인해야 한다.

13.8.3 remove와 remove_if

std::remove()는 컨테이너에서 원소를 직접 지우지 않는다. 유지할 원소를 앞쪽으로 이동시키고 새 논리적 끝을 반환한다.

```
std::vector<int> values{10, 20, 30, 20, 40};
auto newEnd = std::remove(values.begin(), values.end(), 20);
```

호출 직후 values.size()는 여전히 5이다. [values.begin(), newEnd)에는 유지할 원소 10, 30, 40이 놓인다. [newEnd, values.end())의 원소 값은 제거 대상으로 사용할 수 없으며 컨테이너의 실제 원소로 남아 있다.

```
호출 전: 10 20 30 20 40
논리 범위: 10 30 40 | 미지정된 나머지 영역
                    ↑
                  newEnd
```

std::remove_if()는 조건을 만족하는 원소를 논리적으로 제거한다.

```
auto newEnd = std::remove_if(
    values.begin(),
    values.end(),
    [](int value)
    {
        return value % 2 == 0;
    });
```

알고리즘은 컨테이너 타입을 모르므로 size()를 줄이거나 노드를 해제할 수 없다. 반복자 범위 안의 값을 재배치하고 경계 반복자만 반환한다.

13.8.4 erase-remove와 std::erase_if

벡터와 문자열에서 논리적 끝 뒤의 실제 원소를 제거하려면 erase()를 연결한다.

```
std::vector<int> values{10, 20, 30, 20, 40};

values.erase(
    std::remove(values.begin(), values.end(), 20),
    values.end());
```

이 형태를 erase-remove 관용구라고 한다. remove()가 유지할 원소를 앞쪽으로 모으고, erase()가 남은 꼬리 범위를 컨테이너에서 제거한다.
조건 제거도 같은 구조를 사용한다.

```
values.erase(
    std::remove_if(
        values.begin(),
        values.end(),
        [](int value)
        {
            return value < 0;
        }),
    values.end());
```

C++20은 여러 표준 컨테이너에 std::erase()와 std::erase_if()를 제공한다.

```
std::erase(values, 20);

std::erase_if(
    values,
    [](int value)
    {
        return value < 0;
    });
```

표현이 짧고 컨테이너에 맞는 제거 방법이 선택된다. std::list에서는 노드 제거가 사용되고, 벡터에서는 이동과 erase()가 결합된다.
순회 중 직접 제거해야 할 이유가 없다면 std::erase_if()가 의도를 분명하게 표현한다.

13.8.5 unique와 연속 중복 제거

std::unique()는 서로 인접한 동등 원소를 하나로 모은다. 전체 범위에서 같은 값을 모두 찾는 알고리즘이 아니다.

```
std::vector<int> values{10, 10, 20, 20, 10};

auto newEnd = std::unique(values.begin(), values.end());
values.erase(newEnd, values.end());
```

결과는 10, 20, 10이다. 마지막 10은 앞의 10과 인접하지 않았으므로 남는다.

전체 중복을 제거하려면 먼저 정렬할 수 있다.

```
std::sort(values.begin(), values.end());
values.erase(
    std::unique(values.begin(), values.end()),
    values.end());
```

정렬로 같은 값이 인접하게 모이고 unique()가 각 그룹을 하나로 줄인다. 원래 순서를 유지해야 한다면 집합을 이용해 처음 등장한 값만 별도 컨테이너에 복사하는 방식이 필요하다.

사용자 정의 동등 비교도 전달할 수 있다.

```
struct Player
{
    int id;
    std::string name;
};

std::vector<Player> players{
    {1, "Knight"},
    {1, "Knight Copy"},
    {2, "Mage"}
};

auto newEnd = std::unique(
    players.begin(),
    players.end(),
    [](const Player& left, const Player& right)
    {
        return left.id == right.id;
    });

players.erase(newEnd, players.end());
```

unique()도 크기를 직접 줄이지 않으므로 반환된 논리적 끝을 erase()와 연결해야 한다.

13.9 수치 알고리즘

<numeric> 헤더는 합계, 내적, 부분 합, 차분, 축약과 같은 수치 처리를 제공한다. 초기값 타입과 연산 순서가 결과 타입과 정밀도에 영향을 준다.

13.9.1 accumulate

std::accumulate()는 초기값에서 시작해 왼쪽부터 원소를 누적한다.

```
#include <numeric>

std::vector<int> values{10, 20, 30};

int total = std::accumulate(values.begin(), values.end(), 0);
```

초기값 타입이 누적 결과 타입을 결정한다.

```
std::vector<double> values{1.5, 2.5, 3.5};

double correct = std::accumulate(values.begin(), values.end(), 0.0);
int truncated = std::accumulate(values.begin(), values.end(), 0);
```

초기값을 0으로 전달하면 누적 타입이 int가 되어 매 단계에서 소수 부분이 손실될 수 있다.
사용자 정의 누적 연산을 전달할 수 있다.

```
struct Item
{
    std::string name;
    int price;
};

std::vector<Item> items{
    {"Potion", 100},
    {"Ether", 250},
    {"Elixir", 500}
};
```

```
int totalPrice = std::accumulate(
    items.begin(),
    items.end(),
    0,
    [](int total, const Item& item)
    {
        return total + item.price;
    });
```

std::accumulate()는 원소 순서대로 누적하므로 문자열 연결이나 뺄셈처럼 순서에 민감한 연산에도 사용할 수 있다.

13.9.2 inner_product

std::inner_product()는 두 범위의 대응 원소를 곱한 뒤 합한다.

```
std::vector<int> left{1, 2, 3};
std::vector<int> right{10, 20, 30};

int result = std::inner_product(
    left.begin(),
    left.end(),
    right.begin(),
    0);
```

계산 결과는 1 * 10 + 2 * 20 + 3 * 30이다. 첫 범위의 길이를 기준으로 처리하므로 둘째 범위에 충분한 원소가 있어야 한다.
두 결합 연산을 직접 지정할 수도 있다.

```
int matches = std::inner_product(
    left.begin(),
    left.end(),
    right.begin(),
    0,
    std::plus<>{},
    std::equal_to<>{});
```

대응 원소가 같으면 true, 다르면 false가 생성되고 이를 정수 합으로 누적한다.

13.9.3 partial_sum과 adjacent_difference

std::partial_sum()은 앞에서부터 누적한 결과를 목적지에 기록한다.

```
std::vector<int> values{10, 20, 30, 40};
std::vector<int> sums(values.size());

std::partial_sum(
    values.begin(),
    values.end(),
    sums.begin());
```

sums에는 10, 30, 60, 100이 저장된다.

std::adjacent_difference()는 첫 원소를 그대로 기록하고, 이후에는 현재 원소와 바로 앞 원소의 차이를 기록한다.

```
std::vector<int> differences(values.size());

std::adjacent_difference(
    values.begin(),
    values.end(),
    differences.begin());
```

values가 10, 20, 30, 40이면 결과는 10, 10, 10, 10이다.

부분 합 결과에 인접 차분을 적용하면 원래 값을 복원할 수 있다.

```
std::vector<int> restored(sums.size());
```

```
std::adjacent_difference(sums.begin(), sums.end(), restored.begin());
```

사용자 정의 연산을 전달하면 누적 곱이나 비율 변화 같은 처리도 가능하다. 연산의 의미가 알고리즘 이름과 지나치게 멀어지면 직접 반복문이나 transform()이 더 읽기 쉬울 수 있다.

13.9.4 reduce와 transform_reduce

std::reduce()는 지정된 범위의 원소들을 병렬로 혹은 순차적으로 결합하여 하나의 값으로 축소(reduce)하는 연산 이다.

```
#include <numeric>
std::vector<int> values{10, 20, 30, 40};
int total = std::reduce(values.begin(), values.end(), 0);
```

합계만 보면 accumulate()와 비슷하지만 연산 순서에 중요한 차이가 있다. accumulate()는 왼쪽부터 순서대로 누적한다. reduce()는 원소를 묶는 순서를 바꿀 수 있다.

```
accumulate: (((0 + 10) + 20) + 30) + 40
reduce:     (0 + 10) + (20 + 30) + 40 등으로 재배치 가능
```

연산 순서가 바뀌어도 같은 결과를 내려면 연산이 적절한 결합 성질을 가져야 한다. 뺄셈, 문자열의 순서 의존 연결, 부동소수점 합은 순서에 따라 결과가 달라질 수 있다.

```
std::vector<double> values{1.0e20, -1.0e20, 3.14};
double leftFold = std::accumulate(values.begin(), values.end(), 0.0);
double reduced = std::reduce(values.begin(), values.end(), 0.0);
```

두 값은 부동소수점 반올림 순서에 따라 다를 수 있다.

std::transform_reduce()는 원소를 변환한 뒤 축약한다.

```
struct Player
{
    int score;
};

std::vector<Player> players{{100}, {80}, {90}};

int totalScore = std::transform_reduce(
    players.begin(),
    players.end(),
    0,
    std::plus<>{},
    [](const Player& player)
    {
        return player.score;
    });
```

두 범위를 받는 형태는 내적과 비슷하다.

```
int dot = std::transform_reduce(
    left.begin(),
    left.end(),
    right.begin(),
    0);
```

실행 정책을 사용하는 병렬 축약은 연산 순서를 더 자유롭게 바꿀 수 있다. 재배치 가능한 연산인지 확인하지 않고 reduce()를 accumulate()의 빠른 대체로 사용해서는 안 된다.

13.10 std::function과 콜백

콜백(callback)은 특정 시점에 호출하도록 전달하거나 저장하는 동작이다. 함수 포인터, 함수 객체, 람다 표현식은 모두 콜백으로 사용할 수 있다. 호출 대상의 타입을 템플릿으로 유지할지, 하나의 고정된 인터페이스로 저장할지에 따라 설계가 달라진다.

13.10.1 호출 가능한 객체 저장

호출 가능한 객체의 구체 타입을 알고 있다면 auto로 저장할 수 있다.

```
auto callback = [](int value)
{
    std::cout << value << '\n';
};

callback(10);
```

함수 템플릿은 호출 대상 타입을 템플릿 매개변수로 받을 수 있다.

```
template <typename Callback>
void Execute(int value, Callback callback)
{
    callback(value);
}

Execute(10, [](int value)
{
    std::cout << value << '\n';
});
```

이 방식은 호출 대상의 구체 타입이 컴파일 시점에 유지된다. 인라인화에 유리하고 별도의 타입 소거 비용이 없다. 대신 콜백 타입이 달라질 때마다 함수 템플릿의 인스턴스도 달라지고, 서로 다른 타입의 콜백을 하나의 일반 컨테이너에 바로 저장할 수 없다.
함수 포인터는 같은 시그니처의 일반 함수를 저장할 수 있다.

```
using Callback = void (*)(int);

void PrintValue(int value)
{
    std::cout << value << '\n';
}

Callback callback = PrintValue;
```

캡처가 있는 람다는 함수 포인터로 변환되지 않는다.

```
int offset = 10;

// Callback callback = [offset](int value)
// {
//     std::cout << value + offset << '\n';
// };
```

13.10.2 std::function

std::function<Return(Args...)>은 지정한 호출 시그니처와 호환되는 호출 가능한 객체를 저장한다.

```
#include <functional>

std::function<void(int)> callback;
```

일반 함수, 함수 객체, 람다를 같은 타입에 대입할 수 있다.

```
void PrintValue(int value)
{
    std::cout << "function: " << value << '\n';
}

struct Printer
{
    void operator()(int value) const
    {
        std::cout << "object: " << value << '\n';
```

```
    }
};

std::function<void(int)> callback = PrintValue;
callback(10);

callback = Printer{};
callback(20);

int offset = 100;
callback = [offset](int value)
{
    std::cout << "lambda: " << value + offset << '\n';
};
callback(30);
```

반환값과 매개변수는 std::function의 시그니처에 맞게 변환될 수 있다.

```
std::function<double(int, int)> divide = [](int left, int right)
{
    return static_cast<double>(left) / right;
};;
```

호출 대상이 없는 std::function은 빈 상태이다.

```
std::function<void()> callback;

if (callback)
{
    callback();
}
```

빈 std::function을 호출하면 std::bad_function_call 예외가 발생한다.

```
try
{
    callback();
}
catch (const std::bad_function_call&)
{
    std::cout << "empty callback\n";
}
```

콜백이 선택 사항이라면 호출 전에 상태를 검사하는 편이 분명하다.

함수 포인터, 함수자와 std::function은 모두 호출 가능한 대상을 다루지만 저장 방식과 상태 표현에 차이가 있다.

<함수 포인터, 함수자, std::function 비교>

구분	함수 포인터	함수자(함수 객체)	std::function
형태	함수 주소 저장	operator()를 가진 객체	호출 가능한 객체를 저장하는 래퍼

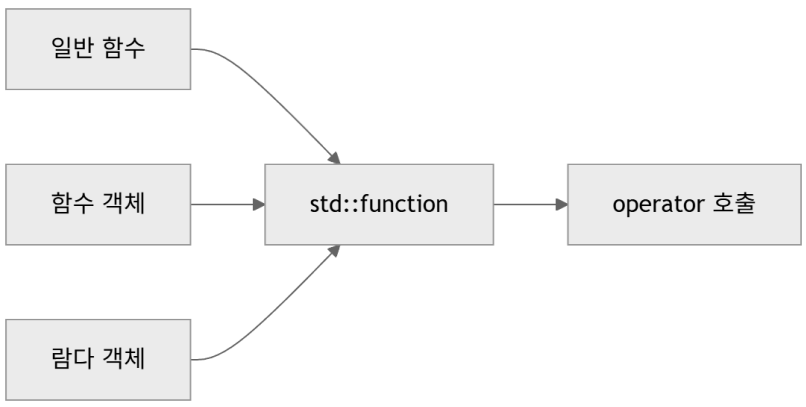
선언 예	void (*callback)(int);	struct FucObj
		{
		  void operator()(int v)
		};
		FucObj callback;
			std::function<void(int)> callback;

저장 예	callback = PrintValue;	Printer callback;	callback = PrintValue;
			callback = Printer{};
			callback = lambda;

상태 저장	X	O	O
호출	callback(10);	callback(10);	callback(10);
주요 내용	같은 시그니처 함수 저장	상태를 가진 호출 객체	함수·함수자·람다를 같은 호출 시그니처로 저장

13.10.3 타입 소거

std::function<void(int)>에 저장되는 람다와 함수 객체의 실제 타입은 서로 다르다. std::function은 호출에 필요한 공통 인터페이스만 남기고 구체 타입을 숨긴다. 이 방식을 타입 소거(type erasure)라고 한다.



사용자는 저장된 대상의 타입을 몰라도 같은 시그니처로 호출할 수 있다.

```
void RunCallback(const std::function<void(int)>& callback)
{
    callback(100);
}
```

target_type()은 저장된 대상의 타입 정보를 제공하고, target<T>()는 정확한 타입이 저장되어 있을 때 포인터를 반환한다.

```
std::function<void(int)> callback = Printer{};

if (Printer* printer = callback.target<Printer>())
{
    (*printer)(10);
}
}
```

일반적인 콜백 코드는 구체 타입을 다시 확인하지 않고 공통 호출 인터페이스만 사용하는 편이 좋다. target()이 자주 필요하다면 타입 소거가 설계에 맞는지 검토할 필요가 있다.

C++20의 std::function에 저장되는 호출 대상은 복사 가능해야 한다.

```
auto pointer = std::make_unique<int>(10);

auto moveOnly = [stored = std::move(pointer)]
{
    std::cout << *stored << '\n';
};

// std::function<void()> callback = std::move(moveOnly);
// ← 복사 불가능한 대상
```

이동 전용 콜백을 저장하려면 별도의 이동 전용 래퍼를 구현하거나 사용하는 라이브러리에서 제공하는 타입을 검토해야 한다. 표준의 std::move_only_function은 C++23 기능이므로 C++20 기준 코드에는 사용할 수 없다.

13.10.4 콜백 인터페이스 설계

이벤트에 하나의 콜백을 등록하는 클래스는 std::function을 멤버로 가질 수 있다.

```
class Button
{
public:
    using ClickHandler = std::function<void()>;

    void SetOnClick(ClickHandler handler)
    {
        onClick = std::move(handler);
    }

    void Click()
    {
```

```
        if (onClick)
        {
            onClick();
        }
    }

private:
    ClickHandler onClick;
};

Button button;
int clickCount = 0;

button.SetOnClick([&clickCount]
{
    ++clickCount;
});

button.Click();
```

참조 캡처한 clickCount는 button이 콜백을 호출하는 동안 살아 있어야 한다. 버튼이 더 오래 유지된다면 값을 소유하도록 캡처하거나 수명 관리 객체를 사용해야 한다.

여러 콜백을 등록하면 식별자를 반환해 해제할 수 있게 설계할 수 있다.

```
class Event
{
public:
    using Handler = std::function<void(int)>;
    using HandlerId = std::size_t;

    HandlerId AddHandler(Handler handler)
    {
        const HandlerId id = nextId++;
        handlers.emplace_back(id, std::move(handler));
        return id;
    }

    void RemoveHandler(HandlerId id)
    {
        std::erase_if(
            handlers,
            [id](const auto& entry)
            {
                return entry.first == id;
            });
    }

    void Notify(int value) const
    {
        for (const auto& [id, handler] : handlers)
        {
            (void)id;
            handler(value);
        }
    }

private:
    HandlerId nextId = 1;
    std::vector<std::pair<HandlerId, Handler>> handlers;
};
```

콜백 실행 중 등록 목록을 변경할 수 있게 허용하면 반복자 무효화와 호출 순서 문제가 생긴다. 변경 요청을 별도 대기열에 저장한 뒤 통지 종료 후 반영하거나, 콜백 목록의 스냅샷을 사용하는 등 정책을 정해야 한다.

콜백 인터페이스에서 확인할 항목은 호출 시점, 호출 횟수, 예외 처리, 스레드, 소유권, 해제 방법이다. 타입 선언만으로 드러나지 않는 규칙은 함수 이름과 문서에 포함해야 한다.

13.10.5 std::function의 비용과 선택 기준

std::function 호출에는 저장된 대상의 구체 타입을 통하지 않는 간접 호출이 필요하다. 작은 호출 대상은 객체 내부 저장 공간에 보관될 수 있지만, 이 최적화의 크기와 적용 여부는 구현에 따라 다르다. 큰 함수 객체는 동적 메모리 할당을 일으킬 수 있다.

복사할 때는 저장된 호출 대상도 복사된다.

```
std::function<void()> first = [data = std::vector<int>(1000)]
{
    std::cout << data.size() << '\n';
};

std::function<void()> second = first;
```

큰 상태를 가진 람다를 복사하면 비용이 커질 수 있다. 공유 상태가 맞다면 shared_ptr를 캡처할 수 있지만 소유권 의미가 달라진다.

호출 경로가 성능에 민감하고 콜백 타입이 컴파일 시점에 알려져 있다면 함수 템플릿이 적합하다.

```
template <typename Callback>
void Process(const std::vector<int>& values, Callback callback)
{
    for (int value : values)
    {
        callback(value);
    }
}
```

일반 함수만 저장하면 함수 포인터가 간단하다. 다양한 호출 가능 타입을 런타임에 교체하고 하나의 멤버나 컨테이너에 저장해야 하면 std::function의 타입 소거가 유용하다.

요구	우선 검토할 방식
컴파일 시점에 호출 타입이 결정됨	함수 템플릿, auto
일반 함수 주소만 필요	함수 포인터
다양한 복사 가능한 호출 대상을 같은 타입으로 저장	std::function
호출 대상의 소유권 이동이 필요	이동 전용 콜백 래퍼 검토
호출 비용이 매우 민감	템플릿 또는 함수 포인터와 측정

13.11 Ranges

C++20 Ranges는 반복자 쌍뿐 아니라 범위 객체 자체를 알고리즘에 전달하고, View를 조합해 처리 과정을 표현한다. 기존 알고리즘을 없애는 기능이 아니라 반복자와 범위 인터페이스를 확장한 기능이다.

13.11.1 Range의 개념

Range는 시작과 끝을 얻을 수 있는 대상이다. 표준 컨테이너, 배열, std::string, View가 범위로 사용될 수 있다.

```
#include <ranges>
#include <vector>

std::vector<int> values{10, 20, 30};

static_assert(std::ranges::range<decltype(values)>);
```

std::ranges::begin(range)와 std::ranges::end(range)는 범위의 시작과 끝을 얻는다.

```
auto first = std::ranges::begin(values);
auto last = std::ranges::end(values);
```

끝 위치의 타입이 시작 반복자 타입과 같지 않아도 된다. Ranges에서는 끝을 나타내는 센티널(sentinel)을 별도 타입으로 사용할 수 있다. 특정 조건에서 종료되는 범위를 반복자 타입에 맞춰 억지로 표현하지 않아도 된다.

```
for (int value : values)
{
    std::cout << value << ' ';
}
```

범위 기반 for문과 Range는 모두 시작과 끝을 중심으로 순회하지만 표준의 세부 요구와 지원 기능은 동일한 하나의 문법으로만 보아서는 안 된다.

13.11.2 std::ranges 알고리즘

Ranges 알고리즘은 범위 객체를 직접 받을 수 있다.

```
#include <algorithm>
#include <ranges>

std::vector<int> values{40, 10, 30, 20};
std::ranges::sort(values);
```

기존 알고리즘과 비교하면 begin()과 end()를 반복해서 작성하지 않아도 된다.

```
std::sort(values.begin(), values.end());
std::ranges::sort(values);
```

반복자와 센티널을 직접 전달하는 형태도 지원한다.

```
std::ranges::sort(values.begin(), values.end());
```

검색 결과는 반복자를 반환한다.

```
auto position = std::ranges::find(values, 30);

if (position != values.end())
{
    std::cout << *position << '\n';
}
```

Ranges 알고리즘은 Concepts로 요구 조건을 표현하므로 비교 함수나 반복자 조건이 맞지 않을 때 진단이 더 직접적일 수 있다.

```
std::list<int> values{30, 10, 20};
// std::ranges::sort(values); // 임의 접근 범위가 아님
```

리스트는 자체 sort() 멤버 함수를 사용한다.

13.11.3 Projection

Projection은 알고리즘이 원소를 비교하거나 판정하기 전에 원소에서 사용할 값을 선택하는 함수이다.

```
struct Player
{
    std::string name;
    int score;
};

std::vector<Player> players{
    {"Knight", 100},
    {"Mage", 80},
    {"Archer", 90}
};
```

점수를 기준으로 정렬할 때 비교 람다를 직접 작성할 수 있다.

```
std::ranges::sort(
    players,
    [](const Player& left, const Player& right)
```

```
    {
        return left.score < right.score;
    });
```

Projection으로 멤버를 지정하면 비교와 값 선택을 분리할 수 있다.

```
std::ranges::sort(players, std::less<>{}, &Player::score);
```

&Player::score가 각 원소에서 점수를 선택하고 std::less<>가 선택된 점수를 비교한다. std::invoke()로 호출 가능한 대상이라면 Projection으로 사용할 수 있다.

내림차순도 비교 함수만 바꾸면 된다.

```
std::ranges::sort(players, std::greater<>{}, &Player::score);
```

검색에서도 Projection을 사용할 수 있다.

```
auto position = std::ranges::find(players, std::string{"Mage"}, &Player::name);
```

원소 전체와 검색 값을 직접 비교하는 람다 없이 이름 멤버를 기준으로 찾는다.

13.11.4 View와 지연 평가

View는 범위의 원소를 소유하지 않고 순회 방법이나 변환 규칙을 표현하는 가벼운 범위인 경우가 많다. std::views::filter는 조건을 만족하는 원소만 보이게 한다.

```
std::vector<int> values{1, 2, 3, 4, 5, 6};

auto evenValues = values | std::views::filter([](int value)
{
    return value % 2 == 0;
});
```

View를 만들 때 결과 원소를 별도 컨테이너에 복사하지 않는다. 원본을 순회할 때 조건을 검사한다.

```
values[0] = 10;

for (int value : evenValues)
{
    std::cout << value << ' ';
}
```

View를 만든 뒤 첫 순회 전에 원본의 첫 값을 10으로 바꾸었으므로 출력에도 반영된다. 지연 평가(lazy evaluation)는 실제로 원소가 요구될 때 필터와 변환을 수행한다. 중간 결과 컨테이너를 줄일 수 있지만 계산 시점과 원본 의존성을 이해해야 한다.

View에서 반복자를 얻거나 순회를 시작한 뒤 원본 컨테이너를 변경하면 기존 반복자와 View 내부에 저장된 위치 정보가 무효화될 수 있다. 원본을 변경할 때도 해당 컨테이너의 반복자 무효화 규칙을 적용해야 한다.

std::views::transform은 원소를 변환된 값으로 보이게 한다.

```
auto squares = values | std::views::transform([](int value)
{
    return value * value;
});
```

변환 결과가 값이면 View를 통한 수정은 원본을 바꾸지 않는다. 변환 함수가 참조를 반환하면 원본 수정이 가능할 수 있으므로 반환 타입을 확인해야 한다.

13.11.5 View 조합

파이프 연산자 |로 여러 View 어댑터를 연결할 수 있다.

```
std::vector<int> values{1, 2, 3, 4, 5, 6, 7, 8};
```

```
auto result = values
    | std::views::filter([](int value)
      {
          return value % 2 == 0;
      })
    | std::views::transform([](int value)
      {
          return value * value;
      })
    | std::views::take(3);
```

순회 결과는 4, 16, 36이다. 짝수를 고르고 제곱한 뒤 앞의 세 결과만 사용한다.
View 조합 순서는 의미와 비용에 영향을 준다.

```
auto first = values
    | std::views::filter(IsEven{})
    | std::views::take(3);

auto second = values
    | std::views::take(3)
    | std::views::filter(IsEven{});
```

first는 전체 범위에서 처음 만나는 짝수 세 개를 제공한다. second는 원본의 처음 세 원소 중 짝수만 제공한다.
View 결과를 소유 컨테이너로 만들려면 C++20에서는 명시적으로 복사할 수 있다.

```
std::vector<int> copied;
std::ranges::copy(result, std::back_inserter(copied));
```

std::ranges::to는 C++23 기능이므로 C++20 기준에서는 사용할 수 없다.

13.11.6 View와 반환 반복자의 수명

많은 View는 원본 범위를 참조한다. 원본이 소멸하면 View를 사용할 수 없다.

```
auto MakeInvalidView()
{
    std::vector<int> values{1, 2, 3, 4};
    return values | std::views::filter([](int value)
    {
        return value % 2 == 0;
    });
}
```

함수가 끝나면 지역 벡터가 소멸한다. 반환된 View가 지역 벡터를 참조한다면 유효한 원소가 없다. View를 반환하려면 원본 범위도 함께 소유하거나, 결과를 별도 컨테이너에 저장하거나, 호출자가 원본의 수명을 유지하도록 인터페이스를 정해야 한다.

원본을 소유하는 컨테이너에 이름을 붙이고 View보다 오래 유지하면 소유 관계가 분명해진다. 임시 범위를 직접 연결하는 표현보다 원본의 수명을 코드에 드러내는 방식이 안전하다.

Ranges 알고리즘은 임시 비소유 범위에서 반복자를 반환하면 댕글링 반복자가 생길 수 있다. 표준은 안전하지 않은 반복자 반환 위치에 std::ranges::dangling을 사용할 수 있다.

```
using Result = decltype(std::ranges::find(
    std::vector<int>{1, 2, 3},
    2));

static_assert(std::is_same_v<Result, std::ranges::dangling>);
```

이름이 있는 벡터는 호출 뒤에도 살아 있으므로 반복자를 반환한다.

```
std::vector<int> values{1, 2, 3};
auto position = std::ranges::find(values, 2);
```

View와 View에서 얻은 반복자는 대부분 원본 범위의 수명에 의존한다. 반환값을 오래 저장할수록 어떤 객체가 원소를 소유하는지 확인해야 한다.

13.12 알고리즘 선택

표준 알고리즘을 선택할 때는 원하는 결과만 보지 않고 반복자 범주, 범위의 정렬 상태, 목적지 크기, 원소 이동 비용, 메모리 사용, 반복자 무효화를 함께 확인한다.

13.12.1 반복자 범주와 사전 조건

알고리즘마다 요구하는 반복자 능력이 다르다.

알고리즘	필요한 대표 반복자 능력
find, count, accumulate	입력 반복자
copy의 목적지	출력 반복자
unique, remove	순방향 반복자 이상
reverse	양방향 반복자 이상
sort, nth_element	임의 접근 반복자

컴파일된다는 사실만으로 논리적 사전 조건이 충족된 것은 아니다.

- binary_search() 범위는 같은 비교 규칙으로 정렬되어 있어야 한다.
- copy() 목적지에는 충분한 원소가 있거나 삽입 반복자를 사용해야 한다.
- 두 범위를 결합하는 알고리즘은 둘째 범위에 충분한 원소가 있어야 한다.
- 비교 함수는 엄격 약순서를 만족해야 한다.
- 판정 함수 실행 중 알고리즘 범위를 무효화해서는 안 된다.
- 반환된 반복자와 참조는 원본 범위가 유효한 동안만 사용한다.

알고리즘 이름과 타입 요구 조건뿐 아니라 호출 전 상태까지 인터페이스의 일부로 보아야 한다.

13.12.2 시간 복잡도와 메모리 사용

같은 결과를 내는 알고리즘도 작업량과 추가 메모리가 다를 수 있다.

목적	선택 후보	특징
전체 정렬	sort	평균적으로 빠른 범용 정렬
안정 정렬	stable_sort	동등 원소 순서 유지, 추가 메모리 가능
상위 n개 정렬	partial_sort	앞부분만 정렬
n번째 원소와 분할	nth_element	전체 순서 불필요
정렬 범위 존재 확인	binary_search	로그 비교, 위치 반환 없음
조건 원소 제거	erase_if	컨테이너에 맞는 제거

이론적 복잡도만으로 실제 성능을 결정할 수 없다. 벡터의 연속 메모리와 캐시 지역성, 리스트의 노드 할당, 함수 객체 인라인화, 원소 이동 비용이 영향을 준다.

전체 정렬 후 한 원소를 찾는 작업을 한 번만 수행한다면 선형 find_if()가 더 저렴할 수 있다. 같은 데이터에서 탐색을 반복한다면 정렬 비용을 먼저 지불하고 이진 탐색을 반복하는 구조가 유리할 수 있다.

13.12.3 직접 반복문과 표준 알고리즘

표준 알고리즘은 작업의 목적을 이름으로 표현한다.

```
bool hasNegative = std::any_of(
    values.begin(),
    values.end(),
    [](int value)
    {
        return value < 0;
    });
```

직접 반복문으로도 같은 작업을 작성할 수 있다.

```
bool hasNegative = false;

for (int value : values)
{
    if (value < 0)
    {
        hasNegative = true;
        break;
    }
}
```

한 가지 검색, 변환, 복사, 정렬 작업이라면 표준 알고리즘이 의도를 더 분명하게 드러내는 경우가 많다. 여러 상태를 동시에 갱신하거나 복잡한 제어 흐름이 필요하다면 직접 반복문이 더 읽기 쉬울 수 있다.

```
int positiveCount = 0;
int negativeCount = 0;
long long total = 0;

for (int value : values)
{
    total += value;

    if (value >= 0)
    {
        ++positiveCount;
    }
    else
    {
        ++negativeCount;
    }
}
```

여러 알고리즘을 억지로 연결해 한 번의 순회를 여러 번으로 늘리기보다 직접 반복문 하나가 더 분명할 수 있다. 반대로 직접 반복문 안에서 검색과 제거와 정렬을 섞기보다 각 목적에 맞는 알고리즘으로 단계를 나누는 편이 안전할 수 있다.

13.12.4 가독성과 성능

알고리즘 선택에서는 코드 길이보다 의도가 드러나는지를 먼저 판단한다.

```
std::erase_if(players, [](const Player& player)
{
    return player.hp == 0;
});
```

이 코드는 체력이 0인 플레이어를 제거한다는 목적이 함수 이름과 판정식에 드러난다.

Ranges와 View 조합도 짧다고 항상 읽기 쉬운 것은 아니다.

```
auto result = values
    | std::views::filter(IsValid{})
    | std::views::transform(ToScore{})
    | std::views::take(10);
```

각 단계의 이름과 타입이 목적을 설명한다면 유용하다. 복잡한 캡처와 중첩 람다, 수명 의존성이 많아지면 중간 범위에 이름을 붙이거나 일반 함수로 분리하는 편이 좋다.

성능은 추측보다 측정으로 판단한다. 표준 알고리즘은 구현 최적화의 이점을 받을 수 있지만, 원소 타입과 데이터 배치, 호출 가능한 객체, 컴파일 옵션에 따라 결과가 달라진다. 성능 측정 전에는 올바른 알고리즘과 자료 구조를 선택하고 사전 조건과 수명을 지키는 것이 우선이다.

13.13 학습 정리

반복자는 컨테이너와 알고리즘 사이에서 원소 위치와 범위를 표현한다. 포인터와 비슷한 연산을 제공하지만 컨테이너의 저장 구조에 맞는 별도 타입일 수 있다. [first, last) 반 개방 구간은 빈 범위와 부분 범위를 일관된 방식으로 나타낸다.

입력과 출력 반복자는 단일 순회를 전제로 한다. 순방향 반복자부터 다중 순회를 보장하고, 양방향 반복자는 뒤로 이동할 수 있다. 임의 접근 반복자는 일정 거리 이동과 거리 계산을 상수 시간에 수행하며, 연속 반복자는 원소의 연속 메모리 배치까지 보장한다.

std::advance(), std::next(), std::prev(), std::distance()는 반복자 범주에 맞게 이동과 거리를 처리한다. 같은 호출이라도 리스트에서는 이동 거리에 비례한 시간이 들고 벡터에서는 상수 시간에 처리될 수 있다. 삽입 반복자는 알고리즘의 대입을 컨테이너 삽입으로 연결하고, 스트림 반복자는 입력과 출력을 반복자 범위로 표현한다.

반복자 무효화는 컨테이너 변경 후 기존 위치를 사용할 수 있는지를 결정한다. 무효화된 반복자를 사용하면 프로그램의 동작은 보장되지 않는다. 순회 중 제거에는 erase()의 반환 반복자를 사용하고, 판정 함수 안에서 알고리즘이 순회 중인 컨테이너를 변경하지 않는다.

일반 함수, 함수 포인터, 함수 객체, 람다 표현식, 멤버 포인터는 호출 가능한 객체이다. 함수 객체는 상태를 저장할 수 있고 표준 알고리즘에 비교와 판정 규칙을 전달한다. std::invoke()는 여러 호출 대상과 멤버 포인터를 통합된 형식으로 호출한다.

람다 표현식은 고유한 클로저 타입의 객체를 만든다. 값 캡처는 값을 복사하고 참조 캡처는 바깥 객체를 참조한다. mutable은 값 캡처된 멤버를 람다 내부에서 변경하게 하지만 바깥 변수를 변경하지 않는다. 초기화 캡처는 계산 결과와 이동 전용 객체를 람다 상태로 저장한다.

참조 캡처와 [this] 캡처는 대상 객체의 수명에 의존한다. 저장형 콜백과 비동기 작업에서는 값 캡처, *this 캡처, 스마트 포인터 캡처 중 어떤 소유권이 필요한지 판단해야 한다.

검색 알고리즘은 값을 찾거나 조건을 검사한다. 정렬과 이진 탐색 알고리즘은 비교 규칙과 정렬 사전 조건을 지켜야 한다. partial_sort()는 앞부분만 정렬하고 nth_element()는 경계 원소와 분할만 만든다.

remove(), remove_if(), unique()는 컨테이너 크기를 줄이지 않고 유지할 원소를 앞쪽으로 재배치해 새 논리적 끝을 반환한다. 실제 제거에는 erase()를 연결하거나 C++20의 std::erase()와 std::erase_if()를 사용한다. unique()는 인접한 중복만 처리한다.

accumulate()는 왼쪽부터 순서대로 누적한다. reduce()와 transform_reduce()는 연산 순서를 바꿀 수 있으므로 재배치 가능한 연산인지 확인해야 한다. 초기값 타입은 누적 결과 타입과 정밀도에 영향을 준다.

std::function은 여러 복사 가능한 호출 대상을 하나의 시그니처로 저장하는 타입 소거 래퍼이다. 다양한 콜백을 런타임에 저장하고 교체할 수 있지만 간접 호출과 복사, 메모리 할당 비용이 생길 수 있다. 호출 타입이 컴파일 시점에 정해지면 함수 템플릿이나 함수 포인터가 더 간단할 수 있다.

C++20 Ranges 알고리즘은 범위를 직접 받고 Projection으로 원소에서 비교할 값을 분리한다. View는 필터와 변환을 지연 평가하고 파이프 연산자로 조합한다. 많은 View와 반환 반복자는 원본 범위의 수명에 의존하므로 원소를 소유하는 객체가 충분히 오래 유지되어야 한다.

13.14 학습 과제

과제 **1.** 반복자 범주 비교
std::forward_list<int>, std::list<int>, std::deque<int>, std::vector<int>를 만들고 각 컨테이너 반복자에서 사용할 수 있는 이동 연산을 확인한다. ++, --, +=, 반복자 뺄셈, std::advance(), std::distance()의 사용 가능 여부를 표로 정리한다. 거리 계산 비용이 컨테이너마다 달라지는 이유도 기록한다.

과제 **2.** 반 개방 구간 분할
정수 벡터를 두 부분 범위 [begin, middle)과 [middle, end)로 나눈다. 앞부분은 오름차순, 뒷부분은 내림차순으로 정렬한다.
원소 수가 홀수일 때 중앙 원소가 어느 범위에 포함되는지 설명하고, 빈 벡터에서도 범위가 올바르게 표현되는지 확인한다.

과제 **3. const_iterator** 구분
수정 가능한 벡터에 대해 iterator, const_iterator, iterator const, const_iterator const 변수를 선언한다. 반복자 이동과 원소 수정 가능 여부를 컴파일 주석으로 확인한다.
포인터의 int*, const int*, int* const, const int* const와 대응 관계도 작성한다.

과제 **4.** 삽입 반복자를 이용한 복사
정수 벡터에서 짝수만 골라 빈 std::vector<int>에 복사한다. std::copy_if()와 std::back_inserter()를 사용한다.
같은 원소를 std::deque<int>의 앞쪽에 복사하여 입력 순서가 뒤집히는 이유를 설명한다. 순서를 유지하면서 덱 앞쪽에 넣는 방법도 구현한다.

과제 **5.** 반복 중 안전한 제거
정수 벡터에서 음수를 제거하는 코드를 세 방식으로 작성한다.

- 반복자와 erase() 반환값 사용
- erase-remove 관용구
- C++20 std::erase_if()

각 코드에서 무효화되는 반복자 범위와 순회 중 사용할 반복자를 설명한다.

과제 **6.** 상태를 가진 함수 객체
기준 점수와 허용 오차를 멤버로 가진 IsNearScore 함수 객체를 작성한다. 전달된 점수가 기준 점수에서 허용 오차 이내인지 판정한다.
std::count_if()로 조건을 만족하는 점수 수를 계산하고, 같은 동작을 람다 표현식으로도 작성한다.

과제 **7.** 람다 캡처와 수명
값 캡처, 참조 캡처, 초기화 캡처를 사용하는 람다를 각각 작성한다. 바깥 변수를 변경한 뒤 각 람다의 출력이 어떻게 달라지는지 확인한다.
지역 변수를 참조 캡처한 람다를 함수 밖으로 반환하는 잘못된 코드는 실행하지 않고, 댕글링 참조가 생기는 시점을 주석으로 설명한다.

과제 **8. this**와 ***this** 캡처
멤버 변수 name과 score를 가진 Player 클래스에 두 콜백 생성 함수를 작성한다.

- [this]로 현재 객체를 참조하는 콜백
- [*this]로 현재 객체를 복사하는 콜백

원본 객체의 값을 변경한 뒤 두 콜백의 결과를 비교한다. 원본 객체가 소멸한 뒤 [this] 콜백을 호출할 수 없는 이유도 설명한다.

과제 **9.** 정렬과 안정 정렬
이름, 점수, 등록 순서를 가진 학생 구조체 벡터를 작성한다. 점수 내림차순으로 std::sort()와 std::stable_sort()를 각각 적용한다.
동점 학생의 상대 순서를 비교하고, 점수와 이름을 모두 비교하는 함수로 전체 순서를 명시하는 코드도 작성한다.

과제 **10.** 상위 점수 선택
점수 100개가 저장된 벡터에서 상위 10개를 구한다.

- 전체 sort()
- partial_sort()
- nth_element() 후 앞부분 정렬

세 방식의 결과와 필요한 정렬 범위를 비교한다. 실행 시간을 측정할 때는 난수 입력을 여러 번 생성하고 최적화 설정을 기록한다.

과제 **11.** 이진 탐색 사전 조건
오름차순 벡터와 내림차순 벡터에서 lower_bound(), upper_bound(), binary_search()를 사용한다.
내림차순 범위에 기본 비교를 사용한 잘못된 호출은 실행 결과에 의존하지 않고 사전 조건 위반으로 설명한다. 정렬과 탐색에 같은 비교 객체를 전달하도록 수정한다.

과제 **12. remove**와 **unique**
중복과 음수가 섞인 정수 벡터에 remove_if()와 unique()를 적용한다. 각 알고리즘 호출 직후의 size()와 논리적 범위를 출력한다.
erase()까지 적용한 후 실제 크기를 비교하고, 전체 중복 제거를 위해 정렬이 필요한 이유를 설명한다.

과제 **13.** 수치 알고리즘
상품 가격과 수량을 별도 벡터에 저장한다. inner_product()로 총액을 계산하고, 상품 구조체 벡터에서는 accumulate()로 같은 결과를 구한다.
초기값을 0, 0.0, long long{0}으로 바꾸어 결과 타입을 비교한다.

과제 **14. accumulate**와 **reduce** 비교
큰 값과 작은 부동소수점 값이 섞인 벡터를 작성한다. accumulate()와 reduce() 결과를 여러 최적화 설정에서 비교한다.
연산 순서가 결과에 영향을 주는 이유를 부동소수점의 유한 정밀도와 연결해 설명한다. 뺄셈을 축약 연산으로 사용했을 때 결과를 안정적으로 예상할 수 없는 이유도 기록한다.

과제 **15.** 콜백 이벤트
정수 값을 전달하는 Event 클래스를 작성한다. 여러 std::function<void(int)> 콜백을 등록하고 식별자로 해제할 수 있게 한다.
통지 중 콜백이 자신을 제거하거나 새 콜백을 추가할 때 발생할 수 있는 반복자 무효화 문제를 분석하고, 변경 요청을 통지 종료 후 처리하는 방식으로 개선한다.

과제 **16. std::function** 선택 보고서
함수 포인터, 함수 템플릿, std::function을 사용한 콜백 인터페이스를 각각 작성한다.
캡처 람다 지원, 호출 대상 복사, 실행 파일 코드 증가 가능성, 간접 호출, 동적 할당 가능성, 이동 전용 대상 지원 여부를 비교한다.

과제 **17. Ranges Projection**
이름, 레벨, 체력을 가진 캐릭터 구조체 벡터를 작성한다. std::ranges::sort()와 멤버 포인터 Projection을 사용해 레벨 순서와 이름 순서로 각각 정렬한다.
std::ranges::find()와 Projection으로 특정 이름의 캐릭터를 찾는다. 비교 람다를 직접 작성한 코드와 길이와 역할 분리를 비교한다.

과제 **18. View** 조합과 지연 평가
정수 벡터에서 양수만 선택하고 제곱한 뒤 앞의 다섯 결과만 제공하는 View를 작성한다.
View 생성 후 원본 벡터의 값을 수정하고 원소를 추가하여 순회 결과가 어떻게 달라지는지 확인한다. 원본 벡터의 재할당이 View 순회 중이 아니라 순회 전에 끝나야 하는 이유도 설명한다.

과제 **19. View** 수명 분석
지역 벡터를 참조하는 View를 반환하는 잘못된 함수를 작성하되 반환된 View를 실제로 순회하지 않는다. 원본이 소멸하는 시점과 View가 무효 상태가 되는 이유를 주석으로 기록한다.
결과 벡터를 반환하는 방식, 원본 범위를 함수 인수로 받는 방식, 소유 객체와 View를 함께 보관하는 방식으로 인터페이스를 각각 개선한다.

과제 **20.** 알고리즘 선택
아래 작업에 적합한 알고리즘이나 직접 반복문을 선택하고 근거를 작성한다.

- 첫 번째 음수 검색
- 모든 체력이 양수인지 판정
- 상위 5개 점수 정렬
- 중앙값 계산
- 정렬된 점수에서 80점 구간 찾기
- 죽은 캐릭터 제거
- 인접한 중복 로그 합치기
- 가격 총합 계산

- 필터, 변환, 개수 제한을 지연 평가로 연결

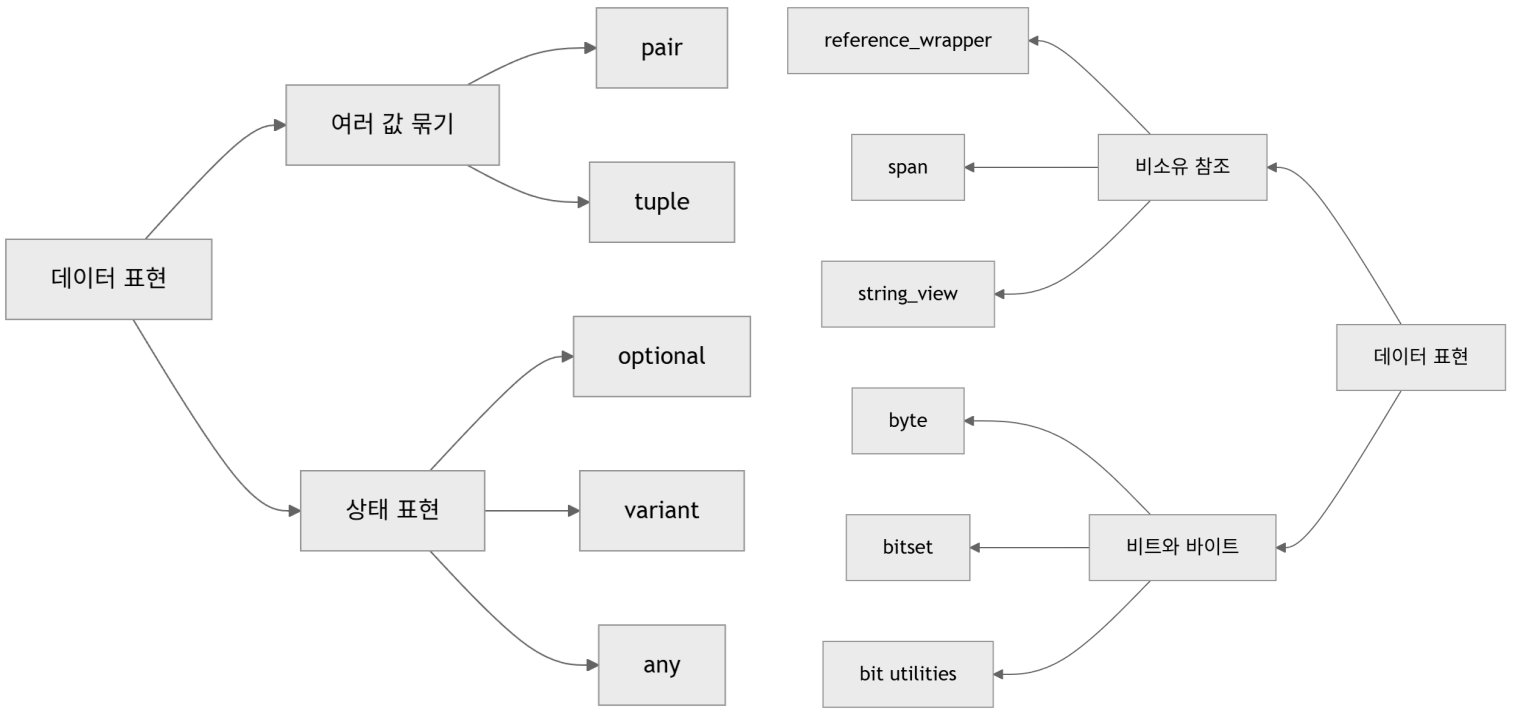
반복자 범주, 정렬 사전 조건, 시간 복잡도, 추가 메모리, 원본 범위 수명, 반복자 무효화를 각 선택에 반영한다.

14. 유틸리티 타입과 데이터 표현

프로그램의 데이터는 값 하나만으로 표현되지 않는 경우가 많다. 함수가 두 결과를 함께 반환해야 할 수 있고, 검색 결과에 값이 없을 수도 있다. 하나의 변수에 여러 타입 중 하나를 저장하거나, 원본 데이터를 복사하지 않고 일정 구간만 참조해야 할 때도 있다.

표준 라이브러리는 자주 반복되는 데이터 표현을 타입으로 제공한다. std::pair와 std::tuple은 여러 값을 하나로 묶는다. std::optional은 값의 존재 여부를 타입에 포함하고, std::variant는 정해진 여러 타입 중 하나를 저장한다. std::any는 저장 타입을 실행 시간에 결정한다.

std::reference_wrapper, std::span, std::string_view는 원본 객체나 메모리를 소유하지 않고 참조한다. 복사 비용을 줄이고 인터페이스를 단순하게 만들 수 있지만 원본의 수명과 무효화 규칙을 지켜야 한다.



유틸리티 타입을 선택할 때는 저장할 값의 종류뿐 아니라 소유권, 수명, 오류 상태, 타입 검사 시점까지 함께 판단해야 한다.

14.1 std::pair와 std::tuple

두 값이나 여러 값을 하나의 객체로 묶어야 할 때 별도 구조체를 정의할 수 있다.

```
struct SearchResult
{
    int index;
    bool found;
};
```

구조체는 멤버 이름으로 의미를 명확하게 표현할 수 있다. 데이터의 의미가 일회적이거나 범용 알고리즘의 중간 결과처럼 짧게 사용된다면 std::pair와 std::tuple을 활용할 수 있다.

14.1.1 두 값과 여러 값을 묶는 타입

std::pair<T1, T2>는 서로 다른 타입의 값 두 개를 저장한다.

```cpp
#include <string>
#include <utility>

std::pair<int, std::string> item{1001, "Potion"};
```

첫 번째 값은 first, 두 번째 값은 second로 접근한다.

```cpp
std::cout << item.first << '\n';
std::cout << item.second << '\n';
```

std::make_pair()는 전달한 인수에서 타입을 추론한다.

```cpp
std::pair<int, std::string> firstItem{1001, "Potion"};
auto secondItem = std::make_pair(1002, std::string{"Ether"});
```

C++17부터 클래스 템플릿 인수 추론을 사용할 수 있으므로 타입을 직접 쓰지 않고 생성할 수도 있다.

```
std::pair thirdItem{1003, std::string{"Elixir"}};
```

std::tuple<Ts...>은 두 개보다 많은 값을 저장할 수 있다.

```
#include <tuple>

std::tuple<int, std::string, int> player{1, "Knight", 100};
```

std::make_tuple()도 인수 타입을 추론한다.

```
auto monster = std::make_tuple(10, std::string{"Slime"}, 30);
```

pair와 tuple은 원소 타입을 직접 멤버로 저장한다. 값 타입을 넣으면 값이 복사되거나 이동되고, 참조 타입을 명시하면 외부 객체를 참조할 수도 있다.

```
int score = 100;
std::tuple<int&, const char*> data{score, "Alice"};

std::get<0>(data) = 200;
```

std::get<0>(data)는 score를 참조하므로 대입 후 score도 200이 된다. 참조를 저장한 튜플은 참조 대상의 수명에 의존한다.

14.1.2 생성과 std::get

std::tuple은 원소 이름을 제공하지 않으므로 std::get<I>()로 위치를 지정한다.

```
std::tuple<int, std::string, int> player{1, "Knight", 100};
int id = std::get<0>(player);
std::string name = std::get<1>(player);
int hp = std::get<2>(player);
```

같은 타입이 한 번만 등장하면 타입으로도 접근할 수 있다.

```
std::tuple<int, std::string, double> data{10, "Potion", 2.5};
std::string& name = std::get<std::string>(data);
```

같은 타입이 여러 번 들어 있으면 타입 기반 접근은 어느 원소를 선택할지 결정할 수 없다.

```
std::tuple<int, int, std::string> position{10, 20, "Field"};
// int x = std::get<int>(position); // 오류: int가 두 번 등장함
```

std::pair도 튜플 인터페이스를 지원한다.

```
std::pair<int, std::string> item{1001, "Potion"};
int id = std::get<0>(item);
std::string& name = std::get<1>(item);
```

std::tuple_size_v<T>는 원소 개수를 컴파일 시점 상수로 제공하고, std::tuple_element_t<I, T>는 지정한 위치의 원소 타입을 얻는다.

```
using Data = std::tuple<int, std::string, double>;
static_assert(std::tuple_size_v<Data> == 3);
static_assert(std::is_same_v<std::tuple_element_t<1, Data>, std::string>);
```

템플릿 코드에서 튜플 계열 타입의 구조를 검사할 때 활용할 수 있다.

14.1.3 std::tie, std::ignore와 구조적 바인딩

std::tie()는 기존 변수의 참조를 튜플로 묶는다.

```
int id = 0;
std::string name;
int hp = 0;
```

```
std::tie(id, name, hp) = std::make_tuple(1,
std::string{"Knight"}, 100);
```

오른쪽 튜플의 각 값이 왼쪽 변수에 대입된다. 필요 없는 원소는 std::ignore로 건너뛸 수 있다.

```
std::tie(id, std::ignore, hp) =
    std::make_tuple(2, std::string{"Mage"}, 80);
```

구조적 바인딩은 원소마다 이름을 부여해 분해한다.

```
auto [playerId, playerName, playerHp] =
    std::make_tuple(1, std::string{"Knight"}, 100);
```

auto만 사용하면 원소를 새 변수로 복사하거나 이동한다.

```
std::pair<int, std::string> item{1001, "Potion"};
auto [idCopy, nameCopy] = item;
nameCopy = "Ether";
```

nameCopy를 변경해도 item.second는 바뀌지 않는다. 원본 원소를 수정하려면 참조 구조적 바인딩을 사용한다.

```
auto& [idReference, nameReference] = item;
nameReference = "Elixir";
```

읽기만 한다면 상수 참조가 불필요한 복사를 막고 수정도 제한한다.

```
const auto& [idView, nameView] = item;
```

컨테이너 순회에서도 같은 차이가 발생한다.

```
std::map<int, std::string> itemNames{
    {1001, "Potion"},
    {1002, "Ether"}
};

for (const auto& [id, name] : itemNames)
{
    std::cout << id << ": " << name << '\n';
}
```

std::map의 원소 타입은 std::pair<const Key, T>이므로 id는 수정할 수 없다. name은 순회 변수가 상수 참조이므로 수정할 수 없다.

14.1.4 여러 반환값과 std::apply

함수는 pair, tuple, 구조체를 반환해 여러 결과를 전달할 수 있다.

```
std::pair<int, int> DivideWithRemainder(int value, int divisor)
{
    return {value / divisor, value % divisor};
}

auto [quotient, remainder] = DivideWithRemainder(17, 5);
```

두 결과의 의미가 함수 이름과 사용 문맥에서 충분히 드러난다면 pair가 간단하다. 반환값이 여러 곳에서 사용되거나 각 원소의 의미가 중요하다면 이름 있는 구조체가 더 분명하다.

```
struct DivisionResult
{
    int quotient;
    int remainder;
};

DivisionResult Divide(int value, int divisor)
{
```

```
    return {value / divisor, value % divisor};
}
```

std::apply()는 튜플의 원소를 함수 인수로 펼쳐 호출한다.

```
#include <functional>
#include <tuple>

int Add(int left, int right)
{
    return left + right;
}

std::tuple<int, int> arguments{10, 20};
int result = std::apply(Add, arguments);
```

람다와도 함께 사용할 수 있다.

```
auto player = std::make_tuple(std::string{"Knight"}, 5, 100);

std::apply(
    [](const std::string& name, int level, int hp)
    {
        std::cout << name << ' ' << level << ' ' << hp << '\n';
    },
    player);
```

pair와 tuple은 짧은 묶음에 적합하지만 원소가 많아질수록 위치 번호만으로 의미를 파악하기 어렵다. 도메인에서 반복 사용하는 데이터는 이름 있는 구조체로 정의하는 편이 유지보수에 유리하다.

14.2 std::optional

함수가 값을 항상 만들 수 있는 것은 아니다. 검색 실패, 설정 누락, 입력 부재를 특정 값으로 표시하면 정상 데이터와 실패 표시가 충돌할 수 있다.

```
int FindIndex(const std::vector<int>& values, int target)
{
    for (std::size_t index = 0; index < values.size(); ++index)
    {
        if (values[index] == target)
        {
            return static_cast<int>(index);
        }
    }

    return -1;
}
```

-1은 실패를 나타내지만 반환 타입만 보아서는 특별한 의미를 알 수 없다. std::optional<T>는 T 값이 있거나 없는 상태를 타입으로 표현한다.

14.2.1 값의 부재

```
#include <optional>
#include <vector>

std::optional<std::size_t> FindIndex(
    const std::vector<int>& values, int target)
{
    for (std::size_t index = 0; index < values.size(); ++index)
    {
        if (values[index] == target)
        {
            return index;
        }
```

```
    }

    return std::nullopt;
}
```

호출자는 반환값의 존재 여부를 확인한다.

```
auto index = FindIndex(values, 30);

if (index)
{
    std::cout << "index: " << *index << '\n';
}
else
{
    std::cout << "not found\n";
}
```

std::nullopt는 값이 없는 상태를 명시한다. std::optional<std::size_t>는 모든 std::size_t 값을 정상 결과로 사용할 수 있으므로 별도 센티널 값이 필요하지 않다.

optional은 실패 이유까지 표현하지 않는다. 값이 없다는 사실만 중요할 때 적합하다. 파일 열기 실패와 형식 오류를 구분해야 한다면 오류 타입을 함께 반환하는 방식이 필요하다.

14.2.2 생성, 대입, reset과 emplace

기본 생성된 optional에는 값이 없다.

```
std::optional<int> score;
```

std::nullopt로 빈 상태를 명시할 수도 있다.

```
std::optional<int> score = std::nullopt;
```

값으로 초기화하면 내부에 해당 값을 저장한다.

```
std::optional<int> score = 100;
std::optional<std::string> name{std::string{"Alice"}};
```

std::in_place는 저장 공간에서 객체를 직접 생성한다.

```
std::optional<std::string> message{
    std::in_place,
    5,
    '*'
};
```

message에는 별표 다섯 개로 구성된 문자열이 저장된다.

대입으로 상태를 바꿀 수 있다.

```
std::optional<int> value;
value = 10;
value = 20;
value = std::nullopt;
```

reset()은 저장된 값을 파괴하고 빈 상태로 만든다.

```
value = 30;
value.reset();
```

emplace()는 기존 값이 있으면 파괴한 뒤 전달한 인수로 새 값을 생성한다.

```
std::optional<std::string> text;
```

```
text.emplace(4, 'A');
text.emplace("updated");
```

첫 호출 후 문자열은 "AAAA", 둘째 호출 후 "updated"가 된다.

14.2.3 값 접근과 기본값

has_value()는 값의 존재 여부를 반환한다.

```
if (value.has_value())
{
    std::cout << *value << '\n';
}
```

optional은 조건식에서도 사용할 수 있다.

```
if (value)
{
    std::cout << *value << '\n';
}
```

operator*와 operator->는 저장된 값에 직접 접근한다.

```
std::optional<std::string> name = "Alice";
std::cout << *name << '\n';
std::cout << name->size() << '\n';
```

값이 없는 optional을 operator*나 operator->로 접근하면 프로그램의 동작은 보장되지 않는다. 존재 여부를 이미 확인한 코드에서 사용해야 한다.
value()는 값이 없으면 std::bad_optional_access 예외를 발생시킨다.

```
try
{
    std::cout << value.value() << '\n';
}
catch (const std::bad_optional_access&)
{
    std::cout << "no value\n";
}
```

value_or()는 값이 있으면 저장된 값을, 없으면 전달한 기본값을 반환한다.

```
std::optional<int> port;
int selectedPort = port.value_or(8080);
```

value_or()는 참조를 반환하지 않고 값을 반환한다. 큰 객체를 기본값으로 제공하면 복사나 이동 비용이 생길 수 있다.

```
std::optional<std::string> title;
std::string selected = title.value_or("Untitled");
```

값의 존재 여부에 따라 서로 다른 작업이 필요하다면 조건문이 더 분명하다.

```
if (title)
{
    UseTitle(*title);
}
else
{
    UseDefaultTitle();
}
```

14.2.4 포인터와 optional의 차이

포인터와 optional은 모두 값이 없음을 표현할 수 있지만 의미가 다르다.

표현	값의 위치	소유 관계	빈 상태
T*	외부 객체	보통 비 소유	nullptr
std::unique_ptr<T>	동적 객체	단독 소유	nullptr
std::shared_ptr<T>	동적 객체	공유 소유	nullptr
std::optional<T>	optional 내부	값 자체를 소유	std::nullopt

optional<T>는 T를 내부에 직접 저장한다. 값이 존재하면 별도 동적 할당 없이 optional 객체 안에 T의 수명이 시작된다.

```
std::optional<Player> player;
player.emplace("Knight", 100);
```

기존 객체를 가리키는 검색 결과라면 포인터가 자연스럽다.

```
Player* FindPlayer(std::vector<Player>& players, int id)
{
    for (Player& player : players)
    {
        if (player.GetId() == id)
        {
            return &player;
        }
    }

    return nullptr;
}
```

값을 새로 계산해 반환하거나 독립된 값의 부재를 표현한다면 optional<T>가 적합하다.
C++20의 std::optional<T>는 참조 타입을 원소로 사용할 수 없다.

```
// std::optional<Player&> result; // C++20에서는 사용할 수 없음
```

참조 결과가 필요하면 포인터 또는 std::reference_wrapper<T>를 optional에 저장할 수 있다.

```
std::optional<std::reference_wrapper<Player>>
FindPlayerReference(std::vector<Player>& players, int id)
{
    for (Player& player : players)
    {
        if (player.GetId() == id)
        {
            return std::ref(player);
        }
    }

    return std::nullopt;
}
```

반환된 래퍼는 원본 Player를 소유하지 않는다. 컨테이너 재할당이나 원소 제거로 참조가 무효화될 수 있다.

14.3 std::variant

프로그램의 한 위치에 서로 다른 타입의 값이 들어갈 수 있지만 가능한 타입 집합은 미리 정해져 있을 수 있다. 설정 값이 정수, 실수, 문자열 중 하나이거나 이벤트 데이터가 여러 종류 중 하나인 경우가 해당한다.
std::variant<Ts...>는 지정한 대안 타입 중 하나를 저장한다.

14.3.1 여러 타입 중 하나

```cpp
#include <string>
#include <variant>

std::variant<int, double, std::string> value;
```

기본 생성하면 첫 번째 대안 타입의 기본값을 저장하므로 value에는 int{}가 들어 있다.

```cpp
std::variant<int, double, std::string> value = 10;
value = 3.5;
value = std::string{"hello"};
```

대입할 때 선택 가능한 대안 타입이 결정되고 기존 값의 수명은 끝난다.
대안 타입이 중복되면 타입만으로 생성하거나 접근하기 어려울 수 있다.

```cpp
std::variant<int, int> duplicated{std::in_place_index<0>, 10};
```

타입의 의미가 다르다면 별도 래퍼 타입을 정의하는 편이 안전하다.

```cpp
struct PlayerId
{
    int value;
};
struct ItemId
{
    int value;
};
std::variant<PlayerId, ItemId> id = PlayerId{100};
```

두 타입 모두 내부에 int를 저장하지만 컴파일러는 서로 다른 의미 타입으로 구분한다.

14.3.2 대안 타입과 std::monostate

첫 번째 대안 타입이 기본 생성될 수 없으면 variant도 기본 생성할 수 없다.

```cpp
class Command
{
public:
    explicit Command(std::string text)
        : text(std::move(text))
    {
    }

private:
    std::string text;
};

// std::variant<Command, int> value; // 첫 대안이 기본 생성 불가
```

std::monostate를 첫 대안으로 두면 명시적인 빈 상태를 만들 수 있다.

```cpp
using Result = std::variant<std::monostate, int, std::string>;

Result result;
```

기본 상태는 std::monostate이다.

```cpp
if (std::holds_alternative<std::monostate>(result))
{
    std::cout << "empty\n";
}
```

std::monostate는 저장할 데이터가 없는 상태를 대안 타입 하나로 표현한다. 단순히 값이 있거나 없는 구조라면 std::optional<T>가 더 간단하다. 서로 다른 여러 상태가 각기 다른 데이터를 가질 때 variant가 적합하다.

14.3.3 index, holds_alternative, std::get과 std::get_if

index()는 현재 활성 대안의 위치를 반환한다.

```
std::variant<int, double, std::string> value = 3.5;
std::cout << value.index() << '\n'; // 1
```

std::holds_alternative<T>()는 지정한 타입이 활성 상태인지 확인한다.

```
if (std::holds_alternative<double>(value))
{
    std::cout << "double\n";
}
```

std::get<T>()와 std::get<I>()는 저장된 값에 참조로 접근한다.

```
double number = std::get<double>(value);
double sameNumber = std::get<1>(value);
```

활성 대안과 다른 타입을 std::get()으로 요구하면 std::bad_variant_access 예외가 발생한다.

```
try
{
    std::cout << std::get<std::string>(value) << '\n';
}
catch (const std::bad_variant_access&)
{
    std::cout << "wrong alternative\n";
}
```

std::get_if<T>()는 variant의 주소를 받고 타입이 일치하면 원소 포인터를, 일치하지 않으면 nullptr을 반환한다.

```
if (double* number = std::get_if<double>(&value))
{
    std::cout << *number << '\n';
}
```

예외 없이 분기하고 싶을 때 유용하다.

14.3.4 std::visit

활성 타입마다 다른 작업을 수행하려면 std::visit()을 사용할 수 있다.

```
std::variant<int, double, std::string>
value = std::string{"hello"};

std::visit(
    [](const auto& stored)
    {
        std::cout << stored << '\n';
    },
    value);
```

제네릭 람다의 매개변수 타입은 활성 대안에 맞게 결정된다. 모든 대안에서 람다 본문이 올바르게 컴파일되어야 한다.
타입마다 별도 동작이 필요하면 if constexpr을 사용할 수 있다.

```
std::visit(
    [](const auto& stored)
    {
        using T = std::remove_cvref_t<decltype(stored)>;

        if constexpr (std::is_same_v<T, int>)
        {
            std::cout << "integer: " << stored << '\n';
        }
```

```
        else if constexpr (std::is_same_v<T, double>)
        {
            std::cout << "real: " << stored << '\n';
        }
        else
        {
            std::cout << "text: " << stored << '\n';
        }
    },
    value);
```

여러 람다의 operator()를 하나의 방문자 객체로 결합할 수도 있다.

```
template <typename... Functions>
struct Overloaded : Functions...
{
    using Functions::operator()...;
};

template <typename... Functions>
Overloaded(Functions...) -> Overloaded<Functions...>;

std::visit(
    Overloaded{
        [](int number)
        {
            std::cout << "int: " << number << '\n';
        },
        [](double number)
        {
            std::cout << "double: " << number << '\n';
        },
        [](const std::string& text)
        {
            std::cout << "string: " << text << '\n';
        }
    },
    value);
```

방문자에 대안 타입 처리가 빠지면 컴파일 오류로 확인할 수 있다. 가능한 타입 집합이 컴파일 시점에 고정되어 있다는 장점이 드러난다.

14.3.5 상태 표현

상태에 따라 필요한 데이터가 다르면 여러 bool과 선택적 멤버를 한 구조체에 섞기보다 variant로 상태와 데이터를 결합할 수 있다.

```
struct Idle
{
};
struct Moving
{
    float speed;
};
struct Attacking
{
    int targetId;
    int damage;
};

using CharacterState = std::variant<Idle, Moving, Attacking>;

CharacterState state = Idle{};
state = Moving{3.5f};
state = Attacking{42, 15};
```

각 상태는 자신에게 필요한 데이터만 가진다. Idle 상태에서 공격 대상이나 이동 속도를 잘못 읽는 구조 자체가 만들어지지 않는다.

```
void UpdateState(CharacterState& state)
```

```
{
    std::visit(
        Overloaded{
            [](Idle&)
            {
                std::cout << "idle\n";
            },
            [](Moving& moving)
            {
                std::cout << "speed: " << moving.speed << '\n';
            },
            [](Attacking& attacking)
            {
                std::cout << "target: " << attacking.targetId << '\n';
            }
        },
        state);
}
```

상태를 추가하면 방문자 처리도 함께 수정해야 한다. 컴파일 오류는 누락된 상태 처리를 찾는 데 도움이 된다.

14.3.6 valueless_by_exception

variant는 보통 하나의 대안을 저장한다. 현재 대안을 제거하고 새 대안을 생성하는 과정에서 예외가 발생하면 어떤 대안도 저장하지 못하는 상태가 될 수 있다.

```
if (value.valueless_by_exception())
{
    std::cout << "no active alternative\n";
}
```

값을 잃은 상태에서 index()는 std::variant_npos를 반환한다.

```
if (value.index() == std::variant_npos)
{
    std::cout << "invalid variant state\n";
}
```

모든 대안 타입의 이동 생성과 상태 변경 연산이 예외를 발생시키지 않도록 설계하면 발생 가능성을 낮출 수 있다. 일반 코드에서 자주 만나는 상태는 아니지만 variant가 절대로 비어 있지 않다고 단정해서는 안 된다.
std::monostate는 사용자가 정의한 정상 대안이고, valueless_by_exception()은 대안 교체 실패로 생길 수 있는 예외 상태이다. 두 상태의 의미는 다르다.

14.3.7 상속 기반 다형성과의 차이

상속 기반 다형성은 공통 인터페이스를 가진 객체 계층을 포인터나 참조로 다룬다.

```
class Shape
{
public:
    virtual ~Shape() = default;
    virtual double Area() const = 0;
};
```

variant는 가능한 타입 목록을 한 타입 정의 안에 나열한다.
using ShapeValue = std::variant<Circle, Rectangle, Triangle>;

기준	상속 기반 다형성	std::variant
타입 집합	새 파생 타입으로 확장 가능	대안 목록에 고정
호출 방식	가상 함수	std::visit()
저장 방식	보통 포인터·참조	값 직접 저장 가능

공통 인터페이스	기반 클래스에 정의	방문자가 연산 정의
새 연산 추가	기반 인터페이스 변경 가능	방문자 추가로 처리 가능
새 타입 추가	파생 클래스 추가	variant와 관련 방문자 수정

플러그인처럼 타입이 계속 추가되고 각 타입이 공통 동작을 제공해야 한다면 상속이 자연스럽다. 타입 집합은 고정되어 있고 상태별 처리 함수를 자주 추가해야 한다면 variant가 적합할 수 있다.

14.4 std::any

std::variant는 저장 가능한 타입을 템플릿 인수로 미리 나열한다. 설정 시스템이나 편집기 속성처럼 저장 타입을 실행 시간에 결정해야 한다면 std::any를 사용할 수 있다.

14.4.1 복사 가능한 임의 타입 저장

```
#include <any>
#include <string>

std::any value = 10;
value = 3.5;
value = std::string{"hello"};
```

std::any 객체 하나에 서로 다른 타입을 차례로 저장할 수 있다. 대입할 때 기존 객체는 파괴되고 새 타입의 객체가 저장된다. C++20에서 std::any에 저장하는 타입은 복사 생성 가능해야 한다.

```
std::any number = 10;
// std::any pointer = std::make_unique<int>(10);
// 이동 전용 타입은 저장 불가
```

이동 전용 객체를 저장해야 한다면 std::variant에 타입을 명시하거나 소유권 구조를 별도로 설계해야 한다.

14.4.2 빈 상태와 저장 타입 확인

기본 생성한 std::any는 비어 있다.

```
std::any value;
```

has_value()로 값의 존재 여부를 확인한다.

```
if (!value.has_value())
{
    std::cout << "empty\n";
}
```

reset()은 저장된 객체를 파괴하고 빈 상태로 만든다.

```
value = std::string{"hello"};
value.reset();
```

emplace<T>()는 지정한 타입의 객체를 내부 저장 공간에 생성한다.

```
value.emplace<std::string>(5, '*');
```

type()은 저장된 타입의 std::type_info를 반환한다.

```
if (value.type() == typeid(std::string))
{
    std::cout << "string stored\n";
}
```

빈 any의 type()은 typeid(void)를 반환한다. 타입 이름 문자열은 구현에 따라 달라질 수 있으므로 프로그램 로직을 type().name() 결과에 의존시키지 않는다.

14.4.3 any_cast

std::any_cast<T>()는 저장 타입과 요청 타입이 일치할 때 값을 얻는다.

```
std::any value = std::string{"hello"};
std::string text = std::any_cast<std::string>(value);
```

참조 타입을 요구하면 저장된 객체를 복사하지 않고 접근한다.

```
std::string& text = std::any_cast<std::string&>(value);
text += " world";
```

값 또는 참조 형식의 any_cast에서 타입이 일치하지 않으면 std::bad_any_cast 예외가 발생한다.

```
try
{
    int number = std::any_cast<int>(value);
}
catch (const std::bad_any_cast&)
{
    std::cout << "type mismatch\n";
}
```

포인터 형식은 실패 시 nullptr을 반환한다.

```
if (std::string* text = std::any_cast<std::string>(&value))
{
    std::cout << *text << '\n';
}
```

타입이 불확실한 입력을 검사할 때 포인터 형식이 분기 코드와 잘 맞는다.

```
void PrintAny(const std::any& value)
{
    if (const int* number = std::any_cast<int>(&value))
    {
        std::cout << *number << '\n';
    }
    else if (const double* number = std::any_cast<double>(&value))
    {
        std::cout << *number << '\n';
    }
    else if (const std::string* text = std::any_cast<std::string>(&value))
    {
        std::cout << *text << '\n';
    }
    else
    {
        std::cout << "unsupported type\n";
    }
}
```

지원 타입이 고정되어 있다면 같은 분기를 variant와 visit으로 컴파일 시점에 검사할 수 있다.

14.4.4 variant와 any의 차이

```
using SettingValue = std::variant<int, double, std::string>;
std::any dynamicValue;
```

기준	std::variant	std::any

저장 타입	템플릿 인수로 제한	실행 시간에 결정
처리 누락	방문자 컴파일 오류로 발견 가능	실행 시간 검사 필요
타입 정보	대안 목록이 타입에 드러남	std::any만 보아서는 알 수 없음
이동 전용 타입	대안 조건이 허용하면 저장 가능	C++20에서는 저장 불가
용도	닫힌 타입 집합	개방된 속성 저장소

설정 값이 정수, 실수, 문자열 세 종류로 제한된다면 variant가 더 안전하다. 외부 모듈이 임의 타입의 값을 등록하는 속성 저장소라면 any가 유연하다.
any를 사용하면서 호출부마다 타입 문자열이나 수동 태그를 함께 관리하면 타입 안전성이 약해진다. 가능한 타입 집합을 알고 있다면 variant나 명시적인 클래스 계층을 우선 검토한다.

14.4.5 런타임 타입 저장의 비용

std::any는 저장 타입을 실행 시간에 관리하고 any_cast에서 타입을 확인한다. 저장 객체의 크기와 구현 전략에 따라 동적 메모리 할당이 발생할 수 있다. 작은 객체를 내부 버퍼에 저장하는 최적화가 사용될 수 있지만 표준이 모든 타입의 무할당 저장을 보장하지는 않는다.
복사 가능한 타입만 저장할

```
std::any original = std::string{"large text"};
std::any copied = original;t"};
std::any copied = original;
```

호출 빈도가 높은 내부 루프에서 타입 확인과 동적 할당 가능성이 문제가 된다면 구체 타입, variant, 템플릿 인터페이스를 비교해야 한다. 확장성과 런타임 유연성이 더 중요한 경계 계층에서는 비용을 감수할 수 있다.

14.5 std::reference_wrapper

C++ 참조는 선언 후 다른 객체를 가리키도록 바꿀 수 없고, 참조 자체를 컨테이너 원소 타입으로 사용할 수도 없다.

```
int first = 10;
int second = 20;

int& reference = first;
reference = second;
```

마지막 대입은 참조 대상을 second로 바꾸지 않는다. first에 second의 값인 20을 대입한다.

```
// std::vector<int&> references; // 사용할 수 없음
```

std::reference_wrapper<T>는 T&를 복사 가능하고 대입 가능한 객체 형태로 감싼다.

14.5.1 참조를 복사 가능한 객체로 저장

```
#include <functional>

int value = 10;
std::reference_wrapper<int> reference = std::ref(value);
```

get()은 원본 참조를 반환한다.

```
reference.get() = 20;
std::cout << value << '\n';
```

reference_wrapper<T>는 T&로 변환될 수 있다.

```
int& alias = reference;
alias = 30;
```

래퍼를 복사하면 원본 값을 복사하지 않고 같은 객체를 가리키는 래퍼가 만들어진다.

```
auto copied = reference;
copied.get() = 40;
```

reference와 copied가 모두 value를 가리키므로 value는 40이 된다.
함수 객체나 일반 함수도 감쌀 수 있다. reference_wrapper는 함수 호출 연산자를 제공하므로 감싼 호출 대상을 직접 호출할 수 있다.

```
int AddOne(int value)
{
    return value + 1;
}

std::reference_wrapper<int(int)> function = std::ref(AddOne);
int result = function(10);
```

14.5.2 컨테이너와 참조

서로 독립적으로 존재하는 객체를 한 컨테이너에서 참조하고 싶다면 reference_wrapper를 원소 타입으로 사용할 수 있다.

```
#include <functional>
#include <string>
#include <vector>

struct Player
{
    std::string name;
    int hp;
};

Player knight{"Knight", 100};
Player mage{"Mage", 80};
Player archer{"Archer", 90};

std::vector<std::reference_wrapper<Player>> party{
    knight,
    mage,
    archer
};

for (Player& player : party)
{
    player.hp += 10;
}
```

컨테이너는 Player 객체를 복사하지 않는다. 각 래퍼가 기존 객체를 가리킨다.
포인터 컨테이너로도 같은 관계를 표현할 수 있다.

```
std::vector<Player*> pointers{&knight, &mage, &archer};
```

포인터는 nullptr을 저장할 수 있고 포인터 연산 문법을 사용한다. reference_wrapper는 생성 시 유효한 참조가 필요하고 값처럼 복사하면서 참조 의미를 유지한다. 빈 참조가 필요한 구조라면 포인터나 optional<reference_wrapper<T>>가 적합하다.

14.5.3 std::ref, std::cref와 get

std::ref()는 수정 가능한 참조 래퍼를 만든다.

```
int score = 100;
auto scoreReference = std::ref(score);
```

std::cref()는 상수 참조 래퍼를 만든다.

```
const std::string name = "Alice";
auto nameReference = std::cref(name);

// nameReference.get() += "!"; // 오류
```

템플릿 함수가 인수를 값으로 복사하는 경우에도 std::ref()를 사용해 참조 의미를 전달할 수 있다.

```
template <typename Function>
void ExecuteTwice(Function function)
{
    function();
    function();
}

struct Counter
{
    int value = 0;

    void operator()()
    {
        ++value;
    }
};

Counter counter;
ExecuteTwice(std::ref(counter));
```

ExecuteTwice()의 매개변수는 값으로 전달되지만 복사되는 것은 작은 참조 래퍼이다. 두 호출은 원본 counter를 수정한다.
std::ref()에 이미 reference_wrapper가 전달되면 래퍼를 겹쳐 만들지 않고 같은 참조 의미를 유지한다.

14.5.4 대상 객체의 수명

reference_wrapper는 대상을 소유하지 않는다. 대상이 소멸하면 래퍼는 더 이상 사용할 수 없다.

```
std::reference_wrapper<int> MakeInvalidReference()
{
    int local = 10;
    return std::ref(local); // 지역 변수 소멸 후 댕글링
}
```

MakeInvalidReference()는 컴파일될 수 있지만 반환된 래퍼가 가리키는 local은 함수 종료 시 소멸한다.
컨테이너 원소를 참조한 래퍼도 원본 컨테이너의 무효화 규칙을 따른다.

```
std::vector<Player> players;
players.reserve(2);
players.push_back({"Knight", 100});
players.push_back({"Mage", 80});

std::reference_wrapper<Player> selected = players[0];
players.push_back({"Archer", 90});
```

세 번째 삽입에서 재할당이 발생하면 selected가 가리키던 원소의 주소가 바뀌고 래퍼는 무효가 된다. 래퍼 타입 자체에는 무효화 여부를 검사하는 기능이 없다.
참조 래퍼를 저장하기 전에 원본 객체의 소유 위치, 이동 가능성, 컨테이너 재할당 여부를 확인해야 한다.

14.6 std::span

배열을 함수에 전달할 때 포인터만 받으면 원소 수를 알 수 없다.

```
void PrintValues(const int* values, std::size_t count)
{
    for (std::size_t index = 0; index < count; ++index)
    {
        std::cout << values[index] << ' ';
    }
}
```

포인터와 길이는 하나의 범위를 이루지만 서로 독립된 인수라서 잘못된 길이를 전달할 수 있다. std::span<T>은 연속 메모리 구간의 시작 위치와 원소 수를 하나의 비 소유 객체로 묶는다.

14.6.1 포인터와 길이를 묶는 비 소유 구간

```
#include <span>
void PrintValues(std::span<const int> values)
{
    for (int value : values)
    {
        std::cout << value << ' ';
    }
}
```

span은 원소를 소유하거나 복사하지 않는다. 원본 메모리를 가리키는 작은 범위 객체이다.

```
int values[]{10, 20, 30};
PrintValues(values);
```

size(), empty(), data()로 범위 상태를 확인할 수 있다.

```
std::span<int> view{values};
std::size_t count = view.size();
bool isEmpty = view.empty();
int* firstAddress = view.data();
```

operator[], front(), back()으로 원소에 접근한다.

```
view[0] = 100;
view.front() = 200;
view.back() = 300;
```

C++20의 std::span::operator[]는 범위를 검사하지 않는다. 인덱스가 유효하다는 전제가 필요하다.

14.6.2 배열과 컨테이너 전달

C 배열, std::array, std::vector는 원소가 연속 메모리에 저장되므로 span으로 전달할 수 있다.

```
#include <array>
#include <vector>

int first[]{10, 20, 30};
std::array<int, 3> second{40, 50, 60};
std::vector<int> third{70, 80, 90};

PrintValues(first);
PrintValues(second);
PrintValues(third);
```

std::list는 원소가 연속 메모리에 저장되지 않으므로 span으로 만들 수 없다.

```
std::list<int> values{10, 20, 30};
// PrintValues(values); // 오류
```

함수 하나가 여러 연속 컨테이너를 받을 수 있으므로 포인터와 길이 오버로드를 반복해서 정의할 필요가 줄어든다.
수정 가능한 범위를 받으면 원본 원소를 변경할 수 있다.

```
void DoubleValues(std::span<int> values)
{
    for (int& value : values)
    {
        value *= 2;
    }
}
```

읽기 전용 함수는 std::span<const T>로 표현한다.

14.6.3 정적 크기와 동적 크기

std::span<T>의 두 번째 템플릿 인수는 원소 수인 extent이다. 생략하면 std::dynamic_extent가 사용된다.

```
std::span<int> dynamicView{values};s};
```

원소 수가 컴파일 시점에 고정된 인터페이스에는 정적 extent를 지정할 수 있다.

```
void NormalizeColor(std::span<float, 4> color)
{
    // RGBA 네 원소만 받음
}
std::array<float, 4> color{1.0f, 0.5f, 0.25f, 1.0f};
NormalizeColor(color);
```

크기가 다른 배열은 호출할 수 없다.

```
std::array<float, 3> position{1.0f, 2.0f, 3.0f};
// NormalizeColor(position); // 오류
```

정적 extent는 인터페이스의 크기 요구를 타입에 포함한다. 동적 extent는 실행 시간에 크기가 달라지는 벡터와 부분 구간을 받는 데 적합하다.
정적 extent의 size()도 컴파일 시점 상수로 사용할 수 있다.

```
std::span<float, 4> colorView{color};
static_assert(colorView.extent == 4);
```

14.6.4 span의 const와 원소의 const

span 객체의 상수성과 원소의 상수성은 별개이다.

```
int values[]{10, 20, 30};
const std::span<int> fixedView{values};
fixedView[0] = 100;
```

fixedView가 상수이므로 다른 범위를 가리키도록 대입할 수 없지만 element_type은 int이므로 원소는 수정할 수 있다.

```
std::span<const int> readOnlyView{values};
// readOnlyView[0] = 100; // 오류
```

읽기 전용 인터페이스를 만들 때는 const std::span<int>가 아니라 std::span<const int>를 사용한다.

선언	span을 다른 범위로 대입	원소 수정
std::span<int>	가능	가능
const std::span<int>	불가	가능
std::span<const int>	가능	불가
const std::span<const int>	불가	불가

함수 매개변수는 값으로 전달되므로 const std::span<const int>처럼 span 객체 자체를 상수로 제한할 필요는 거의 없다. 원소 수정 가능 여부가 인터페이스 의미에 더 중요하다.

14.6.5 부분 구간

first<N>(), last<N>(), subspan()으로 부분 범위를 만들 수 있다.

```
std::array<int, 6> values{10, 20, 30, 40, 50, 60};
std::span<int> all{values};

auto firstThree = all.first<3>();
auto lastTwo = all.last<2>();
auto middle = all.subspan(1, 3);

// firstThree: 10, 20, 30
```

```
// lastTwo: 50, 60
// middle: 20, 30, 40
```

부분 span도 원본 데이터를 참조한다.

```
middle[0] = 200;
```

values[1]이 200으로 바뀐다.
실행 시간 인수도 사용할 수 있다.
std::size_t offset = 2;
std::size_t count = 3;
auto selected = all.subspan(offset, count);

오프셋과 길이가 원본 범위를 벗어나지 않아야 한다. span은 범위의 소유권이나 자동 확장을 제공하지 않는다.

14.6.6 수명과 무효화

span은 원본 메모리보다 오래 존재할 수 없다.

```
std::span<int> MakeInvalidSpan()
{
    int values[]{10, 20, 30};
    return values;
}
```

반환된 span이 가리키는 배열은 함수 종료 시 소멸한다.
벡터의 재할당도 span을 무효화한다.

```
std::vector<int> values;
values.reserve(3);
values.push_back(10);
values.push_back(20);
values.push_back(30);

std::span<int> view{values};

values.push_back(40); // 재할당 가능
```

재할당이 발생하면 기존 원소 주소가 바뀌므로 view의 data()가 더 이상 유효하지 않다. span은 벡터의 크기 변화를 자동으로 추적하지 않는다. 재할당이 없더라도 span 생성 후 벡터 크기가 증가하면 기존 span의 size()는 바뀌지 않는다.

```
std::vector<int> values{10, 20, 30};
values.reserve(10);

std::span<int> view{values};
values.push_back(40);

std::cout << view.size() << '\n'; // 3
```

view는 생성 시점의 세 원소 범위를 유지한다. 새 원소까지 포함하려면 span을 다시 만들어야 한다.

원본 컨테이너를 변경하지 않는 함수 호출 동안만 사용하는 매개변수 span은 수명 관리가 단순하다. span을 멤버에 저장하거나 반환할 때는 원본 소유 객체가 더 오래 유지된다는 조건을 인터페이스에 분명히 해야 한다.

14.7 std::string_view

문자열을 읽기만 하는 함수에서 const std::string&를 사용하면 std::string 객체를 받아 복사를 피할 수 있다.

```
void PrintName(const std::string& name)
{
    std::cout << name << '\n';
}
```

문자열 리터럴이나 다른 문자 버퍼를 전달하면 std::string 임시 객체가 생성될 수 있다. std::string_view는 연속된 문자 구간을 소유하지 않고 참조한다.

14.7.1 비 소유 문자열

```
#include <string_view>
void PrintName(std::string_view name)
{
    std::cout << name << '\n';
}
```

문자열 리터럴과 std::string을 모두 받을 수 있다.
PrintName("Alice");

std::string name = "Bob";
PrintName(name);

string_view는 문자 포인터와 길이를 저장한다. 원본 문자를 복사하지 않으므로 생성 비용이 작고 부분 문자열도 상수 시간에 표현할 수 있다.
std::string_view text = "Hello, World";
std::cout << text.size() << '\n';
std::cout << text[0] << '\n';

remove_prefix()와 remove_suffix()는 뷰의 시작 위치와 길이를 조정한다. 원본 문자열은 변경하지 않는다.
std::string_view text = "[message]";
text.remove_prefix(1);
text.remove_suffix(1);

std::cout << text << '\n'; // message

14.7.2 문자열 복사 방지

문자열을 값으로 받으면 호출 때마다 복사가 생길 수 있다.

```
bool StartsWithA(std::string text)
{
    return !text.empty() && text.front() == 'A';
}
```

읽기만 하는 함수는 std::string_view로 여러 문자열 표현을 통합할 수 있다.

```
bool StartsWithA(std::string_view text)
{
    return !text.empty() && text.front() == 'A';
}
```

반환값이나 멤버에 문자열을 보관해야 한다면 소유 문자열로 복사해야 할 수 있다.

```
class Player
{
public:
    explicit Player(std::string_view name)
        : name(name)
    {
    }

private:
    std::string name;
};
```

생성자 매개변수는 비 소유 뷰지만 멤버 std::string은 문자를 복사해 소유한다. 생성 이후 원본 문자열의 수명에 의존하지 않는다. 모든 문자열 매개변수를 string_view로 바꾸는 것이 정답은 아니다. 함수가 전달된 std::string을 이동해 보관하려면 값 매개변수나 std::string&&가 더 적합할 수 있다.

14.7.3 부분 문자열과 함수 매개변수

substr()은 원본 문자를 복사하지 않고 부분 뷰를 반환한다.

```
std::string_view path = "assets/characters/player.png";
std::string_view folder = path.substr(0, 17);
std::string_view fileName = path.substr(18);
```

folder와 fileName은 path가 가리키는 문자 배열의 일부를 참조한다.
구분자를 찾아 뷰를 나눌 수 있다.

```
std::pair<std::string_view, std::string_view> SplitOnce(
    std::string_view text,
    char delimiter)
{
    std::size_t position = text.find(delimiter);

    if (position == std::string_view::npos)
    {
        return {text, {}};
    }

    return {
        text.substr(0, position),
        text.substr(position + 1)
    };
}

auto [key, value] = SplitOnce("width=1920", '=');
```

반환된 두 뷰는 호출 인수의 문자 저장 공간을 참조한다. 문자열 리터럴은 프로그램 실행 동안 유지되므로 SplitOnce("width=1920", '=')에서 얻은 뷰는 유효하다. 지역 std::string에서 만든 뷰를 함수 밖에서 보관하면 원본 수명을 확인해야 한다.

14.7.4 널 종료와 data

string_view가 가리키는 구간은 널 문자로 끝난다고 보장할 수 없다.

```
std::string text = "hello world";
std::string_view word{text.data(), 5};
```

word의 길이는 5이고 내용은 "hello"이지만 word.data()[5]에는 원본 문자열의 공백이 있다. word.data()를 널 종료 문자열을 요구하는 함수에 전달하면 뷰의 경계를 무시하고 뒤 문자까지 읽을 수 있다.

```
// std::printf("%s\n", word.data()); // hello world가 출력될 수 있음
```

스트림의 operator<<에 string_view 자체를 전달하면 길이를 사용해 올바른 범위만 출력한다.

```
std::cout << word << '\n';
```

C API에 널 종료 문자열을 전달해야 한다면 소유 문자열로 변환할 수 있다.

```
std::string nullTerminated{word};
UseCString(nullTerminated.c_str());
```

빈 string_view의 data()가 항상 사용할 수 있는 널 종료 주소라고 가정해서도 안 된다. 빈 문자열 여부는 empty()로 판단한다.

14.7.5 댕글링 뷰

임시 std::string으로 string_view를 초기화하면 문장이 끝날 때 문자열이 소멸한다.

```
std::string_view name = std::string{"Alice"};
```

// name은 소멸한 임시 문자열을 가리킴

지역 문자열을 참조하는 뷰를 반환해도 같은 문제가 생긴다.

```
std::string_view MakeInvalidName()
{
```

```
    std::string name = "Alice";
    return name;
}
```

소유 문자열을 반환하거나 호출자가 소유한 문자열을 인수로 받아 부분 뷰를 반환해야 한다.

```
std::string_view FileName(std::string_view path)
{
    std::size_t position = path.find_last_of("/\\");

    if (position == std::string_view::npos)
    {
        return path;
    }

    return path.substr(position + 1);
}

std::string path = "assets/player.png";
std::string_view fileName = FileName(path);
```

fileName은 path보다 오래 사용할 수 없다.
원본 std::string이 재할당되면 기존 뷰가 무효화될 수 있다.

```
std::string text = "short";
std::string_view view = text;
text += " text that may cause reallocation";
```

재할당이 발생하면 view.data()는 이전 저장 공간을 가리킨다. 원본 문자를 수정하면 뷰에서 보이는 내용도 바뀔 수 있다. 뷰는 문자열의 스냅샷이나 복사본이 아니다.

14.8 비트와 바이트

정수의 비트 연산은 플래그, 마스크, 프로토콜 필드, 압축 데이터에 사용된다. 메모리의 원시 바이트를 다룰 때는 숫자 값과 객체 표현을 구분해야 한다. C++ 표준 라이브러리는 std::byte, std::bitset, <bit>의 유틸리티를 제공한다.

14.8.1 std::byte

std::byte는 바이트 단위의 메모리 데이터를 표현하는 열거형 타입이다.

```
#include <cstddef>
std::byte value{0x2A};
```

char, unsigned char와 달리 일반 정수처럼 덧셈이나 곱셈을 수행하지 않는다.

```
// value += std::byte{1}; // 오류
```

비트 연산은 사용할 수 있다.

```
std::byte flags{0b0000'0101};
flags |= std::byte{0b0000'1000};
flags &= std::byte{0b0000'1111};
```

정수로 읽으려면 std::to_integer<T>()를 사용한다.

```
unsigned int number = std::to_integer<unsigned int>(flags);
```

std::byte는 숫자 한 개보다 메모리 한 바이트라는 의미를 분명히 한다. 바이너리 파일, 네트워크 패킷, 직렬화 버퍼에서 원시 데이터와 수치 데이터를 구분하는 데 유용하다.

```
std::vector<std::byte> buffer(1024);
```

std::as_bytes()는 연속된 객체 범위를 읽기 전용 바이트 범위로 본다.

```
std::array<std::uint32_t, 2> values{0x12345678u, 0x90ABCDEFu};
std::span<const std::byte> bytes = std::as_bytes(std::span{values});
```

std::as_writable_bytes()는 수정 가능한 객체 범위를 바이트 범위로 제공한다.

```
std::span<std::byte> writable =
    std::as_writable_bytes(std::span{values});
```

바이트를 임의로 바꾸면 원래 타입에서 의미 없는 값이나 유효하지 않은 표현이 만들어질 수 있다. 객체 형식의 규칙을 알고 있는 저수준 코드에서만 수정한다.
std::byte 배열에 객체의 바이트를 복사했다고 해서 다른 타입 객체가 자동으로 생성되지는 않는다. 객체 수명, 정렬, 타입 별칭 규칙을 지켜야 한다.

14.8.2 std::bitset

std::bitset<N>은 컴파일 시점에 크기가 정해진 비트 묶음을 저장한다.

```
#include <bitset>
std::bitset<8> permissions{0b0010'0101};
```

인덱스로 비트를 읽거나 수정할 수 있다.

```
bool firstBit = permissions.test(0);
permissions.set(1);
permissions.reset(2);
permissions.flip(5);
```

operator[]도 제공하지만 test()는 인덱스 범위를 검사하고 범위를 벗어나면 std::out_of_range를 발생시킨다.

```
permissions[0] = false;
```

전체 비트 상태를 확인하는 함수도 있다.

```
std::size_t enabledCount = permissions.count();
bool hasAny = permissions.any();
bool hasNone = permissions.none();
bool hasAll = permissions.all();
```

문자열로 변환하면 사람이 읽기 쉬운 비트열을 얻을 수 있다.

```
std::string bits = permissions.to_string();
std::cout << bits << '\n';
```

정수로 변환할 때 대상 타입의 범위를 넘으면 std::overflow_error가 발생할 수 있다.

```
unsigned long raw = permissions.to_ulong();
```

bitset은 크기가 타입에 포함되므로 실행 시간에 비트 수를 바꿀 수 없다. 동적 크기의 비트 집합이 필요하면 별도 컨테이너나 라이브러리를 검토해야 한다.

14.8.3 비트 연산과 비트 플래그

열거형 값에 비트 위치를 부여하면 여러 상태를 정수 하나에 조합할 수 있다.

```
enum class Permission : unsigned int
{
    None    = 0,
    Read    = 1u << 0,
    Write   = 1u << 1,
    Execute = 1u << 2
};
```

enum class는 정수와 암시적으로 섞이지 않으므로 필요한 비트 연산자를 정의한다.

```
constexpr Permission operator|(Permission left, Permission right)
{
    return static_cast<Permission>(
        static_cast<unsigned int>(left) |
        static_cast<unsigned int>(right));
}
constexpr Permission operator&(Permission left, Permission right)
{
    return static_cast<Permission>(
        static_cast<unsigned int>(left) &
        static_cast<unsigned int>(right));
}
constexpr bool HasPermission(Permission value, Permission flag)
{
    return (value & flag) == flag;
}

Permission permissions = Permission::Read | Permission::Write;

if (HasPermission(permissions, Permission::Write))
{
    std::cout << "writable\n";
}
```

플래그 값은 서로 겹치지 않는 한 비트로 정의해야 한다. 조합 값을 열거자로 제공하려면 정수 비트식을 사용한다.

```
enum class Permission : unsigned int
{
    None    = 0,
    Read    = 1u << 0,
    Write   = 1u << 1,
    Execute = 1u << 2,
    All     = (1u << 0) | (1u << 1) | (1u << 2)
};
```

14.8.4 C++20 비트 유틸리티

<bit>에는 부호 없는 정수의 비트 패턴을 다루는 함수가 있다.

```
#include <bit>
#include <cstdint>

std::uint32_t value = 0b0010'1000;
```

std::popcount()는 1인 비트 수를 센다.

```
int count = std::popcount(value); // 2
```

std::has_single_bit()는 값이 2의 거듭제곱인지 확인한다.

```
bool powerOfTwo = std::has_single_bit(std::uint32_t{64});
```

std::bit_width()는 값을 표현하는 데 필요한 비트 수를 반환한다.

```
int width = std::bit_width(std::uint32_t{100}); // 7
```

std::bit_floor()와 std::bit_ceil()은 값 이하 또는 이상에서 가장 가까운 2의 거듭제곱을 구한다.

```
auto lower = std::bit_floor(std::uint32_t{100}); // 64
auto upper = std::bit_ceil(std::uint32_t{100});  // 128
```

std::countl_zero(), std::countr_zero()는 앞쪽과 뒤쪽의 연속된 0 비트 수를 센다.

```
int leading = std::countl_zero(value);
int trailing = std::countr_zero(value);
```

std::rotl()과 std::rotr()은 비트를 순환 이동한다. 일반 시프트와 달리 밀려난 비트가 반대쪽으로 돌아온다.

```
std::uint32_t rotated = std::rotl(value, 3);
```

함수 대부분은 부호 없는 정수 타입을 대상으로 한다. 부호 있는 정수의 오른쪽 시프트나 최상위 비트 해석에 의존하기보다 명확한 부호 없는 타입을 사용하는 편이 안전하다.

14.8.5 std::bit_cast

std::bit_cast<To>(from)은 원본 객체의 비트 표현을 같은 크기의 대상 타입으로 복사한다.

```
#include <bit>
#include <cstdint>

float value = 1.0f;
std::uint32_t bits = std::bit_cast<std::uint32_t>(value);
```

대상 타입과 원본 타입의 크기가 같아야 하고 두 타입은 trivially copyable 조건을 만족해야 한다.

```
static_assert(sizeof(float) == sizeof(std::uint32_t));
```

reinterpret_cast로 다른 타입 포인터를 만들어 역참조하는 방식은 타입 별칭 규칙과 정렬 문제를 일으킬 수 있다.

```
// std::uint32_t bits =
*reinterpret_cast<std::uint32_t*>(&value); // 사용하지 않음
```

bit_cast는 비트를 복사해 새 값을 만들기 때문에 포인터 별칭을 만들지 않는다.
수치 변환과 비트 변환은 결과가 다르다.

```
float value = 1.0f;
std::uint32_t numeric = static_cast<std::uint32_t>(value); // 정수 1
std::uint32_t representation = std::bit_cast<std::uint32_t>(value);
```

representation은 float의 객체 표현을 담는다. 특정 비트값은 부동소수점 형식에 따라 달라질 수 있다. bit_cast 결과를 파일 형식으로 그대로 저장하면 플랫폼 간 호환성이 자동으로 보장되지 않는다.

14.8.6 엔디언 (Endian)

여러 바이트로 구성된 정수를 메모리에 배치하는 순서는 시스템에 따라 다를 수 있다.
값: 0x12345678

리틀 엔디언 메모리 순서: 78 56 34 12
빅 엔디언 메모리 순서: 12 34 56 78

C++20의 std::endian은 구현의 기본 바이트 순서를 나타낸다.

```
#include <bit>
if constexpr (std::endian::native == std::endian::little)
{
    std::cout << "little endian\n";
}
else if constexpr (std::endian::native == std::endian::big)
{
    std::cout << "big endian\n";
}
else
{
    std::cout << "mixed endian\n";
}
```

네트워크 프로토콜이나 파일 형식은 바이트 순서를 별도로 정한다. 메모리의 정수 값을 바이트 배열로 복사하는 것만으로 규격에 맞는 직렬화가 끝나지 않는다. 저장 형식의 엔디언에 맞춰 바이트 순서를 변환해야 한다.

C++20에는 표준 byteswap()이 없다. 직접 마스크와 시프트를 사용하거나 플랫폼 API를 이용할 수 있다. std::byteswap()은 C++23에서 추가되었다.

14.8.7 객체 표현 시 주의점

객체의 메모리 바이트 전체를 객체 표현이라고 한다. 값 표현에 사용되지 않는 패딩 비트가 포함될 수 있고, 구조체 멤버 사이에도 패딩 바이트가 들어갈 수 있다.

```
struct Data
{
    char code;
    int value;
};
```

sizeof(Data)가 sizeof(char) + sizeof(int)와 같다고 보장할 수 없다. 정렬을 맞추기 위해 멤버 사이에 패딩이 들어갈 수 있다.
구조체 메모리를 그대로 파일에 기록하면 여러 문제가 생긴다.

- 컴파일러와 ABI에 따른 패딩 차이
- 엔디언 차이
- 정수와 부동소수점 표현 차이
- 포인터 값과 프로세스 내부 주소
- 클래스 불변식과 객체 수명
- 버전 변경으로 인한 멤버 배치 변화

직렬화 형식은 각 필드의 크기, 부호, 바이트 순서, 문자열 인코딩을 명시해야 한다.

```
struct PacketHeader
{
    std::uint16_t type;
    std::uint32_t payloadSize;
};
```

PacketHeader의 메모리를 그대로 전송하지 않고 필드별로 규격에 맞는 바이트를 기록하는 편이 안전하다.

std::memcmp()로 일반 구조체의 값 동등성을 판단하는 것도 안전하지 않다. 같은 값을 가진 두 객체의 패딩 바이트가 다를 수 있고, 포인터나 문자열 같은 멤버는 주소만 비교될 수 있다. 값 비교는 멤버 의미에 맞는 operator==를 사용한다.

14.9 타입 안전한 데이터 표현

타입 안전한 데이터 표현은 허용되는 상태를 타입으로 제한하고 호출부에서 값의 의미를 드러내는 설계이다. 단순한 기본형만으로 여러 의미를 표현하면 잘못된 인수 순서와 유효하지 않은 조합을 컴파일러가 구분하기 어렵다.

14.9.1 불리언 플래그의 한계

두 개 이상의 bool 인수는 호출부에서 의미를 파악하기 어렵다.

```
void OpenWindow(bool border, bool resizable);
OpenWindow(true, false);
```

true와 false가 무엇을 뜻하는지 선언을 확인해야 한다. 인수 순서를 바꾸어도 컴파일된다.
열거형으로 의미를 분리할 수 있다.

```
enum class BorderMode
{
    Hidden,
    Visible
};
enum class ResizeMode
{
    Fixed,
    Resizable
};

void OpenWindow(BorderMode border, ResizeMode resize);
OpenWindow(BorderMode::Visible, ResizeMode::Fixed);
```

호출부에 의미가 드러나고 서로 다른 열거형을 바꾸어 전달하면 컴파일 오류가 발생한다.
상태가 단순한 참·거짓이고 함수 이름으로 의미가 충분히 드러나는 경우에는 bool이 적합할 수 있다.

```
player.SetVisible(true);
```

문제는 bool 자체가 아니라 같은 타입의 값 여러 개가 서로 다른 의미를 가지면서 호출부에서 구분되지 않는 구조이다.

14.9.2 enum class와 의미 타입

같은 기본형이라도 의미가 다르면 작은 래퍼 타입으로 구분할 수 있다.

```
struct PlayerId
{
    int value;
};
struct ItemId
{
    int value;
};

void EquipItem(PlayerId playerId, ItemId itemId);

PlayerId playerId{100};
ItemId itemId{200};

EquipItem(playerId, itemId);
// EquipItem(itemId, playerId); // 오류
```

두 타입은 내부 표현이 같아도 서로 암시적으로 변환되지 않는다.
단위도 타입으로 분리할 수 있다.

```
struct Seconds
{
    float value;
};
struct Meters
{
    float value;
};
void Move(Meters distance, Seconds duration);
```

의미 타입은 값 검사와 불변식도 한곳에 모을 수 있다.

```
class Health
{
public:
    explicit Health(int value)
        : value(value < 0 ? 0 : value)
    {
    }

    int Get() const
    {
        return value;
    }

private:
    int value;
};
```

원시 int가 프로그램 전체를 이동하면서 음수 체력이 만들어지는 문제를 줄일 수 있다.
enum class는 닫힌 선택지를 표현하고, 래퍼 클래스는 값과 검증 규칙을 함께 표현한다.

14.9.3 optional, variant와 expected의 선택

세 타입은 상태를 값으로 표현하지만 목적이 다르다.

표현하려는 상태	타입
값이 있거나 없음	std::optional<T>
정해진 여러 타입 중 하나	std::variant<Ts...>
성공값 또는 오류 정보	std::expected<T, E>

사용자 이름 검색에서 찾지 못함이 정상적인 결과이고 이유가 필요 없다면 optional이 적합하다.

```
std::optional<Player> FindPlayer(int id);
```

설정 값이 정수, 실수, 문자열 중 하나라면 variant가 적합하다.

```
using SettingValue = std::variant<int, double, std::string>;
```

파일 읽기에서 파일 없음, 권한 부족, 형식 오류를 구분해야 한다면 성공값과 오류 정보를 함께 표현해야 한다.

```
// C++23
std::expected<FileData, FileError> LoadFile(std::string_view path);
```

optional에 별도 전역 오류 값을 붙이거나 variant<T, Error>를 직접 구성할 수도 있지만 expected는 성공과 실패라는 역할을 인터페이스에 직접 나타낸다.
예외가 더 적합한 경우도 있다. 호출자가 대부분의 오류를 현 위치에서 처리할 수 없고 여러 호출 단계를 건너뛰어 처리해야 한다면 예외가 코드 흐름을 단순하게 만들 수 있다. expected는 오류를 일반 반환값처럼 명시적으로 검사하고 전달할 때 적합하다.

14.9.4 유효하지 않은 상태 줄이기

여러 불리언과 선택적 필드를 한 구조체에 모으면 서로 모순되는 상태가 만들어질 수 있다.

```
struct RequestResult
{
    bool succeeded;
    bool timedOut;
    std::string data;
    std::string errorMessage;
};
```

succeeded가 true이면서 timedOut도 true일 수 있고, 성공했는데 오류 메시지만 있거나 실패했는데 데이터가 남아 있을 수도 있다. 타입이 모순된 조합을 막지 못한다.

성공과 실패 데이터를 별도 타입으로 분리할 수 있다.

```
struct Success
{
    std::string data;
};
struct Timeout
{
    int elapsedMilliseconds;
};
struct NetworkError
{
    int errorCode;
};

using RequestResult = std::variant<Success, Timeout,
NetworkError>;
```

한 시점에는 한 상태만 활성화된다. 상태별 데이터도 해당 타입 안에만 존재한다.
객체 생성 단계에서 유효성을 보장하는 방법도 있다.

```
class Percentage
{
public:
```

```
    static std::optional<Percentage> Create(int value)
    {
        if (value < 0 || value > 100)
        {
            return std::nullopt;
        }
        return Percentage{value};
    }

    int Get() const
    {
        return value;
    }

private:
    explicit Percentage(int value)
        : value(value)
    {
    }

    int value;
};
```

생성에 성공한 Percentage 객체는 항상 0부터 100 사이의 값을 가진다. 각 멤버 함수에서 범위를 반복 검사할 필요가 줄어든다.
타입 안전한 표현은 모든 오류를 없애지는 않는다. 값의 의미와 허용 상태를 타입에 반영하면 잘못된 조합이 만들어지는 위치를 줄이고 컴파일러가 검사할 수 있는 범위를 넓힌다.

14.10 학습 정리

std::pair는 두 값을, std::tuple은 여러 값을 하나의 객체로 묶는다. std::get()은 위치나 고유 타입으로 원소에 접근하고, std::tie()는 기존 변수를 참조 튜플로 묶는다. 구조적 바인딩의 auto, auto&, const auto&는 복사와 참조 여부를 결정한다. 여러 반환값의 의미가 오래 유지된다면 위치 기반 튜플보다 이름 있는 구조체가 더 분명하다.

std::optional<T>는 T 값이 있거나 없는 상태를 표현한다. operator*와 operator->는 값 존재를 전제로 하고, value()는 값이 없을 때 예외를 발생시키며, value_or()는 기본값을 값으로 반환한다. optional은 값을 내부에 소유하고 실패 이유는 저장하지 않는다.

std::variant<Ts...>는 정해진 대안 타입 중 하나를 저장한다. index(), holds_alternative(), get(), get_if()로 활성 대안을 확인하고 접근한다. std::visit()은 활성 타입에 맞는 연산을 적용한다. 대안 교체 중 예외가 발생하면 valueless_by_exception() 상태가 될 수 있다.

std::any는 실행 시간에 저장 타입을 결정한다. C++20에서는 복사 생성 가능한 타입을 저장하며 any_cast로 타입을 검사해 접근한다. 가능한 타입 집합을 알고 있다면 variant가 더 많은 오류를 컴파일 시점에 발견할 수 있다.

std::reference_wrapper<T>는 참조를 복사 가능한 객체로 감싼다. 컨테이너에 기존 객체의 참조를 저장하거나 값 전달 템플릿에 참조 의미를 전달할 수 있다. 대상을 소유하지 않으므로 대상 소멸과 컨테이너 재할당 후에는 사용할 수 없다.

std::span<T>은 연속 메모리의 포인터와 길이를 하나의 비 소유 범위로 묶는다. C 배열, std::array, std::vector를 같은 함수 인터페이스로 받을 수 있다. const std::span<int>는 원소를 수정할 수 있지만 std::span<const int>는 수정할 수 없다. 원본 소멸과 재할당은 span을 무효화한다.

std::string_view는 문자 구간을 복사하지 않고 참조한다. 부분 문자열을 저렴하게 표현할 수 있지만 널 종료를 보장하지 않는다. 임시 문자열, 지역 문자열, 재할당된 문자열을 가리키는 뷰는 댕글링 상태가 된다.

std::byte는 원시 메모리의 한 바이트를 표현하고, std::bitset<N>은 고정 크기 비트 집합을 제공한다. C++20 <bit> 함수는 비트 개수, 2의 거듭제곱, 순환 이동, 객체 표현 변환, 엔디언 확인을 지원한다. 객체 메모리를 그대로 직렬화하면 패딩, 엔디언, 표현 차이 때문에 호환성이 깨질 수 있다.

타입 안전한 데이터 표현은 원시 기본형에 여러 의미를 겹쳐 넣지 않는다. enum class, 의미 래퍼 타입, optional, variant, expected를 이용하면 호출 인수와 상태의 의미를 타입에 포함하고 유효하지 않은 조합을 줄일 수 있다.

14.11 학습 과제

과제 **1. pair**와 구조체 비교
상품 ID와 수량을 반환하는 함수를 std::pair<int, int>로 작성한다. 같은 함수를 이름 있는 ItemCountResult 구조체로 다시 작성한다.
구조적 바인딩을 이용해 두 반환값을 받고, 호출부에서 각 값의 의미를 파악하기 쉬운 쪽을 설명한다.

과제 **2. tuple** 분해와 **apply**
이름, 레벨, 체력, 이동 속도를 저장하는 튜플을 만든다. std::get(), 구조적 바인딩, std::apply()를 각각 사용해 모든 원소를 출력한다.
구조적 바인딩을 값, 참조, 상수 참조로 작성하고 원본 수정 여부와 복사 발생 가능성을 비교한다.

과제 **3. optional** 검색 결과
학생 이름 벡터에서 지정한 이름의 인덱스를 찾는 함수를 작성한다. 찾으면 std::optional<std::size_t>에 인덱스를 저장하고 찾지 못하면 std::nullopt를 반환한다.
operator*, value(), value_or()를 사용한 호출 코드를 각각 작성하고 값이 없을 때의 차이를 설명한다.

과제 **4. optional**과 포인터
std::vector<Player>에서 ID로 플레이어를 찾는 함수를 포인터 반환형과 std::optional<Player> 반환형으로 각각 작성한다.
원본 객체 수정, 복사 비용, 값 부재, 컨테이너 재할당 후 유효성 측면에서 두 인터페이스를 비교한다.

과제 **5. variant** 설정 값
정수, 실수, 문자열을 저장할 수 있는 SettingValue를 std::variant로 정의한다. std::get_if()로 각 값을 출력하는 함수와 std::visit()으로 출력하는 함수를 작성한다.
지원하지 않는 대안 타입을 하나 추가한 뒤 방문자 코드에서 누락이 어떻게 발견되는지 확인한다.

과제 **6. variant** 상태 모델
캐릭터 상태를 Idle, Moving, Attacking, Dead 구조체로 나누고 std::variant로 묶는다. 각 상태에 필요한 데이터만 저장한다.
std::visit()으로 상태별 갱신 함수를 작성하고 여러 bool 멤버로 상태를 표현한 구조와 유효하지 않은 상태 가능성을 비교한다.

과제 **7. any** 속성 저장소
문자열 키와 std::any 값을 저장하는 속성 테이블을 작성한다. 정수, 실수, 문자열 값을 등록하고 포인터 형식 std::any_cast()로 안전하게 읽는다.
잘못된 타입을 요청했을 때 값 형식과 포인터 형식의 차이를 확인한다. std::unique_ptr<int>를 직접 저장할 수 없는 이유도 설명한다.

과제 **8. reference_wrapper** 파티 구성
서로 독립적으로 생성된 Player 객체 세 개를 std::vector<std::reference_wrapper<Player>>에 저장한다. 컨테이너 순회로 모든 플레이어의 체력을 변경한다.
std::vector<Player*>와 비교해 nullptr, 문법, 소유권, 수명 측면의 차이를 정리한다.

과제 **9. span** 배열 처리
정수 원소를 두 배로 만드는 함수를 std::span<int>로 작성한다. C 배열, std::array, std::vector를 각각 전달한다.
std::span<const int>를 받는 합계 함수도 작성하고 const std::span<int>와 원소 수정 가능 여부를 비교한다.

과제 **10. span** 부분 구간과 무효화
정수 벡터에서 first(), last(), subspan()으로 부분 범위를 만든다. 부분 span을 통해 원본 원소를 수정한다.
벡터 재할당 전후의 data() 주소를 출력하고 기존 span을 재할당 후 사용할 수 없는 이유를 반복자 무효화와 연결해 설명한다.

과제 **11. string_view** 분리 함수
key=value 형식 문자열을 두 개의 std::string_view로 나누는 함수를 작성한다. 구분자가 없을 때의 반환 규칙도 정한다.
문자열 리터럴, 지역 std::string, 임시 std::string을 전달했을 때 반환된 뷰의 수명을 분석한다.

과제 **12.** 널 종료 확인
긴 std::string의 앞부분 다섯 문자만 가리키는 std::string_view를 만든다. 스트림에 뷰 자체를 출력한 결과와 data()를 C 문자열로 출력한 결과를 비교한다.
C API에 안전하게 전달하기 위해 std::string으로 변환하는 코드를 작성한다.

과제 **13. bitset** 권한 관리
읽기, 쓰기, 실행 권한을 std::bitset<3>으로 표현한다. 각 권한 설정, 해제, 반전, 검사 함수를 작성한다.
같은 권한을 enum class 비트 플래그로도 구현하고 타입 표현과 출력 편의성을 비교한다.

과제 **14. C++20** 비트 함수
여러 std::uint32_t 값에 std::popcount(), std::has_single_bit(), std::bit_floor(), std::bit_ceil(), std::bit_width()를 적용한다.
입력값 0, 1, 63, 64, 65의 결과를 표로 정리하고 각 함수의 용도를 설명한다.

과제 **15. bit_cast**와 수치 변환
float 값을 static_cast<std::uint32_t>()와 std::bit_cast<std::uint32_t>()로 변환한다. 두 결과가 다른 이유를 수치값과 객체 표현의 차이로 설명한다.
bit_cast 대상 타입의 크기와 trivially copyable 조건을 static_assert로 확인한다.

과제 **16.** 엔디언 직렬화
std::uint32_t 값을 항상 빅 엔디언 네 바이트로 저장하는 함수를 작성한다. 시프트와 마스크를 사용하고 호스트 엔디언에 의존하지 않게 한다.
저장한 네 바이트를 다시 정수로 복원하는 함수도 작성한다.

과제 **17.** 의미 타입
플레이어 ID, 아이템 ID, 수량을 모두 int로 받는 함수를 작성한 뒤 각 값을 별도 래퍼 타입으로 변경한다.
인수 순서를 잘못 전달했을 때 원시 정수 버전과 의미 타입 버전의 컴파일 결과를 비교한다.

과제 **18.** 유효한 상태 설계
네트워크 요청 결과를 성공, 시간 초과, 서버 오류로 나누고 std::variant로 표현한다. 각 상태에 필요한 데이터만 포함한다.
불리언 플래그와 선택적 문자열을 사용하는 단일 구조체 버전도 작성하고 만들어질 수 있는 모순 상태를 열거한다.

과제 **19.** 데이터 표현 선택
작업별로 pair, 구조체, optional, variant, any, reference_wrapper, span, string_view 중 적합한 타입을 선택하고 근거를 작성한다.

- 두 좌표 반환
- 사용자 검색 실패
- 정수·실수·문자열 설정
- 확장 모듈의 임의 속성
- 기존 객체 목록 참조
- 연속 배열 일부 전달
- 문자열 일부 읽기
- 성공값과 오류 코드 반환

소유권, 타입 집합의 고정 여부, 수명, 오류 정보, 복사 비용을 판단 기준에 포함한다.

과제 **20.** 댕글링 분석
std::reference_wrapper, std::span, std::string_view가 댕글링 상태가 되는 코드를 각각 작성하되 무효 객체를 실제로 역참조하거나 순회하지 않는다.
대상 객체 소멸, 컨테이너 재할당, 임시 객체 소멸 중 어떤 원인으로 무효화되는지 주석으로 기록하고 안전한 인터페이스로 수정한다.

참고**. C++23**의 **std::expected**
std::optional<T>는 값이 없는 상태를 표현하지만 실패 이유를 저장하지 않는다. std::expected<T, E>는 성공값 T 또는 오류값 E 중 하나를 저장한다.

```
#include <expected>
#include <string>
#include <string_view>

enum class ParseError
{
    Empty,
    InvalidCharacter,
    OutOfRange
};

std::expected<int, ParseError>
ParsePositiveInt(std::string_view text);
```

성공하면 값에 접근한다.

```
auto result = ParsePositiveInt("120");

if (result)
{
    std::cout << *result << '\n';
}
```

실패하면 error()로 오류값을 얻는다.

```
if (!result)
{
    ParseError error = result.error();
}
```

오류 결과는 std::unexpected로 만든다.

```
std::expected<int, ParseError> ParsePositiveInt(std::string_view text)
{
    if (text.empty())
    {
        return std::unexpected(ParseError::Empty);
    }

    int value = 0;

    for (char character : text)
    {
        if (character < '0' || character > '9')
        {
            return std::unexpected(ParseError::InvalidCharacter);
        }
```

```
        int digit = character - '0';

        if (value > (std::numeric_limits<int>::max() - digit) / 10)
        {
            return std::unexpected(ParseError::OutOfRange);
        }

        value = value * 10 + digit;
    }

    return value;
}
```

value()는 성공값을 반환하고 실패 상태이면 std::bad_expected_access<E>를 발생시킨다.

```
try
{
    int value = result.value();
}
catch (const std::bad_expected_access<ParseError>& exception)
{
    ParseError error = exception.error();
}
```

반환할 성공 데이터가 없고 성공 여부와 오류만 필요하면 std::expected<void, E>를 사용할 수 있다.

```
std::expected<void, SaveError> SaveSettings(const Settings& settings)
{
    if (!CanOpenFile())
    {
        return std::unexpected(SaveError::OpenFailed);
    }

    WriteSettings(settings);
    return {};
}

auto result = SaveSettings(settings);

if (!result)
{
    HandleSaveError(result.error());
}
```

C++23의 expected는 성공과 실패 흐름을 연결하는 연산도 제공한다.

```
auto result = LoadText("config.txt")
    .and_then(ParseConfig)
    .transform(BuildSettings)
    .or_else(LogLoadError);
```

```
and_then(): 성공값으로 expected를 반환하는 함수를 호출
transform(): 성공값을 다른 값으로 변환
or_else(): 오류 상태를 처리하거나 다른 expected로 변환
transform_error(): 오류값의 타입이나 내용을 변환
```

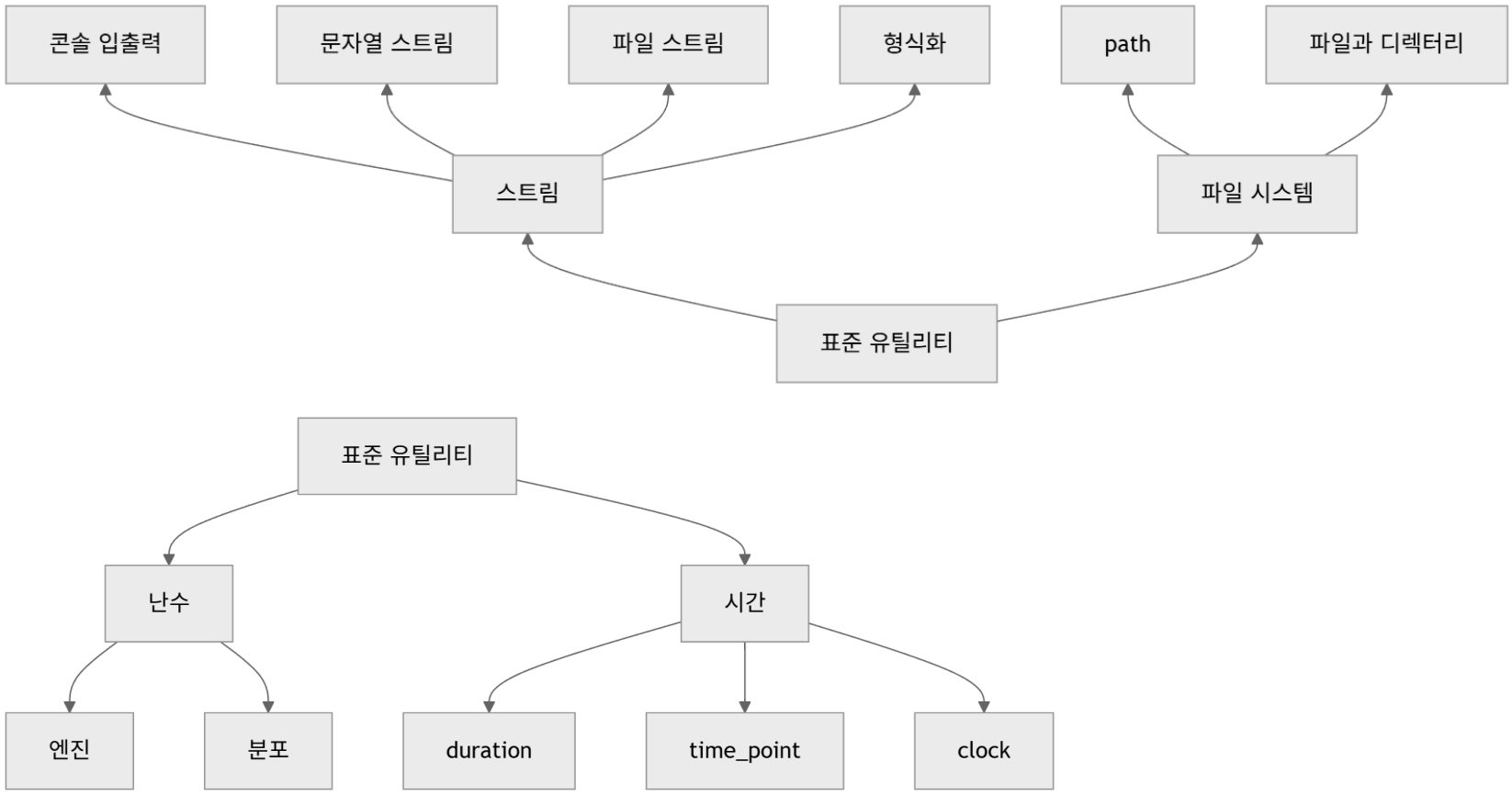
연결된 각 연산은 앞 단계가 성공했을 때만 성공 경로를 실행한다. 오류가 발생하면 남은 성공 연산을 건너뛰고 오류 상태를 전달한다.

expected는 예외를 자동으로 대체하지 않는다. 오류가 함수의 정상적인 결과에 가깝고 호출자가 현 위치에서 검사하거나 전달할 때 적합하다. 메모리 부족이나 내부 불변식 위반처럼 일반적인 반환 흐름으로 처리하기 어려운 실패까지 모두 expected로 바꾸면 호출부의 검사 코드가 과도해질 수 있다.

C++23 모드와 <expected>를 지원하는 표준 라이브러리가 필요하다.

15. 입출력과 표준 유틸리티

프로그램은 실행 중에 만들어진 값만으로 동작하지 않는다. 키보드에서 값을 입력받고 화면에 결과를 출력하며, 파일에 데이터를 저장하고 다시 읽는다. 설정 파일과 로그 파일을 처리하고, 디렉터리를 순회하며, 난수와 시간을 이용해 실행 흐름을 제어하기도 한다.
C++ 표준 라이브러리는 입출력을 스트림이라는 공통 모델로 표현한다. 콘솔, 문자열, 텍스트 파일과 바이너리 파일은 대상이 서로 다르지만 읽기와 쓰기라는 동일한 인터페이스를 사용한다. 파일 시스템 라이브러리는 경로와 디렉터리를 다루고, 난수 라이브러리는 난수 생성 과정과 확률 분포를 분리한다. std::chrono는 시간의 길이와 시점을 타입으로 구분한다.



입출력 코드는 값만 처리하지 않는다. 실패 상태, 버퍼, 파일 위치, 문자 인코딩, 저장 형식과 외부 자원의 수명도 함께 관리해야 한다. 난수와 시간도 단순한 함수 호출보다 재현성, 단위와 기준 시계의 선택이 중요하다.

15.1 스트림 입출력

스트림은 데이터가 순서대로 흐르는 통로를 표현한다. 입력 스트림은 외부 데이터에서 값을 읽고, 출력 스트림은 값을 외부 대상으로 보낸다. std::cin과 std::cout은 콘솔에 연결된 대표적인 스트림 객체이다.

15.1.1 입력 스트림과 출력 스트림

출력 연산자 operator<<는 값을 출력 스트림에 삽입한다.

```
#include <iostream>
#include <string>

int main()
{
    int level = 12;
    std::string name = "Knight";

    std::cout << "Name: " << name << '\n';
    std::cout << "Level: " << level << '\n';
}
```

연속된 operator<< 호출은 왼쪽에서 오른쪽으로 같은 스트림 객체를 전달한다.

```
std::cout << "Level: " << level << '\n';
```

이 표현은 std::cout에 문자열을 출력한 뒤 반환된 스트림에 level과 줄바꿈 문자를 차례로 출력한다.
입력 연산자 operator>>는 입력 스트림에서 값을 추출한다.

```
int level = 0;
std::string name;
std::cin >> name >> level;
```

형식 입력은 원소 타입에 맞게 문자를 해석한다. int에는 정수 형식이 필요하고, double에는 부동소수점 형식이 필요하다. std::string에 operator>>를 사용하면 앞쪽 공백을 건너뛴 뒤 다시 공백이 나올 때까지 읽는다.

```
// Dark Knight 20 할 때

std::string name;
int level = 0;

std::cin >> name >> level;
```

name에는 "Dark"가 저장되고 level 입력은 "Knight"에서 실패한다. 공백을 포함한 한 줄은 std::getline()으로 읽는다.

```
std::string name;
std::getline(std::cin, name);
```

std::cerr는 오류 메시지, std::clog는 진단 로그에 사용할 수 있는 표준 출력 스트림이다.

```
std::cerr << "Failed to load file\n";
std::clog << "Loading configuration\n";
```

std::cerr는 출력 후 즉시 처리되도록 설정된 스트림이며, std::clog는 일반적인 버퍼링을 사용한다. 로그 양이 많다면 매 출력마다 플러시되는 스트림보다 버퍼링된 스트림이 유리할 수 있다.

15.1.2 스트림 상태와 오류 플래그

스트림은 입출력 결과를 상태 플래그로 기록한다. 한 번 실패한 입력 스트림은 상태를 복구하기 전까지 후속 입력도 수행하지 않는다.

```
if (int value = 0; std::cin >> value)
{
    std::cout << "Input: " << value << '\n';
}
else
{
    std::cout << "Invalid input\n";
}
```

스트림 객체를 조건식에 사용하면 치명적인 오류나 형식 오류가 없는지 확인한다. 내부적으로 fail() 결과와 연결되며, 단순히 파일 끝에 도달했는지만 검사하는 표현은 아니다.

<스트림 상태 플래그>

상태	의미
std::ios::goodbit	오류 상태가 없음
std::ios::eofbit	입력 과정에서 파일 끝에 도달함
std::ios::failbit	형식 변환 실패처럼 복구 가능한 입력 오류가 발생함
std::ios::badbit	장치 오류처럼 스트림 자체의 심각한 오류가 발생함

상태 검사 함수는 서로 완전히 배타적이지 않다.

```
if (stream.good())
{
    // 오류 플래그가 하나도 없음
}

if (stream.eof())
{
    // eofbit가 설정됨
}

if (stream.fail())
{
    // failbit 또는 badbit가 설정됨
```

```
}

if (stream.bad())
{
    // badbit가 설정됨
}
```

파일에서 값을 반복해서 읽을 때는 입력 연산 자체를 반복 조건으로 사용한다.

```
int value = 0;
while (input >> value)
{
    std::cout << value << '\n';
}
```

while (!input.eof())는 적절한 반복 조건이 아니다. 파일 끝 플래그는 파일의 마지막 값을 성공적으로 읽은 직후가 아니라, 그 뒤의 읽기 시도가 실패하면서 설정될 수 있다. 읽기 전에 eof()를 검사하면 이미 실패한 입력을 한 번 더 처리하거나 이전 값을 다시 사용할 위험이 있다.
반복이 끝난 원인을 구분하려면 상태를 별도로 검사한다.

```
while (input >> value)
{
    Process(value);
}

if (!input.eof())
{
    std::cerr << "Invalid data format\n";
}
```

정상적으로 파일 끝에 도달한 경우에는 eofbit와 함께 failbit가 설정될 수 있다. 파일 끝이 아닌 형식 오류라면 !input.eof() 조건으로 구분할 수 있다.

15.1.3 입력 실패 복구

잘못된 입력을 다시 받으려면 상태 플래그와 입력 버퍼를 모두 처리해야 한다.

```
#include <iostream>
#include <limits>

int ReadInteger()
{
    int value = 0;

    while (!(std::cin >> value))
    {
        std::cout << "정수를 입력하세요: ";

        std::cin.clear();
        std::cin.ignore(
            std::numeric_limits<std::streamsize>::max(),
            '\n');
    }

    return value;
}
```

clear()는 오류 플래그를 초기화한다. 입력 버퍼에 남아 있는 잘못된 문자는 제거하지 않는다. 잘못된 문자가 그대로 남아 있으면 같은 입력이 다시 실패하므로 ignore()로 현재 행의 나머지를 버린다.

ignore()의 첫 번째 인수는 버릴 수 있는 최대 문자 수이고, 두 번째 인수는 중단할 구분 문자이다.

```
std::cin.ignore(
    std::numeric_limits<std::streamsize>::max(),
    '\n');
```

입력 성공 뒤에도 같은 행의 나머지를 버려야 할 수 있다.

```cpp
int age = 0;
std::cin >> age;
std::cin.ignore(
    std::numeric_limits<std::streamsize>::max(),
    '\n');
```

입력 스트림에 예외 처리를 설정할 수도 있다.

```cpp
std::cin.exceptions(std::ios::badbit);
```

형식 오류가 자주 발생할 수 있는 사용자 입력에서는 failbit를 예외로 바꾸기보다 상태를 직접 확인하고 복구하는 방식이 대체로 단순하다.
복구하기 어려운 장치 오류에는 예외 정책을 적용할 수 있다.

15.1.4 형식 지정과 스트림 상태

출력 스트림은 숫자의 진법, 정렬, 폭, 채움 문자, 부동소수점 표기 방식을 상태로 보관한다.

```cpp
#include <iomanip>
#include <iostream>

int main()
{
    int value = 255;

    std::cout << std::dec << value << '\n';
    std::cout << std::hex << value << '\n';
    std::cout << std::oct << value << '\n';
}
```

std::hex를 적용한 뒤에는 다른 진법 조정자를 지정할 때까지 16진수 상태가 유지된다.

```cpp
std::cout << std::hex << 255 << '\n';
std::cout << 16 << '\n';       // 10
std::cout << std::dec << 16 << '\n';
```

std::setw()는 바로 뒤의 한 필드에만 적용된다.

```cpp
std::cout << std::setw(8) << 10;
std::cout << std::setw(8) << 20 << '\n';
```

부동소수점 정밀도는 표기 방식과 함께 해석해야 한다.

```cpp
#include <iomanip>
#include <iostream>

int main()
{
    double value = 123.456789;

    std::cout << std::setprecision(4) << value << '\n';
    std::cout << std::fixed
              << std::setprecision(4)
              << value << '\n';
}
```

일반 표기에서 setprecision(4)는 유효 숫자 수를 의미한다. std::fixed와 함께 사용하면 소수점 이하 자릿수를 의미한다.
스트림 상태를 임시로 바꾼 뒤 원래 상태를 복원해야 하는 함수에서는 플래그와 정밀도를 저장할 수 있다.

```cpp
void PrintHex(std::ostream& output, int value)
{
    const auto oldFlags = output.flags();
    const auto oldFill = output.fill();

    output << std::showbase
           << std::hex
```

```
            << std::setfill('0')
            << value;

    output.flags(oldFlags);
    output.fill(oldFill);
}
```

출력 함수가 전달받은 스트림의 상태를 영구적으로 바꾸면 호출 이후의 출력 결과까지 달라질 수 있다. 라이브러리 성격의 함수는 상태를 복원하거나, 형식 상태를 변경한다는 계약을 인터페이스에 드러내야 한다.

15.1.5 사용자 정의 타입의 입출력

사용자 정의 타입에 operator<<를 정의하면 기본형과 같은 출력 문법을 사용할 수 있다.

```
#include <iostream>
#include <string>

struct Item
{
    int id;
    std::string name;
    int count;
};

std::ostream&
operator<<(std::ostream& output, const Item& item)
{
    output << item.id << ' '
           << item.name << ' '
           << item.count;

    return output;
}

int main()
{
    Item item{1001, "Potion", 10};
    std::cout << item << '\n';
}
```

반환 타입이 std::ostream&이므로 연속 출력이 가능하다.

```
std::cout << item << " stored\n";
```

입력 연산자는 입력 실패 시 객체가 일부만 변경되지 않도록 임시 변수에 읽는 편이 안전하다.

```
std::istream& operator>>(std::istream& input, Item& item)
{
    int id = 0;
    std::string name;
    int count = 0;

    if (input >> id >> name >> count)
    {
        item = Item{id, std::move(name), count};
    }

    return input;
}
```

세 값이 모두 성공적으로 입력된 경우에만 item을 갱신한다. 중간 입력이 실패하면 스트림 상태는 실패로 남고 기존 item은 유지된다.
형식 자체의 의미 검사가 필요하면 failbit를 직접 설정할 수 있다.

```
std::istream& operator>>(std::istream& input, Item& item)
{
    int id = 0;
    std::string name;
```

```
    int count = 0;

    if (!(input >> id >> name >> count))
    {
        return input;
    }

    if (id <= 0 || count < 0)
    {
        input.setstate(std::ios::failbit);
        return input;
    }

    item = Item{id, std::move(name), count};
    return input;
}
```

출력 형식은 사람이 읽기 위한 표현인지 다시 읽을 저장 형식인지 구분해야 한다. 공백으로 구분한 단순 출력은 이름에 공백이 포함되거나 필드가 추가될 때 파싱이 깨질 수 있다. 지속적으로 저장할 데이터라면 형식 규칙과 버전을 별도로 설계해야 한다.

15.1.6 버퍼링과 출력 시점

출력 스트림은 보통 데이터를 버퍼에 모은 뒤 일정 시점에 장치로 보낸다. 작은 출력마다 장치 접근을 반복하지 않아 성능을 높일 수 있다.

```
std::cout << "Loading...";
```

줄바꿈이 없으면 메시지가 즉시 화면에 보인다고 보장할 수 없다. 즉시 출력이 필요하면 std::flush를 사용한다.

```
std::cout << "Loading..." << std::flush;
```

std::endl은 줄바꿈 문자를 출력하고 스트림을 플러시한다.

```
std::cout << "Completed" << std::endl;
```

단순 줄바꿈에는 \n이 적합하다.

```
std::cout << "Completed\n";
```

반복문에서 매번 std::endl을 사용하면 불필요한 플러시가 누적될 수 있다.

```
for (int i = 0; i < 10000; ++i)
{
    std::cout << i << '\n';
}
```

std::cin은 기본적으로 std::cout과 묶여 있다. 입력을 수행하기 전에 연결된 출력 스트림이 플러시되므로 프롬프트가 먼저 표시된다.

```
std::cout << "Value: ";
std::cin >> value;
```

입출력 성능을 위해 C 표준 입출력과의 동기화를 끌 수 있다.

```
std::ios::sync_with_stdio(false);
std::cin.tie(nullptr);
```

동기화를 끈 뒤 printf()와 std::cout을 섞으면 출력 순서를 예측하기 어려울 수 있다. cin.tie(nullptr)는 입력 전 자동 플러시도 제거하므로 필요한 지점에서 직접 플러시해야 한다. 프로그램 전체의 입출력 정책을 이해한 상태에서 적용해야 한다.

15.1.7 형식 입력과 행 입력의 혼합

operator>>는 값을 읽은 뒤 구분 문자를 입력 버퍼에 남길 수 있다. 정수를 읽고 바로 std::getline()을 호출하면 남아 있던 줄바꿈 문자만 읽고 빈 문자열이 반환될 수 있다.

```
int age = 0;
std::string name;

std::cin >> age;
```

std::getline(std::cin, name); // 남은 줄바꿈을 읽을 수 있음

현재 행의 나머지를 버린 뒤 행 입력을 수행할 수 있다.

```
std::cin >> age;
std::cin.ignore(
    std::numeric_limits<std::streamsize>::max(),
    '\n');
std::getline(std::cin, name);
```

앞쪽 공백을 제거해도 되는 문자열이라면 std::ws를 사용할 수 있다.

```
std::cin >> age;
std::getline(std::cin >> std::ws, name);
```

std::ws는 줄바꿈뿐 아니라 이어지는 모든 공백 문자를 제거한다. 사용자가 의도적으로 입력한 앞쪽 공백을 보존해야 한다면 ignore()로 현재 줄바꿈만 처리하는 방식이 더 적합하다.

15.2 문자열 스트림

문자열 스트림은 std::string을 입력 또는 출력 장치처럼 다룬다. 콘솔이나 파일 없이도 스트림의 형식 변환 기능을 사용할 수 있어 문자열 파싱, 메시지 조립, 테스트에 유용하다.

15.2.1 std::istringstream

std::istringstream은 문자열에서 형식화된 값을 읽는다.

```
#include <sstream>
#include <string>

int main()
{
    std::string text = "1001 Potion 10";
    std::istringstream input{text};

    int id = 0;
    std::string name;
    int count = 0;

    if (input >> id >> name >> count)
    {
        // id = 1001, name = "Potion", count = 10
    }
}
```

문자열 안의 공백은 형식 입력에서 구분자로 사용된다. 행 전체를 다시 구분자 기준으로 나누려면 std::getline()과 문자열 스트림을 함께 사용할 수 있다.

```
std::string record = "1001,Potion,10";
std::istringstream input{record};

std::string idText;
std::string name;
std::string countText;

std::getline(input, idText, ',');
std::getline(input, name, ',');
std::getline(input, countText);
```

숫자 필드는 별도의 문자열 스트림이나 std::from_chars()로 변환할 수 있다. 형식이 단순하고 예외 없는 숫자 변환이 필요하다면 std::from_chars()가 적합하고, 여러 타입과 공백 규칙을 스트림 방식으로 처리하려면 문자열 스트림이 편리하다.

15.2.2 std::ostringstream

std::ostringstream은 여러 값을 문자열로 조립한다.

```
#include <sstream>
#include <string>

std::string
MakeItemText(int id, const std::string& name, int count)
{
    std::ostringstream output;
    output << "Item{" << id
           << ", " << name
           << ", " << count
           << '}';

    return output.str();
}
```

str()은 현재 출력 버퍼의 문자열을 반환한다.

```
std::ostringstream output;
output << "Score: " << 1500;

std::string result = output.str();
```

스트림 조정자도 사용할 수 있다.

```
std::ostringstream output;
output << std::fixed
       << std::setprecision(2)
       << 12.3456;

std::string result = output.str(); // "12.35"
```

단순한 문자열 연결에는 std::string의 operator+나 append()가 더 직접적일 수 있다. 서로 다른 타입을 동일한 출력 문법으로 조립하고 형식 상태를 적용해야 할 때 문자열 스트림의 장점이 커진다.

15.2.3 문자열 파싱과 조립

하나의 문자열에 저장된 레코드를 구조체로 변환할 수 있다.

```
#include <sstream>
#include <string>

struct PlayerData
{
    int id;
    std::string name;
    int hp;
};

bool ParsePlayer(const std::string& text, PlayerData& player)
{
    std::istringstream input{text};

    int id = 0;
    std::string name;
    int hp = 0;

    if (!(input >> id >> name >> hp))
    {
        return false;
    }
```

```
    player = PlayerData{id, std::move(name), hp};
    return true;
}
```

이 함수는 앞의 세 필드만 올바르면 성공한다. 뒤에 불필요한 문자가 있어도 검사하지 않는다.

```
PlayerData player{};
bool result = ParsePlayer("1 Knight 100 extra", player);
```

입력 전체가 하나의 레코드 형식과 일치해야 한다면 남은 문자를 확인한다.

```
bool ParsePlayer(const std::string& text, PlayerData& player)
{
    std::istringstream input{text};

    int id = 0;
    std::string name;
    int hp = 0;

    if (!(input >> id >> name >> hp))
    {
        return false;
    }

    input >> std::ws;
    if (!input.eof())
    {
        return false;
    }

    player = PlayerData{id, std::move(name), hp};
    return true;
}
```

std::ws는 끝에 남은 공백을 제거한다. 공백 제거 뒤 파일 끝에 도달했다면 유효한 필드 외의 문자가 없었다는 뜻이다.
문자열 조립에서도 구분자와 이스케이프 규칙을 정해야 한다.

```
std::string SerializePlayer(const PlayerData& player)
{
    std::ostringstream output;
    output << player.id << ','
           << player.name << ','
           << player.hp;

    return output.str();
}
```

이 구현은 이름에 쉼표가 들어가지 않는다는 전제가 필요하다. 외부 데이터 형식은 구분자가 필드 안에 등장하는 경우, 인용 부호, 줄바꿈과 이스케이프 규칙까지 정의해야 한다.

15.2.4 입력 전체와 스트림 상태 확인

문자열을 정수 하나로 변환할 때 입력 연산의 성공만 확인하면 일부 문자열만 변환될 수 있다.

```
int ParseInteger(const std::string& text)
{
    std::istringstream input{text};

    int value = 0;
    input >> value;

    return value;
}
```

"100abc"를 전달하면 정수 100을 읽은 시점까지는 성공한다. 문자열 전체가 정수 형식인지 검사하려면 추가 입력 가능 여부를 확인한다.

```cpp
#include <optional>
#include <sstream>
#include <string>

std::optional<int> ParseInteger(const std::string& text)
{
    std::istringstream input{text};

    int value = 0;
    if (!(input >> value))
    {
        return std::nullopt;
    }

    input >> std::ws;
    if (!input.eof())
    {
        return std::nullopt;
    }

    return value;
}
```

eof()만 먼저 검사하지 않고 실제 추출 결과를 기준으로 처리한다는 스트림 원칙은 문자열 스트림에도 그대로 적용된다.
형식 입력 후 남은 문자 하나를 읽는 방식도 사용할 수 있다.

```cpp
int value = 0;
char extra = '\0';
std::istringstream input{text};

if ((input >> value) && !(input >> extra))
{
    // 공백을 제외한 추가 문자가 없음
}
```

두 방식 모두 공백을 허용할지, 부호와 진법을 어떻게 처리할지 입력 계약에 맞춰 선택해야 한다.

15.2.5 문자열 스트림 재사용

문자열 스트림은 내부 문자열과 상태 플래그를 각각 가진다. 이전 파싱에서 실패한 스트림은 문자열만 교체해도 실패 상태가 남을 수 있다.

```cpp
std::istringstream input{"abc"};
int value = 0;

input >> value; // 실패
input.str("100");
input >> value; // failbit가 남아 있어 다시 실패
```

상태를 초기화한 뒤 새 문자열을 설정한다.

```cpp
input.clear();
input.str("100");
input >> value;
```

출력 문자열 스트림도 내부 문자열과 위치를 함께 고려해야 한다.

```cpp
std::ostringstream output;
output << "first";

output.str("");
output.clear();
output << "second";
```

새 객체 생성 비용이 문제되지 않는 일반 코드에서는 필요한 지점마다 지역 문자열 스트림을 새로 만드는 편이 단순하다. 반복 횟수가 많고 재사용 이유가 분명할 때 상태와 버퍼를 모두 초기화해야 한다.

15.3 형식화된 출력

형식화는 값의 의미를 바꾸지 않고 출력 표현을 정한다. 정렬, 폭, 채움 문자, 정밀도, 진법과 부호 표시를 일관되게 적용하면 로그, 표, 사용자 인터페이스의 가독성이 높아진다.

15.3.1 iomanip

<iomanip>은 스트림 형식을 조정하는 함수와 객체를 제공한다.

```
#include <iomanip>
#include <iostream>

int main()
{
    std::cout << std::left
              << std::setw(12) << "Name"
              << std::right
              << std::setw(8) << "Score"
              << '\n';

    std::cout << std::left
              << std::setw(12) << "Knight"
              << std::right
              << std::setw(8) << 1500
              << '\n';
}
```

std::setfill()은 빈 폭을 채울 문자를 정한다.

```
std::cout << std::setfill('0')
          << std::setw(5)
          << 42
          << '\n'; // 00042
```

진법과 접두사를 함께 표현할 수 있다.

```
int value = 255;

std::cout << std::showbase << std::hex << value << '\n';
std::cout << std::showbase << std::oct << value << '\n';
std::cout << std::dec << value << '\n';
```

논리값은 기본적으로 0과 1로 출력된다.

```
std::cout << std::boolalpha << true << '\n';
std::cout << std::noboolalpha << true << '\n';
```

<대표적인 스트림 형식 조정자>

조정자	역할	적용 범위
std::setw(n)	한 필드의 최소 폭 지정	바로 뒤의 한 필드
std::setfill(c)	채움 문자 지정	변경할 때까지 유지
std::left, std::right	정렬 방향 지정	변경할 때까지 유지
std::fixed, std::scientific	부동소수점 표기 방식 지정	변경할 때까지 유지
std::setprecision(n)	정밀도 지정	변경할 때까지 유지
std::hex, std::oct, std::dec	정수 진법 지정	변경할 때까지 유지
std::showbase, std::showpos	진법 접두사와 양수 부호 표시	해제할 때까지 유지

스트림 상태가 누적되므로 독립적인 형식 문자열을 선호하는 코드에서는 std::format이 더 분명할 수 있다.

15.3.2 std::format

C++20의 std::format()은 형식 문자열에 값을 대입해 std::string을 만든다.

```
#include <format>
#include <string>

int main()
{
    std::string name = "Knight";
    int level = 12;

    std::string text = std::format(
        "Name: {}, Level: {}",
        name,
        level);
}
```

중괄호 {}는 값이 들어갈 교체 필드이다. 인수 위치를 명시할 수도 있다.

```
std::string text = std::format(
    "{1} has level {0}",
    level,
    name);
```

자동 인덱스와 수동 인덱스를 한 형식 문자열에서 섞어 쓰지 않는다.

```
// std::format("{} {1}", 10, 20); // 형식 오류
```

std::format()은 스트림 객체의 형식 상태를 변경하지 않는다. 호출 하나의 형식이 문자열 안에 모두 표현된다.

```
std::string text = std::format(
    "ID={:05}, HP={:6}, Ratio={:.2f}",
    42,
    100,
    0.875);
```

형식 문자열과 인수 타입이 맞지 않거나 문법이 잘못되면 형식 검증 과정에서 오류가 발견된다. 런타임 형식 문자열을 std::vformat()으로 처리할 때는 std::format_error가 발생할 수 있다.

15.3.3 형식 문자열과 서식 지정

교체 필드는 {인수 인덱스:서식 지정} 형태를 사용한다. 인수 인덱스와 서식 지정은 필요하지 않으면 생략할 수 있다.

```
std::format("{}", 42);
std::format("{:8}", 42);
std::format("{:08}", 42);
std::format("{:+}", 42);
std::format("{:#x}", 255);
```

정렬과 채움 문자는 폭과 함께 사용한다.

```
std::format("{:<10}", "left");
std::format("{:>10}", "right");
std::format("{:^10}", "center");
std::format("{:*^10}", "title");
```

부동소수점 표기에는 정밀도와 형식 문자를 지정할 수 있다.

```
std::format("{:.2f}", 12.3456);
std::format("{:.3e}", 12.3456);
std::format("{:8.2f}", 12.3456);
```

정수 진법도 형식 문자열에 포함할 수 있다.

```
std::format("{:b}", 10);   // 1010
std::format("{:#x}", 255); // 0xff
std::format("{:o}", 64);   // 100
```

폭과 정밀도를 인수로 받을 수도 있다.

```
int width = 10;
int precision = 3;

auto text = std::format(
    "{0:{1}.{2}f}",
    12.34567,
    width,
    precision);
```

형식 문자열이 복잡해지면 필드 의미를 별도 함수나 사용자 정의 형식화로 분리할 수 있다. 형식 지정 자체가 비즈니스 규칙을 대신하게 만들면 변경과 테스트가 어려워진다.

15.3.4 std::format_to

std::format()은 새 문자열을 반환한다. 이미 존재하는 문자열이나 출력 반복자에 결과를 추가하려면 std::format_to()를 사용할 수 있다.

```
#include <format>
#include <iterator>
#include <string>

int main()
{
    std::string log;

    std::format_to(
        std::back_inserter(log),
        "Player {} joined with level {}\n",
        "Knight",
        12);

    std::format_to(
        std::back_inserter(log),
        "Current HP: {}\n",
        100);
}
```

std::format_to()는 반환된 출력 반복자로 기록이 끝난 위치를 알려 준다.
고정 크기 버퍼에 기록해야 한다면 std::format_to_n()으로 최대 출력 수를 제한할 수 있다.

```
#include <array>
#include <format>

std::array<char, 32> buffer{};

auto result = std::format_to_n(
    buffer.begin(),
    buffer.size() - 1,
    "Score: {}",
    1500);

const std::size_t written =
    static_cast<std::size_t>(result.out - buffer.begin());

buffer[written] = '\0';
```

result.size는 제한이 없었다면 필요한 전체 문자 수를 나타낸다. result.out은 실제 기록이 끝난 위치이다. 출력이 잘렸는지 판단하려면 두 값을 함께 확인한다.

15.3.5 사용자 정의 형식화 개요

사용자 정의 타입은 std::formatter<T> 특수화로 std::format()에 연결할 수 있다.

```
#include <format>
#include <string>

struct Vector2
{
    float x, y;
};

template <>
struct std::formatter<Vector2>
{
    constexpr auto parse(std::format_parse_context& context)
    {
        return context.begin();
    }

    auto format(
        const Vector2& value,
        std::format_context& context) const
    {
        return std::format_to(
            context.out(),
            "({}, {})",
            value.x,
            value.y);
    }
};

int main()
{
    Vector2 position{10.5f, 20.0f};
    std::string text = std::format("Position: {}", position);
}
```

parse()는 사용자 정의 서식 지정 부분을 해석하고, format()은 값을 출력 반복자에 기록한다. 예제의 parse()는 별도 옵션을 처리하지 않으므로 시작 위치를 그대로 반환한다.

사용자 정의 형식이 한 가지뿐이라면 operator<<나 별도 문자열 변환 함수로도 충분할 수 있다. 폭, 정밀도, 표현 모드처럼 여러 서식 옵션을 통합해야 할 때 std::formatter의 가치가 커진다.

15.4 텍스트 파일 입출력

텍스트 파일은 데이터를 문자 형태로 저장한다. 사람이 직접 읽고 수정하기 쉽고 다른 도구와 교환하기 편리하지만, 숫자를 문자열로 변환하는 비용과 형식 규칙이 필요하다.

15.4.1 std::ifstream과 std::ofstream

std::ifstream은 파일 입력, std::ofstream은 파일 출력을 담당한다.

```
#include <fstream>
#include <string>

int main()
{
    std::ofstream output{"player.txt"};

    if (!output)
    {
        return 1;
    }

    output << 1 << ' '
           << "Knight" << ' '
           << 100 << '\n';
```

```
}
```

파일 스트림 객체가 생성되면서 파일을 연다. 열기에 실패하면 스트림이 실패 상태가 된다.

```
std::ifstream input{"player.txt"};

if (!input)
{
    std::cerr << "Failed to open player.txt\n";
    return 1;
}
```

값을 읽는 방법은 콘솔 입력과 같다.

```
int id = 0;
std::string name;
int hp = 0;

if (input >> id >> name >> hp)
{
    std::cout << id << ' '
              << name << ' '
              << hp << '\n';
}
```

파일 스트림은 소멸할 때 파일을 닫는다. 지역 객체로 만들면 예외나 조기 반환이 발생해도 소멸자를 통해 자원이 정리된다.

```
void SavePlayer(const PlayerData& player)
{
    std::ofstream output{"player.txt"};

    if (!output)
    {
        throw std::runtime_error{"Failed to open file"};
    }

    output << player.id << ' '
           << player.name << ' '
           << player.hp << '\n';
}
```

기록 오류를 확인해야 한다면 출력 이후 상태를 검사한다.

```
output << data;

if (!output)
{
    throw std::runtime_error{"Failed to write file"};
}
```

파일을 성공적으로 열었다는 사실이 모든 쓰기의 성공을 보장하지 않는다. 저장 장치 공간 부족이나 연결된 장치 오류는 기록 과정에서 발생할 수 있다.

15.4.2 std::fstream과 파일 열기 모드

std::fstream은 읽기와 쓰기를 모두 지원한다.

```
#include <fstream>

std::fstream file{
    "data.txt",
    std::ios::in | std::ios::out};
```

파일 열기 모드는 비트 플래그이므로 | 연산자로 결합한다.

<파일 열기 모드>

모드	의미
std::ios::in	읽기 허용
std::ios::out	쓰기 허용
std::ios::app	모든 쓰기를 파일 끝에서 수행
std::ios::ate	파일을 연 직후 위치를 끝으로 이동
std::ios::trunc	기존 내용을 제거
std::ios::binary	텍스트 변환 없이 바이트 단위로 처리

std::ofstream을 기본 모드로 열면 기존 파일 내용이 잘릴 수 있다.

```
std::ofstream output{"log.txt"};
```

기존 내용 뒤에 추가하려면 std::ios::app을 사용한다.

```
std::ofstream output{
    "log.txt",
    std::ios::out | std::ios::app};
```

app에서는 위치를 옮겨도 실제 쓰기는 항상 파일 끝에서 이루어진다. ate는 파일을 열 때만 위치를 끝으로 옮기므로 이후 seekp()로 다른 위치에 쓸 수 있다.

```
std::fstream file{
    "data.txt",
    std::ios::in | std::ios::out | std::ios::ate};
```

읽기와 쓰기를 같은 스트림에서 번갈아 수행할 때는 위치와 상태를 명확히 관리해야 한다.

```
file.seekg(0);
std::string line;
std::getline(file, line);

file.clear();
file.seekp(0, std::ios::end);
file << "new line\n";
```

입력에서 파일 끝에 도달하면 실패 상태가 남을 수 있으므로 위치를 바꾸기 전에 clear()가 필요할 수 있다.

15.4.3 행 단위 입력

설정 파일이나 로그처럼 한 행이 하나의 레코드를 표현한다면 std::getline()이 적합하다.

```
#include <fstream>
#include <iostream>
#include <string>

int main()
{
    std::ifstream input{"config.txt"};
    std::string line;

    while (std::getline(input, line))
    {
        std::cout << line << '\n';
    }

    if (!input.eof())
    {
        std::cerr << "Failed while reading config.txt\n";
    }
}
```

std::getline()은 구분 문자인 줄바꿈을 결과 문자열에 포함하지 않는다. Windows 형식의 텍스트를 다른 환경에서 처리하거나 바이너리 모드로 읽는 경우 행 끝에 \r이 남을 수 있으므로 입력 형식과 모드를 확인해야 한다.

빈 행도 정상적인 데이터일 수 있다.

```
while (std::getline(input, line))
{
    if (line.empty())
    {
        continue;
    }

    ProcessLine(line);
}
```

주석과 앞뒤 공백을 처리하려면 행을 읽은 뒤 별도 파싱 단계를 둔다.

```
if (!line.empty() && line[0] == '#')
{
    continue;
}
```

문자열 스트림을 연결하면 행 단위 오류를 파일 전체 오류와 분리할 수 있다.

```
int lineNumber = 0;

while (std::getline(input, line))
{
    ++lineNumber;

    PlayerData player{};
    if (!ParsePlayer(line, player))
    {
        std::cerr << "Invalid line: "
                  << lineNumber << '\n';
        continue;
    }

    players.push_back(std::move(player));
}
```

15.4.4 구분자 기반 입력

std::getline()의 세 번째 인수로 구분자를 지정할 수 있다.

```
std::string line = "1001,Potion,10";
std::istringstream input{line};

std::string idText;
std::string name;
std::string countText;

if (std::getline(input, idText, ',') &&
    std::getline(input, name, ',') &&
    std::getline(input, countText))
{
    // 세 필드를 읽음
}
```

텍스트 파일에서는 먼저 행을 읽고 행 내부를 다시 분리하는 방식이 오류 위치를 추적하기 쉽다.

```
std::ifstream file{"items.csv"};
std::string line;

while (std::getline(file, line))
```

```
{
    std::istringstream record{line};

    std::string idText;
    std::string name;
    std::string countText;

    if (!std::getline(record, idText, ',') ||
        !std::getline(record, name, ',') ||
        !std::getline(record, countText))
    {
        continue;
    }

    auto id = ParseInteger(idText);
    auto count = ParseInteger(countText);

    if (!id || !count)
    {
        continue;
    }

    items.push_back(Item{*id, std::move(name), *count});
}
```

이 구현은 쉼표가 필드 안에 들어가지 않는 단순 형식만 처리한다. 일반적인 CSV는 인용 부호로 감싼 필드, 필드 내부 쉼표, 줄바꿈, 인용 부호 이스케이프 규칙을 포함할 수 있다. 실제 CSV 호환성이 필요하면 해당 규칙을 구현하거나 검증된 파서를 사용해야 한다.

15.4.5 파일 오류 처리와 자원 정리

파일 처리에는 열기 실패, 읽기 실패, 쓰기 실패, 형식 오류가 존재한다. 오류 종류를 하나의 메시지로 처리하면 원인을 찾기 어렵다.

```
std::ifstream input{"items.txt"};

if (!input.is_open())
{
    std::cerr << "File open failed\n";
    return;
}

Item item{};
while (input >> item)
{
    items.push_back(item);
}

if (input.bad())
{
    std::cerr << "Device error while reading\n";
}
else if (!input.eof())
{
    std::cerr << "Invalid item format\n";
}
```

is_open()은 파일이 연결되어 있는지 확인하고, 스트림 상태는 최근 입출력 연산의 성공 여부를 나타낸다. 두 검사의 목적이 다르다. 파일 저장은 기존 파일을 바로 덮어쓰는 대신 임시 파일에 기록한 뒤 이름을 바꾸는 방식으로 실패 위험을 줄일 수 있다.

```
void SaveTextAtomically(
    const std::filesystem::path& path,
    const std::string& text)
{
    std::filesystem::path temporary = path;
    temporary += ".tmp";

    {
        std::ofstream output{temporary};
        if (!output)
```

```
        {
            throw std::runtime_error{"Temporary file open failed"};
        }

        output << text;
        if (!output)
        {
            throw std::runtime_error{"Temporary file write failed"};
        }
    }

    std::filesystem::rename(temporary, path);
};
}
```

대상 파일이 이미 존재할 때 rename()의 동작은 운영체제와 파일 시스템 조건에 영향을 받을 수 있다. 중요한 저장 기능은 교체 정책, 백업, 동기화와 오류 복구까지 설계해야 한다.

명시적 close()는 같은 스트림 객체로 다른 파일을 열거나 닫기 오류를 즉시 확인해야 할 때 사용할 수 있다.

```
output.close();

if (!output)
{
    throw std::runtime_error{"Failed to close output file"};
}
```

정상적인 지역 파일 스트림은 소멸자가 닫기를 담당하므로 모든 코드 경로에 close()를 반복해서 배치할 필요는 없다.

15.5 바이너리 파일 입출력

바이너리 파일은 값을 사람이 읽는 문자로 변환하지 않고 바이트 형식으로 저장한다. 파일 크기와 변환 비용을 줄일 수 있지만 저장 형식의 크기, 바이트 순서, 버전과 호환성을 직접 정의해야 한다.

15.5.1 바이너리 모드

파일을 바이너리 모드로 열려면 std::ios::binary를 지정한다.

```
std::ofstream output{
    "data.bin",
    std::ios::out | std::ios::binary};
```

텍스트 모드에서는 운영체제에 따라 줄바꿈 문자나 특수 문자가 변환될 수 있다. 바이너리 모드는 파일에 기록한 바이트와 읽은 바이트 사이의 텍스트 변환을 막는다.

```
std::ifstream input{
    "data.bin",
    std::ios::in | std::ios::binary};
```

바이너리 모드가 정수와 구조체를 자동으로 직렬화하지는 않는다. 저장할 값을 바이트열로 바꾸고 다시 복원하는 규칙은 프로그램이 정의해야 한다.

15.5.2 read와 write

write()는 메모리의 바이트를 출력 스트림에 기록한다.

```
#include <cstdint>
#include <fstream>

int main()
{
    std::int32_t score = 1500;

    std::ofstream output{
        "score.bin",
```

```
        std::ios::binary};

    output.write(
        reinterpret_cast<const char*>(&score),
        sizeof(score));
}
```

read()는 파일의 바이트를 메모리에 읽는다.

```
std::int32_t score = 0;

std::ifstream input{
    "score.bin",
    std::ios::binary};

input.read(
    reinterpret_cast<char*>(&score),
    sizeof(score));

if (!input)
{
    std::cerr << "Failed to read score\n";
}
```

이 코드는 같은 환경에서 저장하고 읽는 간단한 예제이다. 정수의 바이트 순서와 크기를 파일 형식으로 명시하지 않았으므로 플랫폼 독립 형식은 아니다.
read()가 요청한 크기보다 적게 읽었을 때 gcount()로 실제 읽은 바이트 수를 확인할 수 있다.

```
std::array<char, 128> buffer{};
input.read(buffer.data(), buffer.size());

std::streamsize readCount = input.gcount();
```

고정 크기 블록을 반복해서 읽을 수 있다.

```
std::array<char, 4096> buffer{};

while (input)
{
    input.read(buffer.data(), buffer.size());
    const std::streamsize count = input.gcount();

    if (count > 0)
    {
        ProcessBytes(buffer.data(), count);
    }
}

if (input.bad())
{
    std::cerr << "Binary read error\n";
}
```

마지막 블록은 버퍼 크기보다 작을 수 있으므로 gcount()만큼만 처리한다.

15.5.3 파일 위치

입력 위치는 tellg()와 seekg(), 출력 위치는 tellp()와 seekp()로 다룬다. 이름의 g는 get 위치, p는 put 위치를 뜻한다.

```
std::ifstream input{
    "data.bin",
    std::ios::binary};

std::streampos position = input.tellg();
```

절대 위치로 이동할 수 있다.

```
input.seekg(0, std::ios::beg);
input.seekg(16, std::ios::beg);
```

현재 위치나 파일 끝을 기준으로 이동할 수도 있다.

```
input.seekg(8, std::ios::cur);
input.seekg(-4, std::ios::end);
```

파일 크기를 구할 때 끝으로 이동한 뒤 위치를 확인할 수 있다.

```
input.seekg(0, std::ios::end);
const std::streampos end = input.tellg();
input.seekg(0, std::ios::beg);
```

std::filesystem::file_size()를 사용할 수 있는 파일이라면 경로 기반 크기 조회가 더 직접적이다. 스트림 위치는 파일 내부 레코드 이동이나 부분 읽기에 적합하다.

입력 실패 상태가 남아 있으면 seekg()가 동작하지 않을 수 있다.

```
input.clear();
input.seekg(0, std::ios::beg);
```

텍스트 모드에서는 문자 수와 바이트 위치가 단순히 일치한다고 가정하지 않는다. 임의 위치 접근이 필요한 파일 형식은 바이너리 모드와 명시적인 레코드 배치를 사용하는 편이 안전하다.

15.5.4 고정 길이 데이터와 문자열

고정 크기 정수형은 파일 필드의 크기를 명시하는 데 도움이 된다.

```
#include <cstdint>

std::uint32_t id = 1001;
std::int32_t hp = 100;
```

하지만 std::uint32_t를 메모리 그대로 기록하면 바이트 순서는 여전히 환경에 의존한다. 파일 형식에서 리틀 엔디언을 사용하기로 정했다면 바이트를 명시적으로 분리할 수 있다.

```
#include <array>
#include <cstdint>
#include <ostream>

void WriteUint32LE(std::ostream& output, std::uint32_t value)
{
    std::array<unsigned char, 4> bytes{
        static_cast<unsigned char>(value & 0xFFu),
        static_cast<unsigned char>((value >> 8) & 0xFFu),
        static_cast<unsigned char>((value >> 16) & 0xFFu),
        static_cast<unsigned char>((value >> 24) & 0xFFu)
    };

    output.write(
        reinterpret_cast<const char*>(bytes.data()),
        static_cast<std::streamsize>(bytes.size()));
}
```

읽기 함수는 네 바이트를 같은 순서로 조립한다.

```
#include <istream>
#include <optional>

std::optional<std::uint32_t> ReadUint32LE(std::istream& input)
{
    std::array<unsigned char, 4> bytes{};
```

```
    input.read(
        reinterpret_cast<char*>(bytes.data()),
        static_cast<std::streamsize>(bytes.size()));

    if (!input)
    {
        return std::nullopt;
    }

    std::uint32_t value =
        static_cast<std::uint32_t>(bytes[0]) |
        (static_cast<std::uint32_t>(bytes[1]) << 8) |
        (static_cast<std::uint32_t>(bytes[2]) << 16) |
        (static_cast<std::uint32_t>(bytes[3]) << 24);

    return value;
}
```

가변 길이 문자열은 길이와 바이트를 함께 기록할 수 있다.

```
void WriteString(std::ostream& output, const std::string& text)
{
    if (text.size() > std::numeric_limits<std::uint32_t>::max())
    {
        throw std::length_error{"String is too long"};
    }

    WriteUint32LE(
        output,
        static_cast<std::uint32_t>(text.size()));

    output.write(
        text.data(),
        static_cast<std::streamsize>(text.size()));
}
```

읽을 때는 파일에 기록된 길이를 그대로 신뢰하지 않는다. 최대 허용 길이를 검사해야 손상되거나 악의적인 파일이 과도한 메모리를 요구하는 문제를 막을 수 있다.

```
std::optional<std::string> ReadString(std::istream& input)
{
    constexpr std::uint32_t maxLength = 1024 * 1024;

    auto length = ReadUint32LE(input);
    if (!length || *length > maxLength)
    {
        return std::nullopt;
    }

    std::string text(*length, '\0');
    input.read(
        text.data(),
        static_cast<std::streamsize>(text.size()));

    if (!input)
    {
        return std::nullopt;
    }

    return text;
}
```

문자열 바이트가 UTF-8인지 다른 인코딩인지도 파일 형식에 포함해야 한다. 바이트 길이는 사용자에게 보이는 문자 개수와 같지 않을 수 있다.

15.5.5 객체를 그대로 저장할 때의 문제

구조체 전체를 sizeof만큼 기록하는 코드는 간단해 보인다.

```
struct PlayerRecord
{
    int id;
    std::string name;
    int hp;
};

PlayerRecord player{1, "Knight", 100};

output.write(
    reinterpret_cast<const char*>(&player),
    sizeof(player));
```

std::string 객체 안에는 문자 데이터 자체가 아니라 관리 정보와 내부 포인터가 포함될 수 있다. 파일에 기록되는 값은 문자열 내용이 아니며, 다시 읽어 객체로 사용하면 객체의 불변식과 소유권이 깨진다.
단순한 기본형 멤버만 있는 구조체도 이식 가능한 저장 형식이 되는 것은 아니다.

```
struct SimpleRecord
{
    std::uint32_t id;
    std::uint16_t level;
    std::uint32_t score;
};
```

멤버 사이에 패딩 바이트가 들어갈 수 있고, 정수의 바이트 순서도 다를 수 있다. 컴파일러 옵션이나 구조체 정렬 규칙이 달라지면 크기와 배치가 바뀔 수 있다.

가상 함수를 가진 객체, 포인터 멤버, 참조 멤버, 동적 메모리를 소유한 객체는 메모리 덤프 방식으로 저장해서는 안 된다.

```
class Character
{
public:
    virtual ~Character() = default;

private:
    std::string name;
    std::vector<int> items;
};
```

파일에는 객체의 메모리 표현이 아니라 복원에 필요한 논리적 필드를 기록해야 한다.

```
WriteUint32LE(output, player.id);
WriteString(output, player.name);
WriteUint32LE(output, player.hp);
```

15.5.6 파일 형식과 버전 관리

지속적으로 사용하는 바이너리 파일은 필드 순서만 정해서는 부족하다. 파일 종류와 버전을 식별할 수 있는 헤더가 필요하다.

```
struct FileHeader
{
    std::array<char, 4> magic;
    std::uint32_t version;
};
```

실제 저장에서는 구조체 전체를 기록하지 않고 각 필드를 정해 둔 바이트 순서로 기록한다.

```
output.write("PLYR", 4);
WriteUint32LE(output, 1);
```

읽기 과정에서 매직 값과 버전을 확인한다.

```
std::array<char, 4> magic{};
input.read(magic.data(), magic.size());
```

```
if (!input || magic != std::array<char, 4>{'P', 'L', 'Y', 'R'})
{
    throw std::runtime_error{"Invalid file type"};
}

auto version = ReadUint32LE(input);
if (!version)
{
    throw std::runtime_error{"Invalid file header"};
}
```

버전에 따라 읽기 경로를 분리할 수 있다.

```
switch (*version)
{
case 1:
    LoadVersion1(input);
    break;

case 2:
    LoadVersion2(input);
    break;

 default:
    throw std::runtime_error{"Unsupported file version"};
}
```

파일 형식에는 최소한 필드 타입과 크기, 바이트 순서, 문자열 인코딩, 필드 순서, 반복 원소의 개수 표현, 버전과 오류 검증 방법이 정의되어야 한다. 외부 입력을 읽을 때는 길이와 개수의 상한을 검사하고 모든 읽기 연산의 성공 여부를 확인해야 한다.

15.6 파일 시스템

<filesystem>은 경로, 파일, 디렉터리와 파일 속성을 다루는 기능을 제공한다. 문자열을 직접 이어 붙이는 대신 std::filesystem::path를 사용하면 운영체제별 경로 구분과 구성 요소를 타입으로 처리할 수 있다.

```
#include <filesystem>
namespace fs = std::filesystem;
```

짧은 별칭은 파일 시스템 코드를 읽기 쉽게 만들지만, 헤더의 전역 영역에 광범위하게 공개하기보다 구현 파일이나 좁은 범위에서 사용하는 편이 좋다.

15.6.1 std::filesystem::path

std::filesystem::path는 경로 문자열을 저장하고 구성 요소를 조작한다.

```
fs::path directory = "save";
fs::path fileName = "player.dat";
fs::path fullPath = directory / fileName;
```

연산자는 운영체제에 맞는 경로 구분자를 사용해 경로를 결합한다. 문자열에 "/"나 "\\"를 직접 넣는 방식보다 이식성이 높다.

```
fs::path path = "assets/characters/knight.png";

std::cout << path.filename() << '\n';
std::cout << path.stem() << '\n';
std::cout << path.extension() << '\n';
std::cout << path.parent_path() << '\n';
```

각 함수의 결과는 path 객체이다.

```
filename()    → knight.png
stem()        → knight
extension()   → .png
parent_path() → assets/characters
```

확장자를 교체하거나 파일명을 바꿀 수 있다.

```
fs::path path = "data/player.txt";
path.replace_extension(".bin");
path.replace_filename("monster.bin");
```

경로는 절대 경로와 상대 경로로 구분된다.

```
fs::path relative = "data/player.txt";
fs::path absolute = fs::absolute(relative);
```

lexically_normal()은 파일 시스템에 접근하지 않고 .과 .. 구성 요소를 어휘 규칙으로 정리한다.

```
fs::path path = "assets/characters/../items/potion.png";
fs::path normalized = path.lexically_normal();
```

심볼릭 링크와 실제 파일 위치까지 반영한 경로가 필요하면 canonical()이나 weakly_canonical()을 사용할 수 있다. 두 함수는 파일 시스템 접근과 오류 처리가 필요하다.

15.6.2 파일과 디렉터리 확인

파일 존재 여부와 종류를 검사할 수 있다.

```
fs::path path = "data/player.txt";
if (fs::exists(path))
{
    if (fs::is_regular_file(path))
    {
        std::cout << "regular file\n";
    }
    else if (fs::is_directory(path))
    {
        std::cout << "directory\n";
    }
}
```

대표적인 상태 조회 함수에는 is_regular_file(), is_directory(), is_symlink(), is_empty()가 있다.
파일 크기는 file_size()로 구한다.

```
std::uintmax_t size = fs::file_size(path);
```

마지막 수정 시각은 last_write_time()으로 얻는다.

```
auto writeTime = fs::last_write_time(path);
```

exists(path)가 false라고 해서 항상 파일이 존재하지 않는 것은 아니다. 권한 문제나 경로 접근 오류가 발생했을 수 있다. 오류 원인을 구분해야 한다면 std::error_code 오버로드를 사용한다.

```
std::error_code error;
bool exists = fs::exists(path, error);

if (error)
{
    std::cerr << error.message() << '\n';
}
else if (!exists)
{
    std::cout << "File not found\n";
}
```

15.6.3 디렉터리 생성과 순회

단일 디렉터리는 create_directory(), 중간 경로까지 포함한 디렉터리는 create_directories()로 만든다.

```
fs::create_directory("save");
fs::create_directories("save/profile/slot1");
```

반환값은 실제로 새 디렉터리를 만들었는지 나타낸다. 이미 존재하면 false가 될 수 있다.

```
if (fs::create_directories("save/profile/slot1"))
{
    std::cout << "Directory created\n";
}
```

한 디렉터리의 직접 자식은 directory_iterator로 순회한다.

```
for (const fs::directory_entry& entry :
     fs::directory_iterator{"assets"})
{
    std::cout << entry.path() << '\n';
}
```

하위 디렉터리까지 순회하려면 recursive_directory_iterator를 사용한다.

```
for (const fs::directory_entry& entry :
     fs::recursive_directory_iterator{"assets"})
{
    if (entry.is_regular_file())
    {
        std::cout << entry.path() << '\n';
    }
}
```

디렉터리 순회 순서는 정렬되어 있다고 보장되지 않는다. 이름 순서가 필요하면 경로를 컨테이너에 모은 뒤 정렬한다.

```
std::vector<fs::path> paths;

for (const fs::directory_entry& entry :
     fs::directory_iterator{"assets"})
{
    paths.push_back(entry.path());
}

std::ranges::sort(paths);
```

순회 중 권한 오류나 삭제된 파일을 만날 수 있다. 실패를 건너뛸 정책이 필요하면 directory_options::skip_permission_denied와 std::error_code 기반 반복을 고려할 수 있다.

15.6.4 파일 복사, 이름 변경과 삭제

단일 파일은 copy_file()로 복사한다.

```
fs::copy_file(
    "save/player.dat",
    "backup/player.dat",
    fs::copy_options::overwrite_existing);
```

대상 파일 처리 방식은 copy_options로 정한다.

```
fs::copy_options::skip_existing
fs::copy_options::overwrite_existing
fs::copy_options::update_existing
```

디렉터리와 내부 항목을 복사하려면 copy()와 재귀 옵션을 사용할 수 있다.

```
fs::copy(
    "assets",
    "assets_backup",
```

```
    fs::copy_options::recursive |
    fs::copy_options::overwrite_existing);
```

파일이나 디렉터리의 이름 변경은 rename()으로 처리한다.

```
fs::rename("save/slot1.dat", "save/slot2.dat");
```

같은 파일 시스템 안에서는 경로 이동에도 사용할 수 있다.

```
fs::rename("temp/player.dat", "save/player.dat");
```

서로 다른 파일 시스템이나 볼륨 사이에서는 rename()이 실패할 수 있다. 복사 후 원본 삭제가 필요한 이동은 두 작업 사이의 실패 가능성까지 처리해야 한다.
파일 하나 또는 빈 디렉터리는 remove()로 삭제한다.

```
bool removed = fs::remove("save/player.dat");
```

디렉터리 내부까지 모두 삭제하려면 remove_all()을 사용한다.

```
std::uintmax_t count = fs::remove_all("cache");
```

삭제 경로는 되돌리기 어려우므로 사용자 입력으로 만든 경로를 곧바로 전달하지 않는다. 기준 디렉터리 확인, 정규화, 허용 범위 검사가 필요하다.

15.6.5 플랫폼 독립 경로

path는 운영체제의 기본 경로 표현을 내부적으로 사용한다. 문자열 변환이 꼭 필요하지 않다면 path 상태로 전달하는 편이 안전하다.

```
void LoadTexture(const fs::path& path);
```

경로를 조립할 때 구분자를 직접 붙이지 않는다.

```
fs::path path = baseDirectory / "textures" / fileName;
```

string()은 환경의 일반 문자열 표현, native()는 기본 경로 문자열 타입을 반환한다.

```
auto nativePath = path.native();
std::string text = path.string();
std::string generic = path.generic_string();
```

generic_string()은 /를 구분자로 사용하는 일반 형식을 만든다. 파일 시스템 API 호출은 변환된 문자열보다 path를 직접 사용하는 편이 문자 인코딩과 플랫폼 표현 문제를 줄인다.

경로 비교에는 단순 문자열 비교만으로 충분하지 않을 수 있다. 대소문자 구분, 심볼릭 링크, .과 .., 서로 다른 문자열로 표현된 같은 파일이 운영체제와 파일 시스템에 따라 달라진다. 실제 같은 파일인지 검사해야 한다면 equivalent()를 사용할 수 있다.

```
if (fs::equivalent(firstPath, secondPath))
{
    std::cout << "Same file\n";
}
```

두 경로가 존재하지 않거나 접근할 수 없으면 오류가 발생할 수 있으므로 오류 처리와 함께 사용한다.

15.6.6 예외와 std::error_code

파일 시스템 함수는 기본적으로 실패 시 std::filesystem::filesystem_error를 발생시킬 수 있다.

```
try
{
    fs::copy_file(
        source,
        destination,
        fs::copy_options::overwrite_existing);
```

```
}
catch (const fs::filesystem_error& error)
{
    std::cerr << error.what() << '\n';
    std::cerr << error.path1() << '\n';
    std::cerr << error.path2() << '\n';
}
```

반복 처리처럼 일부 실패를 직접 분기하려면 std::error_code 오버로드를 사용할 수 있다.

```
std::error_code error;
fs::copy_file(
    source,
    destination,
    fs::copy_options::overwrite_existing,
    error);

if (error)
{
    std::cerr << "Copy failed: "
              << error.message() << '\n';
}
```

예외 방식은 실패 시 정상 흐름을 중단하고 상위 계층에서 처리하기 좋다. error_code 방식은 여러 파일을 처리하면서 항목별 실패를 기록하거나 예외를 사용할 수 없는 경로에 적합하다.
오류 코드를 무시하면 파일이 없는 상태와 권한 오류를 구분하지 못한다.

```
std::error_code error;
bool exists = fs::exists(path, error);

if (error)
{
    ReportFilesystemError(path, error);
}
else if (exists)
{
    Process(path);
}
```

파일 시스템은 프로그램 밖에서 동시에 변경될 수 있다. exists()로 확인한 직후 다른 프로세스가 파일을 삭제할 수 있으므로 검사 결과만 믿고 후속 작업의 오류 처리를 생략해서는 안 된다.

15.7 난수

컴퓨터가 만드는 난수는 대부분 결정적인 계산으로 생성되는 의사 난수이다. 같은 엔진 상태에서 시작하면 같은 수열이 만들어진다. 난수 라이브러리는 수열을 생성하는 엔진과 결과를 원하는 확률 분포로 바꾸는 분포 객체를 분리한다.

15.7.1 의사 난수

의사 난수 엔진은 내부 상태를 이용해 수열을 생성한다.

```
#include <random>

std::mt19937 engine{1234};

std::uint32_t first = engine();
std::uint32_t second = engine();
```

같은 시드 1234로 만든 std::mt19937은 같은 호출 순서에서 같은 수열을 생성한다.

```
std::mt19937 firstEngine{1234};
std::mt19937 secondEngine{1234};

assert(firstEngine() == secondEngine());
assert(firstEngine() == secondEngine());
```

의사 난수는 결과를 미리 정할 수 없다는 의미의 진정한 무작위와 다르다. 시드와 엔진 상태를 알면 이후 수열을 재현할 수 있다. 게임 플레이, 시뮬레이션과 테스트에서는 재현성이 장점이 되며, 암호 키나 보안 토큰에는 일반 난수 엔진을 사용해서는 안 된다.

15.7.2 난수 엔진과 분포

엔진은 넓은 정수 범위의 값을 만들고 분포는 원하는 의미의 값으로 변환한다.

```
#include <iostream>
#include <random>

int main()
{
    std::mt19937 engine{1234};
    std::uniform_int_distribution<int> dice{1, 6};

    for (int i = 0; i < 10; ++i)
    {
        std::cout << dice(engine) << ' ';
    }
}
```

std::uniform_int_distribution<int>{1, 6}은 양 끝을 포함한 정수 범위 [1, 6]에서 균등하게 값을 만든다.
실수 범위에는 std::uniform_real_distribution을 사용할 수 있다.

```
std::uniform_real_distribution<double> ratio{0.0, 1.0};
double value = ratio(engine);
```

분포 객체는 엔진을 인수로 받아 값을 만든다.

```
엔진 상태 → 엔진 출력 → 분포 변환 → 프로그램에서 사용할 값
```

정수 범위를 만들기 위해 엔진 결과에 %를 적용하는 방식은 피한다.

```
int value = static_cast<int>(engine() % 6) + 1;
```

엔진 출력 범위의 개수가 6의 배수가 아니면 일부 결과가 더 자주 나타날 수 있다. std::uniform_int_distribution은 엔진 범위와 목표 범위를 고려해 균등 분포를 만든다.

15.7.3 시드와 엔진 상태

시드는 엔진의 초기 상태를 결정한다.

```
std::mt19937 engine{1234};
```

고정 시드는 실행마다 같은 수열이 필요할 때 사용한다.

```
constexpr std::uint32_t testSeed = 1234;
std::mt19937 engine{testSeed};
```

실행마다 다른 시작 상태가 필요하면 std::random_device에서 시드 재료를 얻을 수 있다. std::random_device는 클래스이며 operator()를 제공하므로 함수자처럼 호출하여 난수 값을 얻는다.

```
std::random_device randomDevice;
std::mt19937 engine{randomDevice()};
```

임시 객체를 사용하면 한 문장으로 작성할 수도 있다.

```
std::mt19937 engine{std::random_device{}()};
```

std::random_device 자체도 난수 값을 생성하지만 std::mt19937과 목적이 다르다.

구분	std::random_device	std::mt19937

역할	시드 재료 생성	의사 난수 수열 생성
재현성	일반적으로 사용하지 않음	같은 시드로 재현 가능
일반적인 사용	엔진 초기화	반복적인 난수 생성

std::random_device는 구현 환경에 따라 비결정적 난수원을 사용할 수도 있고 결정적으로 구현될 수도 있다. 일반적인 프로그램에서는 매 난수마다 호출하기보다 엔진의 시드 재료를 얻는 용도로 사용한다.

여러 값을 std::seed_seq에 전달하면 엔진의 초기 상태를 구성할 수 있다.

```
std::random_device randomDevice;

std::seed_seq seed{
    randomDevice(),
    randomDevice(),
    randomDevice(),
    randomDevice()
};

std::mt19937 engine{seed};
```

엔진은 상태를 가진 객체이므로 값을 복사하면 같은 시점의 상태도 복사된다.

```
std::mt19937 first{1234};
std::mt19937 second = first;
assert(first() == second());
```

함수마다 엔진을 값으로 전달하면 복사된 엔진만 진행되고 원본 상태는 변하지 않는다.

```
int RollDice(std::mt19937 engine)
{
    std::uniform_int_distribution<int> dice{1, 6};
    return dice(engine);
}
```

하나의 수열을 계속 사용하려면 참조로 전달한다.

```
int RollDice(std::mt19937& engine)
{
    std::uniform_int_distribution<int> dice{1, 6};
    return dice(engine);
}
```

15.7.4 재현 가능한 난수

버그를 재현하려면 난수 결과 전체보다 시드와 호출 순서를 기록하는 편이 효율적이다.

```
std::uint32_t seed = 1234;
std::mt19937 engine{seed};
std::clog << "Random seed: " << seed << '\n';
```

같은 실행을 재현하려면 같은 엔진 종류, 시드, 분포 매개변수와 호출 순서가 필요하다. 코드 변경으로 난수 호출 횟수가 바뀌면 같은 시드라도 이후 결과가 달라진다.

엔진 상태 전체를 스트림으로 저장하고 복원할 수도 있다.

```
std::ofstream output{"random_state.txt"};
output << engine;

std::ifstream input{"random_state.txt"};
input >> engine;
```

상태 저장은 실행 중간부터 정확히 이어서 재생할 때 유용하다. 장기간 저장 형식으로 사용할 경우 표준 라이브러리 구현과 엔진 타입의 호환성 정책을 함께 고려해야 한다.

테스트에서는 엔진을 외부에서 주입하면 결과를 통제하기 쉽다.

```
class DamageCalculator
{
public:
    explicit DamageCalculator(std::mt19937& engine)
        : engine(engine)
    {
    }

    int Calculate()
    {
        std::uniform_int_distribution<int> damage{10, 20};
        return damage(engine);
    }

private:
    std::mt19937& engine;
};
```

15.7.5 대표 분포와 셔플

확률 모델에 맞는 분포를 선택해야 한다.

```
std::uniform_int_distribution<int> integerRange{1, 100};
std::uniform_real_distribution<double> realRange{0.0, 1.0};
std::bernoulli_distribution criticalHit{0.2};
std::normal_distribution<double> height{170.0, 8.0};
```

std::bernoulli_distribution은 지정한 확률로 true를 반환한다.

```
if (criticalHit(engine))
{
    damage *= 2;
}
```

정규 분포는 평균 주변의 값이 더 자주 나타나는 모델에 사용할 수 있다. 분포 선택은 데이터의 실제 의미와 맞아야 하며, 단순히 값이 무작위로 보인다는 이유로 선택하지 않는다.
컨테이너 원소 순서를 섞을 때는 std::shuffle()을 사용한다.

```
#include <algorithm>
#include <random>
#include <vector>

std::vector<int> cards{1, 2, 3, 4, 5};
std::shuffle(cards.begin(), cards.end(), engine);;
```

원소 하나를 무작위로 선택할 수 있다.

```
if (!items.empty())
{
    std::uniform_int_distribution<std::size_t>
            index{0, items.size() - 1};

    Item& selected = items[index(engine)
```

빈 컨테이너에서는 size() - 1이 잘못된 범위를 만들 수 있으므로 먼저 원소 존재 여부를 확인한다.

15.7.6 rand와 random 라이브러리의 차이

C의 std::rand()는 전역 상태를 사용하는 단순한 난수 함수이다.

```cpp
#include <cstdlib>
#include <ctime>

std::srand(static_cast<unsigned int>(std::time(nullptr)));
int value = std::rand();
```

범위 변환에 %를 자주 사용하지만 균등 분포를 보장하기 어렵다.

```cpp
int dice = std::rand() % 6 + 1;
```

<random>은 엔진과 분포를 타입으로 분리하고, 엔진 객체별 상태 관리와 고정 시드 재현을 지원한다.

<rand와 random 라이브러리 비교>

항목	std::rand()	<random>
상태	전역 상태	엔진 객체별 상태
범위 변환	사용자가 직접 처리	분포 객체가 처리
재현성	전역 호출 순서에 의존	시드와 엔진을 명시적으로 관리
알고리즘 선택	구현에 의존	엔진 타입을 선택 가능
확률 분포	별도 구현 필요	여러 표준 분포 제공

새로운 C++ 코드에서는 <random>을 기본으로 사용한다. 엔진을 프로그램의 적절한 소유 객체에 두고 필요한 함수에 참조로 전달하면 전역 상태 의존을 줄일 수 있다.

15.8 시간

std::chrono는 시간을 숫자 하나가 아니라 단위와 기준을 포함한 타입으로 표현한다. 시간의 길이는 duration, 특정 시점은 time_point, 시점을 생성하는 기준은 시계 타입이 담당한다.

15.8.1 std::chrono::duration

std::chrono::duration<Rep, Period>는 시간의 길이를 표현한다. Rep은 값을 저장할 타입이고 Period는 한 틱의 단위이다.

```cpp
#include <chrono>

std::chrono::seconds seconds{3};
std::chrono::milliseconds milliseconds{1500};
std::chrono::microseconds microseconds{250};
```

count()는 현재 단위의 수치를 반환한다.

```cpp
std::cout << seconds.count() << '\n';
std::cout << milliseconds.count() << '\n';
```

서로 다른 단위끼리 산술 연산을 수행하면 공통 단위로 변환된다.

```cpp
using namespace std::chrono_literals;

auto total = 2s + 500ms;
```

total은 두 값을 정확히 표현할 수 있는 밀리초 단위가 된다.

```cpp
static_assert(
    std::is_same_v<decltype(total), std::chrono::milliseconds>);
```

시간 리터럴을 사용하려면 std::chrono_literals 이름 공간을 열 수 있다.

```cpp
using namespace std::chrono_literals;
auto frame = 16ms;
```

```
auto timeout = 5s;
auto delay = 250us;
```

사용 범위가 넓어지지 않도록 함수나 구현 파일의 좁은 범위에서 사용하는 편이 좋다.
실수 기반 시간도 만들 수 있다.

```
using Seconds = std::chrono::duration<double>;

Seconds elapsed{0.016};
```

게임 루프처럼 소수 초 단위가 필요한 계산에서는 duration<double>을 사용할 수 있다. 파일 형식이나 네트워크 메시지처럼 정확한 정수 단위가 필요한 데이터는 밀리초나 마이크로초 정수형으로 저장하는 편이 명확하다.

15.8.2 std::chrono::time_point

std::chrono::time_point<Clock, Duration>은 시계의 기준점에서 떨어진 위치를 표현한다.

```
using Clock = std::chrono::steady_clock;
Clock::time_point start = Clock::now();
```

두 시점의 차이는 duration이다.

```
Clock::time_point end = Clock::now();
auto elapsed = end - start;
```

시점에 시간 길이를 더하면 새로운 시점이 된다.

```
auto deadline = Clock::now() + std::chrono::seconds{5};
```

현재 시각과 비교할 수 있다.

```
if (Clock::now() >= deadline)
{
    std::cout << "Timeout\n";
}
```

서로 다른 시계의 time_point는 직접 비교하지 않는다.

```
std::chrono::steady_clock::time_point steadyNow =
    std::chrono::steady_clock::now();

std::chrono::system_clock::time_point systemNow =
    std::chrono::system_clock::now();
```

두 시계는 기준점과 목적이 다르다. 같은 시계에서 얻은 시점끼리 차이를 계산해야 한다.

15.8.3 시계의 종류와 선택

표준 라이브러리는 대표적으로 세 시계를 제공한다.

<표준 시계의 용도>

시계	특성	적합한 용도
std::chrono::system_clock	시스템의 달력 시각과 연결됨	파일 시각, 로그 시각, 외부 시간 표현
std::chrono::steady_clock	시간이 역행하지 않음	경과 시간, 제한 시간, 성능 측정
std::chrono::high_resolution_clock	짧은 틱 주기를 목표로 하는 구현 별칭	구현 특성을 확인한 특수 측정

system_clock은 사용자가 시간을 바꾸거나 운영체제의 시간 동기화가 수행되면 값이 앞으로 또는 뒤로 조정될 수 있다. 실행 구간의 길이를 측정하는 데 사용하면 결과가 왜곡될 수 있다.

```
using Clock = std::chrono::steady_clock;
```

```
auto start = Clock::now();
RunTask();
auto end = Clock::now
```

steady_clock::is_steady는 단조 증가 특성을 나타낸다.
static_assert(std::chrono::steady_clock::is_steady);

high_resolution_clock이 별도 하드웨어 시계를 의미한다고 가정해서는 안 된다. 구현에 따라 system_clock이나 steady_clock의 별칭일 수 있으며 단조 증가 여부는 타입 특성으로 확인해야 한다.

달력 시각이 필요하면 system_clock을 사용한다.

```
auto now = std::chrono::system_clock::now();
std::time_t time = std::chrono::system_clock::to_time_t(now);
```

std::time_t를 지역 시각 문자열로 변환하는 과정은 시간대와 스레드 안전성 문제를 포함한다. 단순 경과 시간 측정과 달력·시간대 처리는 목적을 분리해야 한다.

15.8.4 시간 단위 변환

정수 단위 사이의 명시적 변환에는 std::chrono::duration_cast()를 사용한다.

```
using namespace std::chrono;
milliseconds value{2500};
seconds wholeSeconds = duration_cast<seconds>(val
```

wholeSeconds에는 2가 저장된다. 정수 단위로 변환하면서 남는 500밀리초는 절삭된다.

std::cout << wholeSeconds.count() << '\n';

나머지를 분리할 수 있다.

```
auto secondsPart = duration_cast<seconds>(value);
auto millisecondsPart = value - secondsPa
```

C++17 이후에는 반올림 목적에 맞는 함수도 제공된다.

```
auto down = std::chrono::floor<std::chrono::seconds>(value);
auto up = std::chrono::ceil<std::chrono::seconds>(value);
auto nearest = std::chrono::round<std::chrono::seconds>(value);
```

실수 단위로 변환하면 소수 부분을 보존할 수 있다.

```
std::chrono::duration<double> secondsValue = value;
std::cout << secondsValue.count() << '\n'; //
```

time_point의 내부 단위를 바꾸려면 time_point_cast()를 사용할 수 있다.

```
auto now = std::chrono::steady_clock::now();
auto millisecondsPoint =
    std::chrono::time_point_cast<std::chrono::milliseconds>(now);
```

단위 변환은 표현 정밀도를 바꾸는 작업이다. 실제 시간이 흐르거나 시계 기준이 바뀌는 것은 아니다.

15.8.5 경과 시간 측정

함수 실행 시간을 측정할 때는 시작과 종료 시점을 같은 steady_clock에서 얻는다.

```
#include <chrono>
#include <iostream>

int main()
{
```

```
    using Clock = std::chrono::steady_clock;

    const auto start = Clock::now();

    RunTask();

    const auto end = Clock::now();
    const auto elapsed = end - start;

    const auto milliseconds =
        std::chrono::duration<double, std::milli>{elapsed};

    std::cout << milliseconds.count() << " ms\n";
}
```

한 번의 측정에는 캐시 상태, 운영체제 스케줄링, 최적화와 다른 프로세스의 영향이 크게 섞일 수 있다. 매우 짧은 함수는 여러 번 반복하고 전체 시간을 나누는 방식이 더 안정적이다.

```
constexpr int repeatCount = 1000;

const auto start = Clock::now();

for (int i = 0; i < repeatCount; ++i)
{
    RunTask();
}

const auto end = Clock::now();
auto average = (end - start) / repeatCount;
```

컴파일러가 결과가 사용되지 않는 계산을 제거할 수 있으므로 측정 대상이 실제로 수행되는지 확인해야 한다. 정밀한 벤치마크에는 워밍업, 반복 횟수, 통계 처리와 최적화 방지 기능을 갖춘 벤치마크 도구가 적합하다.
측정 코드를 RAII 객체로 분리할 수 있다.

```
class ScopeTimer
{
public:
    explicit ScopeTimer(std::string label)
        : label(std::move(label)),
          start(std::chrono::steady_clock::now())
    {
    }

    ~ScopeTimer()
    {
        const auto end = std::chrono::steady_clock::now();
        const auto elapsed =
            std::chrono::duration<double, std::milli>{end - start};

        std::clog << label << ": "
                  << elapsed.count() << " ms\n";
    }

private:
    std::string label;
    std::chrono::steady_clock::time_point start;
};
```

함수 시작 지점에 객체를 두면 정상 반환과 예외 경로 모두에서 소멸자가 시간을 기록한다.

15.8.6 시간 기반 처리

프레임 횟수에 따라 상태를 변경하면 실행 속도에 따라 결과가 달라진다.

```
position += speed;
```

속도가 초당 이동량이라면 경과 시간을 곱한다.

```
position += speed * delta Seconds;
```

deltaSeconds를 duration<double>로 유지하면 단위를 타입으로 표현할 수 있다.

```
using Seconds = std::chrono::duration<double>;

void Update(Seconds deltaTime)
{
    position += speed * deltaTime.count();
}

쿨다운은 완료 시점을 저장하는 방식으로 구현할 수 있다.
class Cooldown
{
public:
    explicit Cooldown(std::chrono::milliseconds interval)
        : interval(interval),

readyTime(std::chrono::steady_clock::time_point::min())
    {
    }

    bool TryActivate()
    {
        const auto now = std::chrono::steady_clock::now();

        if (now < readyTime)
        {
            return false;
        }

        readyTime = now + interval;
        return true;
    }

private:
    std::chrono::milliseconds interval;
    std::chrono::steady_clock::time_point readyTime;
};
```

일정 간격으로 갱신하는 처리에서는 누적 시간을 사용할 수 있다.

```
using Seconds = std::chrono::duration<double>;

Seconds accumulator{0.0};
constexpr Seconds fixedStep{1.0 / 60.0};

void Tick(Seconds frameTime)
{
    accumulator += frameTime;

    while (accumulator >= fixedStep)
    {
        FixedUpdate(fixedStep);
        accumulator -= fixedStep;
    }
}
```

}

프레임이 매우 오래 지연되면 반복 횟수가 급격히 늘 수 있다. 최대 프레임 시간을 제한하거나 한 프레임에서 허용할 고정 업데이트 횟수를 정하는 정책이 필요하다.
스레드를 일정 시간 멈추려면 std::this_thread::sleep_for()와 sleep_until()을 사용할 수 있다.

```
#include <thread>
std::this_thread::sleep_for(std::chrono::milliseconds{100});
```

수면 함수는 지정한 시간보다 일찍 깨어나지 않는 방향의 대기이며 정확한 실시간 스케줄을 보장하지 않는다. 운영체제 스케줄러 지연으로 더 늦게 깨어날 수 있다.

15.9 학습 정리

스트림은 콘솔, 문자열과 파일을 공통된 읽기·쓰기 인터페이스로 다룬다. 입력 연산의 성공 여부는 실제 추출 연산으로 확인해야 하며, 실패 상태를 복구할 때는 clear()로 상태 플래그를 초기화하고 ignore()로 잘못된 입력을 제거한다. operator>>와 std::getline()을 함께 사용할 때는 입력 버퍼에 남은 줄바꿈을 처리해야 한다.

스트림 형식 조정자는 일부 상태가 이후 출력에도 유지된다. std::setw()는 한 필드에만 적용되지만 진법, 정렬과 정밀도 설정은 변경할 때까지 남을 수 있다. std::format()은 호출 하나에 형식 규칙을 모아 문자열을 만들며, std::format_to()는 기존 출력 대상에 형식 결과를 기록한다. 사용자 정의 타입은 스트림 연산자 또는 std::formatter로 표준 출력 문법에 연결할 수 있다.

텍스트 파일은 사람이 읽기 쉽지만 구분자, 인용 부호, 문자 인코딩과 버전 규칙이 필요하다. 바이너리 모드는 텍스트 변환을 막을 뿐 객체를 자동으로 직렬화하지 않는다. 지속 가능한 바이너리 형식은 필드 크기, 바이트 순서, 문자열 길이, 파일 종류와 버전을 명시해야 한다. 포인터와 동적 메모리를 포함한 객체를 메모리 그대로 저장해서는 안 된다.

std::filesystem::path는 운영체제 경로를 타입으로 표현하고 / 연산자로 구성 요소를 결합한다. 파일 시스템 함수는 예외 방식과 std::error_code 방식을 제공하며, 파일은 검사 직후에도 외부 요인으로 변경될 수 있으므로 실제 작업의 실패를 항상 처리해야 한다.

난수 라이브러리는 엔진과 분포를 분리한다. 엔진은 상태를 가진 의사 난수 수열을 만들고 분포는 원하는 범위와 확률 모델로 변환한다. 고정 시드는 테스트와 재현에 사용하고, 실행마다 다른 수열이 필요하면 시드 재료를 외부에서 얻는다. 엔진 결과에 %를 적용하기보다 표준 분포를 사용해야 한다.

std::chrono::duration은 시간의 길이, time_point는 특정 시계의 시점을 표현한다. 달력 시각은 system_clock, 경과 시간과 제한 시간은 steady_clock을 사용한다. 단위 변환 과정에서는 정수 단위의 절삭을 확인하고, 시간 기반 처리는 프레임 횟수가 아니라 실제 경과 시간을 기준으로 설계한다.

15.10 학습 과제

과제 **1.** 안전한 정수 입력
정수를 입력받는 ReadInteger() 함수를 작성한다. 문자열이나 실수를 입력하면 오류 상태와 현재 행의 나머지를 제거하고 다시 입력받는다. 최솟값과 최대값을 인수로 전달해 범위를 벗어난 값도 다시 입력받도록 확장한다.

과제 **2.** 사용자 정의 타입 입출력
Item 구조체에 id, name, count, price를 정의하고 operator<<와 operator>>를 구현한다. 입력 과정에서 일부 필드가 실패하면 기존 객체의 값이 변경되지 않아야 한다. id <= 0, count < 0, price < 0은 failbit를 설정해 거부한다.

과제 **3.** 문자열 전체 파싱
문자열을 double 하나로 변환하는 ParseDouble() 함수를 작성한다. 앞뒤 공백은 허용하지만 숫자 뒤에 다른 문자가 남으면 실패해야 한다. 반환 타입은 std::optional<double>을 사용한다.

과제 **4.** 형식화된 목록 출력
여러 Item을 이름, 수량, 단가, 총액 열로 출력한다. 스트림 조정자를 사용하는 버전과 std::format()을 사용하는 버전을 각각 작성한다. 이름은 왼쪽 정렬, 숫자는 오른쪽 정렬하고 금액은 소수점 이하 두 자리로 표시한다.

과제 **5.** 텍스트 파일 저장과 불러오기
PlayerData 배열을 텍스트 파일에 저장하고 다시 읽는다. 한 행에 하나의 객체를 기록하고 잘못된 행은 행 번호와 함께 오류로 출력한다. 파일을 열지 못한 경우, 형식 오류, 장치 읽기 오류를 구분한다.

과제 **6.** 단순 **CSV**와 제한 조건
쉼표로 구분된 아이템 파일을 읽어 std::vector<Item>으로 변환한다. 각 행의 필드 수와 숫자 변환 성공 여부를 검사한다. 이름에 쉼표와 줄바꿈이 들어가지 않는다는 제한을 문서화하고, 일반 CSV와 다른 점을 설명한다.

과제 **7.** 바이너리 레코드 형식
매직 값, 버전, 아이템 개수와 아이템 필드를 포함하는 바이너리 형식을 설계한다. 정수는 리틀 엔디언으로 기록하고 문자열은 길이와 UTF-8 바이트를 기록한다. 읽을 수 있는 최대 아이템 개수와 최대 문자열 길이를 정해 파일 손상에 대비한다.

과제 **8.** 파일 버전 변환
버전 1에는 id, name, hp, 버전 2에는 level 필드가 추가된 플레이어 파일을 구현한다. 버전 1 파일을 읽으면 level에 기본값을 적용한다. 알 수 없는 버전은 오류로 처리한다.

과제 **9.** 파일 검색
지정한 디렉터리를 재귀적으로 순회해 확장자가 .png인 파일을 찾는다. 결과 경로를 정렬하고 파일 크기와 함께 출력한다. 접근 권한 오류는 기록한 뒤 다른 항목의 순회를 계속한다.

과제 **10.** 안전한 임시 파일 저장
원본 파일을 직접 덮어쓰지 않고 임시 파일에 저장한 뒤 교체하는 함수를 작성한다. 임시 파일 기록 실패, 이름 변경 실패와 기존 백업 파일 처리 정책을 구분한다.

과제 **11.** 난수 주사위
std::mt19937과 std::uniform_int_distribution으로 주사위를 백만 번 던지고 각 눈의 출현 횟수를 출력한다. engine() % 6 + 1 방식의 결과와 비교하고 분포 차이를 분석한다.

과제 **12.** 재현 가능한 던전 생성
정수 시드를 받아 방의 위치와 아이템 배치를 생성한다. 같은 시드에서는 같은 결과가 만들어져야 한다. 실행 로그에 시드를 기록하고 사용자가 이전 시드를 입력해 같은 던전을 다시 생성할 수 있도록 구성한다.

과제 **13.** 무작위 셔플과 추첨
참가자 이름을 저장한 벡터를 std::shuffle()로 섞고 중복 없이 당첨자를 선택한다. 빈 목록, 당첨 인원이 참가자 수보다 많은 경우를 처리한다.

과제 **14.** 실행 시간 측정
같은 데이터에 대해 선형 검색과 정렬 후 이진 탐색의 실행 시간을 측정한다. steady_clock을 사용하고 여러 번 반복해 평균 시간을 구한다. 데이터 생성 시간은 검색 시간에서 제외한다.

과제 **15.** 쿨다운과 고정 업데이트
std::chrono 타입을 사용하는 쿨다운 클래스를 구현한다. 별도의 고정 업데이트 예제에서는 프레임 시간을 누적하고 60분의 1초 간격으로 상태를 갱신한다. 한 프레임에서 수행할 최대 업데이트 횟수를 제한한다.

참고. C++23의 std::print
C++23의 std::print()는 std::format 형식 문자열을 사용해 표준 출력에 직접 기록한다.

```cpp
#include <print>
int main()
{
    std::print("Name: {}, Level: {}\n", "Knight", 12);
}
```

std::print()는 줄바꿈을 자동으로 추가하지 않는다. std::println()은 출력 끝에 줄바꿈을 추가한다.
std::println("Name: {}, Level: {}", "Knight", 12);

형식화된 결과를 먼저 문자열로 만들 필요가 없다.

```cpp
std::cout << std::format(
    "Score: {}\n",
    1500);

std::print("Score: {}\n", 15
```

FILE*에도 출력할 수 있다.

```cpp
#include <cstdio>
#include <print>

int main()
{
    std::FILE* file = std::fopen("log.txt", "w");

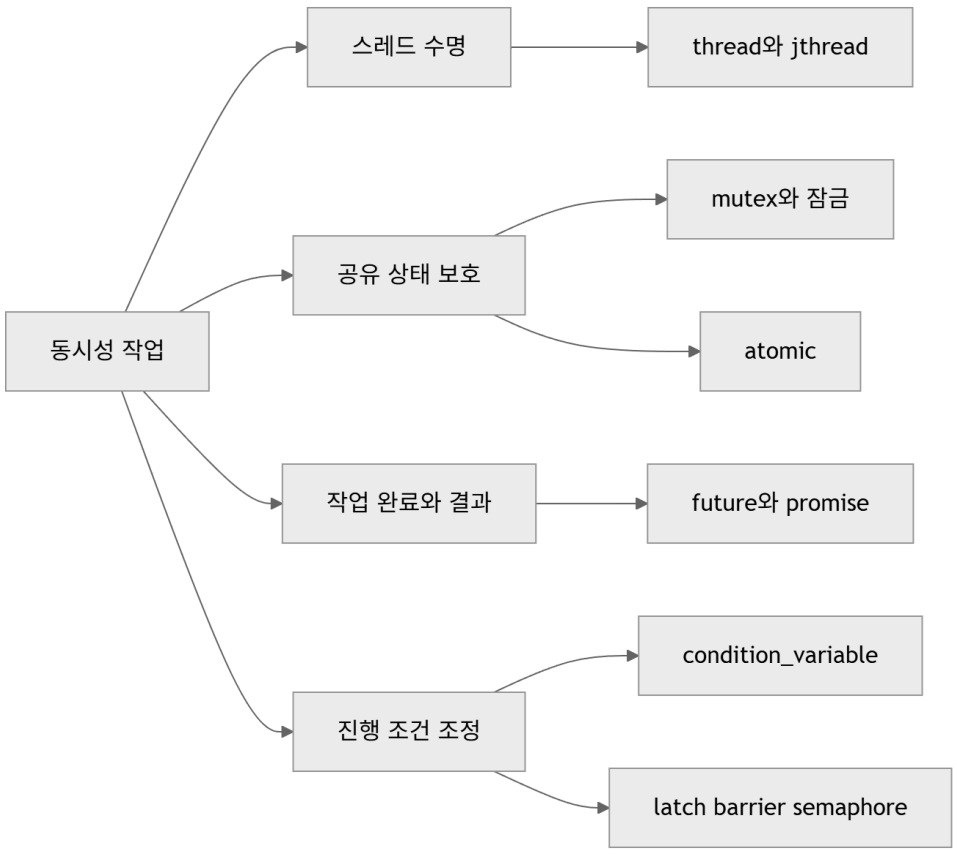
    if (file == nullptr)
    {
        return 1;
    }

    std::println(file, "Player {} joined", "Knight");
    std::fclose(file);
}
```

std::print와 std::println은 C++23 기능이다. C++20을 기준으로 하는 코드에서는 스트림 출력이나 std::format()으로 만든 문자열을 사용한다.

16. 동시성 프로그래밍

프로그램의 모든 작업을 하나의 실행 흐름에서 처리하면 긴 연산이 끝날 때까지 다른 작업이 지연될 수 있다. 파일을 읽는 동안 입력 처리가 멈추거나, 복잡한 계산 때문에 화면 갱신이 늦어지는 현상이 대표적이다. 서로 독립적인 작업을 여러 실행 흐름으로 나누면 한 작업이 대기하는 동안 다른 작업을 진행하거나 여러 처리 장치에서 작업을 동시에 실행할 수 있다.



여러 스레드가 같은 데이터를 사용하면 실행 순서를 예측하기 어려워진다. 공유 변수의 값을 읽고 변경하는 코드가 한 줄로 보이더라도 실제로는 여러 연산으로 나뉠 수 있다. 적절한 동기화 없이 같은 메모리 위치를 동시에 변경하면 데이터 경쟁이 발생하며 프로그램의 동작은 보장되지 않는다.
동시성 프로그래밍은 스레드를 많이 만드는 기술이 아니다. 작업을 나누고 데이터의 소유권을 정하며, 공유 상태에 접근하는 순서와 프로그램 종료 시점을 설계하는 작업이다. 이 장에서는 C++20 표준 라이브러리의 스레드와 동기화 도구를 중심으로 안전한 실행 흐름을 구성한다.

16.1 동시성과 병렬성

동시성(concurrency)은 여러 작업의 실행 구간이 겹치도록 구성하는 성질이고, 병렬성(parallelism)은 여러 작업이 실제로 같은 시간에 실행되는 성질이다. 한 처리 장치에서도 작업을 번갈아 실행하면 동시성을 구성할 수 있지만 실제 병렬 실행에는 둘 이상의 실행 자원이 필요하다.

16.1.1 프로세스와 스레드

프로세스는 실행 중인 프로그램의 자원과 주소 공간을 관리하는 단위이다. 서로 다른 프로세스는 일반적으로 독립된 주소 공간을 사용하므로 데이터를 공유하려면 운영체제가 제공하는 통신 수단이 필요하다.

스레드는 한 프로세스 안에서 실행되는 흐름이다. 같은 프로세스의 스레드는 코드와 전역 데이터, 동적 메모리와 운영체제 자원을 공유한다. 각 스레드는 자신의 호출 스택과 현재 실행 위치를 가진다.

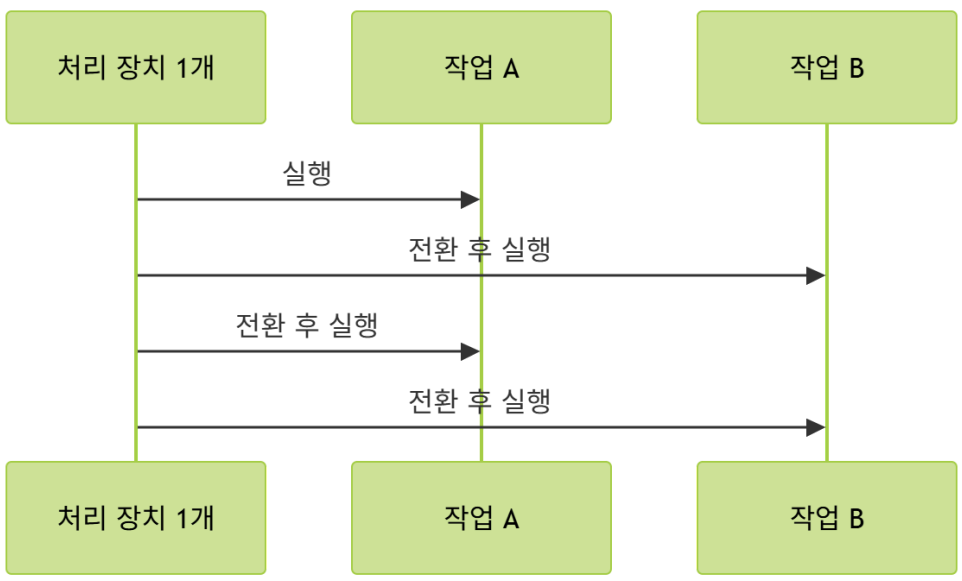
구분	프로세스	스레드
주소 공간	일반적으로 분리	같은 프로세스 안에서 공유
데이터 교환	프로세스 간 통신 필요	같은 메모리에 직접 접근 가능
격리	비교적 강함	한 스레드의 오류가 프로세스 전체에 영향 가능
생성과 전환 비용	상대적으로 큼	상대적으로 작음

스레드가 메모리를 쉽게 공유한다는 특성은 장점이면서 위험 요소이다. 데이터 복사 없이 협력할 수 있지만 접근 규칙을 잘못 설계하면 서로의 작업을 방해한다.

16.1.2 동시성과 병렬성의 차이

한 명의 작업자가 두 작업을 번갈아 처리해도 두 작업은 함께 진행된다. 작업 A를 잠시 수행하고 작업 B로 전환한 뒤 다시 작업 A로 돌아오는 구조는 동시성이 있지만 병렬성은 없다.

두 명의 작업자가 각 작업을 같은 시각에 수행하면 동시성과 병렬성이 모두 존재한다. 운영체제는 실행 가능한 스레드를 처리 장치에 배치하므로 프로그램이 만든 스레드 수와 실제 병렬 실행 수는 같다고 보장되지 않는다.



작업을 여러 스레드로 분리해도 항상 빨라지지는 않는다. 스레드 생성, 작업 분배, 잠금, 캐시 이동과 결과 결합에도 비용이 든다. 작업이 충분히 크고 서로 독립적일 때 병렬 실행의 이점이 나타난다.

16.1.3 공유 상태와 데이터 경쟁

두 스레드가 하나의 정수 변수를 증가시키는 코드를 생각해 보자.

```
int counter = 0;
void Increase()
{
    for (int i = 0; i < 100'000; ++i)
    {
        ++counter;
    }
}
```

++counter는 메모리에서 값을 읽고, 1을 더하고, 결과를 다시 기록하는 과정으로 처리될 수 있다. 두 스레드가 같은 값을 읽은 뒤 각자 증가시켜 기록하면 한 번의 증가가 사라질 수 있다.

```
counter의 값: 10

스레드 A가 10을 읽음
스레드 B가 10을 읽음
스레드 A가 11을 기록함
스레드 B가 11을 기록함
```

두 번 증가했지만 최종값은 11이다. 더 중요한 문제는 잘못된 숫자에 그치지 않는다. 둘 이상의 스레드가 같은 메모리 위치에 접근하고, 하나 이상의 접근이 쓰기이며, 접근 사이에 적절한 동기화가 없으면 데이터 경쟁이 발생한다. 데이터 경쟁이 있는 C++ 프로그램의 동작은 정의되지 않는다.

읽기만 수행하는 공유 데이터는 여러 스레드가 동시에 접근할 수 있다. 한 스레드라도 변경한다면 모든 관련 접근을 같은 동기화 규칙으로 보호해야 한다.

```
std::vector<int> values;

void AddValue(int value)
{
    values.push_back(value); // 동기화가 없다면 안전하지 않음
}

int ReadFirst()
{
    return values.front();   // 동시에 push_back이 실행되면
안전하지 않음
}
```

컨테이너의 서로 다른 멤버 함수를 호출했다는 사실만으로 스레드 안전성이 생기지 않는다. 내부 크기와 저장 공간처럼 같은 상태를 읽고 변경할 수 있기 때문이다.

16.1.4 메모리 가시성과 happens-before

한 스레드가 데이터를 기록한 뒤 준비 완료 플래그를 변경하고, 다른 스레드가 플래그를 확인한 뒤 데이터를 읽는 구조가 있다.

```
int result = 0;
bool ready = false;

void Produce()
{
    result = 100;
    ready = true;
}

void Consume()
{
    if (ready)
    {
        Use(result);
    }
}
```

두 변수에 대한 동기화가 없으므로 이 코드는 안전하지 않다. 컴파일러와 처리 장치는 단일 스레드의 의미를 유지하는 범위에서 메모리 연산을 재배치할 수 있으며, 다른 스레드가 기록을 관찰하는 시점도 별도로 보장되지 않는다.
동기화 연산은 한 스레드의 작업이 다른 스레드에서 관찰될 수 있도록 순서 관계를 만든다. 한 연산이 다른 연산보다 happens-before 관계에 있으면 앞선 연산의 효과를 뒤 연산에서 관찰할 수 있다.

```
std::mutex mutex;
int result = 0;
bool ready = false;

void Produce()
{
    std::lock_guard lock{mutex};
    result = 100;
    ready = true;
}

void Consume()
{
    std::lock_guard lock{mutex};

    if (ready)
    {
        Use(result);
    }
}
```

같은 뮤텍스의 잠금과 해제는 공유 데이터 접근을 직렬화하고 메모리 가시성도 연결한다. std::thread::join(), 조건 변수, 원자적 연산도 정해진 규칙에 따라 스레드 사이의 순서를 형성한다.

16.2 std::thread와 std::jthread

std::thread는 하나의 실행 스레드를 나타내는 이동 전용 객체이다. 생성자에 호출 가능한 객체와 인수를 전달하면 새 실행 흐름이 시작된다. <thread> 헤더가 필요하다.

16.2.1 스레드 생성과 함수 실행

일반 함수를 새 스레드에서 실행할 수 있다.

```
#include <iostream>
#include <thread>

void PrintMessage()
```

```
{
    std::cout << "worker thread\n";
}

int main()
{
    std::thread worker{PrintMessage};
    worker.join();
}
```

std::thread 생성이 완료되면 새 스레드는 PrintMessage()를 실행할 수 있다. 생성한 스레드와 현재 스레드 중 어느 쪽이 먼저 코드를 수행할지는 정해져 있지 않다.
람다와 함수 객체도 전달할 수 있다.

```
std::thread worker {
    []
    {
        std::cout << "lambda thread\n";
    }};

worker.join();
```

인수를 받는 함수는 호출 대상 뒤에 인수를 나열한다.

```
void PrintRange(int first, int last)
{
    for (int value = first; value <= last; ++value)
    {
        std::cout << value << ' ';
    }
}

std::thread worker{PrintRange, 1, 5};
worker.join();
```

스레드 함수에서 처리되지 않은 예외가 밖으로 빠져나오면 std::terminate()가 호출된다. 스레드 내부에서 처리하거나 std::promise, std::future 같은 결과 전달 수단을 사용해야 한다.

```
void Worker()
{
    try
    {
        RunTask();
    }
    catch (const std::exception& exception)
    {
        LogError(exception.what());
    }
}
```

16.2.2 인수 전달과 객체 수명

std::thread는 전달받은 함수와 인수를 내부에 저장한 뒤 새 스레드에서 호출한다. 일반 인수는 복사되거나 이동되어 저장된다.

```
void SetValue(int& target, int value)
{
    target = value;
}

int number = 0;

std::thread worker{SetValue, std::ref(number), 100};
worker.join();
```

SetValue()의 첫 매개변수는 참조이지만 std::thread에 number만 전달하면 참조 전달이 되지 않는다. std::ref(number)가 복사 가능한 참조 래퍼를 만들어 원본 객체를 가리키게 한다.

상수 참조는 std::cref()로 명시할 수 있다.

```
void PrintText(const std::string& text);

std::string message = "loading";
std::thread worker{PrintText, std::cref(message)};
worker.join();
```

참조로 전달한 객체는 스레드가 사용하는 동안 살아 있어야 한다.

```
std::thread StartInvalidThread()
{
    int localValue = 10;

    return std::thread{
        [&]
        {
            std::cout << localValue << '\n';
        }};
}
```

함수가 반환되면 localValue가 소멸하므로 새 스레드의 참조는 무효가 된다. 값 캡처나 소유 객체 전달로 수명을 분리해야 한다.

```
std::thread StartThread()
{
    int localValue = 10;

    return std::thread{
        [localValue]
        {
            std::cout << localValue << '\n';
        }};
}
```

멤버 함수는 객체 포인터 또는 참조와 함께 전달한다.

```
class Loader
{
public:
    void Load(std::string fileName)
    {
        std::cout << fileName << '\n';
    }
};

Loader loader;
std::thread worker{&Loader::Load, &loader, "data.bin"};
worker.join();
```

loader도 스레드 실행이 끝날 때까지 살아 있어야 한다.

16.2.3 join, joinable과 detach

join()은 대상 스레드가 종료될 때까지 현재 스레드를 기다리게 한다.

```
std::thread worker{RunTask};
worker.join();
```

join()이 끝나면 worker는 더 이상 실행 스레드를 나타내지 않는다. joinable()로 결합 가능한 상태인지 확인할 수 있다.

```
if (worker.joinable())
{
    worker.join();
}
```

기본 생성된 std::thread, 이동된 원본 객체, 이미 join() 또는 detach()가 호출된 객체는 결합 가능한 스레드를 나타내지 않는다.

```
std::thread first{RunTask};
std::thread second{std::move(first)};

if (!first.joinable())
{
    std::cout << "first is empty\n";
}

second.join();
```

detach()는 std::thread 객체와 실행 스레드의 연결을 끊는다.

```
std::thread worker{RunBackgroundTask};
worker.detach();
```

분리된 스레드는 나중에 join()할 수 없다. 프로세스가 끝나면 작업 완료 여부와 관계없이 실행이 종료될 수 있으며, 참조한 객체의 수명도 자동으로 연장되지 않는다. 소유권과 종료 시점을 별도로 보장할 수 없는 코드에서는 detach()를 피하는 편이 안전하다.

16.2.4 예외 경로와 스레드 수명

결합 가능한 스레드를 나타내는 std::thread 객체가 소멸하면 std::terminate()가 호출된다.

```
void Process()
{
    std::thread worker{RunTask};

    // worker.join() 없이 범위를 벗어나면 프로그램 종료
}
```

예외가 발생해 join() 호출을 건너뛰는 경로도 같은 문제를 만든다.

```
void Process()
{
    std::thread worker{RunTask};

    PerformOperation(); // 예외가 발생할 수 있음

    worker.join();
}
```

C++20에서는 자동 결합을 제공하는 std::jthread가 일반적인 작업 스레드에 더 적합하다. std::thread를 사용해야 한다면 모든 정상 경로와 예외 경로에서 결합 상태를 정리해야 한다.

스레드 함수의 예외는 생성한 스레드로 자동 전달되지 않는다. 호출 대상 내부에서 예외가 빠져나오면 프로그램이 종료되므로 작업 결과와 예외를 std::future로 전달하는 방식이 필요할 수 있다.

16.2.5 std::jthread와 자동 결합

std::jthread는 std::thread와 비슷하게 스레드를 시작하지만 소멸 시 결합 가능한 스레드에 중지를 요청하고 join()한다.

```
#include <iostream>
#include <thread>

void PrintMessage()
{
    std::cout << "jthread worker\n";
}

int main()
{
    std::jthread worker{PrintMessage};
}
```

main()이 끝나면서 worker가 소멸하면 실행 중인 스레드가 끝날 때까지 기다린다. 명시적인 join()도 사용할 수 있지만 단순한 범위 기반 수명 관리에는 자동 결합이 유용하다.

자동 결합이 작업을 즉시 끝내 주는 것은 아니다. 스레드 함수가 긴 작업을 수행하거나 무한 반복한다면 소멸자는 작업이 반환될 때까지 기다릴 수 있다. 종료 요청을 확인하는 협력적 중지가 필요하다.

16.2.6 stop_token을 이용한 협력적 중지

std::jthread는 호출 가능한 객체의 첫 매개변수가 std::stop_token을 받을 수 있으면 토큰을 전달한다.

```
#include <chrono>
#include <iostream>
#include <stop_token>
#include <thread>

using namespace std::chrono_literals;

void Worker(std::stop_token stopToken)
{
    while (!stopToken.stop_requested())
    {
        std::cout << "working\n";
        std::this_thread::sleep_for(100ms);
    }

    std::cout << "stopped\n";
}

int main()
{
    std::jthread worker{Worker};

    std::this_thread::sleep_for(250ms);
    worker.request_stop();
}
```

request_stop()은 중지 상태를 설정한다. 실행 중인 코드를 강제로 중단하지 않는다. 작업 함수가 stop_requested()를 확인하고 정리 작업을 마친 뒤 반환해야 한다.

긴 반복문에서는 적절한 간격으로 중지 요청을 확인한다.

```
void Calculate(std::stop_token stopToken)
{
    for (std::size_t i = 0; i < workCount; ++i)
    {
        if (stopToken.stop_requested())
        {
            return;
        }

        ProcessWork(i);
    }
}
```

중지 시점의 상태 규칙도 정해야 한다. 처리 중인 작업을 끝낸 뒤 중지할지, 큐에 남은 작업까지 처리할지, 즉시 버릴지를 인터페이스에서 결정해야 한다.

16.3 뮤텍스와 잠금

뮤텍스(mutex)는 여러 스레드가 하나의 임계 구역을 동시에 실행하지 못하도록 상호 배제를 제공한다. 공유 데이터에 접근하는 모든 코드가 같은 뮤텍스를 사용해야 보호가 성립한다.

16.3.1 std::mutex와 임계 구역

```
#include <mutex>
```

```cpp
#include <thread>
#include <vector>

int counter = 0;
std::mutex counterMutex;

void Increase()
{
    for (int i = 0; i < 100'000; ++i)
    {
        counterMutex.lock();
        ++counter;
        counterMutex.unlock();
    }
}
```

lock()에 성공한 스레드만 보호 구역에 들어간다. 다른 스레드는 뮤텍스가 해제될 때까지 대기한다.
수동 lock()과 unlock()은 예외와 조기 반환에 취약하다.

```cpp
counterMutex.lock();

if (!CanUpdate())
{
    return; // unlock()이 호출되지 않음
}

++counter;
counterMutex.unlock();
```

잠금 객체를 사용하면 범위 종료와 잠금 해제를 연결할 수 있다.

16.3.2 std::lock_guard

std::lock_guard는 생성할 때 뮤텍스를 잠그고 소멸할 때 해제한다.

```cpp
void Increase()
{
    for (int i = 0; i < 100'000; ++i)
    {
        std::lock_guard lock{counterMutex};
        ++counter;
    }
}
```

템플릿 인수는 생성자 인수에서 추론되므로 std::lock_guard<std::mutex>를 직접 적지 않아도 된다.
여러 값을 하나의 불변식으로 관리한다면 같은 잠금 범위에서 함께 변경한다.

```cpp
class Account
{
public:
    void Deposit(int amount)
    {
        std::lock_guard lock{mutex};
        balance += amount;
        ++transactionCount;
    }

    std::pair<int, int> GetState() const
    {
        std::lock_guard lock{mutex};
        return {balance, transactionCount};
    }

private:
    mutable std::mutex mutex;
    int balance = 0;
    int transactionCount = 0;
```

```
};
```

GetState()가 두 값을 한 잠금 안에서 복사하므로 서로 같은 시점의 상태를 반환한다. const 멤버 함수에서도 동기화 객체는 내부 구현 상태를 변경하므로 뮤텍스를 mutable로 선언할 수 있다.

16.3.3 std::scoped_lock

두 계좌 사이에서 금액을 이동하려면 두 객체를 함께 잠가야 한다.

```
class Account
{
public:
    friend void Transfer(Account& from, Account& to, int
amount);

private:
    std::mutex mutex;
    int balance = 0;
};

void Transfer(Account& from, Account& to, int amount)
{
    if (&from == &to)
    {
        return;
    }

    std::scoped_lock lock{from.mutex, to.mutex};

    if (from.balance < amount)
    {
        return;
    }

    from.balance -= amount;
    to.balance += amount;
}
```

std::scoped_lock은 여러 뮤텍스를 교착 상태를 피하는 방식으로 잠근다. 모든 대상 뮤텍스는 잠금 객체가 소멸할 때 해제된다.
하나의 뮤텍스에도 사용할 수 있지만 여러 뮤텍스를 함께 관리할 때 목적이 가장 분명하다.

16.3.4 std::unique_lock

std::unique_lock은 잠금 소유 여부를 상태로 보관하고 이동할 수 있다. 지연 잠금, 명시적 해제와 재잠금, 조건 변수 대기에 사용한다.

```
std::unique_lock lock{counterMutex};
++counter;
lock.unlock();

PerformLongOperation();

lock.lock();
ReadCounter(counter);
```

잠금 시점을 늦출 수도 있다.

```
std::unique_lock lock{counterMutex, std::defer_lock};

PrepareLocalData();
lock.lock();
UpdateSharedData();
```

owns_lock()은 현재 잠금을 소유하는지 확인한다.

```
if (lock.owns_lock())
{
```

```
    std::cout << "locked\n";
}
```

단순히 한 범위 전체를 잠그는 코드에는 std::lock_guard나 std::scoped_lock이 더 분명하다. std::unique_lock은 잠금 상태를 제어해야 할 때 사용한다.

16.3.5 RAII와 잠금 해제

잠금 객체는 함수가 정상적으로 끝나거나 예외로 빠져나가거나 조기 반환하더라도 소멸한다.

```
void Update()
{
    std::lock_guard lock{mutex};

    if (!IsValid())
    {
        return;
    }

    ModifyState(); // 예외가 발생해도 lock의 소멸자가 뮤텍스를 해제
}
```

RAII는 잠금 해제를 보장하지만 임계 구역의 범위가 적절한지는 별도 문제이다. 파일 입출력, 긴 계산, 사용자 콜백 호출을 잠금 안에서 수행하면 다른 스레드의 대기 시간이 길어질 수 있다.

```
Data snapshot;

{
    std::lock_guard lock{mutex};
    snapshot = sharedData;
}

SaveToFile(snapshot);
```

공유 데이터를 잠금 안에서 복사하고 긴 작업은 잠금 밖에서 수행할 수 있다. 복사된 스냅샷으로 충분한지, 저장 도중 공유 상태가 바뀌어도 되는지는 프로그램 요구에 따라 판단한다.

16.4 교착 상태

교착 상태(deadlock)는 둘 이상의 실행 흐름이 서로가 보유한 자원을 기다리며 어느 쪽도 진행하지 못하는 상태이다. 프로그램이 종료되지는 않지만 관련 스레드는 영원히 대기할 수 있다.

16.4.1 교착 상태의 네 조건

교착 상태에는 네 조건이 함께 존재한다.

조건	의미
상호 배제	한 자원을 한 스레드만 사용할 수 있음
점유와 대기	자원을 가진 채 다른 자원을 기다림
비선점	다른 스레드가 보유한 자원을 강제로 빼앗을 수 없음
순환 대기	스레드들이 원형으로 서로의 자원을 기다림

두 스레드가 반대 순서로 뮤텍스를 잠그면 순환 대기가 생길 수 있다.

```
std::mutex firstMutex;
std::mutex secondMutex;

void FirstTask()
{
```

```
    std::lock_guard firstLock{firstMutex};
    std::lock_guard secondLock{secondMutex};
}

void SecondTask()
{
    std::lock_guard secondLock{secondMutex};
    std::lock_guard firstLock{firstMutex};
}
```

FirstTask()가 firstMutex를 보유하고 SecondTask()가 secondMutex를 보유하면 두 함수가 서로의 잠금을 기다린다.

16.4.2 잠금 순서

모든 코드에서 뮤텍스를 같은 순서로 잠그면 순환 대기를 제거할 수 있다.

```
void FirstTask()
{
    std::lock_guard firstLock{firstMutex};
    std::lock_guard secondLock{secondMutex};
}
void SecondTask()
{
    std::lock_guard firstLock{firstMutex};
    std::lock_guard secondLock{secondMutex};
}
```

대규모 코드에서는 잠금 순서를 문서화하고 계층을 정할 수 있다. 높은 계층의 잠금을 보유한 상태에서 낮은 계층의 잠금만 획득하도록 규칙을 통일한다.
객체 주소를 기준으로 순서를 정하는 방법도 있지만 모든 호출부가 같은 규칙을 사용해야 한다. 직접 순서를 관리하기 어려우면 여러 뮤텍스를 함께 잠그는 표준 도구를 사용한다.

16.4.3 여러 뮤텍스의 안전한 잠금

std::scoped_lock은 여러 뮤텍스를 받아 교착 상태를 피하도록 잠근다.

```
void Exchange(Data& left, Data& right)
{
    if (&left == &right)
    {
        return;
    }

    std::scoped_lock lock{left.mutex, right.mutex};
    std::swap(left.value, right.value);
}
```

```
std::lock()과 std::unique_lock을 함께 사용할 수도 있다.
std::unique_lock firstLock{firstMutex, std::defer_lock};
std::unique_lock secondLock{secondMutex, std::defer_lock};

std::lock(firstLock, secondLock);
```

잠금 획득이 끝나면 두 std::unique_lock이 각 뮤텍스의 소유권을 관리한다. 단순한 여러 뮤텍스 잠금에는 std::scoped_lock이 더 간결하다.

16.4.4 잠금 범위와 외부 호출

잠금을 오래 보유하면 경합이 증가한다. 공유 상태를 읽거나 변경하는 데 필요한 범위만 보호하는 편이 좋다.

```
Task task;

{
    std::lock_guard lock{mutex};
    task = tasks.front();
    tasks.pop();
}
```

```
Execute(task);
```

Execute()가 오래 걸려도 큐의 뮤텍스는 이미 해제되어 다른 스레드가 큐에 접근할 수 있다.
잠금 안에서 외부 콜백이나 가상 함수를 호출하면 호출 대상이 어떤 뮤텍스를 획득할지 알기 어렵다.

```
void Notify()
{
    std::lock_guard lock{mutex};
    callback(); // callback 내부의 잠금 순서를 알 수 없음
}
```

필요한 데이터를 잠금 안에서 복사한 뒤 외부 호출은 잠금 밖에서 수행하는 구조가 안전하다.

```
std::function<void()> localCallback;

{
    std::lock_guard lock{mutex};
    localCallback = callback;
}

if (localCallback)
{
    localCallback();
}
```

공유 상태의 불변식을 완성하기 전에 잠금을 해제해서는 안 된다. 잠금 범위는 짧아야 하지만 상태 변경의 원자성을 깨뜨리지 않아야 한다.

16.5 조건 변수

조건 변수(condition variable)는 공유 상태가 특정 조건을 만족할 때까지 스레드를 효율적으로 대기시킨다. 반복문으로 계속 상태를 확인하는 바쁜 대기는 처리 시간을 낭비한다.

16.5.1 std::condition_variable

작업 큐가 비어 있을 때 소비자 스레드가 계속 검사하는 코드는 비효율적이다.

```
while (true)
{
    std::lock_guard lock{mutex};

    if (!tasks.empty())
    {
        Process(tasks.front());
        tasks.pop();
    }
}
```

std::condition_variable은 작업이 추가될 때까지 소비자를 잠들게 할 수 있다. <condition_variable> 헤더가 필요하다.

```
std::mutex mutex;
std::condition_variable condition;
std::queue<Task> tasks;
```

대기에는 std::unique_lock<std::mutex>가 필요하다. 조건 변수는 대기 과정에서 뮤텍스를 해제하고 깨어날 때 다시 잠가야 하기 때문이다.

16.5.2 공유 상태와 조건식

조건 변수에서 핵심은 통지가 아니라 공유 상태이다. 큐에 작업이 있거나 종료가 요청되었다는 상태를 뮤텍스로 보호한다.

```
std::mutex mutex;
std::condition_variable condition;
std::queue<Task> tasks;
bool stopped = false;
```

소비자는 조건식을 사용해 대기한다.

```
std::unique_lock lock{mutex};

condition.wait(lock, [&]
{
    return !tasks.empty() || stopped;
});
```

조건식은 잠금을 보유한 상태에서 검사된다. 조건이 거짓이면 wait()가 잠금을 해제하고 스레드를 대기시킨다. 깨어나면 잠금을 다시 획득하고 조건식을 다시 검사한다.
생산자는 같은 뮤텍스로 공유 상태를 변경한다.

```
{
    std::lock_guard lock{mutex};
    tasks.push(std::move(task));
}

condition.notify_one();
```

상태 변경과 조건 검사가 같은 뮤텍스로 연결되어야 통지 시점과 대기 전환 사이의 경쟁을 피할 수 있다.

16.5.3 대기, 통지와 가짜 깨어남 (spurious wakeup)

wait()는 통지 없이도 반환할 수 있다. 이를 가짜 깨어남(spurious wakeup)이라고 한다. 통지를 받았더라도 다른 소비자가 먼저 작업을 가져가 조건이 다시 거짓이 될 수 있다.
조건식 오버로드는 반복 검사 구조를 내부에서 처리한다.

```
condition.wait(lock, [&]
{
    return !tasks.empty() || stopped;
});
```

조건식 없는 형태를 사용한다면 반복문이 필요하다.

```
while (tasks.empty() && !stopped)
{
    condition.wait(lock);
}
```

notify_one()은 대기 중인 스레드 하나를 깨우고, notify_all()은 모두를 깨운다.

```
condition.notify_one();
condition.notify_all();
```

작업 하나가 추가되었다면 보통 notify_one()이 적합하다. 종료 상태처럼 모든 대기자가 확인해야 하는 변화에는 notify_all()을 사용할 수 있다.
통지는 조건을 참으로 만들지 않는다. 공유 상태를 먼저 변경하고 대기 스레드가 상태를 다시 검사하도록 깨운다.

16.5.4 생산자와 소비자

생산자-소비자(Producer-Consumer) 패턴은 작업을 만드는 스레드와 처리하는 스레드를 분리하는 구조이다. 생산자는 작업을 큐에 넣고, 소비자는 큐에서 작업을 꺼내 처리한다.
소비자는 큐의 상태를 주기적으로 확인하는 폴링 방식을 사용할 수도 있고, 조건 변수나 이벤트를 이용해 작업이 들어올 때까지 대기할 수도 있다.

```
생산자 : [작업 생성] ──▶ [공유 작업 큐에 저장]

소비자 : [공유 작업 큐 확인] ──▶ [작업 꺼내기] ──▶ [작업 처리]
```

이 예제는 조건 변수를 이용해 소비자가 작업이 들어올 때까지 대기하는 방식이다. 생산자와 소비자는 TaskQueue를 통해서만 작업을 주고받으며, 대기와 통지는 공유 큐 내부에서 처리한다.

```
#include <condition_variable>
#include <iostream>
#include <mutex>
```

```cpp
#include <queue>
#include <thread>

// 생산자, 소비자 태스크.
class TaskQueue
{
public:
    void Push(int value)
    {
        {
            std::lock_guard lock{mutex};
            tasks.push(value);
        }

        // 대기 중인 스레드에 큐 상태 변경을 통지.
        condition.notify_one();
    }

    bool WaitPop(int& value)
    {
        std::unique_lock lock{mutex};

        // 작업이 들어오거나 종료 상태가 될 때까지 대기.
        condition.wait(lock, [&]
        {
            return !tasks.empty() || stopped;
        });

        if (tasks.empty())
        {
            return false;
        }

        value = tasks.front();
        tasks.pop();
        return true;
    }

    void Stop()
    {
        {
            std::lock_guard lock{mutex};
            stopped = true;
        }

        // 종료 상태 변경을 모든 대기 스레드에 통지.
        condition.notify_all();
    }

private:
    std::mutex mutex;
    std::condition_variable condition;
    std::queue<int> tasks;
    bool stopped = false;
};
```

```cpp
// 생산자, 소비자 스래드 활용 예.
int main()
{
    TaskQueue queue;

    // 소비자: 큐에서 작업을 꺼내 처리.
    std::jthread consumer{
        [&]
        {
            int value = 0;

            while (queue.WaitPop(value))
            {
```

```
                std::cout << value << '\n';
            }
        }};

    // 생산자: 작업을 만들어 큐에 저장.
    std::jthread producer{
        [&]
        {
            queue.Push(10);
            queue.Push(20);
            queue.Push(30);
            queue.Stop();
        }};
}
```

Push()는 작업을 큐에 저장하고 큐의 상태 변경을 통지한다. WaitPop()은 작업이 있거나 종료 상태가 될 때까지 대기한 뒤 작업을 하나 꺼낸다. Stop()은 종료 상태를 설정하고 대기 중인 스레드에 상태 변경을 통지한다. 큐에 작업이 남아 있다면 종료 요청 이후에도 남은 작업을 먼저 처리한다.

16.5.5 시간 제한 대기

wait_for()는 상대 시간, wait_until()은 절대 시점까지 기다린다.

```
using namespace std::chrono_literals;

bool ready = condition.wait_for(lock, 500ms, [&]
{
    return !tasks.empty() || stopped;
});

if (!ready)
{
    std::cout << "timeout\n";
}
```

조건식 오버로드의 반환값은 조건이 참인지 나타낸다. 시간 초과가 발생했더라도 종료나 작업 추가가 거의 같은 시점에 일어날 수 있으므로 공유 상태를 잠금 안에서 판단해야 한다.

반복되는 상대 시간 대기를 조건식 없이 직접 구성하면 가짜 깨어남마다 제한 시간이 다시 시작되어 전체 대기 시간이 길어질 수 있다. 절대 종료 시점을 계산해 wait_until()을 사용하거나 조건식 오버로드를 사용한다.

16.6 비동기 작업과 결과 전달

스레드가 계산한 값을 호출한 쪽으로 전달하려면 실행 흐름과 결과 저장 위치를 연결해야 한다. std::future와 관련 타입은 결과와 예외를 공유 상태를 통해 전달한다.

16.6.1 std::future와 공유 상태

std::future<T>는 나중에 준비될 T 결과를 참조한다.

```
#include <future>
#include <iostream>

int Calculate()
{
    return 40 + 2;
}

int main()
{
    std::future<int> result = std::async(
        std::launch::async,
        Calculate);

    std::cout << result.get() << '\n';
}
```

get()은 결과가 준비될 때까지 기다린 뒤 값을 반환한다. 작업에서 예외가 발생했다면 get()이 같은 예외를 다시 던진다. get()은 공유 상태에서 결과를 꺼내므로 한 번만 호출할 수 있다. valid()로 결과 상태와 연결되어 있는지 확인할 수 있다.

```
int value = result.get();

if (!result.valid())
{
    std::cout << "no shared state\n";
}
```

여러 곳에서 같은 결과를 읽어야 한다면 std::shared_future<T>를 사용한다.

```
std::shared_future<int> shared = std::move(result);
int first = shared.get();
int second = shared.get();
```

shared_future의 결과가 참조 형태로 반환될 수 있으므로 저장된 결과의 변경 가능성과 수명을 고려해야 한다.

16.6.2 std::promise

std::promise<T>는 생산자가 공유 상태에 값이나 예외를 설정하게 한다. 소비자는 get_future()로 연결된 future를 얻는다.

```
#include <future>
#include <iostream>
#include <thread>

void Produce(std::promise<int> promise)
{
    promise.set_value(100);
}

int main()
{
    std::promise<int> promise;
    std::future<int> result = promise.get_future();

    std::jthread worker{Produce, std::move(promise)};

    std::cout << result.get() << '\n';
}
```

std::promise는 이동 전용이다. 새 스레드에 소유권을 넘기기 위해 std::move()를 사용한다.
예외도 저장할 수 있다.

```
void Produce(std::promise<int> promise)
{
    try
    {
        promise.set_value(Calculate());
    }
    catch (...)
    {
        promise.set_exception(std::current_exception());
    }
}
```

값이나 예외를 설정하지 않은 채 promise가 소멸하면 연결된 future에서 std::future_error가 발생할 수 있다. 오류 코드는 std::future_errc::broken_promise 에 해당한다.

16.6.3 std::packaged_task

std::packaged_task는 호출 가능한 객체와 future 공유 상태를 연결한다.

```
#include <future>
#include <iostream>
```

```cpp
#include <thread>

int Add(int left, int right)
{
    return left + right;
}

int main()
{
    std::packaged_task<int(int, int)> task{Add};
    std::future<int> result = task.get_future();

    std::jthread worker{std::move(task), 10, 20};

    std::cout << result.get() << '\n';
}
```

task를 호출하면 반환값이나 예외가 공유 상태에 저장된다. 작업 큐에는 매개변수가 없는 std::packaged_task<void()> 형태로 감싸 넣을 수 있다.

```cpp
std::packaged_task<int()> task{
    []
    {
        return Calculate();
    }};
```

std::packaged_task는 스레드를 직접 만들지 않는다. 어느 스레드에서 언제 호출할지는 프로그램이 결정한다.

16.6.4 std::async와 실행 정책

std::async()는 호출 가능한 객체를 실행하고 결과를 future로 반환한다.

```cpp
auto result = std::async(
    std::launch::async,
    Calculate);
```

std::launch::async는 별도의 실행 스레드에서 작업하도록 요구한다. std::launch::deferred는 get()이나 wait()가 호출될 때 그 호출을 수행한 스레드에서 작업을 실행한다.

```cpp
auto deferred = std::async(
    std::launch::deferred,
    Calculate);
```

정책을 생략하면 구현은 비동기 실행과 지연 실행을 선택할 수 있다.

```cpp
auto result = std::async(Calculate);
```

동시 실행이 프로그램 의미에 필요하다면 std::launch::async를 명시한다. 기본 정책은 성능 힌트 이상의 의미를 요구하는 코드에 적합하지 않다.
std::async가 만든 공유 상태의 마지막 future가 소멸할 때 비동기 작업이 끝날 때까지 기다릴 수 있다.

```cpp
std::async(std::launch::async, RunTask);
```

반환된 임시 future가 문장 끝에서 소멸하면서 작업 완료를 기다릴 수 있으므로 비동기 작업 두 개를 연속으로 작성해도 실제로 직렬 실행될 수 있다. 결과 객체를 변수에 보관해 수명을 명확히 한다.

```cpp
auto first = std::async(std::launch::async, FirstTask);
auto second = std::async(std::launch::async, SecondTask);

first.get();
second.get();
```

std::async는 표준 스레드 풀 인터페이스가 아니다. 실행 자원 재사용이나 작업 우선순위가 필요하면 별도의 작업 시스템이 필요하다.

16.6.5 결과와 예외 전달

비동기 작업에서 발생한 예외는 공유 상태에 저장되고 get()에서 다시 발생한다.

```
auto result = std::async(
    std::launch::async,
    []() -> int
    {
        throw std::runtime_error{"calculation failed"};
    });

try
{
    int value = result.get();
}
catch (const std::exception& exception)
{
    std::cout << exception.what() << '\n';
}
```

wait()는 준비될 때까지 기다리지만 결과나 예외를 꺼내지 않는다.

```
result.wait();
int value = result.get();
```

wait_for()와 wait_until()로 준비 상태를 확인할 수 있다.

```
using namespace std::chrono_literals;

if (result.wait_for(100ms) == std::future_status::ready)
{
    std::cout << result.get() << '\n';
}
```

지연 실행 상태에서는 std::future_status::deferred가 반환될 수 있다.

16.7 원자적 연산

std::atomic<T>는 해당 객체에 대한 읽기와 쓰기를 원자적으로 수행한다. 다른 스레드가 연산 중간 상태를 관찰하지 못하며, 정해진 메모리 순서에 따라 다른 메모리 접근의 가시성도 연결할 수 있다.

16.7.1 std::atomic

원자적 카운터는 뮤텍스 없이 여러 스레드에서 증가시킬 수 있다.

```
#include <atomic>
#include <iostream>
#include <thread>

std::atomic<int> counter = 0;

void Increase()
{
    for (int i = 0; i < 100'000; ++i)
    {
        ++counter;
    }
}

int main()
{
    std::jthread first{Increase};
    std::jthread second{Increase};

    first.join();
    second.join();
```

```
    std::cout << counter.load() << '\n';
}
```

std::atomic은 복사할 수 없다. 여러 스레드가 같은 원자 객체를 공유해 사용한다.

원자적 타입이 반드시 잠금 없이 구현되는 것은 아니다. is_lock_free()로 특정 객체의 구현 특성을 확인할 수 있으며 is_always_lock_free는 해당 타입이 구현에서 항상 잠금 없는지 나타낸다.

```
std::atomic<long long> value = 0;
std::cout << std::boolalpha << value.is_lock_free() << '\n';
```

잠금 없는 구현이라는 사실만으로 코드가 빠르거나 전체 알고리즘이 대기 없이 진행된다는 의미는 아니다.

16.7.2 원자적 읽기, 쓰기와 읽기-수정-쓰기

load()와 store()는 값을 읽고 쓴다.

```
std::atomic<bool> ready = false;
ready.store(true);
bool state = ready.load();
```

exchange()는 새 값을 저장하고 이전 값을 반환한다.

```
bool wasReady = ready.exchange(true);
```

정수 원자 타입은 읽기-수정-쓰기 연산을 제공한다.

```
std::atomic<int> count = 0;

int oldValue = count.fetch_add(1);
int newValue = ++count;
```

fetch_add(1)은 증가 전 값을 반환하고 전위 ++는 증가 후 값을 반환한다.
원자 객체를 여러 연산으로 나누면 전체 식이 하나의 원자적 연산이 되지 않을 수 있다.

```
std::atomic<int> count = 0;
count = count + 1;
```

읽기와 쓰기는 각각 원자적이지만 그 사이에 다른 스레드가 값을 변경할 수 있다. 증가에는 fetch_add()나 ++를 사용한다.
원자 객체 하나만으로 여러 변수의 관계를 보호할 수 없다.

```
std::atomic<int> x = 0;
std::atomic<int> y = 0;
```

x와 y가 항상 함께 변경되어야 한다면 두 개의 독립된 원자 연산 사이의 상태를 다른 스레드가 볼 수 있다. 여러 값의 불변식에는 뮤텍스가 더 적합하다.

16.7.3 비교와 교환

비교와 교환(compare-and-exchange)은 현재 값이 기대값과 같을 때 새 값으로 변경한다.

```
std::atomic<int> state = 0;
int expected = 0;

bool changed = state.compare_exchange_strong(expected, 1);
```

state가 0이면 1로 변경하고 true를 반환한다. 값이 다르면 변경하지 않고 expected에 실제 값을 기록한 뒤 false를 반환한다.
최대값을 원자적으로 갱신하는 반복문을 만들 수 있다.

```
void UpdateMaximum(std::atomic<int>& maximum, int value)
{
```

```
    int current = maximum.load();

    while (current < value &&
           !maximum.compare_exchange_weak(current, value))
    {
    }
}
```

실패하면 current가 현재 원자값으로 갱신되므로 조건을 다시 검사할 수 있다. compare_exchange_weak()는 값이 같아도 실패할 수 있어 반복문에 적합하다. compare_exchange_strong()은 이러한 허위 실패를 허용하지 않지만 다른 스레드의 경쟁 때문에 여전히 실패할 수 있다. 비교와 교환은 잠금 없는 자료구조의 기반이 되지만 메모리 회수와 ABA 문제까지 해결하지 않는다. 고급 무잠금 구조는 단순한 CAS 반복문보다 훨씬 많은 설계가 필요하다.

16.7.4 atomic wait와 notify

C++20의 원자 객체는 특정 값이 바뀔 때까지 대기할 수 있다.

```
#include <atomic>
#include <iostream>
#include <thread>

std::atomic<int> state = 0;

void Consumer()
{
    state.wait(0);
    std::cout << "state: " << state.load() << '\n';
}

int main()
{
    std::jthread worker{Consumer};

    state.store(1);
    state.notify_one();
}
```

wait(oldValue)는 원자값이 oldValue와 같은 동안 대기한다. 값이 달라지면 반환한다. 값을 변경한 쪽은 notify_one()이나 notify_all()로 대기자를 깨울 수 있다.
값 하나의 변화만 기다리는 구조에는 간결하다. 큐가 비어 있지 않음과 종료 상태처럼 여러 조건을 함께 검사하거나 복합 상태를 보호해야 한다면 뮤텍스와 조건 변수가 더 분명하다.

16.7.5 메모리 순서 개요

원자 연산은 메모리 순서를 지정할 수 있다. 순서를 생략하면 std::memory_order_seq_cst가 사용된다.

메모리 순서	중심 의미
std::memory_order_seq_cst	원자 연산에 가장 강한 전역 순서를 제공하는 기본값
std::memory_order_release	앞선 쓰기를 다른 스레드에 게시
std::memory_order_acquire	release로 게시된 쓰기를 관찰
std::memory_order_acq_rel	읽기-수정-쓰기에서 acquire와 release를 함께 적용
std::memory_order_relaxed	원자성은 유지하지만 다른 메모리 접근의 순서를 연결하지 않음

게시와 관찰은 release와 acquire로 구성할 수 있다.

```
int data = 0;
std::atomic<bool> ready = false;

void Produce()
{
    data = 100;
```

```
    ready.store(true, std::memory_order_release);
}

void Consume()
{
    while (!ready.load(std::memory_order_acquire))
    {
    }

    std::cout << data << '\n';
}
```

acquire 읽기가 release 쓰기의 값을 관찰하면 data = 100도 소비자에서 관찰할 수 있다. data 자체는 원자 타입이 아니지만 동기화 관계가 접근 순서를 연결한다.

memory_order_relaxed는 카운터처럼 그 값의 원자성만 필요하고 다른 데이터의 게시 순서를 표현하지 않을 때 사용할 수 있다.

```
requestCount.fetch_add(1, std::memory_order_relaxed);
```

약한 메모리 순서는 코드의 정확성을 증명할 수 있을 때만 사용해야 한다. 기본 순서가 병목이라는 측정 없이 미리 약화하면 검증 비용과 오류 위험이 커진다.

16.7.6 뮤텍스와 원자적 연산의 선택

원자적 연산은 단일 값의 독립적인 상태 변화에 적합하다.

- 카운터 증가
- 중지 플래그
- 간단한 상태 전환
- 한 포인터의 원자적 교체

뮤텍스는 여러 값의 관계와 복합 연산에 적합하다.

- 컨테이너 삽입과 제거
- 잔액과 거래 횟수의 일관된 변경
- 검사 후 변경을 한 단위로 처리
- 긴 상태 전이와 예외 처리

```
class BoundedValue
{
public:
    bool Add(int amount)
    {
        std::lock_guard lock{mutex};

        if (value + amount > maximum)
        {
            return false;
        }

        value += amount;
        return true;
    }

private:
    std::mutex mutex;
    int value = 0;
    int maximum = 100;
};
```

검사와 변경이 한 잠금 안에 있어야 최대값 불변식이 유지된다. 원자 타입을 사용한다면 CAS 반복문이 필요해 코드가 더 복잡해질 수 있다.

16.8 C++20 동기화 도구

C++20은 작업 진행 시점을 조정하는 std::latch, std::barrier, std::counting_semaphore를 제공한다. 이 도구들은 공유 데이터의 임계 구역을 보호하는 뮤텍스와 목적이 다르다.

16.8.1 std::latch

std::latch는 내부 카운터가 0이 될 때까지 스레드를 대기시킨다. 카운터가 0이 된 뒤에는 다시 초기화할 수 없다.

```
#include <iostream>
#include <latch>
#include <thread>

int main()
{
    std::latch ready{3};

    auto worker = [&](int id)
    {
        std::cout << "prepare " << id << '\n';
        ready.count_down();
        ready.wait();
        std::cout << "start " << id << '\n';
    };

    std::jthread first{worker, 1};
    std::jthread second{worker, 2};
    std::jthread third{worker, 3};
}
```

각 작업은 준비를 끝낸 뒤 카운터를 줄이고 모든 작업이 준비될 때까지 기다린다. count_down()과 wait()를 연속으로 수행하려면 arrive_and_wait()를 사용할 수 있다.

```
ready.arrive_and_wait();
```

latch는 초기화 완료, 여러 파일 로딩 완료, 테스트 작업 시작선처럼 한 번만 열리는 관문에 적합하다.

16.8.2 std::barrier

std::barrier는 정해진 수의 참여자가 도착할 때까지 기다리고, 모두 도착하면 새로운 단계로 재사용된다.

```
#include <barrier>
#include <iostream>
#include <thread>

int main()
{
    std::barrier syncPoint{2};

    auto worker = [&](int id)
    {
        for (int phase = 0; phase < 3; ++phase)
        {
            std::cout << id << ": phase " << phase << '\n';
            syncPoint.arrive_and_wait();
        }
    };

    std::jthread first{worker, 1};
    std::jthread second{worker, 2};
}
```

각 단계에서 두 스레드가 모두 도착해야 이후 반복으로 진행한다. 단계 완료 함수를 지정하면 모든 참여자가 도착한 시점에 한 번 실행할 수 있다.

```
std::barrier syncPoint{
    2,
    []() noexcept
    {
        std::cout << "phase completed\n";
    }};
```

참여자가 이후 단계에서 빠져야 하면 arrive_and_drop()을 사용할 수 있다.

16.8.3 std::counting_semaphore

세마포어는 동시에 통과할 수 있는 실행 흐름의 수를 제한한다.

```
#include <chrono>
#include <iostream>
#include <semaphore>
#include <thread>

using namespace std::chrono_literals;

std::counting_semaphore<2> slots{2};

void UseResource(int id)
{
    slots.acquire();

    std::cout << "enter " << id << '\n';
    std::this_thread::sleep_for(100ms);
    std::cout << "leave " << id << '\n';

    slots.release();
}

int main()
{
    std::jthread first{UseResource, 1};
    std::jthread second{UseResource, 2};
    std::jthread third{UseResource, 3};
}
```

초기값이 2이므로 최대 두 스레드가 자원 구역에 들어갈 수 있다. acquire()는 허가를 얻을 때까지 기다리고 release()는 허가 수를 늘린다. std::binary_semaphore는 최대 카운트가 1인 별칭이다. 세마포어는 소유 스레드 개념이 없으므로 한 스레드가 획득하고 다른 스레드가 해제할 수도 있다. 뮤텍스는 잠근 스레드가 해제해야 하며 임계 구역의 상호 배제를 표현한다.

도구	용도	재사용
std::latch	정해진 작업 수의 1회 완료 대기	불가
std::barrier	여러 단계의 동기화 지점	가능
std::counting_semaphore	동시 사용 가능한 자원 수 제한	가능

16.9 병렬 알고리즘 개요

표준 알고리즘의 일부 오버로드는 실행 정책을 받아 병렬화나 벡터화를 허용한다. <execution> 헤더에 실행 정책 객체가 정의된다.

16.9.1 실행 정책

```
#include <algorithm>
#include <execution>
#include <vector>

std::vector<int> values{5, 4, 3, 2, 1};

std::sort(
    std::execution::par,
    values.begin(),
    values.end());
```

정책	의미
std::execution::seq	호출을 순차적으로 실행

std::execution::par	여러 스레드의 병렬 실행을 허용
std::execution::par_unseq	병렬 실행과 벡터화를 허용
std::execution::unseq	C++20에서 벡터화된 비순차 실행을 허용

실행 정책은 허용 범위를 지정한다. 구현이 실제로 몇 개의 스레드를 만들거나 병렬화할지는 보장되지 않는다.

16.9.2 병렬 실행의 조건

병렬 알고리즘은 각 원소에 대한 작업이 서로 독립적일 때 적합하다.

```
std::transform(
    std::execution::par,
    source.begin(),
    source.end(),
    destination.begin(),
    [](float value)
    {
        return value * value;
    });
```

각 호출은 하나의 입력값으로 하나의 결과를 계산하고 다른 원소의 상태를 변경하지 않는다.
작업량이 작거나 원소 수가 적으면 분할과 동기화 비용이 계산 비용보다 클 수 있다. 메모리 대역폭이 병목인 연산은 처리 장치를 늘려도 성능이 거의 향상되지 않을 수 있다.
병렬 알고리즘 구현 지원도 표준 라이브러리와 빌드 환경에 따라 다를 수 있다. 사용 환경에서 기능과 성능을 직접 확인해야 한다.

16.9.3 부작용, 데이터 경쟁과 예외

호출 함수가 공유 변수에 동기화 없이 기록하면 데이터 경쟁이 발생한다.

```
int total = 0;

std::for_each(
    std::execution::par,
    values.begin(),
    values.end(),
    [&](int value)
    {
        total += value; // 안전하지 않음
    });
```

합계에는 병렬 축약을 지원하는 알고리즘을 사용할 수 있다.

```
#include <numeric>

int total = std::reduce(
    std::execution::par,
    values.begin(),
    values.end(),
    0);
```

std::reduce()는 연산 순서를 바꿀 수 있으므로 결합법칙이 성립하지 않는 연산이나 부동소수점 결과의 정확한 재현성에는 주의해야 한다.
par_unseq와 unseq에서는 여러 호출이 같은 스레드 안에서도 순서 없이 섞일 수 있다. 뮤텍스 잠금, 메모리 할당처럼 진행을 막거나 벡터화에 부적합한 연산을 함수 객체 안에서 수행해서는 안 된다.

표준 실행 정책을 사용하는 병렬 알고리즘에서 사용자 함수가 예외를 밖으로 내보내면 std::terminate()가 호출될 수 있다. 오류를 반환값으로 표현하거나 알고리즘 밖에서 입력을 검증하는 설계가 필요하다.

16.9.4 지원 범위와 성능 측정

병렬화 전에는 순차 구현의 실행 시간을 측정한다. 병렬 버전은 같은 입력과 빌드 설정으로 반복 측정해야 한다.
검토할 항목은 작업 크기, 원소 수, 스레드 생성과 분할 비용, 메모리 접근 패턴, 캐시 적중률, 잠금 경합과 처리 장치 사용률이다.

```
const auto start = std::chrono::steady_clock::now();
```

```
RunAlgorithm();

const auto end = std::chrono::steady_clock::now();
const auto elapsed =
    std::chrono::duration_cast<std::chrono::microseconds>(end - start);
```

한 번의 측정만으로 결론을 내리기 어렵다. 초기 실행 비용과 시스템 부하를 고려해 여러 번 측정하고 중앙값이나 분포를 비교하는 편이 좋다.

16.10 동시성 설계 기준

동시성 오류는 특정 실행 순서에서만 나타날 수 있어 재현이 어렵다. 잠금을 나중에 덧붙이기보다 데이터 소유권과 작업 경계를 먼저 설계해야 한다.

16.10.1 공유 상태 줄이기

공유되지 않은 데이터에는 동기화가 필요하지 않다. 각 스레드가 지역 결과를 계산한 뒤 마지막에 결합하면 공유 쓰기 횟수를 줄일 수 있다.

```
long long SumRange(const std::vector<int>& values,
                   std::size_t first,
                   std::size_t last)
{
    long long total = 0;

    for (std::size_t i = first; i < last; ++i)
    {
        total += values[i];
    }

    return total;
}
```

각 작업은 입력을 읽기만 하고 자신의 결과를 반환한다. 호출자는 future 결과를 모아 합칠 수 있다.
불변 객체, 값 복사, 메시지 큐, 소유권 이동도 공유 가변 상태를 줄이는 수단이다.

```
queue.Push(std::move(task));
```

작업 소유권을 큐로 이동하면 생산자가 같은 작업 객체를 계속 변경하지 않는다는 규칙을 세울 수 있다.

16.10.2 작업 단위와 소유권

스레드보다 작업을 먼저 정의한다. 작업에는 입력 데이터, 출력 위치, 취소 규칙과 완료 통지가 포함된다.

```
struct LoadTask
{
    std::filesystem::path path;
    std::promise<Data> result;
};
```

std::unique_ptr를 작업에 이동하면 단일 소유권을 유지할 수 있다.

```
void Process(std::unique_ptr<Resource> resource);

std::jthread worker{Process, std::move(resource)};
```

공유 소유권이 실제로 필요하면 std::shared_ptr를 사용할 수 있지만 객체 내부의 가변 상태까지 자동으로 보호되지는 않는다. 참조 횟수 변경은 스레드 안전하게 처리되더라도 대상 객체의 멤버 접근에는 별도 동기화가 필요하다.

16.10.3 스레드 안전한 인터페이스

각 멤버 함수가 뮤텍스를 사용해도 여러 호출을 조합한 연산은 안전하지 않을 수 있다.

```
if (!queue.Empty())
{
    Task task = queue.Front();
```

```
    queue.Pop();
}
```

Empty()와 Front() 사이에 다른 스레드가 마지막 원소를 제거할 수 있다. 검사와 제거를 한 함수로 묶는다.

```
std::optional<Task> TryPop()
{
    std::lock_guard lock{mutex};

    if (tasks.empty())
    {
        return std::nullopt;
    }

    Task task = std::move(tasks.front());
    tasks.pop();
    return task;
}
```

내부 포인터나 참조를 반환하면 잠금이 끝난 뒤 다른 스레드가 대상을 변경하거나 제거할 수 있다.

```
const Task& Front() const;
```

값을 복사하거나 소유권을 이동해 반환하는 인터페이스가 더 안전할 수 있다. 큰 객체의 복사 비용과 호출자의 잠금 유지 요구를 함께 고려한다.

16.10.4 종료와 취소

작업 스레드를 가진 객체는 소멸 전에 스레드를 중지하고 결합해야 한다.

```
class Worker
{
public:
    Worker()
        : thread{[this](std::stop_token stopToken)
        {
            Run(stopToken);
        }}
    {
    }

private:
    void Run(std::stop_token stopToken);

    std::jthread thread;
};
```

멤버 소멸은 선언의 역순으로 진행된다. 스레드가 접근하는 데이터는 std::jthread보다 먼저 선언해 스레드 객체가 먼저 소멸하도록 구성하는 것이 안전하다.

```
class Worker
{
private:
    std::mutex mutex;
    std::condition_variable condition;
    std::queue<Task> tasks;
    std::jthread thread; // 역순 소멸에서 thread가 먼저 정리됨
};
```

조건 변수에서 대기 중인 스레드는 중지 요청만으로 깨어나지 않을 수 있다. 종료 상태를 변경하고 notify_all()을 호출하거나 stop_token을 지원하는 대기 방식을 설계해야 한다.

종료 정책에는 처리 중인 작업, 대기 중인 작업, 새 작업 접수 여부가 포함된다.

- 처리 중인 작업을 완료한 뒤 종료
- 큐에 남은 작업까지 모두 처리
- 대기 작업을 취소하고 즉시 종료

정책이 불분명하면 소멸자가 예상보다 오래 대기하거나 일부 작업이 조용히 사라질 수 있다.

16.10.5 성능 측정과 과도한 병렬화 피하기

스레드 수를 늘리면 작업 분배와 문맥 전환 비용도 증가한다. 실행 가능한 스레드가 처리 장치 수보다 지나치게 많으면 오히려 느려질 수 있다. std::thread::hardware_concurrency()는 구현이 제공하는 동시 실행 단위의 참고값을 반환한다.

```
unsigned int count = std::thread::hardware_concurrency();

if (count == 0)
{
    count = 1;
}
```

반환값은 최적의 작업 스레드 수가 아니며 0일 수도 있다. 작업 특성, 다른 프로세스의 부하와 입출력 대기 비율에 따라 적절한 수가 달라진다.
서로 다른 스레드가 같은 캐시 라인에 있는 독립 변수를 자주 변경하면 거짓 공유(false sharing)가 발생할 수 있다. 논리적으로는 공유하지 않아도 캐시 일관성 트래픽 때문에 성능이 떨어진다.

```
struct alignas(64) Counter
{
    std::atomic<long long> value = 0;
};
```

고정 정렬값을 무조건 사용하기보다 std::hardware_destructive_interference_size 지원 여부와 대상 환경을 고려해야 한다. 구조를 변경하기 전에는 프로파일러로 실제 캐시 경합을 확인한다.
동시성 최적화의 기준은 스레드 수가 아니라 전체 처리 시간, 응답 지연, 처리량과 자원 사용량이다.

16.11 학습 정리

동시성은 여러 작업이 겹쳐 진행되도록 구성하는 성질이고 병렬성은 여러 작업이 실제로 같은 시간에 실행되는 성질이다. 스레드는 같은 프로세스의 메모리를 공유하므로 데이터 전달은 쉽지만 공유 가변 상태에 대한 동기화가 필요하다.

데이터 경쟁은 같은 메모리 위치에 대한 동기화되지 않은 읽기와 쓰기가 겹칠 때 발생하며 프로그램의 동작을 보장할 수 없게 만든다.
뮤텍스와 원자적 연산은 상호 배제와 메모리 가시성을 구성한다.

std::thread는 결합 가능한 상태로 소멸하면 프로그램을 종료하므로 수명을 명시적으로 관리해야 한다. std::jthread는 자동 결합과 중지 요청을 제공하지만 작업 함수가 중지 요청을 확인하고 반환해야 한다.

std::lock_guard, std::scoped_lock, std::unique_lock은 잠금 소유권을 객체 수명과 연결한다. 여러 뮤텍스의 잠금 순서를 통일하고 외부 호출을 잠금 밖으로 분리하면 교착 상태와 경합을 줄일 수 있다.

조건 변수는 공유 상태의 조건이 만족될 때까지 스레드를 대기시킨다. 통지는 상태를 저장하지 않으며 대기자는 조건식을 반복해서 확인해야 한다. 생산자·소비자 구조에는 작업 큐뿐 아니라 종료 상태와 대기자 깨우기 규칙이 필요하다.

std::future, std::promise, std::packaged_task, std::async는 작업 결과와 예외를 공유 상태로 전달한다. std::async의 실행 정책과 반환된 future의 수명은 실제 동시 실행 여부에 영향을 줄 수 있다.

std::atomic은 한 객체에 대한 원자적 연산을 제공한다. 여러 값의 관계나 복합 상태 전이에는 뮤텍스가 더 명확할 수 있다. 약한 메모리 순서는 정확성을 증명하고 성능 병목을 측정한 뒤 적용해야 한다.

std::latch, std::barrier, std::counting_semaphore는 작업 완료 시점, 반복 단계와 동시 자원 수를 조정한다. 병렬 알고리즘은 독립적인 원소 작업에 적합하며 실행 정책을 지정해도 실제 성능 향상은 보장되지 않는다.

안전한 동시성 설계는 공유 상태를 줄이고 작업과 소유권을 분리하며 종료와 취소를 인터페이스에 포함하는 데서 시작한다.

16.12 학습 과제

과제 **1.** 스레드 생성과 결합
두 개의 std::thread로 서로 다른 범위의 정수를 출력한다. 출력 순서가 실행마다 달라질 수 있는 이유를 설명하고 모든 스레드를 join()한다.
join() 호출 하나를 제거했을 때 발생하는 결과와 joinable()의 상태 변화를 확인한다.

과제 **2.** 인수 복사와 참조 전달
정수 변수를 변경하는 함수를 스레드에서 실행한다. 값 전달, std::ref(), 람다의 값 캡처와 참조 캡처를 각각 적용해 원본 변수의 변경 여부를 비교한다.
참조한 지역 객체의 수명이 스레드보다 짧아지는 코드는 실행하지 않고 수명 오류가 발생하는 이유만 분석한다.

과제 **3. jthread** 협력적 중지
std::jthread에서 일정 간격으로 작업 번호를 출력한다. 주 스레드가 1초 뒤 request_stop()을 호출하고 작업 스레드가 요청을 확인해 종료하도록 구성한다.
중지 요청 확인을 제거했을 때 자동 결합이 프로그램 종료를 지연시키는 이유를 설명한다.

과제 **4.** 데이터 경쟁과 뮤텍스
두 스레드가 하나의 카운터를 각각 100만 번 증가시키는 프로그램을 작성한다. 동기화가 없는 버전, std::mutex와 std::lock_guard를 사용하는 버전, std::atomic<int>를 사용하는 버전을 비교한다.
실행 시간과 최종값을 여러 번 기록하되 데이터 경쟁 버전의 결과를 정상 동작으로 해석해서는 안 되는 이유를 작성한다.

과제 **5.** 일관된 상태 반환
잔액과 거래 횟수를 멤버로 가진 클래스를 작성한다. 입금 함수는 두 값을 함께 변경하고 조회 함수는 두 값을 한 번의 잠금 안에서 복사해 반환한다.
두 값을 각각 조회하는 함수로 분리했을 때 호출자가 서로 다른 시점의 값을 받을 수 있는 과정을 설명한다.

과제 **6.** 두 객체 사이의 전송
두 계좌 객체 사이에서 금액을 이동하는 함수를 구현한다. std::scoped_lock으로 두 뮤텍스를 함께 잠그고 같은 객체가 두 인수로 전달된 경우도 처리한다.
두 뮤텍스를 반대 순서로 직접 잠그는 버전에서 교착 상태의 네 조건이 어떻게 성립하는지 분석한다.

과제 **7.** 스레드 안전한 큐
Push(), TryPop(), WaitPop(), Stop()을 가진 작업 큐를 작성한다. WaitPop()은 조건 변수로 작업 또는 종료 상태를 기다린다.
종료 시 남은 작업을 모두 처리하는 정책과 즉시 폐기하는 정책을 각각 구현하고 반환 규칙의 차이를 문서화한다.
과제 **8.** 시간 제한 대기
작업 큐에 WaitPopFor()를 추가한다. 제한 시간 안에 작업이 들어오면 값을 반환하고 시간 초과나 종료 상태이면 구분 가능한 결과를 반환한다.
std::optional<Task>만으로 시간 초과와 종료를 구분할 수 있는지 검토하고 필요하면 std::variant나 열거형 결과 타입을 설계한다.

과제 **9. promise**와 **future**
작업 스레드가 파일 크기를 계산해 std::promise<std::uintmax_t>로 전달한다. 파일 접근 실패는 set_exception()으로 전달하고 호출부에서 future::get()의 예외를 처리한다.
값이나 예외를 설정하지 않고 promise가 소멸했을 때의 오류도 확인한다.

과제 **10. packaged_task** 작업 큐
서로 다른 정수 계산을 std::packaged_task<int()>로 만들고 큐에 저장한다. 작업 스레드가 큐에서 하나씩 꺼내 실행하고 호출자는 각 future에서 결과를 받는다.
packaged_task가 스레드를 직접 생성하지 않는다는 점을 코드의 실행 주체와 연결해 설명한다.

과제 **11. async** 실행 정책
같은 계산 함수를 std::launch::async와 std::launch::deferred로 실행한다. 함수 내부와 호출부의 스레드 ID를 출력해 실행 위치를 비교한다.
정책을 생략한 호출은 특정 실행 방식을 가정하지 않고 관찰 결과만 기록한다.

과제 **12.** 원자적 상태 전환
Idle, Running, Completed 상태를 정수 또는 열거형 기반 std::atomic으로 표현한다. compare_exchange_strong()을 이용해 Idle에서 Running으로 한 스레드만 전환하도록 구현한다.
실패할 때 기대값 인수가 실제 상태로 변경되는 과정을 출력한다.

과제 **13. atomic wait**와 **notify**
소비자 스레드가 std::atomic<int> 값이 0에서 바뀔 때까지 wait()하고 생산자가 값을 기록한 뒤 notify_one()을 호출하도록 구성한다.
같은 문제를 조건 변수로 구현하고 상태 표현, 여러 조건 확장과 코드 복잡도를 비교한다.

과제 **14. latch** 시작선
작업 스레드 네 개가 각자 준비 작업을 마친 뒤 동시에 본 작업을 시작하도록 std::latch를 사용한다.
count_down()과 wait()를 분리한 형태와 arrive_and_wait()를 사용한 형태를 비교한다.

과제 **15. barrier** 단계 처리
여러 스레드가 계산, 결과 교환, 후처리의 세 단계를 반복하도록 std::barrier를 사용한다. 어느 스레드도 다른 스레드가 같은 단계를 끝내기 전에 이후 단계로 진행하지 않게 한다.

단계 완료 함수에서 현재 단계 번호를 증가시키도록 구성한다.

과제 **16. semaphore** 자원 제한
동시에 두 작업만 사용할 수 있는 가상 네트워크 연결 풀을 std::counting_semaphore<2>로 표현한다. 다섯 스레드가 연결을 획득하고 일정 시간 사용한 뒤 반환한다.

뮤텍스 하나로 전체 작업을 직렬화한 경우와 동시에 두 작업을 허용하는 세마포어의 차이를 설명한다.

과제 **17.** 병렬 알고리즘의 부작용
큰 벡터의 각 원소를 제곱하는 std::transform을 순차 정책과 병렬 정책으로 실행해 시간을 측정한다.
람다에서 공유 합계를 직접 변경하는 코드가 안전하지 않은 이유를 작성하고 std::reduce()를 이용한 합계로 교체한다.

과제 **18.** 스레드 안전한 인터페이스

Empty(), Front(), Pop()을 각각 잠그는 큐를 작성한 뒤 세 호출을 조합해도 안전하지 않은 실행 순서를 작성한다.
검사와 제거를 하나의 TryPop()으로 합치고 값 또는 빈 상태를 반환하도록 수정한다.

과제 **19.** 종료 가능한 작업자 클래스
내부에 작업 큐, 조건 변수와 std::jthread를 가진 Worker 클래스를 작성한다. 생성자는 작업 스레드를 시작하고 소멸 과정에서는 새 작업 접수를 막고 대기자를 깨운 뒤 스레드가 끝나도록 한다.
멤버 선언 순서가 소멸 순서와 스레드 안전성에 미치는 영향을 설명한다.

과제 **20.** 동시성 설계 검토
파일 로딩, 이미지 디코딩, 화면 객체 생성으로 이루어진 로딩 시스템을 설계한다. 각 단계에서 실행할 스레드, 데이터 소유권, 결과 전달 방식과 취소 정책을 정한다.
공유 상태, 잠금 대상, 조건 변수의 조건식, 프로그램 종료 순서를 UML 다이어그램으로 표현한다.